

CAP 4. CAMADA DE APLICAÇÃO

AULA 1: INTRODUÇÃO

INE5422 REDES DE COMPUTADORES II
PROF. ROBERTO WILLRICH (INE/UFSC)
ROBERTO.WILLRICH@UFSC.BR
[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Capítulo 2

Camada de aplicação

Nota sobre o uso destes slides ppt:

Estamos disponibilizando estes slides gratuitamente a todos (professores, alunos, leitores). Eles estão em formato do PowerPoint para que você possa incluir, modificar e excluir slides (incluindo este) e o conteúdo do slide, de acordo com suas necessidades. Eles obviamente representam *muito* trabalho da nossa parte. Em retorno pelo uso, pedimos apenas o seguinte:

- ❑ Se você usar estes slides (por exemplo, em sala de aula) sem muita alteração, que mencione sua fonte (afinal, gostamos que as pessoas usem nosso livro!).
- ❑ Se você postar quaisquer slides sem muita alteração em um site Web, que informe que eles foram adaptados dos (ou talvez idênticos aos) nossos slides, e inclua nossa nota de direito autoral desse material.

Obrigado e divirta-se! JFK/KWR

Todo o material copyright 1996-2009

J. F Kurose e K. W. Ross, Todos os direitos reservados.



Capítulo 4: Camada de Aplicação

Metas do capítulo:

- Aspectos conceituais e de implementação de protocolos de aplicação em redes
 - modelos de serviço da camada de transporte
 - paradigma cliente servidor
 - paradigma peer-to-peer
- Estudo dos protocolos da camada de aplicação através das aplicações populares
- Protocolos específicos:
 - DNS
 - HTTP
 - FTP
 - Telnet
 - NFS
 - Email: SMTP/ POP3/ IMAP
- Programação de aplicações de rede
 - programação usando a API de sockets

Capítulo 4: Camada de Aplicação

Nesta aula veremos

- Introdução geral à camada de aplicação
- Modelos Cliente-Servidor e Peer-to-Peer
- Conceito de Socket
- Endereçamento de processos

Camada de Aplicação: Introdução

Algumas aplicações de rede

- e-mail
- web
- mensagem instantânea
- login remoto
- compartilhamento de arquivos P2P
- jogos em rede multiusuários
- Streaming de vídeo
- redes sociais
- Voz over IP (VoIP)
- videoconferência em tempo real
- computação em grade

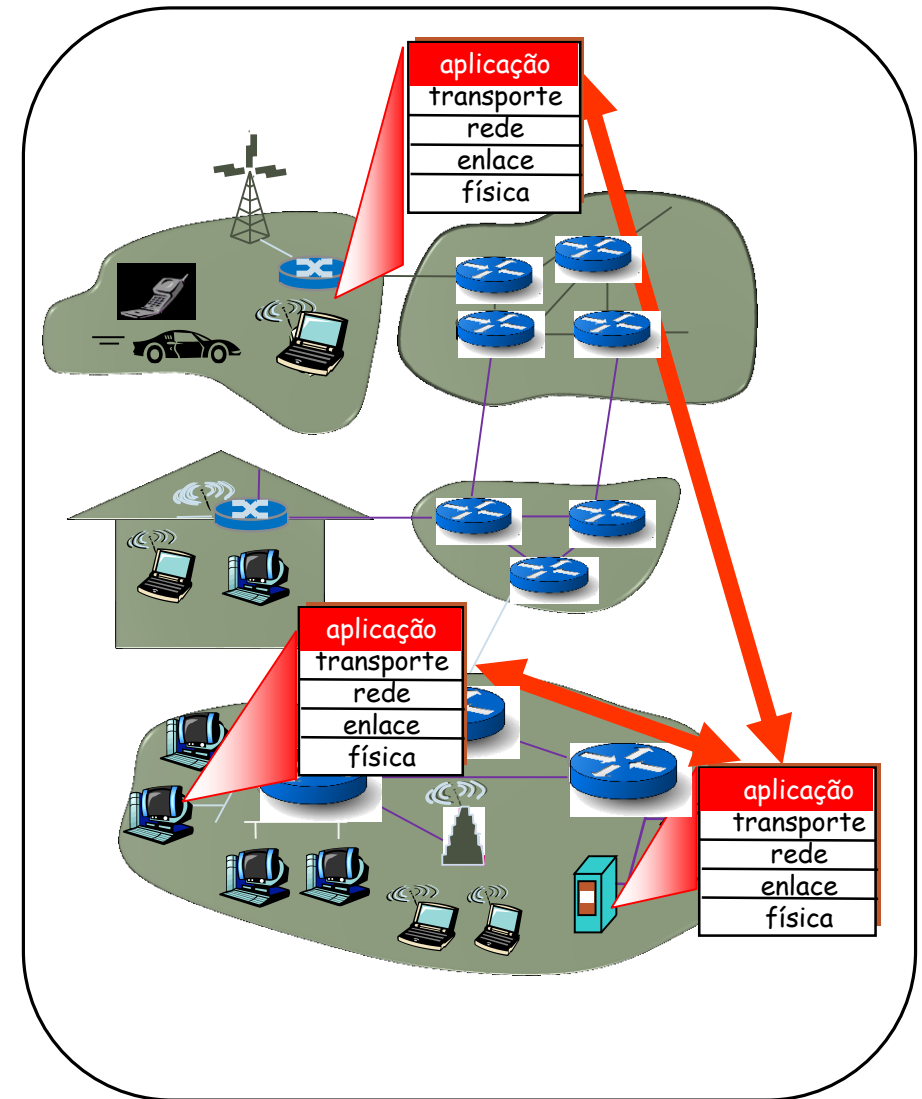
Criando uma aplicação de rede

Programas aplicativos

- Executam em (diferentes) sistemas finais
- Se comunicam pela rede
 - p. e., software de servidor Web se comunica com software de navegador Web

Não é preciso escrever software para dispositivos do núcleo da rede

- Dispositivos do núcleo da rede não executam aplicações do usuário
- As aplicações nos sistemas finais permitem rápido desenvolvimento



Camada de Aplicação: Introdução

- **Os protocolos da camada de aplicação definem**
 - **Tipos de mensagens trocadas**, ex. mensagens de pedido e resposta
 - **Sintaxe dos tipos das mensagens**: campos presentes nas mensagens e como são identificados
 - **Semântica dos campos**, i.e., significado da informação nos campos
 - **Regras** para quando os processos enviam e respondem às mensagens
 - **Protocolos de domínio público**:
 - definidos em RFCs
 - Permitem a interoperação
 - ex, HTTP, FTP, SMTP, DNS,...
 - **Protocolos proprietários**:
 - Ex., KaZaA

Camada de Aplicação: Introdução

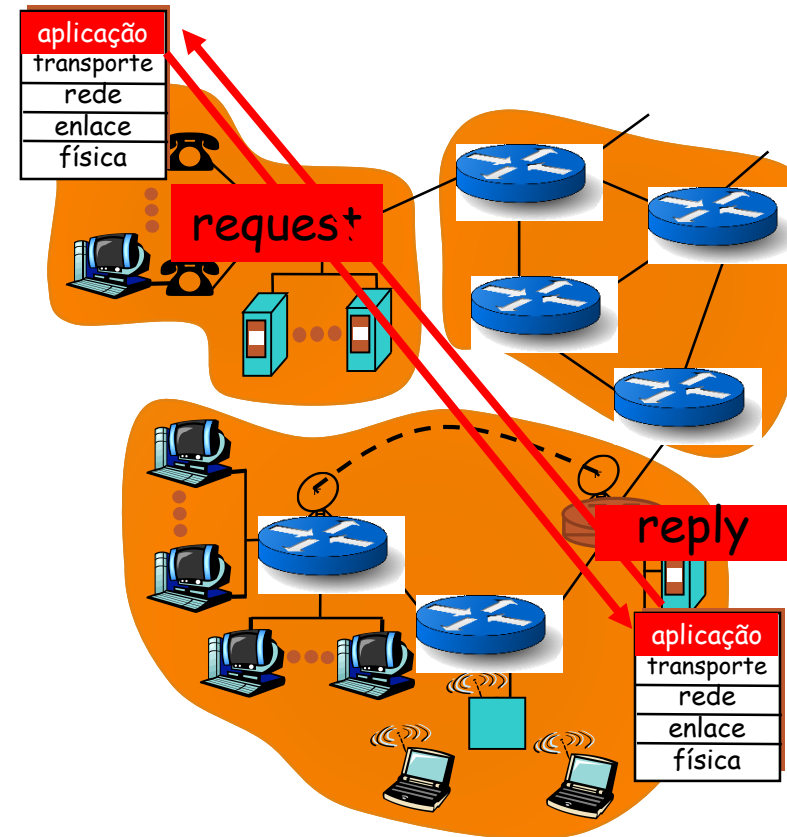
Arquiteturas de aplicação

- Cliente-servidor
 - Incluindo centros de dados/cloud computing
- Peer-to-peer (P2P)
- Híbrida de cliente-servidor e P2P

Camada de Aplicação: Introdução

Arquitetura Cliente Servidor

- Servidor:
 - Fornece o serviço solicitado pelo cliente
 - Hospedeiro sempre ativo
 - Endereço IP permanente
 - Exemplos: servidor Web, servidor de e-mails.
- Cliente:
 - Inicia contato com servidor
 - Normalmente pede um serviço para o servidor
 - Podem ser conectados intermitentemente
 - Podem ter endereço IP dinâmico
 - Não se comunicam diretamente uns com os outros
 - Exemplos: browser, leitor de e-mails



Camada de Aplicação: Introdução

Centros de dados

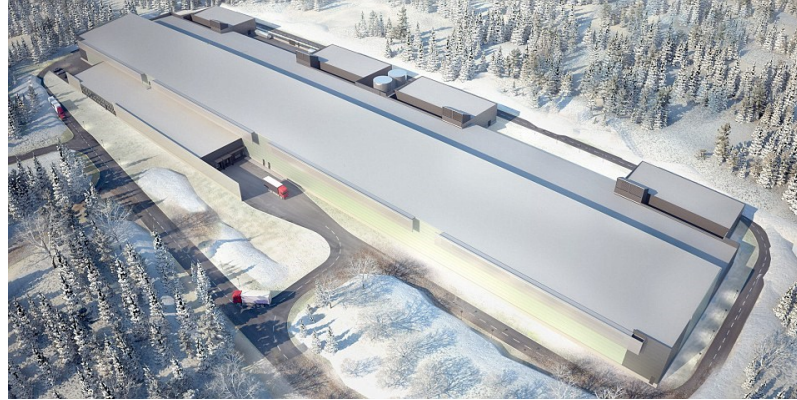
- Infraestrutura de alto custo
- Cada centro de dados consumem muita energia
 - Uma busca no Google gasta energia suficiente para deixar uma lâmpada de 60W acesa durante 17s!



Camada de Aplicação: Introdução

Soluções para datacenters

- Construído no ártico



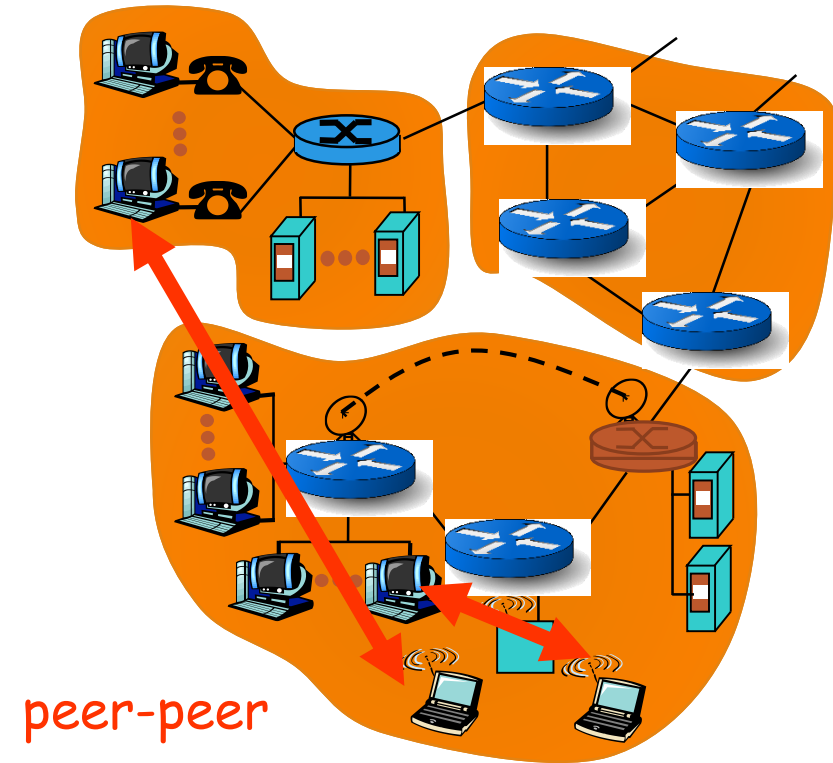
- Microsoft faz testes para um data center submerso



Camada de Aplicação: Introdução

Arquitetura P2P pura

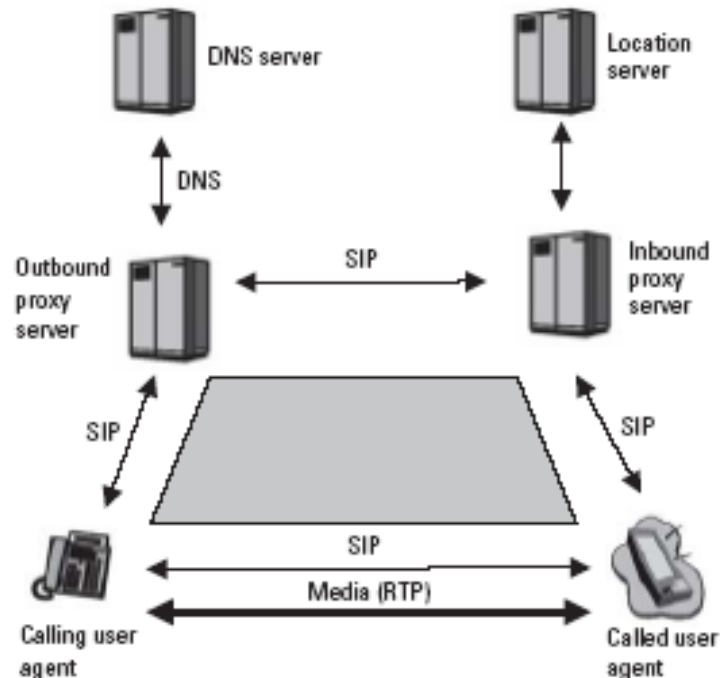
- Nenhum servidor sempre ligado
- Sistemas finais arbitrários se comunicam diretamente
- Pares são conectados intermitentemente e mudam endereços IP
- Altamente escalável, mas difícil de administrar



Híbrido de cliente-servidor e P2P

Mensagem instantânea e outras soluções Voz sobre IP

- Bate-papo entre dois usuários é P2P
- Serviço centralizado: detecção/localização da presença do cliente
 - usuário registra seu endereço IP com servidor central quando entra on-line
 - usuário contacta servidor central para descobrir endereços IP dos parceiros



Aplicações de Rede: Jargão

Um **processo** é um programa que roda em um host

Dentro do mesmo host

- Processos se comunicam usando comunicação interprocessos fornecido pelo S.O.

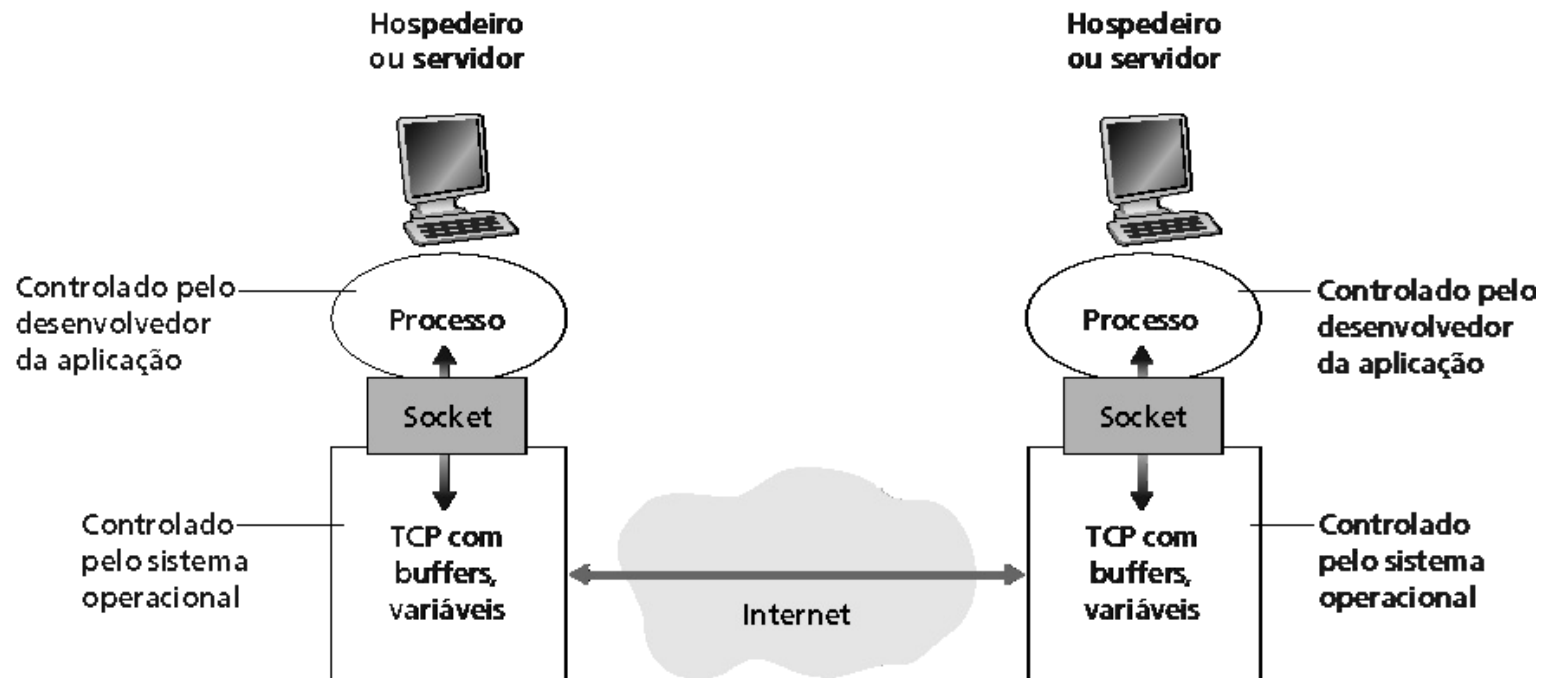
Processos rodando em diferentes hosts

- Se comunicam via protocolo do nível aplicação
- Enviando e recebendo mensagens de suas portas
 - Uma via de acesso ao processo
 - Do outro lado da porta tem processos que enviam a mensagem ao destino

Processos que se comunicam via rede

Socket

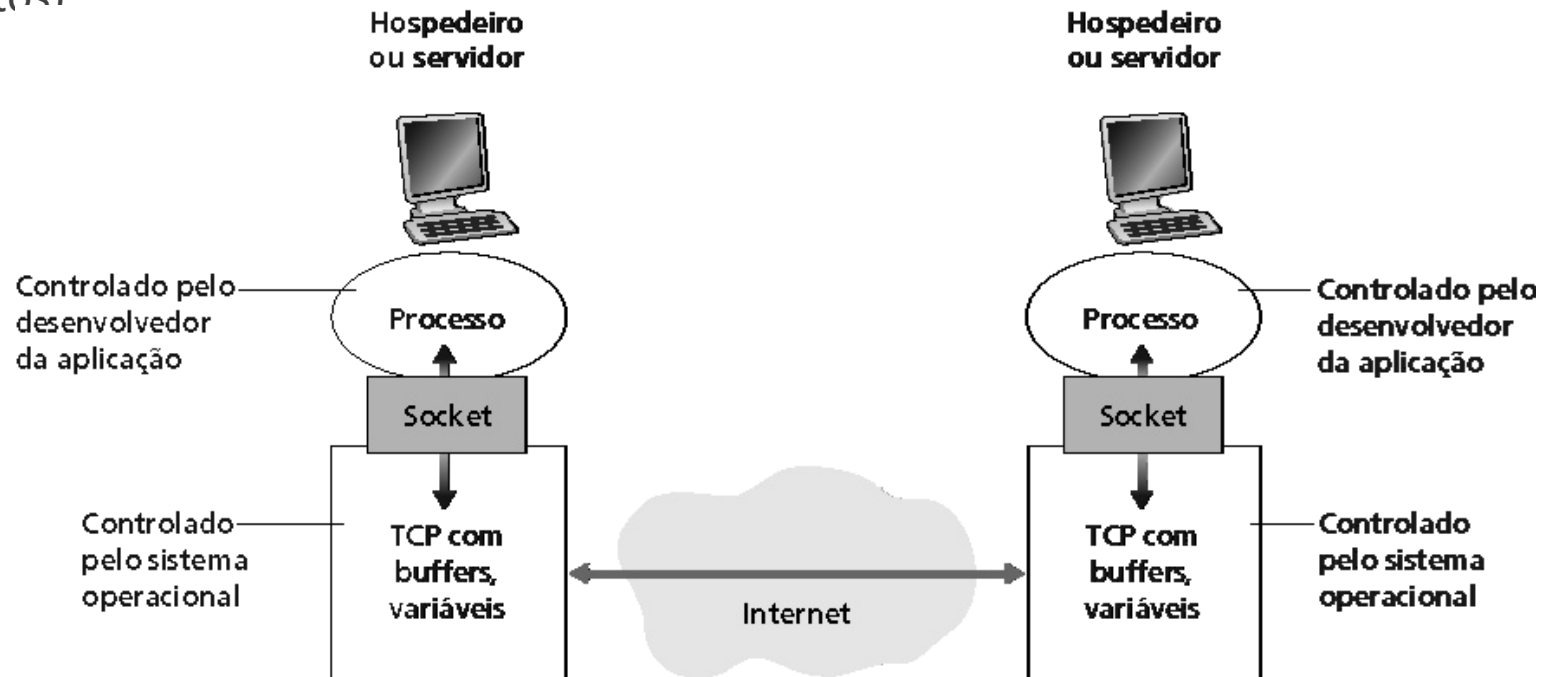
- Um processo envia/recebe mensagens para/de seu socket
 - O socket é análogo a uma porta
 - O processo emissor confia na infraestrutura de transporte no outro lado da porta que leva a mensagem para o socket no processo receptor



Processos que se comunicam via rede

Socket

- API entre a camada de aplicação e a camada de transporte. Desenvolvedor:
 - Cria o programa (processo)
 - Escolhe o protocolo de transporte
 - Fixa alguns parâmetros da camada de transporte (tamanho máximo do buffer e tamanho máximo de segmentos)



Processos que se comunicam via rede

Endereçamento de processos

- Para um processo se comunicar com outro
 - Processo originador tem de identificar o processo destinatário
- Para identificar o processo destinatário deve-se especificar:
 - Nome ou endereço da máquina hospedeira
 - Identificador que especifique a identidade do processo destinatário no hospedeiro de destino
 - O número da porta

Processos que se comunicam via rede

Endereçamento da máquina hospedeira

- Através do endereço IP
 - Valor de 32 bits (IPv4)
 - Ex.: 150.162.60.23
- Identifica unicamente uma máquina na Internet
 - Mais correto: identifica exclusivamente a interface que liga o hospedeiro à rede
- Deve ser gerenciado com cuidado

Processos que se comunicam via rede

Identificação do processo na máquina hospedeira

- Através do número da porta
- Alguns números de portas foram reservados para aplicações mais populares

Serviço	Porta	Descrição
FTP	21	Transferência de Arquivos
Telnet	23	Acesso Remoto
SMTP	25	Envio de Email
DOMAIN	53	Nomes do Domínio
Gopher	70	Browser em modo texto
HTTP	80	WWW
POP3	110	Receber Email
NNTP	119	Newsgroup
IRC	6667	Internet Relay Chat
ICQ	4144	Bate papo
AOL	5190	America On Line
MSN	569	Microsoft Network

Comando netstat (windows&unix)

Mostra status da rede

- Estatísticas e conexões TCP/UDP

Para ver portas usadas e processos

- `netstat -a`

Pontos Importantes

Principais conceitos

- Socket: Interface entre o Processo e o sistema operacional
- Endereçamento de processos (Porta e Endereço IP)

CAP 4. CAMADA DE APLICAÇÃO

AULA 2: SERVIÇOS DA CAMADA DE TRANSPORTE

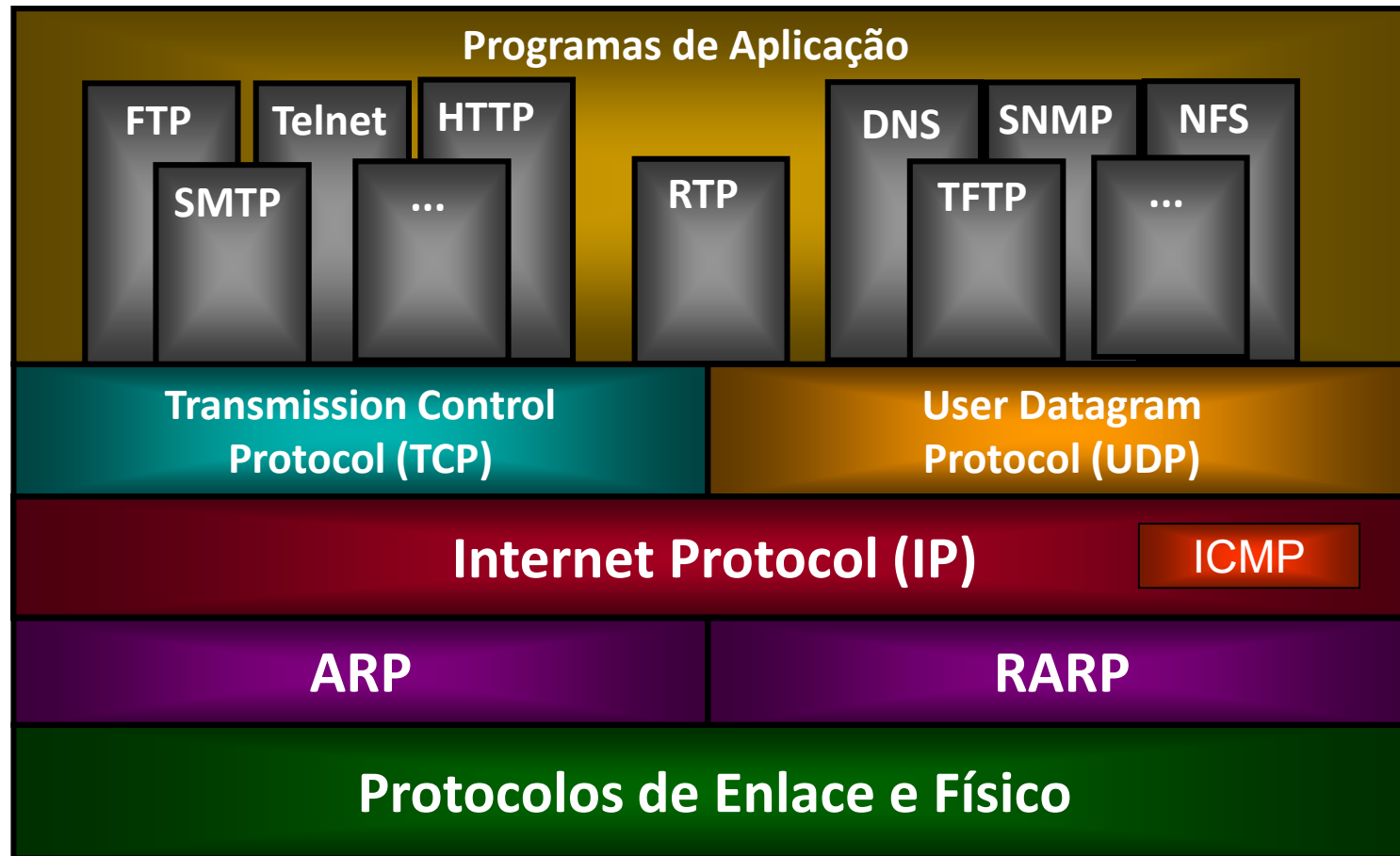
INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Nível Aplicação na Internet



Que tipo de serviço de transporte é necessário?

Confiabilidade

- Algumas aplicações toleram certo nível de perda de dados (p.e. áudio)
- Algumas aplicações necessitam de 100% de confiabilidade (p.e. ftp e telnet)

Taxa de bits

- Algumas aplicações necessitam um valor mínimo de taxa de bits para ser efetiva (p.e. multimídia)
- Outras aplicações precisam apenas de conectividade e não requerem garantias de taxa de bits (aplicações elásticas)

Atraso

- Algumas aplicações requerem pequeno atraso para serem efetivas (p.e. voz a pacotes)

Requisitos de serviços de transporte

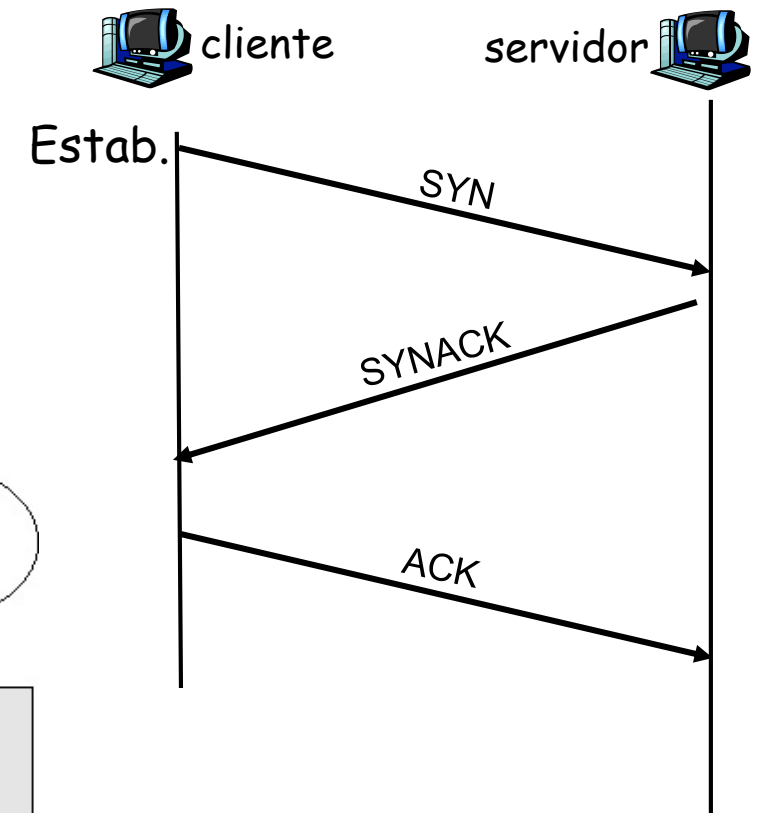
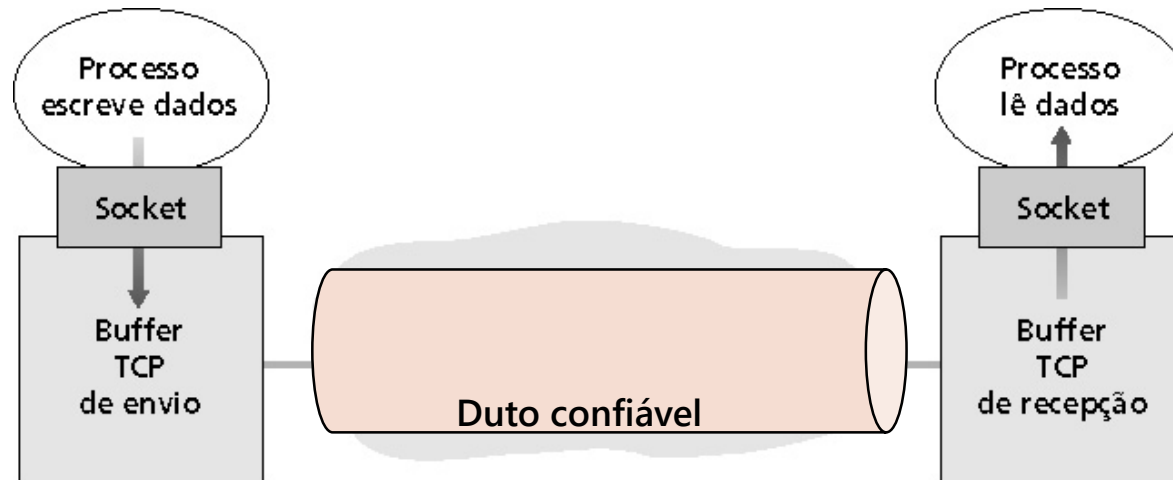
Requisitos de serviços para algumas aplicações comuns

Aplicação	Confiab.	Taxa de bits	Sensível a atraso
ftp	100%	elástica	não
e-mail	100%	elástica	não
documentos Web	100%	elástica	não
audio/video tempo-real	Tol. perdas	áudio: 5Kb-1Mb vídeo:10Kb-5Mb	sim, 100's msec
audio/video armazen.	Tol. perdas	Idem anterior	sim, poucos secs
jogos interativos	Tol. perdas	poucos Kbps	sim, 100's msec
aplic. financeiras	100%	elástica	sim e não

Serviços oferecidos pelos protocolos de transporte

Serviço TCP:

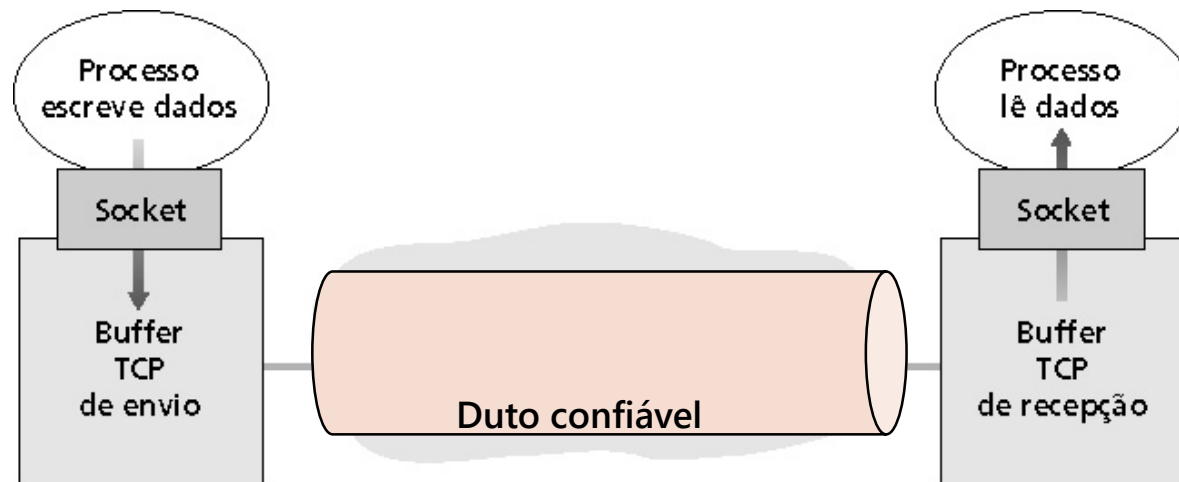
- Orientado a conexão (Criação da conexão TCP)
 - Cliente e servidor trocam informações de controle da camada de transporte antes do envio dos dados
 - Alerta o cliente e servidor para se prepararem para a transmissão dos dados
 - Conexão full-duplex entre os dois processos
 - Após terminada a transmissão a conexão é encerrada



Serviços oferecidos pelos protocolos de transporte

Serviço TCP:

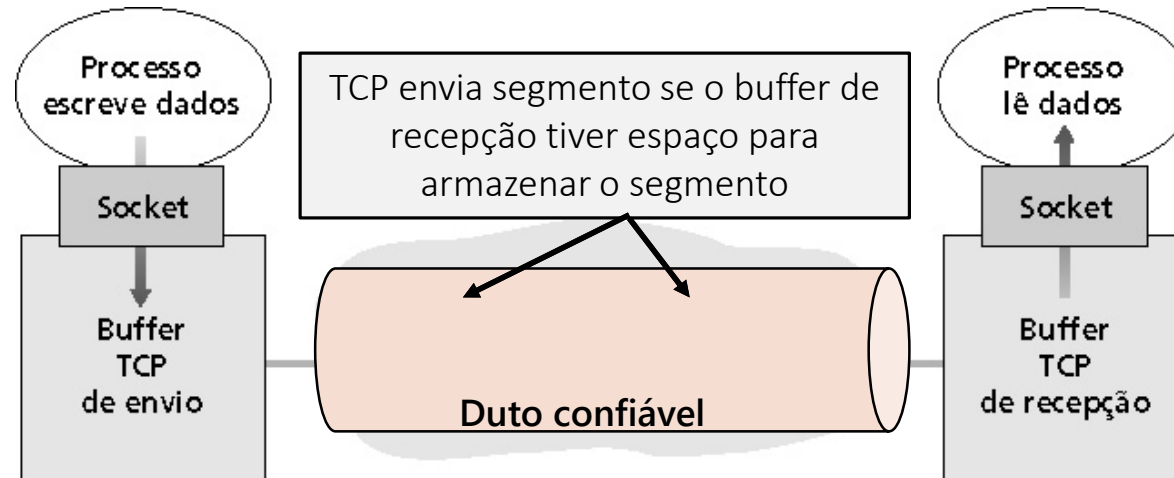
- Transporte confiável entre os processos emissor e receptor
 - Processos confiam no TCP para a entrega dos dados enviados sem erro e na ordem correta



Serviços oferecidos pelos protocolos de transporte

Serviço TCP

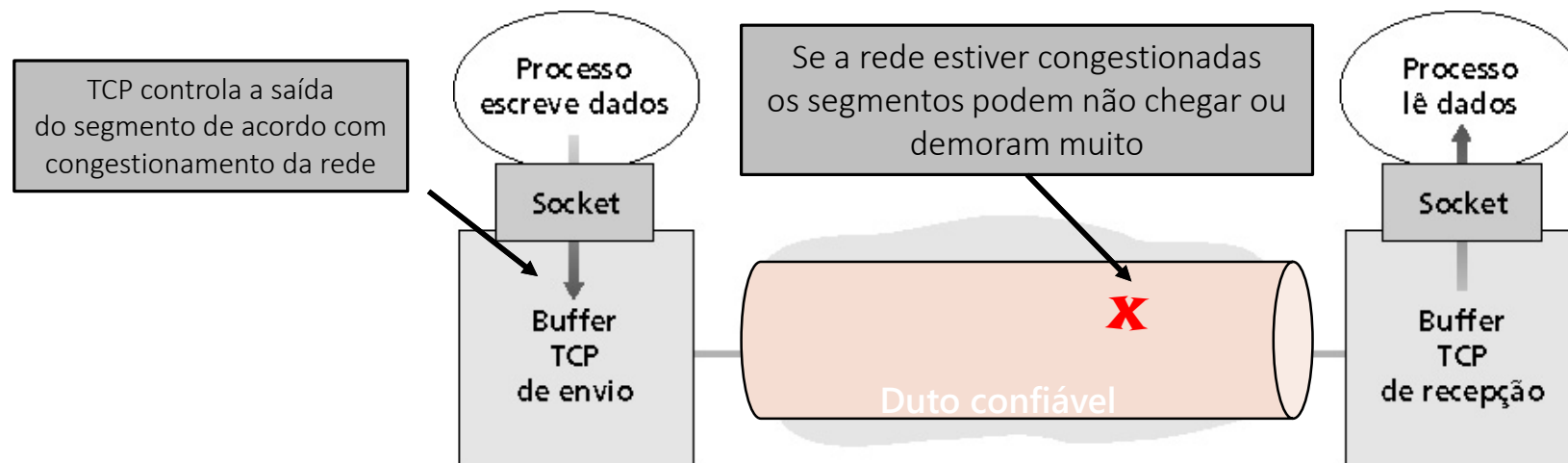
- Controle de fluxo
 - Não deixa um processo mandar mais dados que outro possa consumir



Serviços oferecidos pelos protocolos de transporte

Serviço TCP:

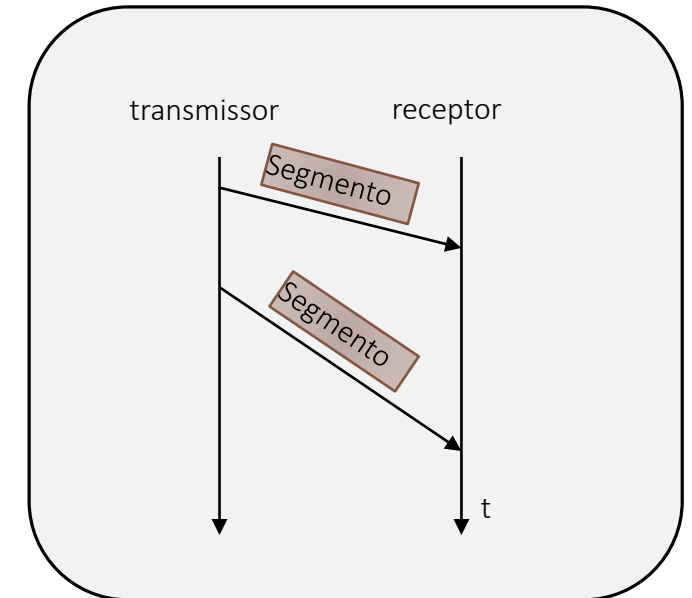
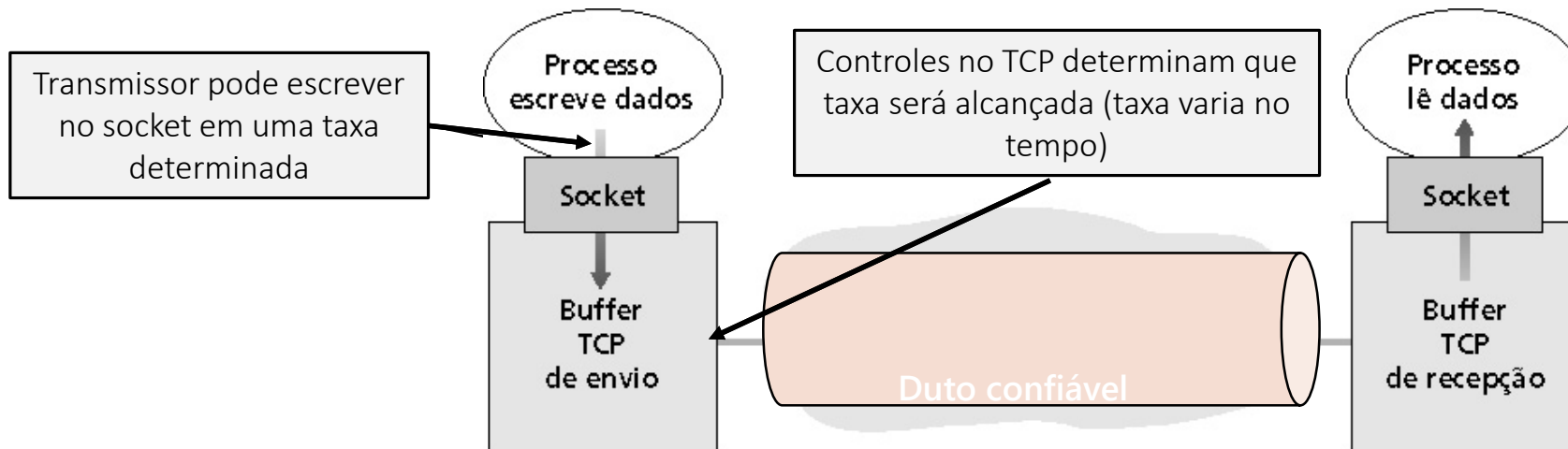
- Controle de congestionamento
 - Serviço mais voltado para o bom funcionamento da Internet do que para os próprios processos comunicantes
 - Limita a capacidade de transmissão de um cliente quando a rede está congestionada
 - Prejudicial para as aplicações de áudio e vídeo em tempo real
 - com exigência de taxa de transferência
 - que são mais flexíveis a problemas de erros



Serviços oferecidos pelos protocolos de transporte

TCP não oferece

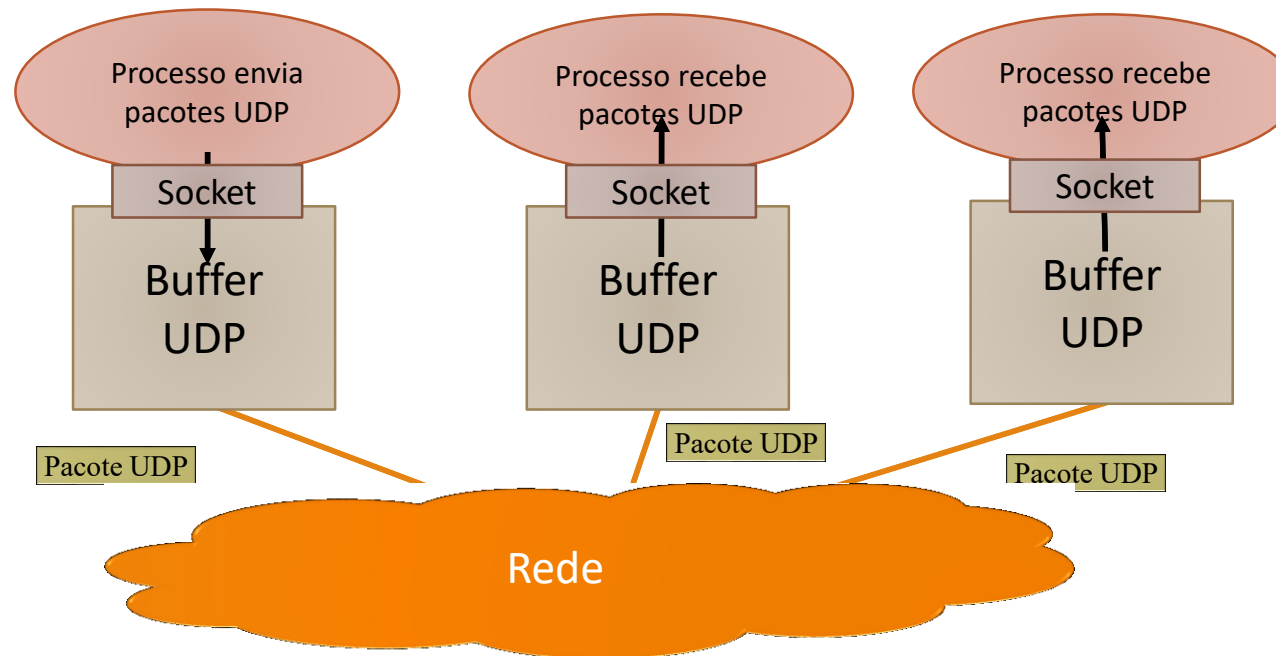
- Não garante taxa de transmissão mínima
 - Taxa de transmissão é regulada pelo controle de fluxo e de congestionamento do TCP
- Não garante atraso na transmissão
 - Não garante o tempo entre o processo transmissor passar os dados para a porta TCP e o processo receptor receber os dados na porta TCP



Serviços oferecidos pelos protocolos de transporte

Serviço UDP

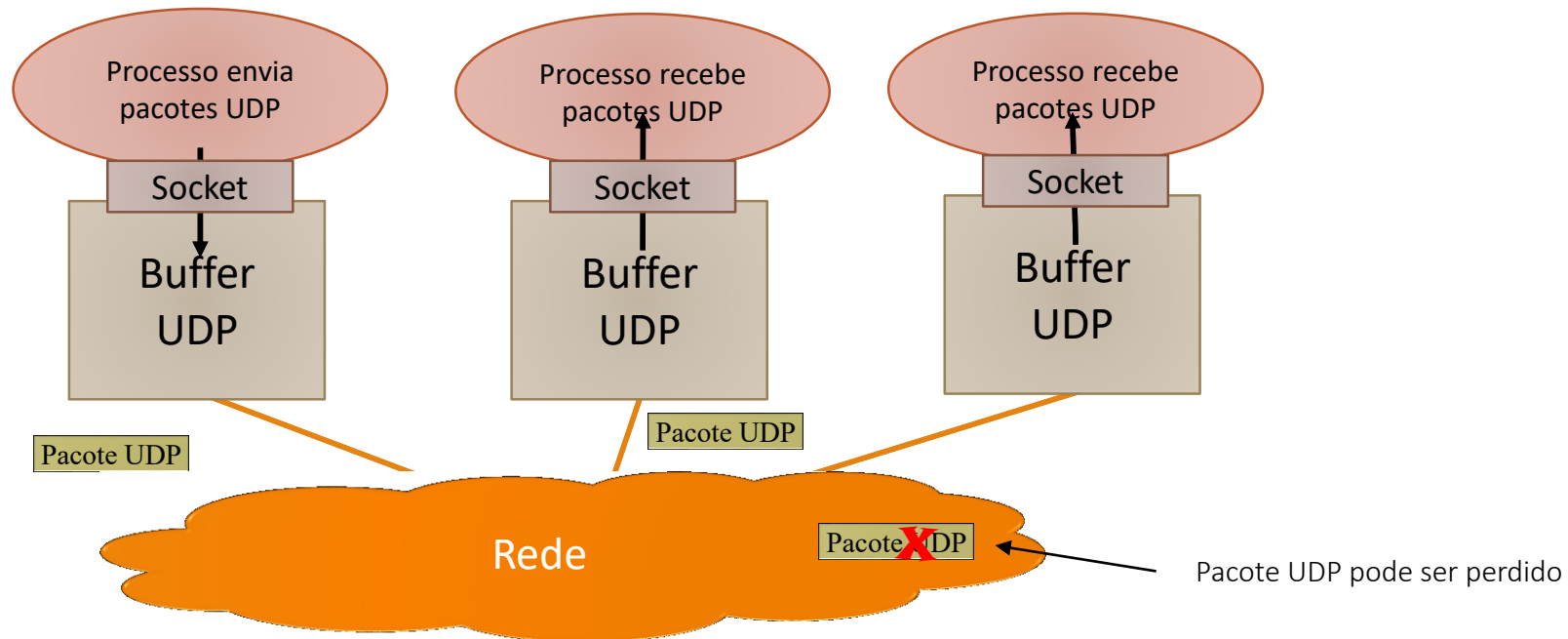
- Protocolo de transporte simples, leve, com modelo de serviço minimalista
- Não é orientado a conexão
 - Não há um contato prévio antes dos dados serem enviados



Serviços oferecidos pelos protocolos de transporte

Serviço UDP

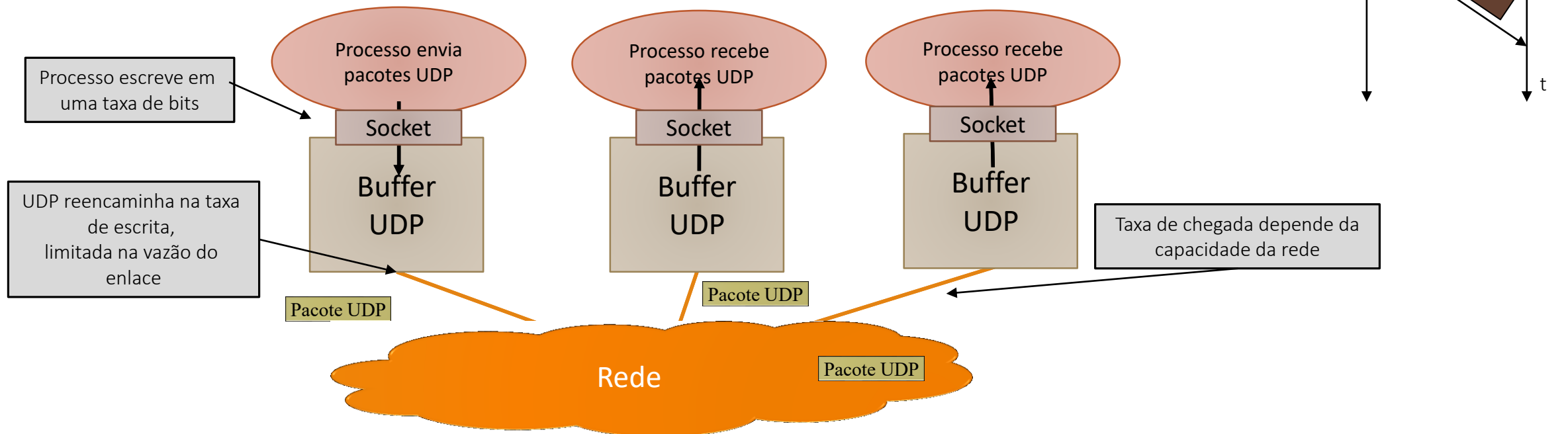
- Transporte não confiável entre os processos emissor e receptor
 - Não há garantias que um dado colocado na porta UDP chegará no destino
 - Podem chegar fora de ordem



Serviços oferecidos pelos protocolos de transporte

Serviço UDP

- Não inclui mecanismos de controle de congestionamento
 - Processo pode tentar transmitir na taxa que quiser
- Desenvolvedores costumam adotar o protocolo UDP para desenvolver aplicações tempo real
- Não fornece garantias de atraso e taxa de bits



Aplicações Internet

Seus protocolos e protocolos de transporte

Aplicação	Protocolo de aplicação	Protocolo de transporte
e-mail	smtp [RFC 821]	TCP
acesso de terminais remotos	telnet [RFC 854]	TCP
Web	http [RFC 2068]	TCP
transferência de arquivos	ftp [RFC 959]	TCP
streaming multimídia	RTP ou proprietário (ex.: RealNetworks)	TCP ou UDP
servidor de arquivos remoto	NSF	TCP ou UDP
telefonia Internet	RTP ou proprietário (ex.: Vocaltec)	tipicamente UDP

Pontos Importantes

Serviços de Transporte

- Entender a escolha do protocolo de transporte baseado em requisitos de confiabilidade, taxa de bits e limites de atraso

CAP 4. CAMADA DE APLICAÇÃO

AULA 3: DNS – SERVIDORES

INE5422 REDES DE COMPUTADORES II
PROF. ROBERTO WILLRICH (INE/UFSC)
ROBERTO.WILLRICH@UFSC.BR
[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

DNS: Domain Name System

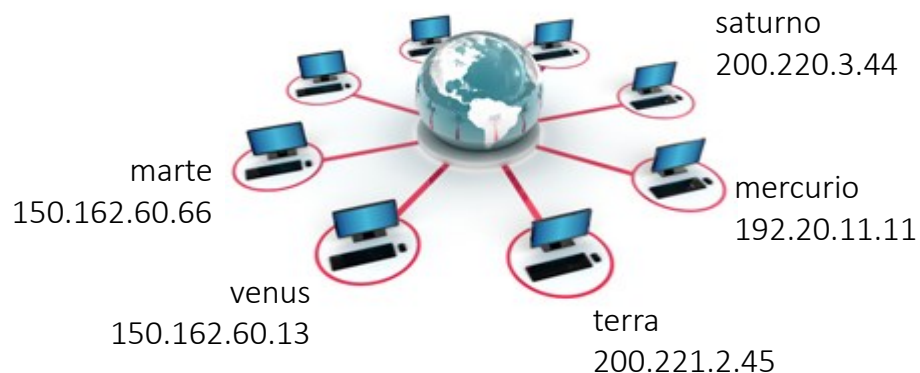
Pessoas: muitos identificadores:

- CPF, nome, ..

Hospedeiros, roteadores Internet :

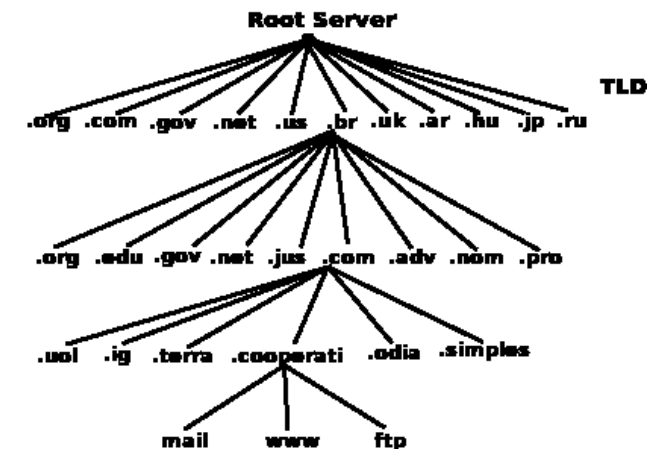
- Endereço IP - usado p/ endereçar datagramas
- “nome”, ex., marte.inf.ufsc.br - usado por gente

Como mapear nomes em endereço IP?



Domain Name System:

- Base de dados distribuída implementada em uma hierarquia de muitos servidores de nomes
- Protocolo de camada de aplicação permite que hospedeiros, roteadores se comuniquem com servidores de nomes para resolver nomes (tradução endereço/nome)
- nota: função imprescindível da Internet implementada como protocolo de camada de aplicação



DNS

Utiliza o protocolo UDP e usa a porta 53

- Especificado nas RFCs 1034 e 1035 e atualizado em outras RFCs.

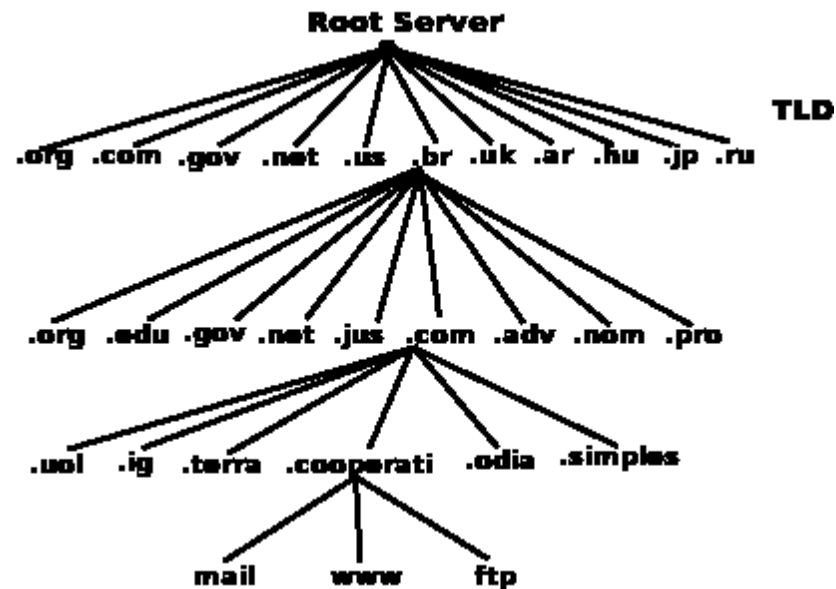
Outros serviços:

- Apelidos para hospedeiros (aliasing)
 - Ex. `www.ufrj.br` é um nome de alias de `ufrj.br` (nome canônico)
- Nome para o servidor de Emails
 - Ex. servidor de envio tem que saber qual é o servidor de nome de destino para mandar o email `roberto.willrich@ufsc.br` -> `mx.mail.ufsc.br`
- Distribuição da carga
 - Servidores Web replicados: seleciona um dos endereços IP dos servidores web com o mesmo conteúdo

DNS (Domain Name System)

Nome DNS é hierárquico

- Similar ao sistema de números de telefone
 - código do país, código da área, código do bairro e código da linha
- Na Internet:
 - um nome do computador que é parte de uma organização, que faz parte de grupo de organizações relacionadas, que está em um país
 - Exemplo: `www.inf.ufsc.br`



DNS (Domain Name System)

Controle de nome é distribuído

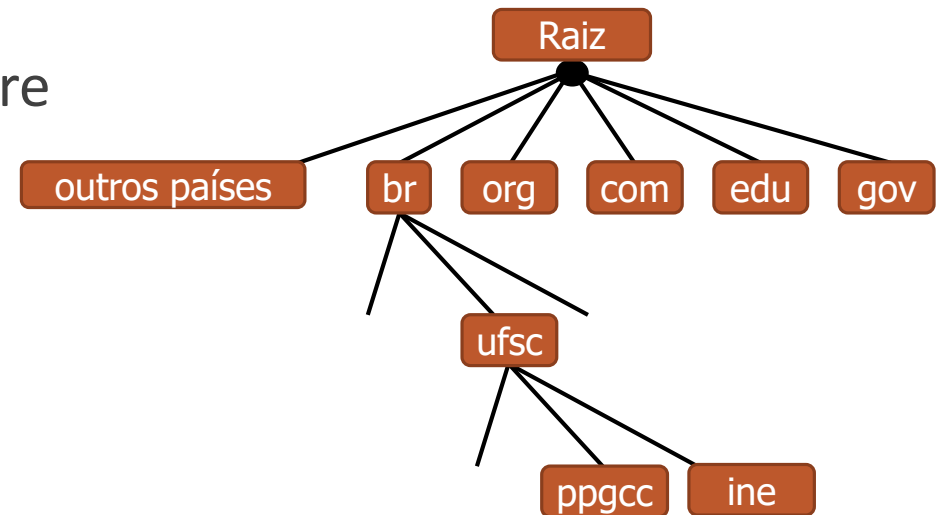
- baseado em uma árvore de nomes
 - Cada nível no sistema de nomes é um Domínio
- uma organização controla uma sessão da árvore
 - é livre para alterar a árvore em sua sessão

Nomes de computadores

- Domínios são separados por ponto:
 - ufsc.br, ux.cso.uiuc.edu, www.tre.gov.br

Controle de nomes é local

- cada organização cria o nome sem pedir a ninguém
 - adiciona o novo nome para sua participação na base de dados mundial
 - Exemplo: www.ine.ufsc.br



Serviços DNS

DNS é empregado pelas aplicações

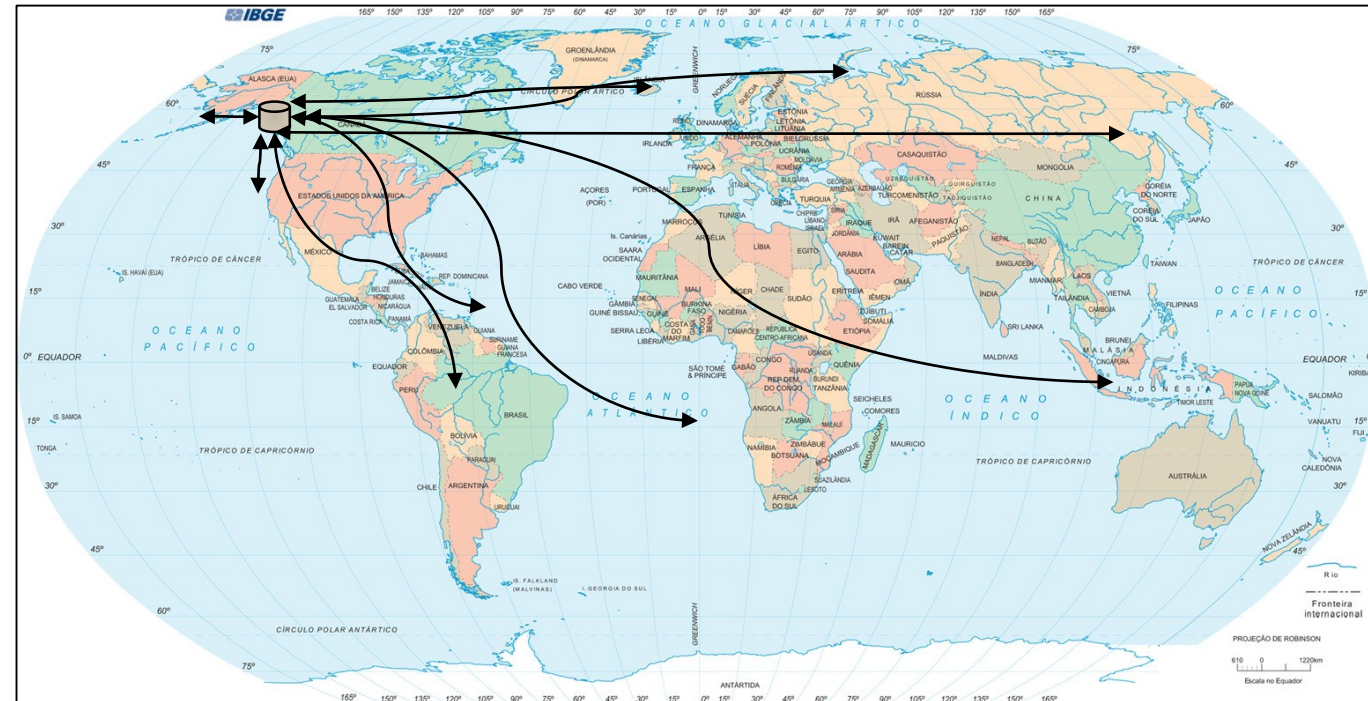
- Para traduzir o nome do hospedeiro fornecido pelo usuário para o endereço IP
- Exemplo HTTP
 - Usuário requisita o URL `www.inf.ufsc.br`
 - Para que o hospedeiro possa solicitar a página web ao servidor é necessário encontrar o endereço IP do servidor `www.inf.ufsc.br`
 - Browser extrai nome do servidor (`www.inf.ufsc.br`) passa para o cliente DNS que solicita ao servidor DNS o seu endereço IP
 - Browser estabelece uma conexão TCP para solicitar a página

Servidores de nomes DNS

Por que não centralizar o DNS?

- Ponto único de falha
- Volume de tráfego
- Base de dados centralizada e distante
- Manutenção (da BD)

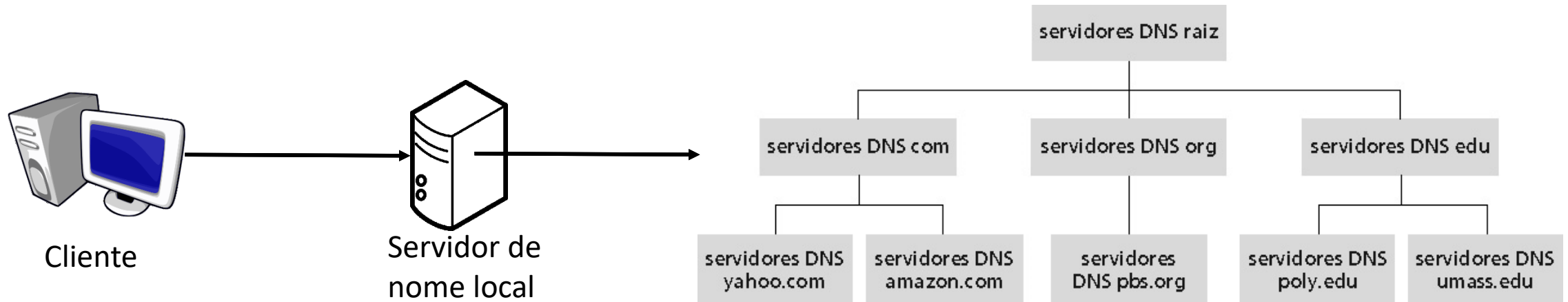
Não é escalável!



Servidores de nomes DNS

Nenhum servidor mantém todos os mapeamento nome-para-endereço IP

- **Servidor de nomes local**
 - Cada provedor, empresa tem servidor de nomes local (default)
 - Pedido DNS de hospedeiro vai primeiro ao servidor de nomes local configurado no hospedeiro
 - No mínimo dois servidores de nome com autoridade para o caso de falha



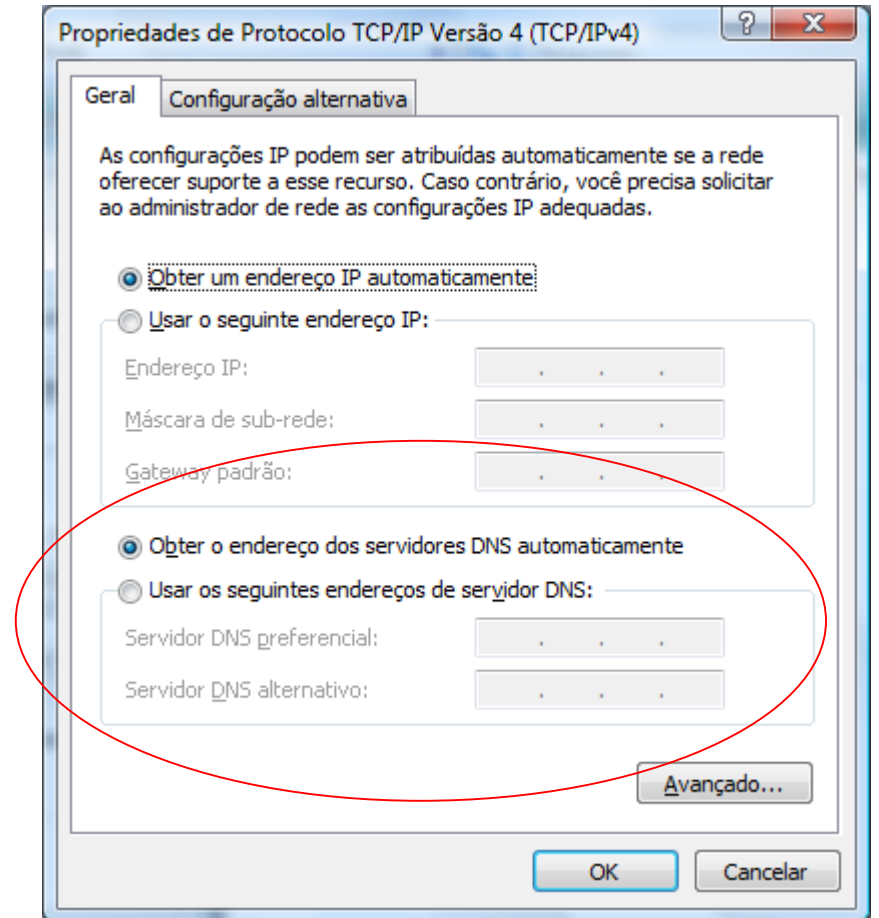
DNS: Servidor de Nomes Local

Conhecido por cada host

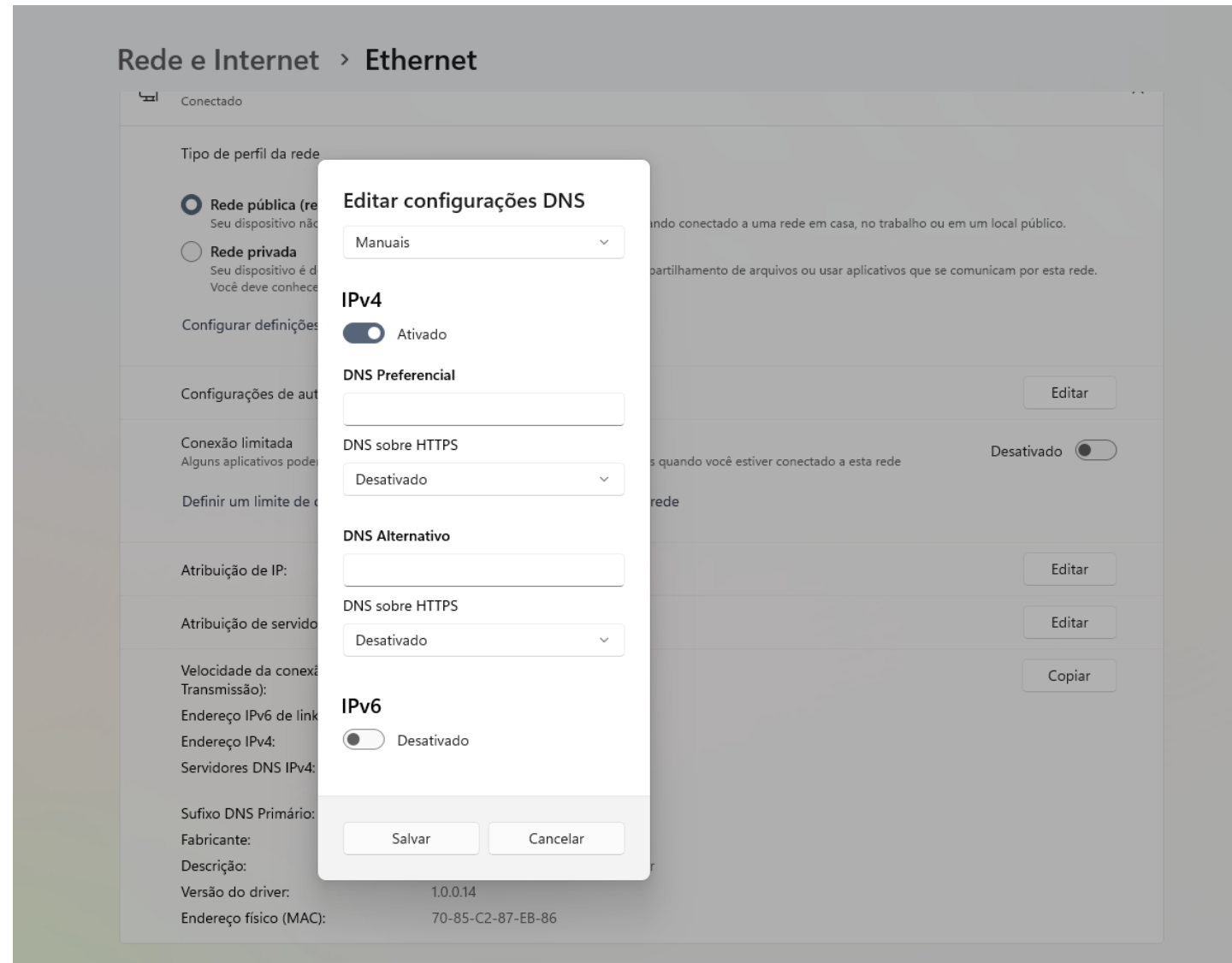
- cada computador deve conhecer o endereço IP do servidor DNS local

Ponto de partida para pedidos de tradução

- Caso o computador não souber o endereço IP de uma máquina ela envia uma solicitação ao servidor DNS local



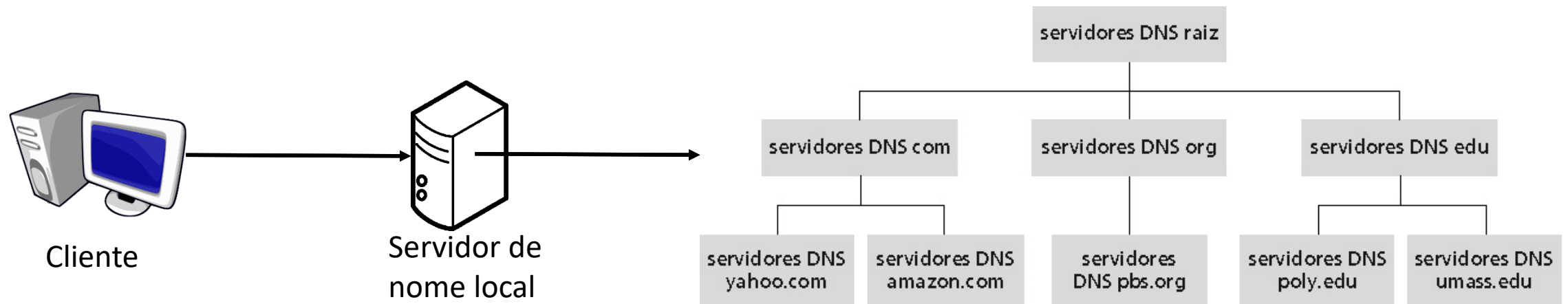
DNS: Servidor de Nomes Local



Servidores de nomes DNS

Nenhum servidor mantém todos os mapeamento nome-para-endereço IP

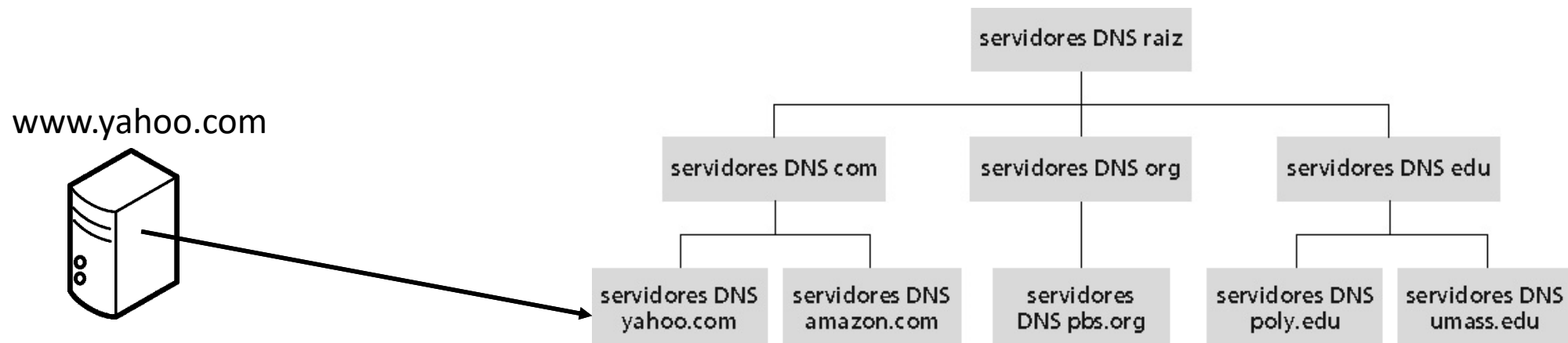
- **Servidor de nomes autorizados**
 - Servidores DNS de organizações, provêem nome de hospedeiro autorizado para mapeamentos IP para servidores de organizações (ex.: Servidores DNS yahoo.com, amazon.com, pbs.org, poly.edu,...).
 - Podem ser mantidos por uma organização ou provedor de serviços



Servidor DNS Autorizado

Todo hospedeiro com nome público está registrado em um servidor de nomes autorizado

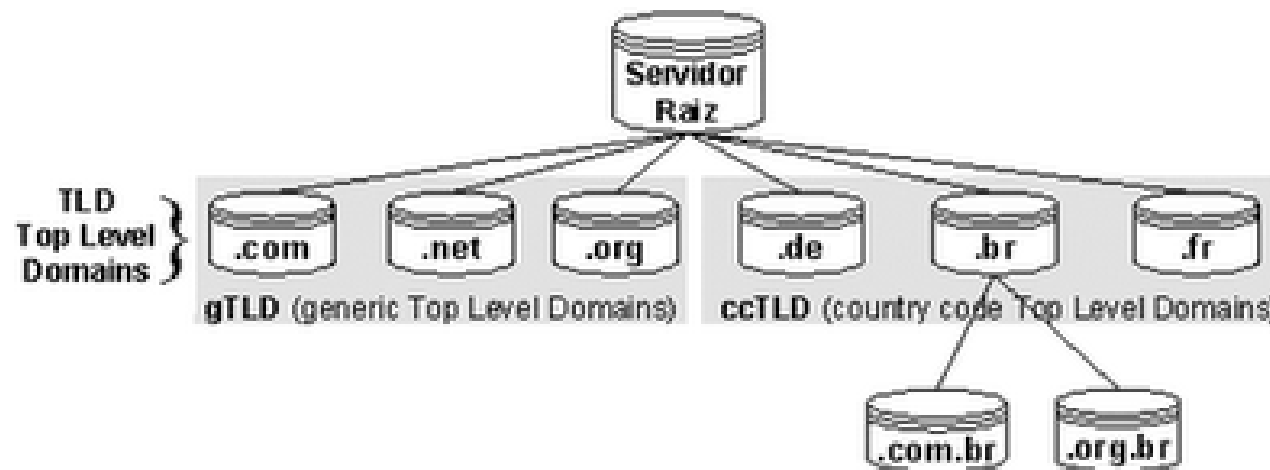
- Um servidor de nomes possui autoridade para um hospedeiro se ele tem sempre um registro DNS que traduz o nome do hospedeiro para o endereço IP do hospedeiro
- Servidores do ufsc.br: slave.ufsc.br; slave1.ufsc.br, slave2.ufsc.br
- Muitos servidores de nomes agem como servidores de nomes locais e também como servidores de nomes autorizados



Servidores DNS

Servidores Top-Level Domain (TLD)

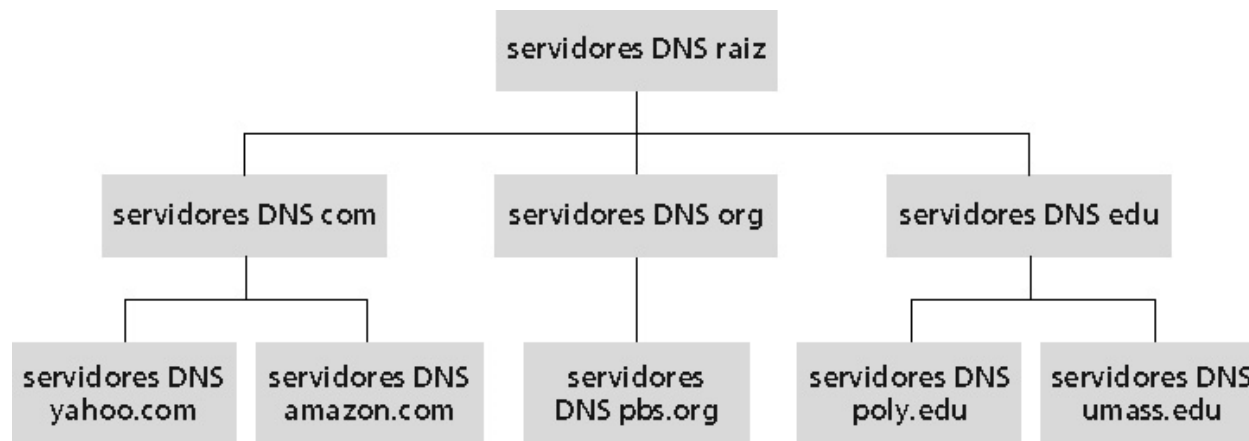
- Responsáveis pelos domínios com, org, net, edu etc. e todos os domínios top-level nacionais uk, fr, ca, jp.
(https://en.wikipedia.org/wiki/List_of_Internet_top-level_domains)
- Tipos:
 - gTLD's (Generic Top Level Domains) domínio genéricos usados no mundo todo
 - ccTLD's (Country Code Top Level Domains) extensões de domínios administrados pelos países.



DNS: Servidores de nome raiz

Para que serve?

- Contactado pelo servidor de nomes local que não consegue resolver o TLD do nome
- O servidor Raiz possui uma tabela que indica qual DNS será responsável pela resolução dos domínios para cada extensão de domínio (TLD) diferente.
 - Pequena tabela que possui apenas uma entrada para cada TLD existente

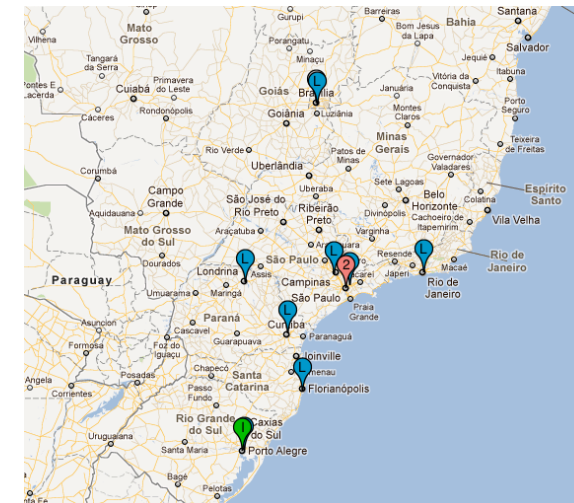


DNS: Servidores de nome raiz

Existem 13 servidores raiz (réplicas)

- Para aumentar a confiabilidade e balanceamento de carga
- 2 vezes ao dia seu conteúdo é automaticamente replicado.
- Cada servidor raiz é chamado por uma letra do alfabeto (Servidor A, Servidor B e etc...).
- Cada servidor raiz é replicado em várias regiões do mundo
 - para assegurar que o tempo para a resolução de um domínio seja rápido (baixa latência).

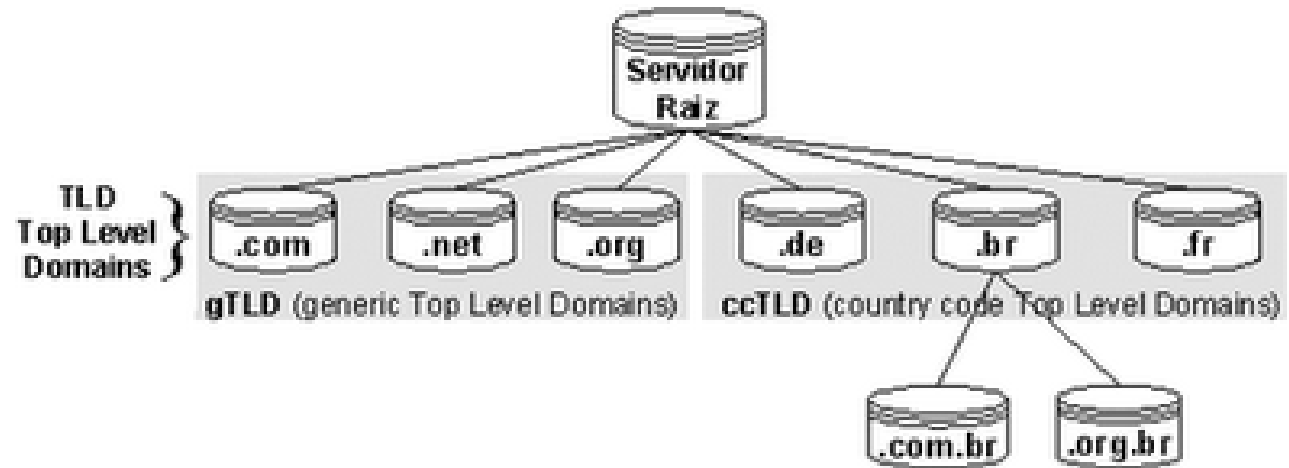
<http://www.root-servers.org/>



Servidores DNS

Organizações que gerenciam

- Servidores Raiz
 - [IANA](#) ([ICANN](#))
- TLD .br
 - CGI/NIC.br



Pontos Importantes

Serviço DNS

- Entendimento geral do serviço

Hierarquia de Servidores DNS

- Saber dos serviços oferecidos por cada tipo de servidor DNS

CAP 4. CAMADA DE APLICAÇÃO

AULA 4: DNS – EXEMPLO DE CONSULTA E TIPOS DE CONSULTAS

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Resumo dos tipos Servidores DNS

Servidores de nome local

- Ponto de partida de uma resolução de nomes
- Não pertence estritamente à hierarquia
- Cada ISP (ISP residencial, empresa, universidade) tem um
- quando hospedeiro faz consulta ao DNS, consulta enviada ao seu servidor DNS local
 - atua como proxy, encaminha consulta para hierarquia

Servidores de nome com autoridade

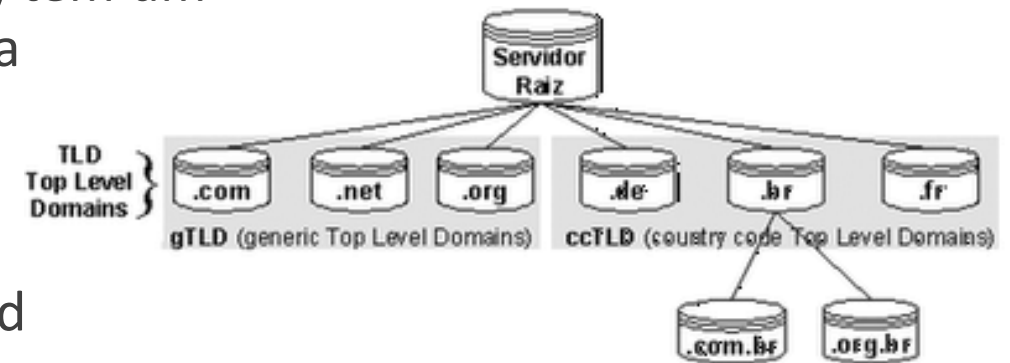
- Autorizado a resolver nomes dos equipamentos d organização

Servidores TLDs

- Mantém registros dos servidores de nome das organizações...

Servidores Raiz

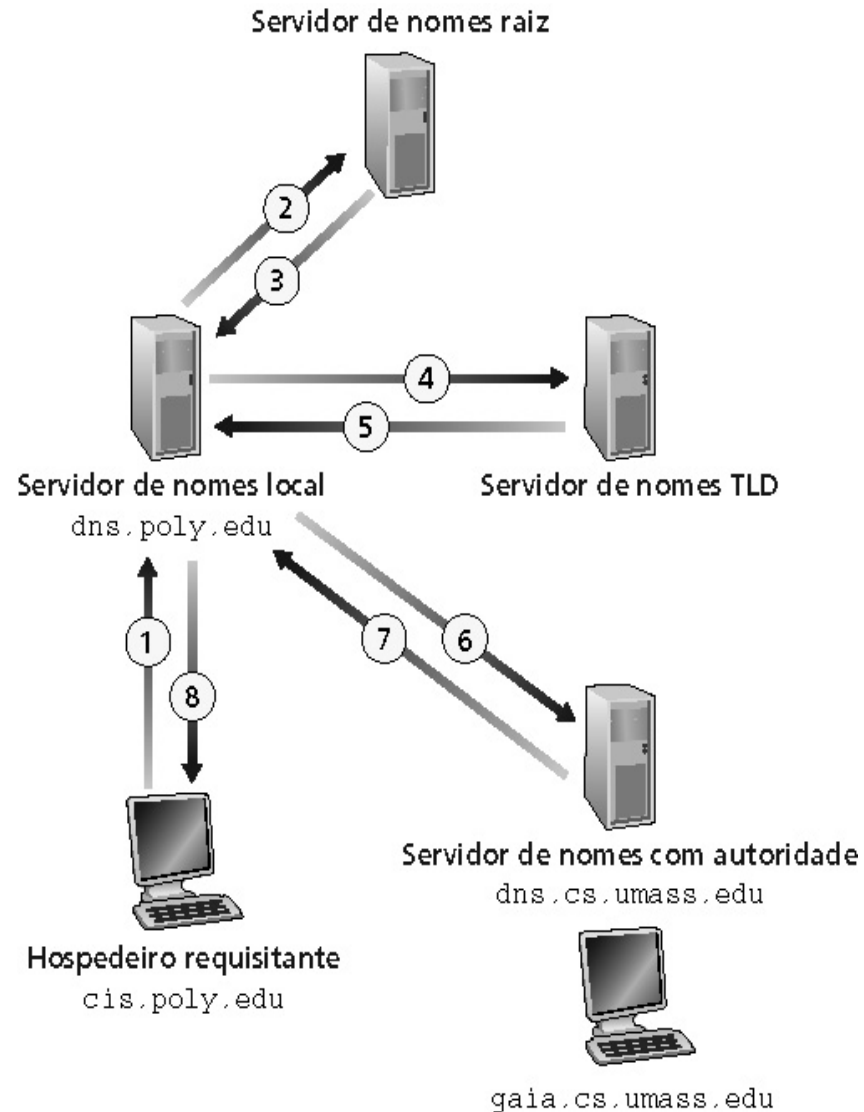
- Mantém registros dos servidores TLDs



Exemplo simples de DNS

Exemplo:

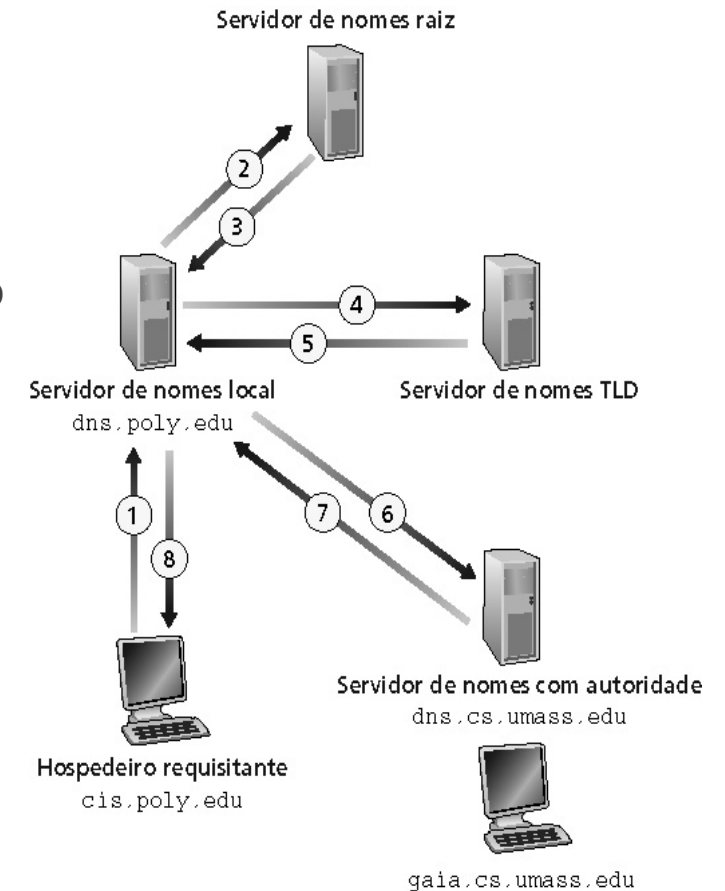
- Hospedeiro em cis.poly.edu quer o endereço IP para gaia.cs.umass.edu



Exemplo simples de DNS

Tipos de consultas

- **Consulta recursiva:**
 - Transfere a tarefa de resolução do nome para o servidor de nomes consultado
 - coloca peso da resolução de nome sobre o servidor de nomes contactado
 - carga pesada?
- **Consulta encadeada (repetida):**
 - Servidor contactado responde com o nome de outro servidor de nomes para contato
 - “Eu não sei isto, mas pergunte a este servidor”

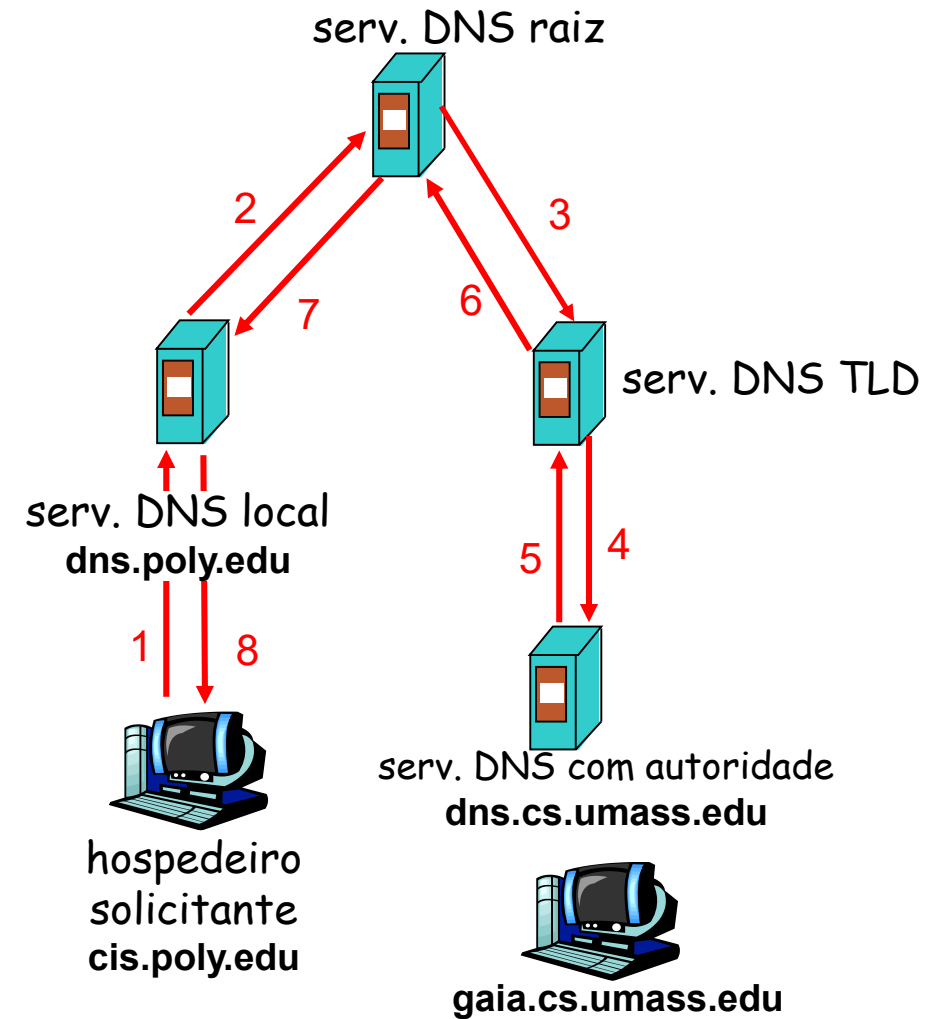


Exemplo simples de DNS

**cis.poly.edu deseja endereço IP de
gaia.cs.umass.edu**

Na consulta recursiva:

- 1. Contacta seu servidor de nomes local, dns.poly.edu
- 2. dns.poly.edu contacta servidor de nomes raiz, se necessário
- 3. Servidor de nomes raiz contacta servidor de nomes TLD .edu, se necessário. TLD sabe o IP do dns.cs.umass.edu e contacta este servidor

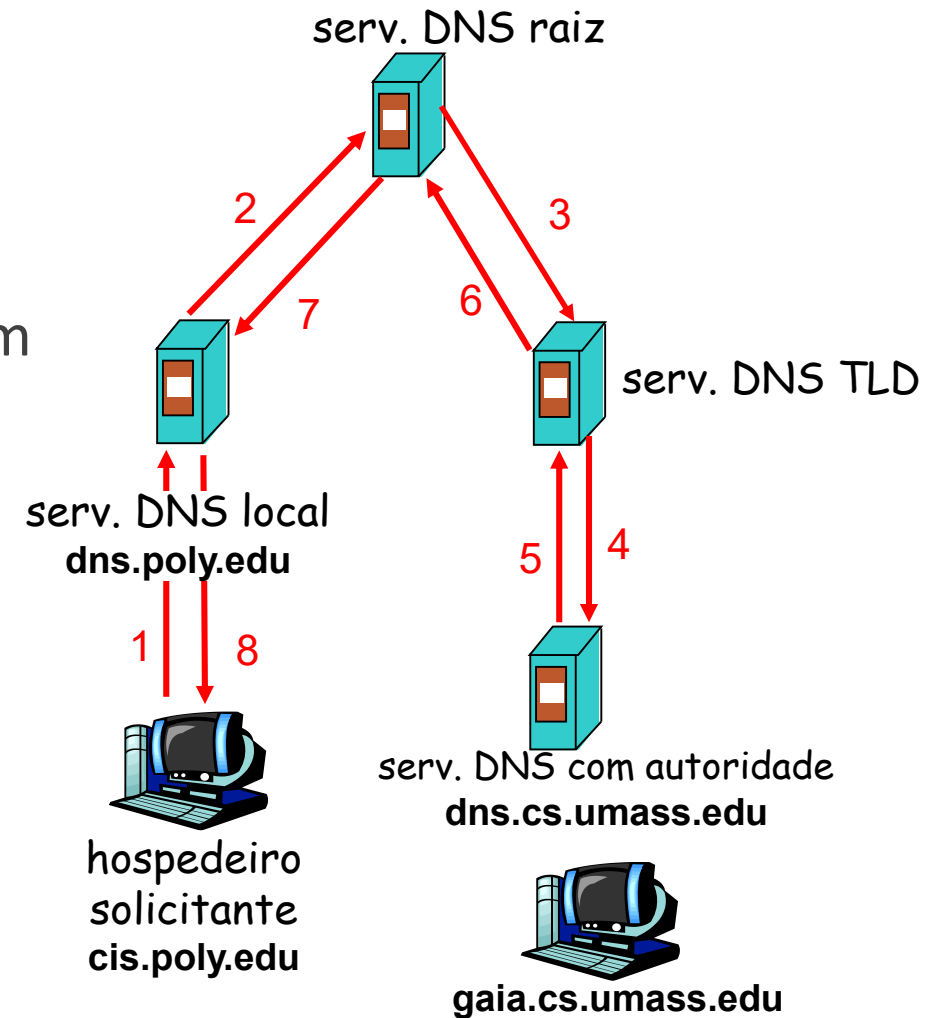


Exemplo simples de DNS

**cis.poly.edu deseja endereço IP de
gaia.cs.umass.edu**

Na consulta recursiva:

- 4. Servidor conhece IP do servidor de nome com autoridade dns.cs.umass.edu e solicita a resolução do nome gaia.cs.umass.edu
- 5. Servidor dns.cs.umass.edu resolve o IP de gaia e responde
- 6., 7, 8, servidores encadeiam a resolução de nome.



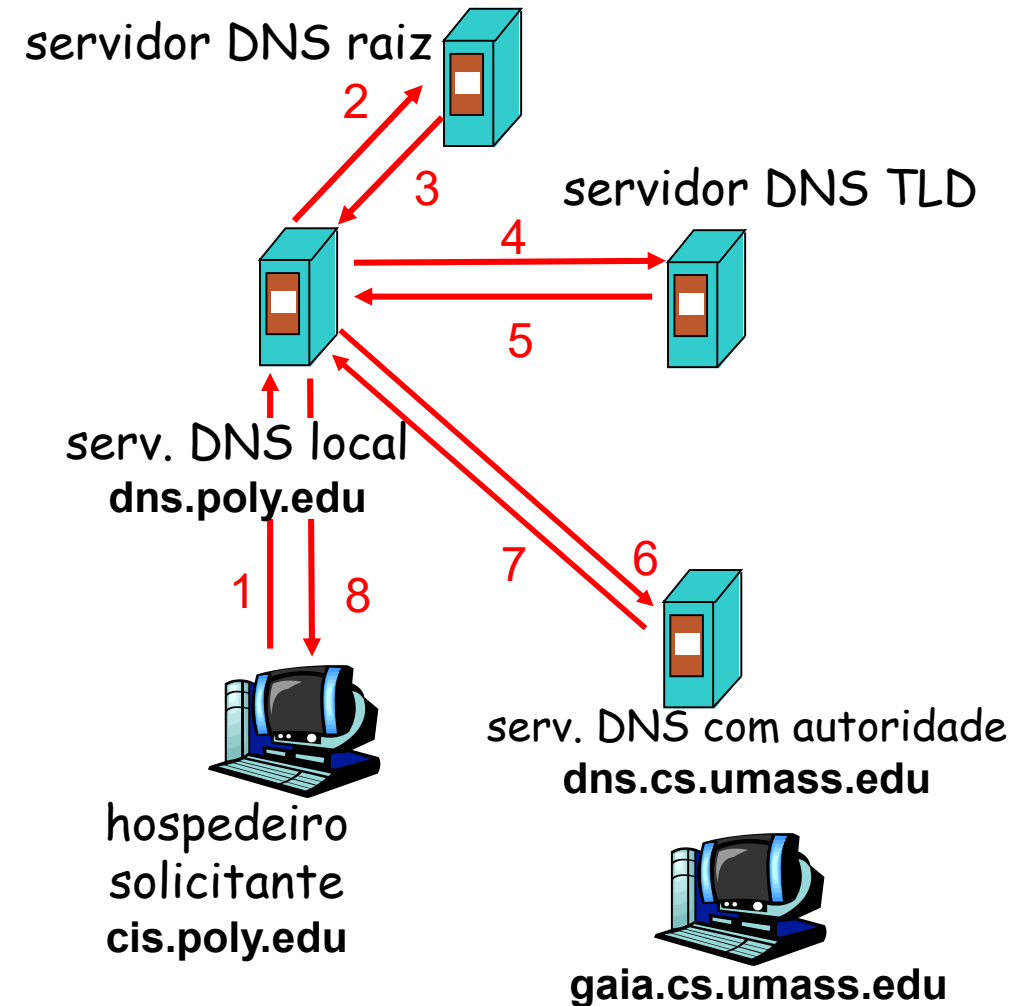
DNS: outro tipo de consulta

Consulta recursiva:

- Servidor local contactado (dns.poly.edu) retorna o nome para contato via consulta recursiva

Outros servidores usam consulta encadeada

- “Eu não conheço este nome, pergunte a este servidor”
- Mais adequado para servidores DNS raiz e TLD
 - Não exigem a manutenção de um estado.



DNS: Cacheamento

Uma vez que um servidor de nomes aprende um mapeamento, ele armazena o mapeamento num registro do tipo cache

- Entradas na cache expiram (dois dias)
- Servidores TLD são tipicamente armazenados em cache nos servidores de nome locais

DNS (Domain Name System)

DNS não se limita a manter e gerenciar endereços Internet

- Cada nome de domínio é um nó em um banco de dados
 - Pode conter informações definindo várias propriedades
 - tipo da máquina e a lista de serviços por ela oferecidos, etc.
 - Permite que seja definido um “alias” (nome alternativo) para o nó
 - É possível usar o DNS para armazenar informações sobre usuários, listas de distribuição e outros objetos

Pontos Importantes

Tipos de consultas DNS

- Entender as consultas recursivas e encadeadas

Passos na resolução de nomes

- Saber os passos na resolução de nomes

CAP 4. CAMADA DE APLICAÇÃO

AULA 5: DNS – REGISTROS E PROTOCOLO

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

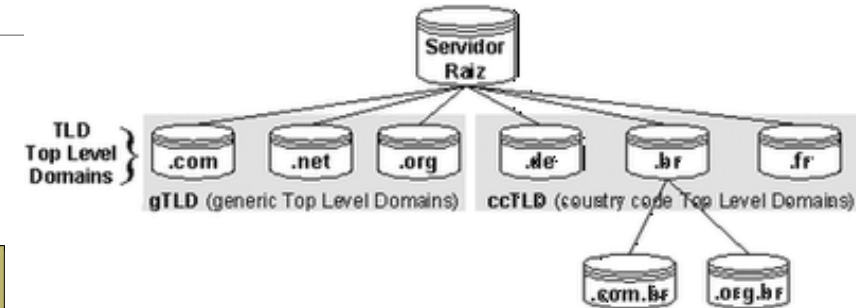
Registros DNS

DNS é uma BD distribuída

- Armazena resource records (RR) no formato:

`(class, name, value, type, ttl)`

- Classes = Internet (IN), Chaosnet (CH), etc.
- ttl = validade do registro



Type	Name	value
A	Nome do host biblio.inf.ufsc.br	Endereço IPv4 do host 150.162.60.154
AAAA	Nome do host	Endereço IPv6 do host
CNAME	Apelido (alias) de algum nome canônico www.ibm.com	Nome canônico www.ibm.com.cs186.net
NS	Domínio ufsc.br	Nome do servidor para o domínio ns67.ufsc.br
MX	Domínio ufsc.br	Nome do servidor de email mx.mail.ufsc.br

nslookup

Quando se acessa uma máquina

- A pesquisa é realizada na hierarquia de servidores partindo pelo topo
- Se você tenta encontrar o endereço de `www.ufrn.br`
 - Seu servidor de nomes deve encontrar o servidor que serve `br`.
 - Servidor local pergunta a um servidor raiz (`.`) a lista de servidores `.br`

```
$ nslookup
```

```
Default Server: ns.inf.ufsc.br
```

```
Address: 150.162.60.5
```

- Definindo o servidor padrão como o raiz (`L`)

```
> server 199.7.83.42
```

```
Address: 199.7.83.42
```


nslookup

Procurando por www.ufrn.br

- Setando o tipo de query para NS (name server records)

```
> set q=ns
```

- Perguntando sobre br.

```
> br.
```

```
br      nameserver = a.dns.br
```

```
br      nameserver = b.dns.br
```

```
br      nameserver = c.dns.br
```

```
br      nameserver = d.dns.br
```

```
br      nameserver = e.dns.br
```

```
br      nameserver = f.dns.br
```

```
a.dns.br      AAAA IPv6 address = 2001:12ff::10
```

```
d.dns.br      AAAA IPv6 address = 2001:12f8:4::10
```

```
e.dns.br      AAAA IPv6 address = 2001:12f8:1::10
```

```
a.dns.br      internet address = 200.160.0.10
```

```
b.dns.br      internet address = 200.189.41.10
```

```
c.dns.br      internet address = 200.192.233.10
```

```
d.dns.br      internet address = 200.219.154.10
```

```
e.dns.br      internet address = 200.229.248.10
```

```
f.dns.br      internet address = 200.219.159.10
```

nslookup

Procurando por www.ufrn.br

- Listagem anterior nos diz que f.DNS.br serve br., assim nós podemos perguntar a f.DNS.br.
 - > server 200.219.159.10
- Agora nós podemos conhecer quem serve o próximo nível do domínio de nome ufrn.br.
- > ufrn.br.
 - Servidor: [200.219.159.10]
 - Address: 200.219.159.10
- ufrn.br nameserver = ns1.ufrn.br
- ufrn.br nameserver = ns2.ufrn.br
- ns2.ufrn.br internet address = 45.55.153.157
- ns1.ufrn.br internet address = 177.20.144.4
- ns1.ufrn.br AAAA IPv6 address = 2801:8c:0:a0::40

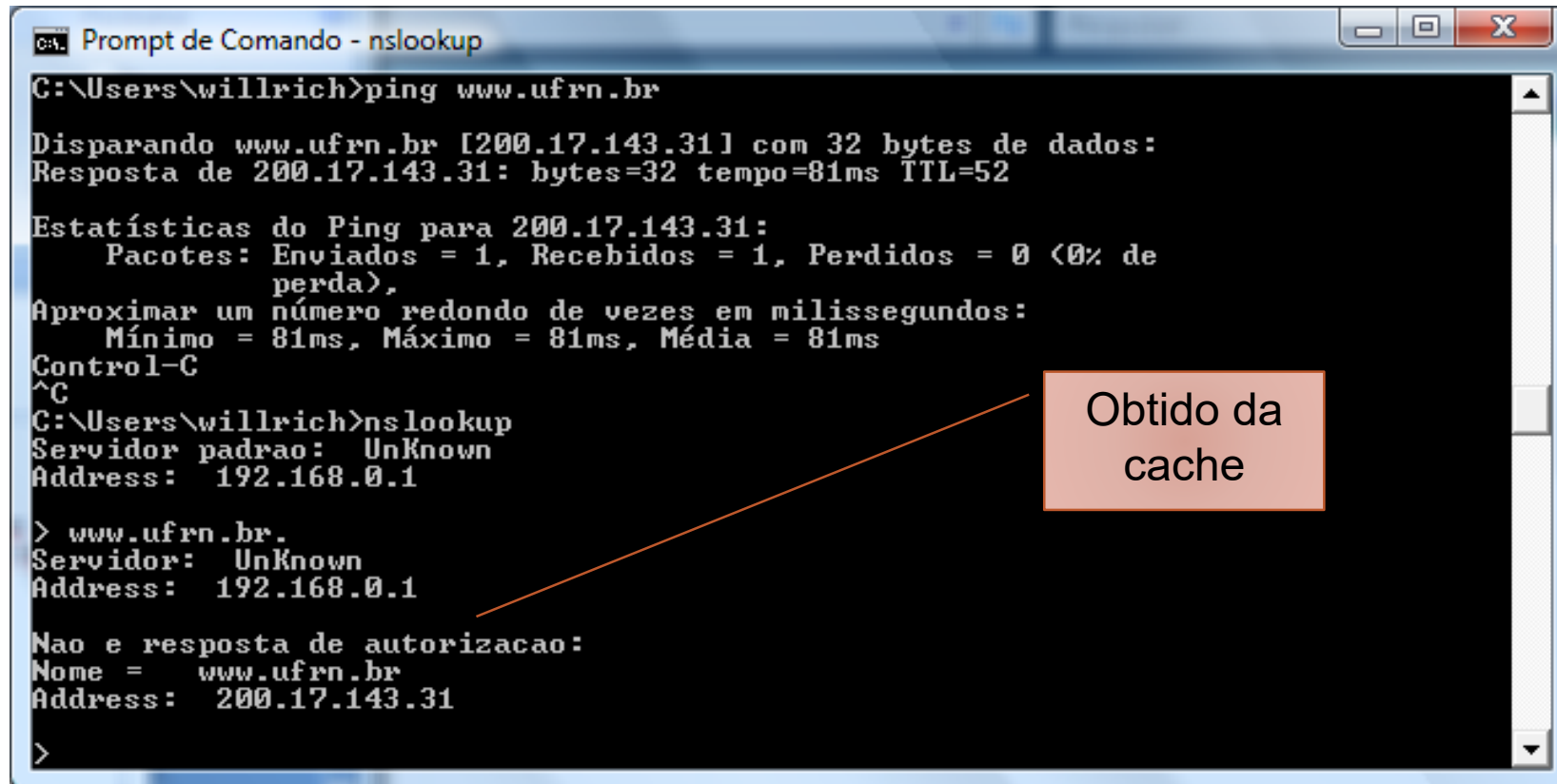
nslookup

Procurando por www.ufrn.br

- DNS.ufrn.br serve ufrn.br, sentando ele como servidor
 - > server 200.17.143.31
- Agora trocamos o tipo de query, para perguntar ao servidor qualquer coisa que ele sabe sobre www.ufrn.br.
 - > set q=any
 - > www.ufrn.br.
 - Servidor: [200.17.143.31]
 - Address: 200.17.143.31
- www.ufrn.br internet address = 200.17.143.31
- ufrn.br nameserver = dns.ufrn.br
- ufrn.br nameserver = dns.ufrnet.br
- dns.ufrn.br internet address = 200.17.143.31
- dns.ufrn.br AAAA IPv6 address = 2001:12f0:702:1e0::31
- dns.ufrnet.br internet address = 200.17.143.130

nslookup

Procurando por www.ufrn.br



```
Prompt de Comando - nslookup
C:\Users\willrich>ping www.ufrn.br

Disparando www.ufrn.br [200.17.143.31] com 32 bytes de dados:
Resposta de 200.17.143.31: bytes=32 tempo=81ms TTL=52

Estatísticas do Ping para 200.17.143.31:
    Pacotes: Enviados = 1, Recebidos = 1, Perdidos = 0 (0% de
    perda),
Aproximar um número redondo de vezes em milissegundos:
    Mínimo = 81ms, Máximo = 81ms, Média = 81ms
Control-C
^C
C:\Users\willrich>nslookup
Servidor padrao: UnKnown
Address: 192.168.0.1

> www.ufrn.br.
Servidor: UnKnown
Address: 192.168.0.1

Nao e resposta de autorizacao:
Nome = www.ufrn.br
Address: 200.17.143.31
>
```

DNS: Protocolo e Mensagens

Protocolo DNS

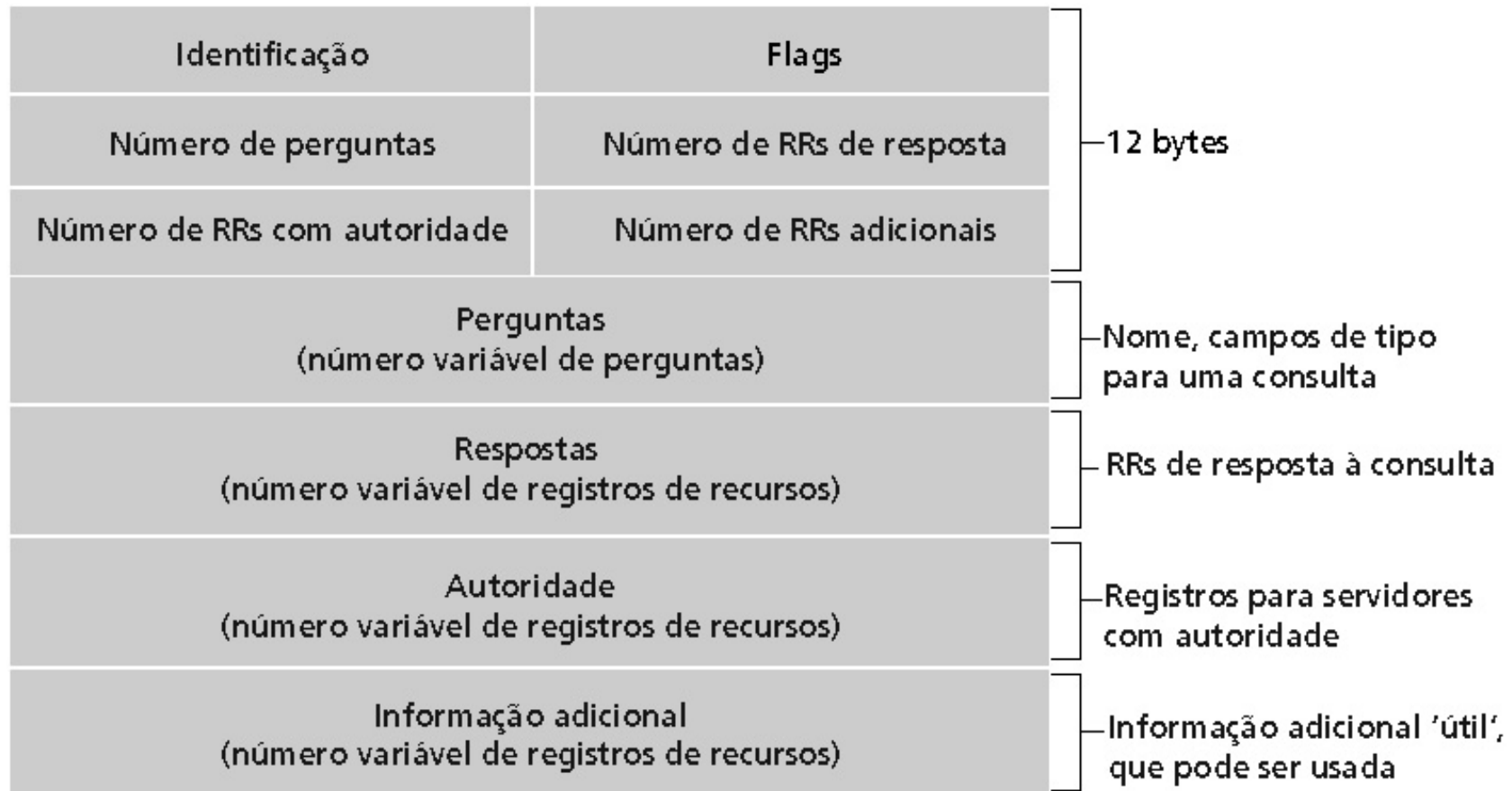
- Mensagens query e reply, com o mesmo formato de mensagem

Cabeçalho DNS

- **identification**: 16 bit # para query, reply para um query usa mesmo #
- **flags**:
 - query ou reply
 - Recursão desejada
 - Recursão disponível
 - reply é autoritativo

Identificação	Flags	
Número de perguntas	Número de RRs de resposta	12 bytes
Número de RRs com autoridade	Número de RRs adicionais	
Perguntas (número variável de perguntas)		Nome, campos de tipo para uma consulta
Respostas (número variável de registros de recursos)		RRs de resposta à consulta
Autoridade (número variável de registros de recursos)		Registros para servidores com autoridade
Informação adicional (número variável de registros de recursos)		Informação adicional 'útil', que pode ser usada

DNS: Protocolo e Mensagens



DNS: Protocolo e Mensagens

The image shows a Wireshark capture of DNS traffic. The packet list at the top shows four packets: a standard query response (No. 12), a standard query (No. 13), an NBIPX packet (No. 14), and another standard query response (No. 15). The selected packet (No. 15) is a DNS standard query response from 150.162.1.9 to 150.162.60.132. The packet details pane shows the following structure:

- User Datagram Protocol, Src Port: 1028 (1028), Dst Port: domain (53)
- Domain Name System (query)
 - [Response In: 15]
 - Transaction ID: 0x0054
 - Flags: 0x0100 (Standard query)
 - 0... .. = Response: Message is a query
 - .000 0... .. = Opcode: Standard query (0)
 -0. = Truncated: Message is not truncated
 -1 = Recursion desired: Do query recursively
 -0... .. = Z: reserved (0)
 -0 = Non-authenticated data OK: Non-authenticated data is unacceptable
 - Questions: 1
 - Answer RRs: 0
 - Authority RRs: 0
 - Additional RRs: 0
 - Queries
 - www.abc.com.br: type A, class IN
 - Name: www.abc.com.br
 - Type: A (Host address)
 - Class: IN (0x0001)

The packet bytes pane at the bottom shows the raw data in hexadecimal and ASCII:

```
0000 00 b0 d0 50 b6 8d 00 08 0d 19 35 66 08 00 45 00 ...P.... ..5f..E.
0010 00 3c 10 48 00 00 80 11 bf 97 96 a2 3c 84 96 a2 <.H.... ..<...
0020 01 09 04 04 00 00 35 00 28 c7 56 00 54 01 00 00 01 .....5.( .V.T...
0030 00 00 00 00 00 00 03 77 77 77 03 61 62 63 03 63 .....w ww.abc.c
0040 6f 6d 02 62 72 00 00 01 00 01 om.br... ..
```

The status bar at the bottom indicates: File: "D:\UFSC\Cursos\INE\INE5615\Pacotes\... Packets: 64 Displayed: 64 Marked: 0 Profile: Default

DNS: Protocolo e Mensagens

The image shows a Wireshark packet capture of a DNS response. The packet list at the top shows two packets: packet 13 is a DNS standard query from 190.102.00.132 to 190.102.1.9, and packet 14 is the corresponding response from 190.102.1.9 to 190.102.00.132. The packet details pane for packet 14 shows the following structure:

- Domain Name System (response)
 - [Request In: 13]
 - [Time: 0.169456000 seconds]
 - Transaction ID: 0x0054
 - Flags: 0x8180 (Standard query response, No error)
 - 1... .. = Response: Message is a response
 - .000 0... .. = opcode: standard query (0)
 -0.. = Authoritative: Server is not an authority for domain
 -0. = Truncated: Message is not truncated
 -1 = Recursion desired: Do query recursively
 - 1... = Recursion available: server can do recursive queries
 -0.. = Z: reserved (0)
 -0. = Answer authenticated: Answer/authority portion was not authenticated by the server
 - 0000 = Reply code: No error (0)
 - Questions: 1
 - Answer RRs: 2
 - Authority RRs: 0
 - Additional RRs: 0
 - Queries
 - Answers
 - www.abc.com.br: type CNAME, class IN, cname wh55m.terraempresas.com.br
 - Name: www.abc.com.br
 - Type: CNAME (Canonical name for an alias)
 - Class: IN (0x0001)
 - Time to live: 6 minutes, 39 seconds
 - Data length: 22
 - Primary name: wh55m.terraempresas.com.br
 - wh55m.terraempresas.com.br: type A, class IN, addr 200.176.131.4

The packet bytes pane at the bottom shows the raw data of the packet, with the first 124 bytes displayed. The data is organized into columns of hexadecimal values and their corresponding ASCII representations.

Frame (frame), 124 bytes Packets: 64 Displayed: 64 Marked: 0 Profile: Default

DNS: Protocolo e Mensagens

The image shows a Wireshark capture of a DNS transaction. The packet list at the top shows two packets: a standard query (No. 13) and a response (No. 14). The response packet is selected, and its details are shown in the right pane. The response contains two resource records: a CNAME record for www.abc.com.br pointing to wh55m.terraempresas.com.br, and an A record for wh55m.terraempresas.com.br with the IP address 200.176.131.4. The packet bytes pane at the bottom shows the raw data of the response packet.

dns - Wireshark

File Edit View Go Capture Analyze Statistics Help

Filter: Expression... Clear Apply

No.	Time	Source	Destination	Protocol	Info
13	3.390018	190.102.00.132	190.102.1.9	DNS	Standard query A www.abc.com.br
14	3.499013	Fd233742.00000000000001	00000000.ffffffffffff	NBIPX	Find name MSHOME<1e>

Authority RRs: 0
Additional RRs: 0

- Queries
- Answers
 - www.abc.com.br: type CNAME, class IN, cname wh55m.terraempresas.com.br
 - Name: www.abc.com.br
 - Type: CNAME (Canonical name for an alias)
 - Class: IN (0x0001)
 - Time to live: 6 minutes, 39 seconds
 - Data length: 22
 - Primary name: wh55m.terraempresas.com.br
 - wh55m.terraempresas.com.br: type A, class IN, addr 200.176.131.4
 - Name: wh55m.terraempresas.com.br
 - Type: A (Host address)
 - Class: IN (0x0001)
 - Time to live: 5 minutes
 - Data length: 4
 - Addr: 200.176.131.4

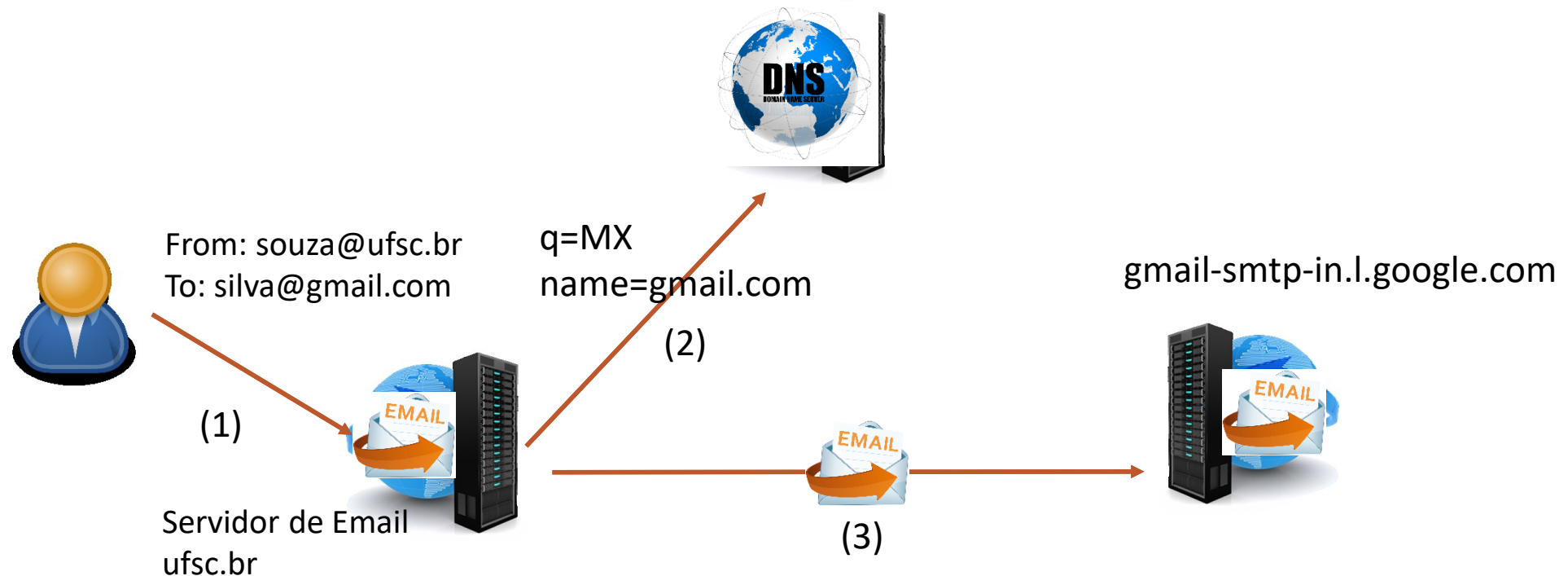
0000 00 08 0d 19 35 66 00 b0 d0 50 b6 8d 08 00 45 005f.. .P....E.
0010 00 6e 1c 27 00 00 7c 11 b7 86 96 a2 01 09 96 a2 .n.'...|.
0020 3c 84 00 35 04 04 00 5a 92 67 00 54 81 80 00 01 <..5...Z .g.T...
0030 00 02 00 00 00 00 03 77 77 77 03 61 62 63 03 63w ww. abc. c
0040 6f 6d 02 62 72 00 01 00 01 c0 0c 00 05 00 01 om.br... ..
0050 00 00 01 8f 00 16 05 77 68 25 25 6d 0d 74 65 72 w h55m tor

Frame (frame), 124 bytes Packets: 64 Displayed: 64 Marked: 0 Profile: Default

DNS (Domain Name System)

DNS não se limita a manter e gerenciar endereços Internet

- Particularmente importante para sistemas de correio eletrônico
- No DNS são definidos registros que identificam a máquina que manipula os emails relativos a um dado nome, identificando assim onde um determinado usuário recebe seus emails
- DNS é usado também para definição de listas para distribuição de emails.



DNS (Domain Name System)

```
Prompt de Comando - nslookup

> set q=mx
> ufsc.br.
Servidor:  dns-r1.ufsc.br
Address:  150.162.1.33

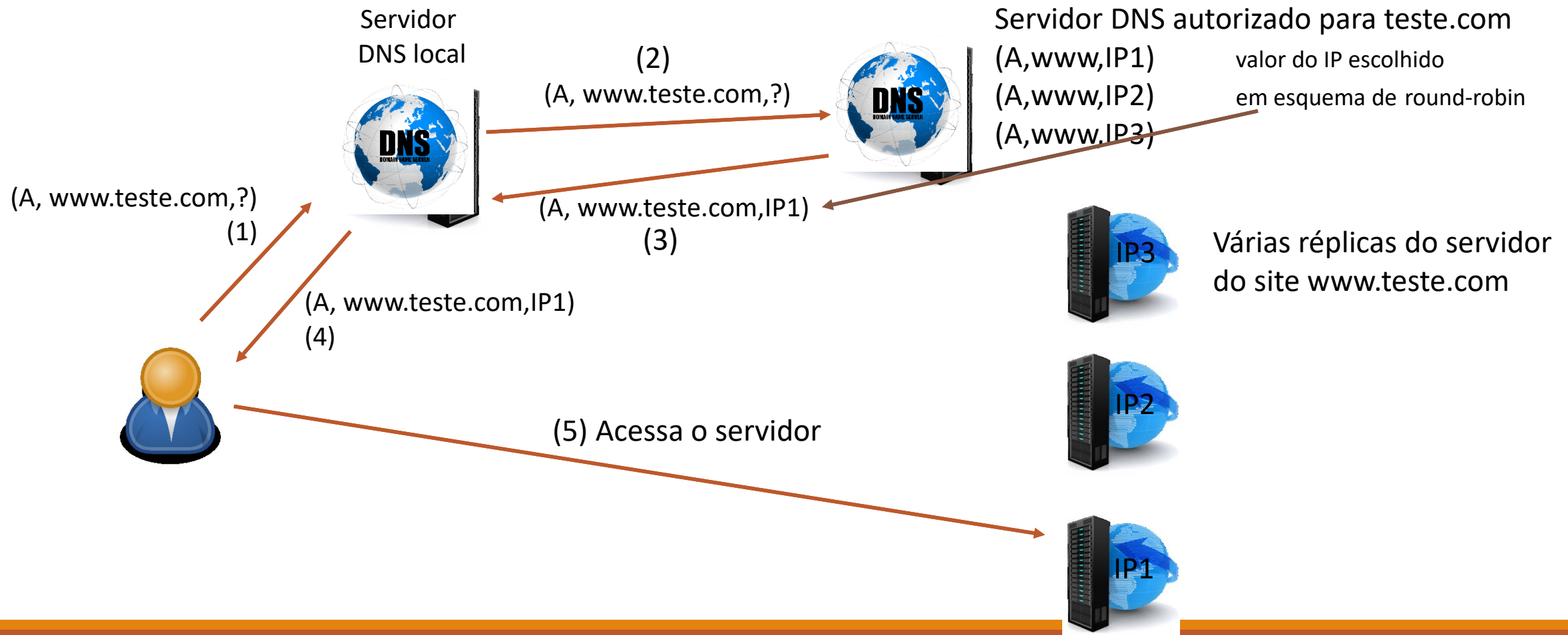
Não é resposta autoritativa:
ufsc.br MX preference = 10, mail exchanger = mx.mail.ufsc.br

ufsc.br nameserver = adufsc04.ufsc.br
ufsc.br nameserver = adufsc03.ufsc.br
ufsc.br nameserver = adns1.pop-sc.rnp.br
ufsc.br nameserver = adufsc01.ufsc.br
ufsc.br nameserver = ns67.ufsc.br
ufsc.br nameserver = adns2.pop-sc.rnp.br
mx.mail.ufsc.br internet address = 150.162.2.76
mx.mail.ufsc.br internet address = 150.162.2.75
mx.mail.ufsc.br AAAA IPv6 address = 2801:84:0:2:150:162:2:76
mx.mail.ufsc.br AAAA IPv6 address = 2801:84:0:2:150:162:2:75
ns67.ufsc.br    internet address = 150.162.1.67
ns67.ufsc.br    AAAA IPv6 address = 2801:84:0:1001::67
adns1.pop-sc.rnp.br    internet address = 200.237.192.10
adns2.pop-sc.rnp.br    internet address = 200.135.14.10
adns2.pop-sc.rnp.br    AAAA IPv6 address = 2001:12f0:200:a014::10
adufsc01.ufsc.br    internet address = 150.162.1.69
adufsc01.ufsc.br    internet address = 150.162.1.98
adufsc01.ufsc.br    AAAA IPv6 address = 2801:84:0:1001::98
adufsc01.ufsc.br    AAAA IPv6 address = 2801:84:0:1001:2c80:356a:4a26:35fa
adufsc01.ufsc.br    AAAA IPv6 address = 2801:84:0:1001::69
adufsc03.ufsc.br    internet address = 150.162.1.67
>
```

DNS (Domain Name System)

DNS não se limita a manter e gerenciar endereços Internet

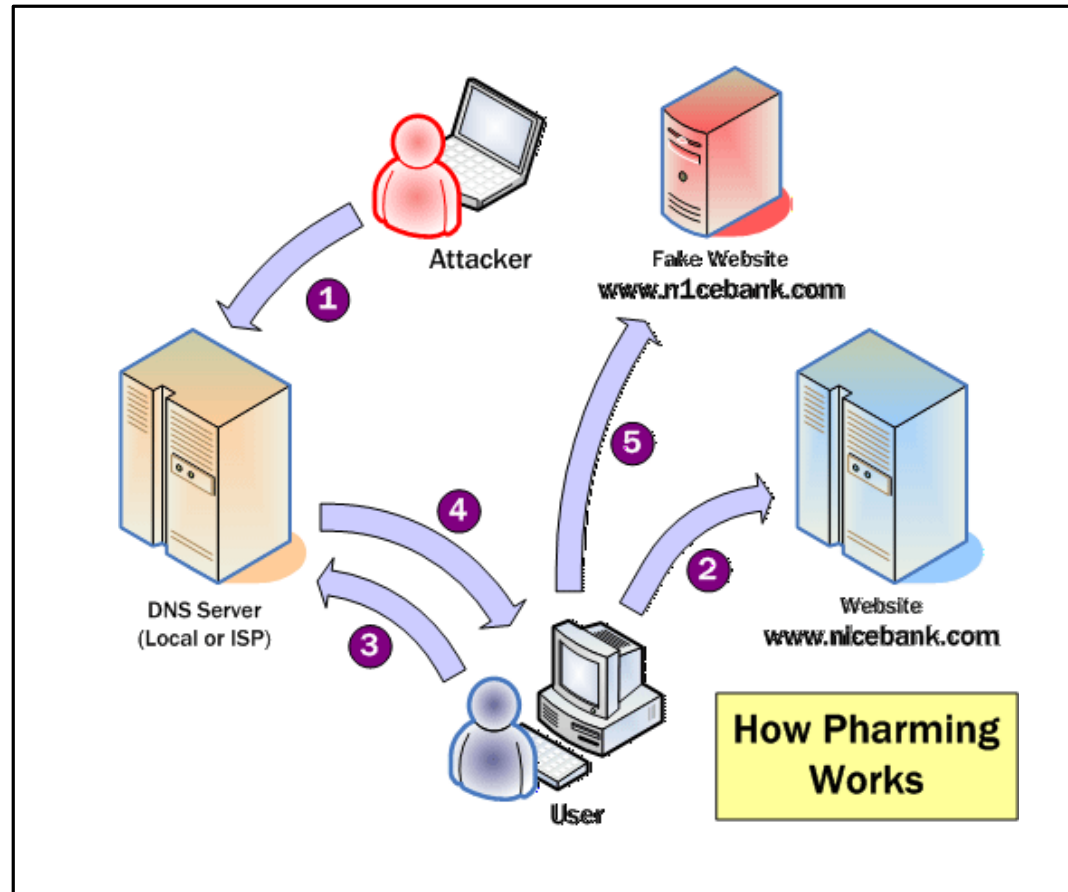
- Também usado como solução para balanceamento de carga



Segurança

DNS pode ser aproveitado para ações ilegais.

- Um modo de fazer isso é induzir um nome a apontar para um endereço diferente do correto (**envenenamento**).



Segurança

DNSSEC

- Devido a problemas como o mencionado anteriormente, a IETF (Internet Engineering Task Force) criou esta extensão do DNS.
- Ela autentica as informações do DNS e garante que as mesmas são autênticas e íntegras.
- Sua adoção depende de cada Top Level Domain.

Pontos Importantes

Registros DNS

- Entender os tipos de registros DNS apresentados (A, AAAA, NS, CNAME, MX)

Protocolo DNS

- Não precisa saber de cor o formato, mas entender o seu uso

Suas aplicações

- Para Email
- Para balanceamento de carga

CAP 4. CAMADA DE APLICAÇÃO

AULA 6: DNS – REGISTRANDO UM DOMÍNIO

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

DNS: Inserindo registros no DNS

Exemplo: Empresa recém-criada “Network Utopia” no domínio “com.br”

- Registrar o nome networkutopia.com.br no “registrar” <http://registro.br/>
- É necessário fornecer ao registrar os nomes e endereços IP do seu servidor de nomes autorizados (primário e secundário). Registrar insere RRs no servidor TLD do domínio com.br.com:
 - (networkutopia.com.br, dns1.networkutopia.com.br, NS)
 - (dns1.networkutopia.com.br, 212.212.212.33, A)
 - (networkutopia.com.br, dns2.networkutopia.com.br, NS)
 - (dns2.networkutopia.com.br, 212.212.212.34, A)
- Nos servidores autorizados, inserir um registro Tipo A para www.networkutopia.com.br e um registro Tipo MX para networkutopia.com.br
 - (www.networkutopia.com.br, 212.212.212.2, A)
 - (networkutopia.com.br, mx1.networkutopia.com.br, MX)
 - (mx1.networkutopia.com.br, 212.212.212.3, A)

Servidores DNS

Implementações

- A maioria das implementações de DNS teve suas raízes nas RFCs 882 e 883, atualizados nas RFCs 1034 e 1035.
- A implementação do DNS foi desenvolvida originalmente para o sistema operacional BSD UNIX 4.3.
- Na Microsoft
 - A implementação do Servidor de DNS Microsoft se tornou parte do sistema operacional Windows NT na versão Server 4.0
 - O DNS passou a ser o serviço de resolução de nomes padrão a partir do Windows 2000 Server
- BIND (Berkeley Internet Name Domain)
 - <https://www.isc.org/bind/>
 - Sistema DNS em código aberto

Pontos Importantes

Registrando um domínio

- Saber dos registros RR necessários para colocar no ar um domínio no DNS

CAP 4. CAMADA DE APLICAÇÃO

AULA 7: HTTP – PEDIDOS

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Web: Jargões

Quase todas as páginas WWW consistem de:

- página base HTML, e
- vários objetos referenciados.
- endereçados por uma URL

URL tem duas partes: nome de hospedeiro, e nome de caminho:

`www.someschool.edu/someDept/pic.gif`

Nome do hospedeiro

Nome do caminho

Agente usuário para WWW é o browser:

- Chrome
- Firefox
- Edge

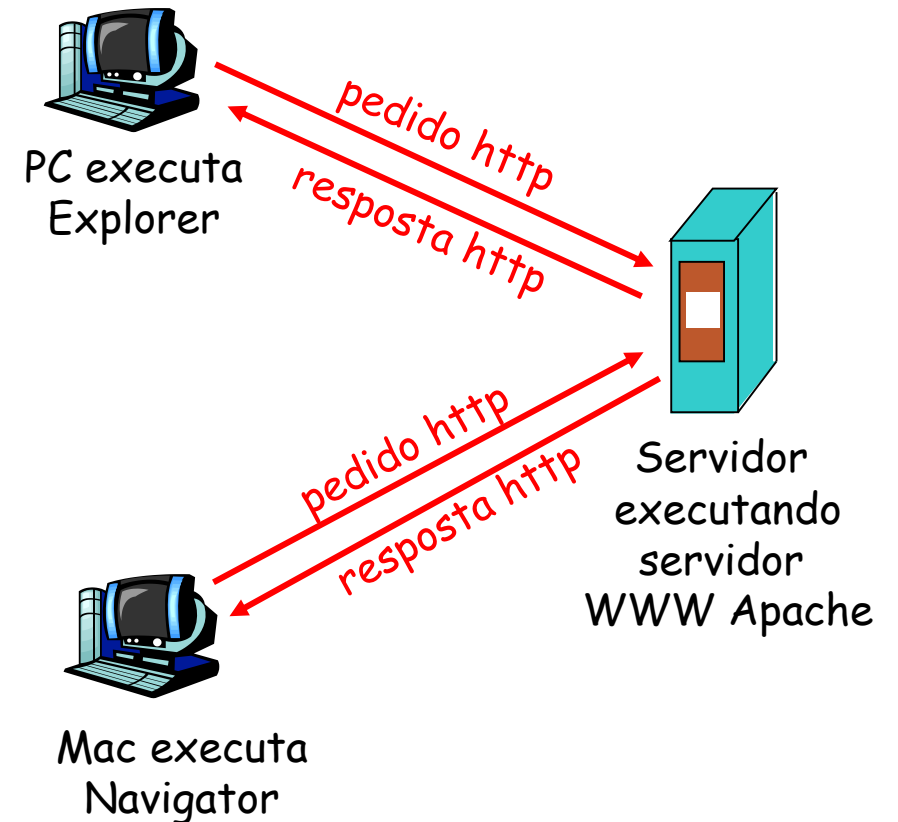
Servidor para WWW é o “servidor Web”:

- Apache HTTP Server (domínio público)
- Internet Information Services (IIS)
- nginx

WWW: o protocolo HTTP

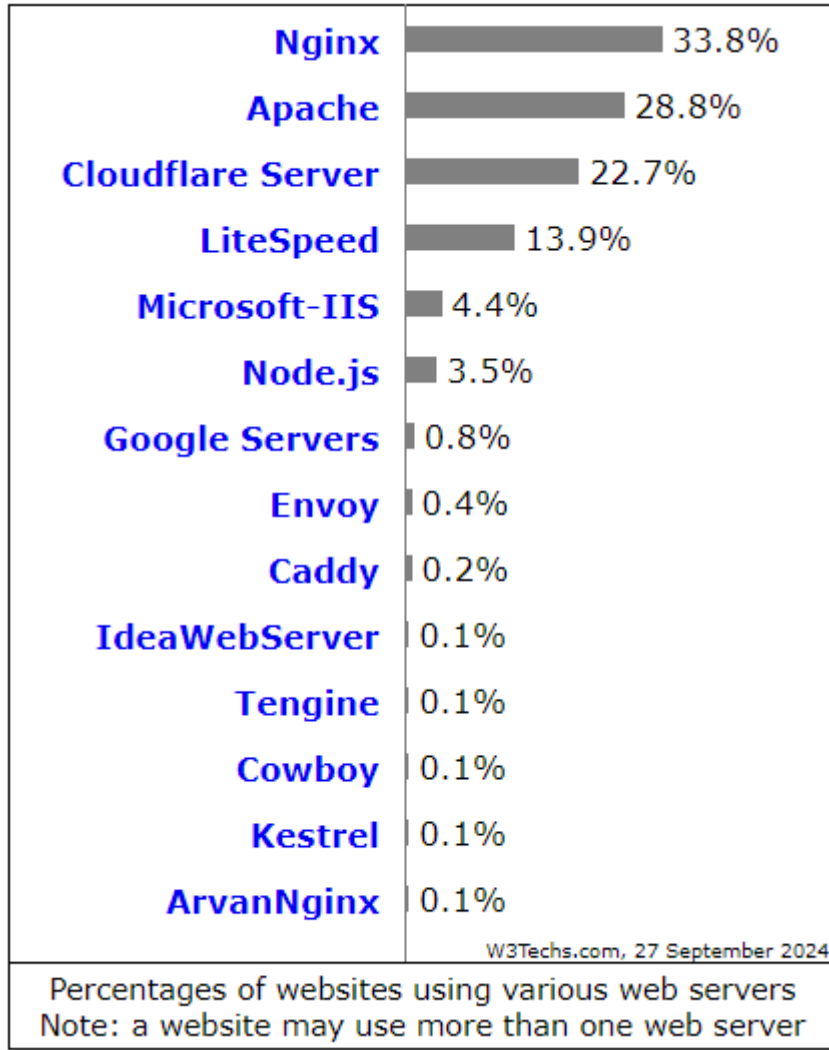
HTTP: Hypertext Transfer Protocol

- protocolo da camada de aplicação para a Web
- Modelo cliente/servidor
 - cliente: browser que pede, recebe, “visualiza” objetos WWW
 - servidor: servidor Web envia objetos em resposta a pedidos
- Versões
 - http1.0: RFC 1945
 - http1.1: RFC 2068
 - http/2: RFC 7540



WWW: o protocolo HTTP

Servidores mais usados



Fonte: https://w3techs.com/technologies/overview/web_server/all

Mais sobre o protocolo HTTP 1.x

Usa serviço de transporte TCP:

- Cliente inicia conexão TCP (cria socket) com servidor, porta 80 (porta default)
- Servidor aceita conexão TCP do cliente
- Mensagens HTTP (mensagens do protocolo da camada de apl) trocadas entre browser (cliente HTTP) e servidor Web (servidor HTTP)
- Encerra conexão TCP

HTTP é “sem estado”

- Servidor não mantém informação sobre pedidos anteriores do cliente
- Simplifica a implementação

Exemplo de Operação http 1.0

Usuário entra com a URL: `www.algumaEscola.edu.br/algumDepto/index.html`

1a. cliente http **inicia conexão TCP** ao servidor http (processo) em `www.algumaEscola.edu.br` Porta 80 é a default para o servidor http .

(contém referência a 10 imagens jpeg)

1b. servidor http no host `www.algumaEscola` esperando pela conexão TCP na porta 80. "aceita" conexão, notificando o cliente

2. cliente http envia **mensagem de pedido** (contendo a URL) para o socket da conexão TCP

3. servidor http recebe mensagem de pedido, forma **mensagem de resposta** contendo o objeto solicitado (`algumDepto/index.index`), envia mensagem para o socket

tempo

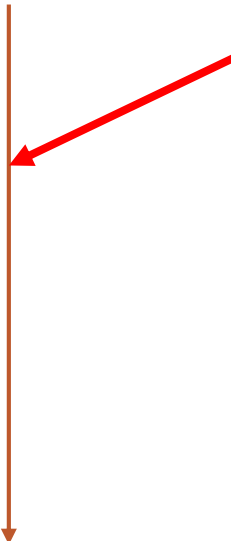
Exemplo (cont.)

5. cliente http recebe mensagem de resposta contendo o arquivo html, apresenta o conteúdo html. Analisando o arquivo html encontra 10 objetos jpeg referenciados

4. servidor http fecha conexão TCP.

6. Passos 1-5 são repetidos para cada um dos 10 objetos jpeg.

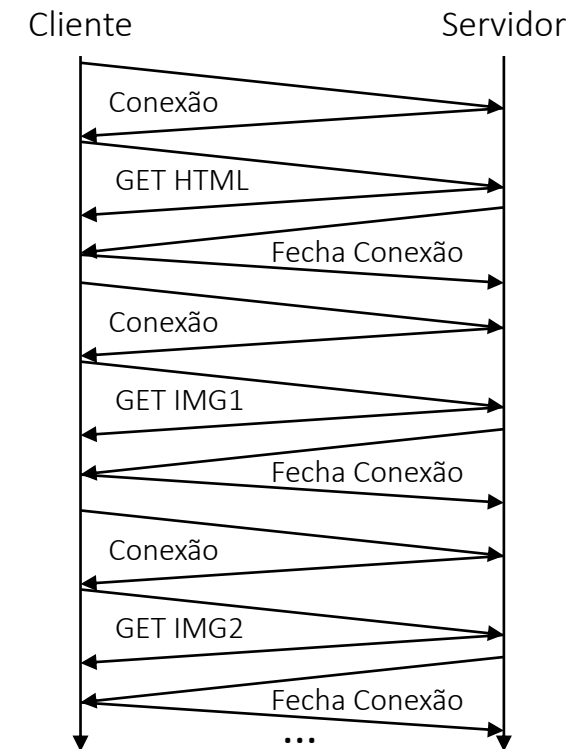
tempo



Conexões Não-persistentes e Persistentes

Conexões não persistentes

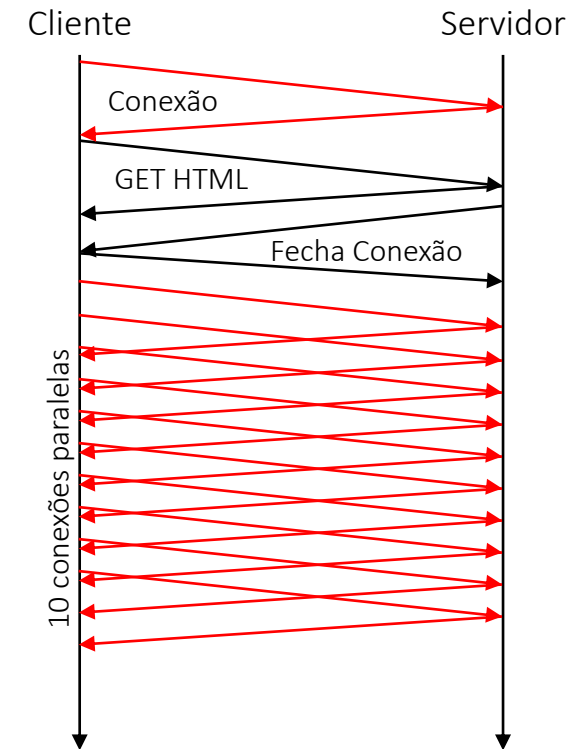
- HTTP/1.0 utiliza conexões não persistentes
- Conexão TCP é fechada após o servidor http enviar o objeto
 - Cada conexão transporta uma requisição e uma resposta
 - No exemplo são necessárias 11 conexões TCP
- Desvantagens: Uma conexão deve ser estabelecida por objeto
 - Necessário alocar buffers no TCP e manter variáveis do TCP tanto no cliente quanto no servidor
 - Atraso na transferência dos objetos
 - Partida lenta do TCP causado pelo controle de congestionamento
 - Conexões podem ser em série ou paralelo
 - Sobrecarrega servidores que atendem vários clientes ao mesmo tempo



Conexões Não-persistentes e Persistentes

Conexões não persistentes

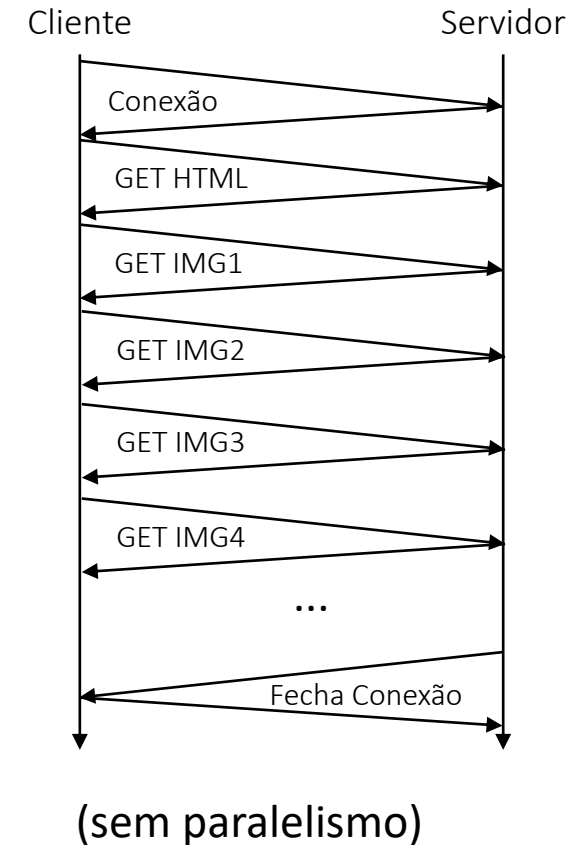
- HTTP/1.0 utiliza conexões não persistentes
- Conexão TCP é fechada após o servidor http enviar o objeto
 - Cada conexão transporta uma requisição e uma resposta
 - No exemplo são necessárias 11 conexões TCP
- Paralelismo reduz algumas desvantagens
 - Reduz latência na carga da página
 - Sofre dos problemas de sobrecarga no servidor e cliente devido a necessidade de buffers
 - Cada conexão sofre da partida lenta: deixa a taxa mais lenta no começo da conexão



Conexões Não-persistentes e Persistentes

Conexões Persistentes (default para o HTTP/1.1)

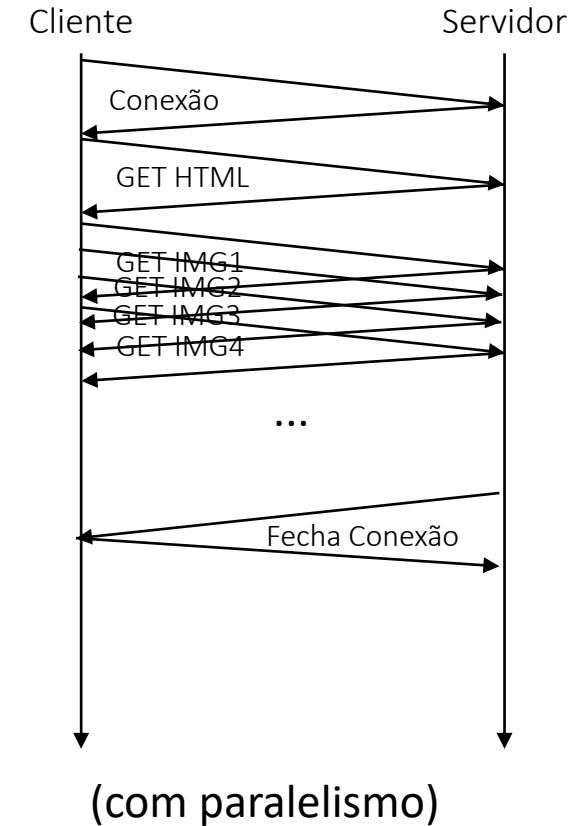
- Servidor deixa a conexão TCP aberta após o envio da resposta
 - Requisições subsequentes são enviadas pela mesma conexão TCP
- Conexão é fechada após um certo tempo de inatividade (configurável)
- Duas versões
 - Sem paralelismo: cliente lança uma nova requisição somente quando a resposta prévia é recebida
 - Com paralelismo: cliente pode lançar pedidos sem respostas prévias (default do HTTP/1.1)



Conexões Não-persistentes e Persistentes

Conexões Persistentes (default para o HTTP/1.1)

- Servidor deixa a conexão TCP aberta após o envio da resposta
 - Requisições subsequentes são enviadas pela mesma conexão TCP
- Conexão é fechada após um certo tempo de inatividade (configurável)
- Duas versões
 - Sem paralelismo: cliente lança uma nova requisição somente quando a resposta prévia é recebida
 - Com paralelismo: cliente pode lançar pedidos sem respostas prévias (default do HTTP/1.1)



Conexões Não-persistentes e Persistentes

Conexões Persistentes (default para o HTTP/1.1)

- Vantagem
 - Reduz o problema de partida lenta do TCP
 - Após ter enviado o primeiro objeto, não tem de enviar o próximo objeto na taxa inicial lenta
 - Reduz o atraso
- Desvantagens de sem paralelismo
 - Conexão TCP fica pendurada enquanto resposta não é recebida
- Vantagens de com paralelismo
 - Conexão fica pendurada por um menor tempo

Modelagem do tempo de resposta

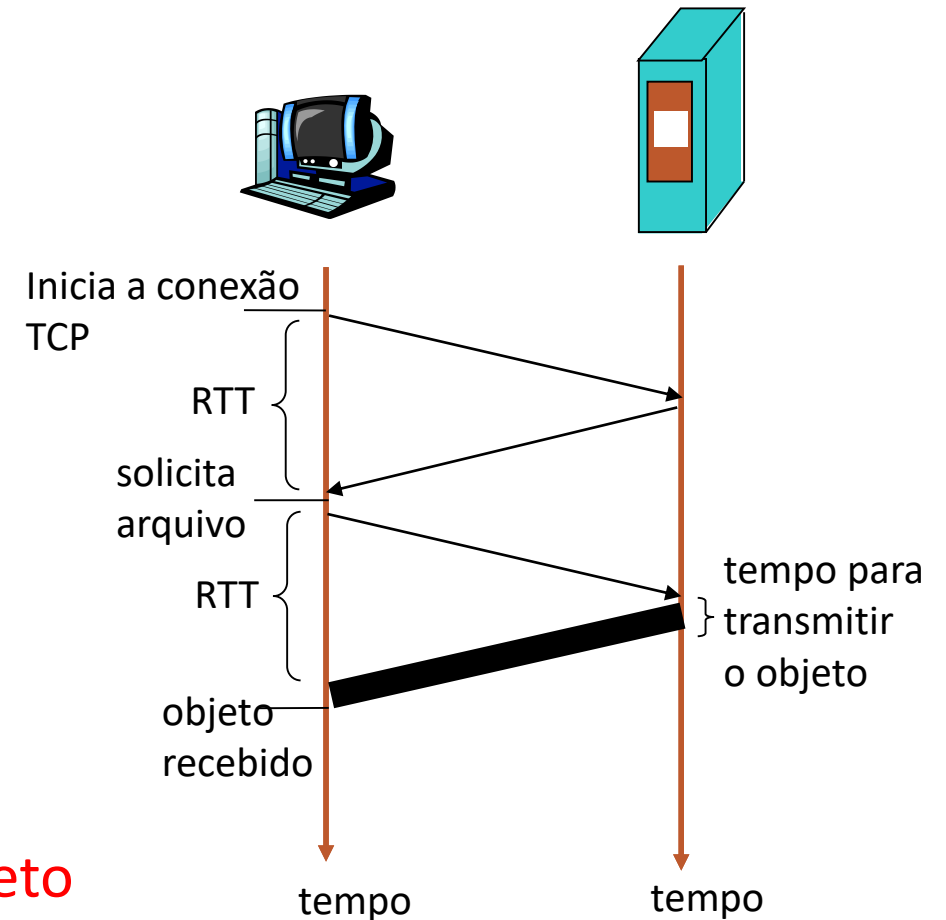
Definição de RTT (Round Trip Time):

- intervalo de tempo entre a ida e a volta de um pacote entre um cliente e um servidor.

Tempo de resposta:

- um RTT para iniciar a conexão TCP
- um RTT para o pedido HTTP e o retorno dos primeiros bytes da resposta HTTP
- tempo de transmissão do objeto (página, imagem, etc.)

$$\text{total} = 2\text{RTT} + \text{tempo de transmissão do objeto}$$



Formato de mensagem HTTP 1.x

Dois tipos de mensagem HTTP: pedido, resposta

Mensagem de pedido HTTP:

- ASCII (formato legível por pessoas)
- Linha de requisição: <método> <URL> <versão do HTTP>

The diagram illustrates the structure of an HTTP request message. It shows a sequence of lines: a request line, header lines, and a final line indicating the end of the message. Annotations in red text with arrows point to these parts: 'linha do pedido (comandos GET, POST, HEAD)' points to the request line; 'linhas do cabeçalho' points to the header lines; and 'Carriage return, line feed indicam fim de mensagem' points to the carriage return and line feed characters at the end of the message.

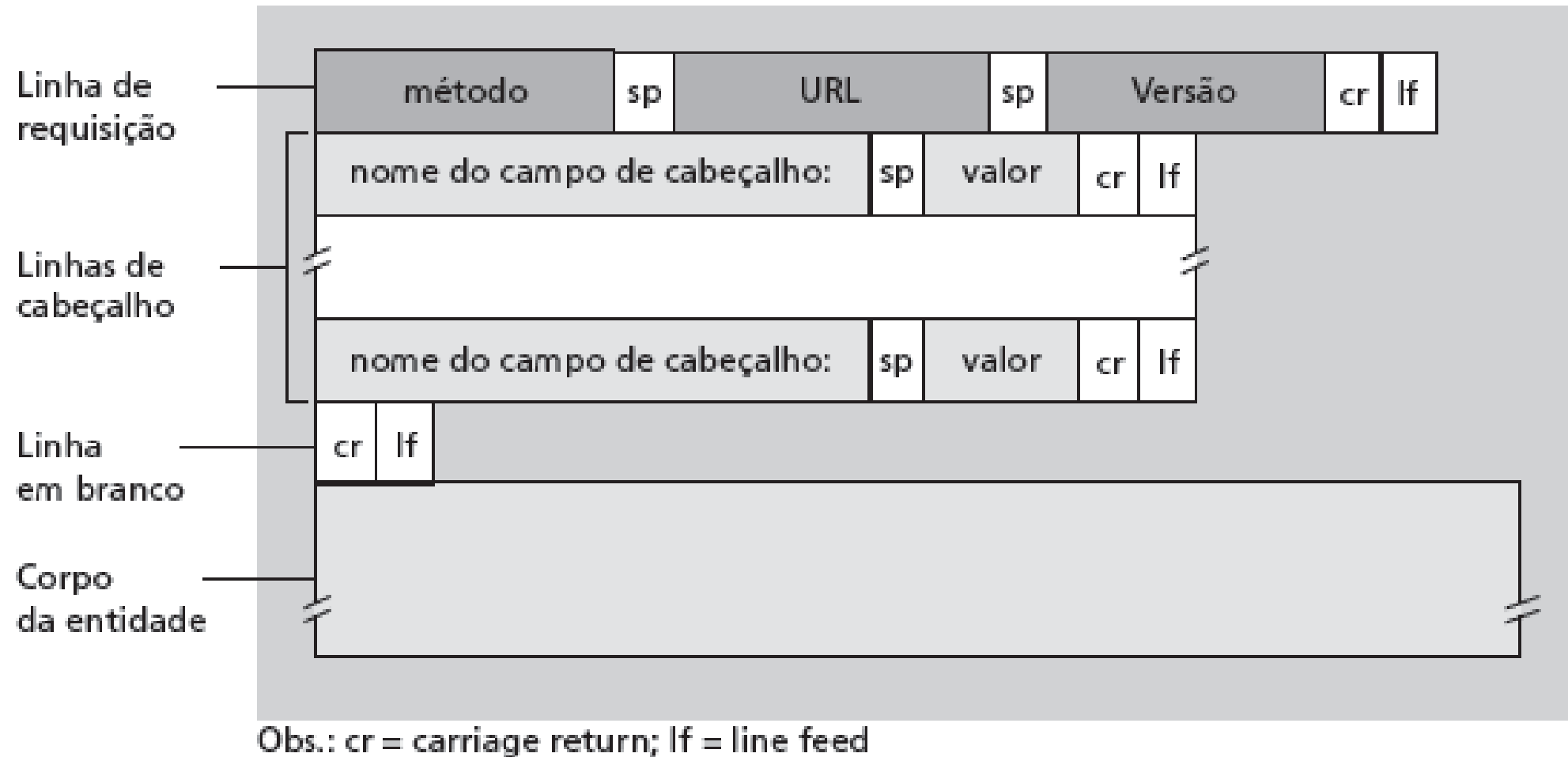
```
GET /algumdir/pagina.html HTTP/1.1
Host: www.algumsite.com
Connection: close
User-agent: Mozilla/4.0
Accept-language:br
(carriage return, line feed)
```

linha do pedido
(comandos GET,
POST, HEAD)

linhas do
cabeçalho

Carriage return,
line feed
indicam fim
de mensagem

Mensagem de Pedido http 1.x: formato Geral



Formato da mensagem de pedido http

HTTP/ 1.0

- Método Get
 - Solicita objeto especificado (URL)
- Método Post
 - Usado pelo cliente http quando o usuário preenche um formulário
 - É uma mensagem de solicitação do objeto
 - Conteúdo específico da página web depende do que o usuário escreveu no formulário
- Método Head
 - Similar ao método Get
 - Mas servidor envia a resposta sem o objeto solicitado
 - Usado para depuração

Formato da mensagem de pedido http

HTTP/1.1

- GET, POST, HEAD
- PUT
 - Envia o arquivo no corpo da entidade para o caminho especificado no campo de URL
- DELETE
 - Apaga o arquivo especificado no campo de URL
- Outros
 - TRACE: Ecoa o pedido, de maneira que o cliente possa saber o que os servidores intermediários estão mudando em seu pedido.
 - OPTIONS: Recupera os métodos HTTP que o servidor aceita.
 - CONNECT: Serve para uso com um proxy que possa se tornar um túnel SSL

Formato da mensagem http 1.x

Método GET

- Linha de comando indica o arquivo a ser transferido ao Browser
- Host indica hospedeiro onde o objeto reside;
- Connection: close indica para servidor usar conexão não persistente
- User-agent: indica o tipo de browser usado
- Accept-language: br indica que usuário prefere uma versão em português do objeto

linhas do
cabeçalho

```
GET /algumdir/pagina.html HTTP/1.1
Host: www.algumsite.com
Connection: close
User-agent: Mozilla/4.0
Accept-language:br
```

(carriage return, line feed)

Formatos HTTP 1.x: Resposta

linha de status
(protocolo
código de status
frase de status)

HTTP/1.1 200 OK

Connection: close

Date: Thu, 06 Aug 1998 12:00:15 GMT

Server: Apache/1.3.0 (Unix)

Last-Modified: Mon, 22 Jun 1998 09:23:24 GMT

Content-Length: 6821

Content-Type: text/html

linhas de
cabeçalho

dados, e.x.,
arquivo html

Objeto solicitado ...

Exemplo de tipo MIME	Descrição
text/plain	Arquivo no formato texto (ASCII)
text/html	Arquivo no formato HTML, utilizado como padrão para documentos Web
Image/gif	Imagem com o formato GIF
Image/jpeg	Imagem com o formato JPEG
application/zip	Arquivo compactado

Formato da mensagem http 1.x: Resposta

Códigos de status (Descritos na RFC 2616)

- 1xx: Informational (Informação) – utilizada para enviar informações para o cliente de que sua requisição foi recebida e está sendo processada;
- 2xx: Success (Sucesso) – indica que a requisição do cliente foi bem sucedida;
 - 200 OK: Pedido com sucesso, objeto pedido está na mensagem
- 3xx: Redirection (Redirecionamento) – informa a ação adicional que deve ser tomada para completar a requisição;
 - 301 Moved Permanently: Objeto pedido movido, nova localização é especificado na mensagem (Location:)
- 4xx: Client Error (Erro no cliente) – avisa que o cliente fez uma requisição que não pode ser atendida;
 - 400 Bad Request: Mensagem de pedido não é entendida pelo servidor
 - 404 Not Found: Documento pedido não foi encontrado no servidor
- 5xx: Server Error (Erro no servidor) – ocorreu um erro no servidor ao cumprir uma requisição válida.
 - 505 HTTP Version Not Supported

Formato da mensagem http 1.x

Método Post

- Usado quando é necessário enviar dados ao servidor
 - para serem processados geralmente por um programa script identificado no Request-URI.
 - informações submetidas são incluídas no corpo da mensagem e formatadas como uma query string, além de conter cabeçalhos adicionais especificando seu tamanho (Content-Length) e seu formato (Content-Type).

```
POST /index.html HTTP/1.0
Accept: text/html
If-modified-since: Sat, 29 Oct 1999 19:43:31 GMT
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
```

```
Nome=NomePessoa&Idade=99&Curso=Computacao
```


HTTP Cliente: faça você mesmo!

1. Telnet para um servidor Web:

```
telnet www.inf.ufsc.br 80
```

Abre conexão TCP para a porta 80 (porta default do servidor http) em `www.inf.ufsc.br`. Qualquer coisa digitada é enviada para a porta 80 em `www.inf.ufsc.br`

2. Digite um pedido GET http:

```
GET /~roberto.willrich/ HTTP/1.1  
Host: www.inf.ufsc.br
```

Digitando isto (tecle carriage return duas vezes), você envia este pedido HTTP GET mínimo (mas completo) ao servidor http

3. Examine a mensagem de resposta enviada pelo servidor http!

Pontos Importantes

Protocolo HTTP

- Utiliza o protocolo TCP Porta 80
- Entender os tipos de conexões persistentes e não persistentes
- Pedidos básicos: GET e POST

CAP 4. CAMADA DE APLICAÇÃO

AULA 8: HTTP – OUTRAS FUNCIONALIDADES

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

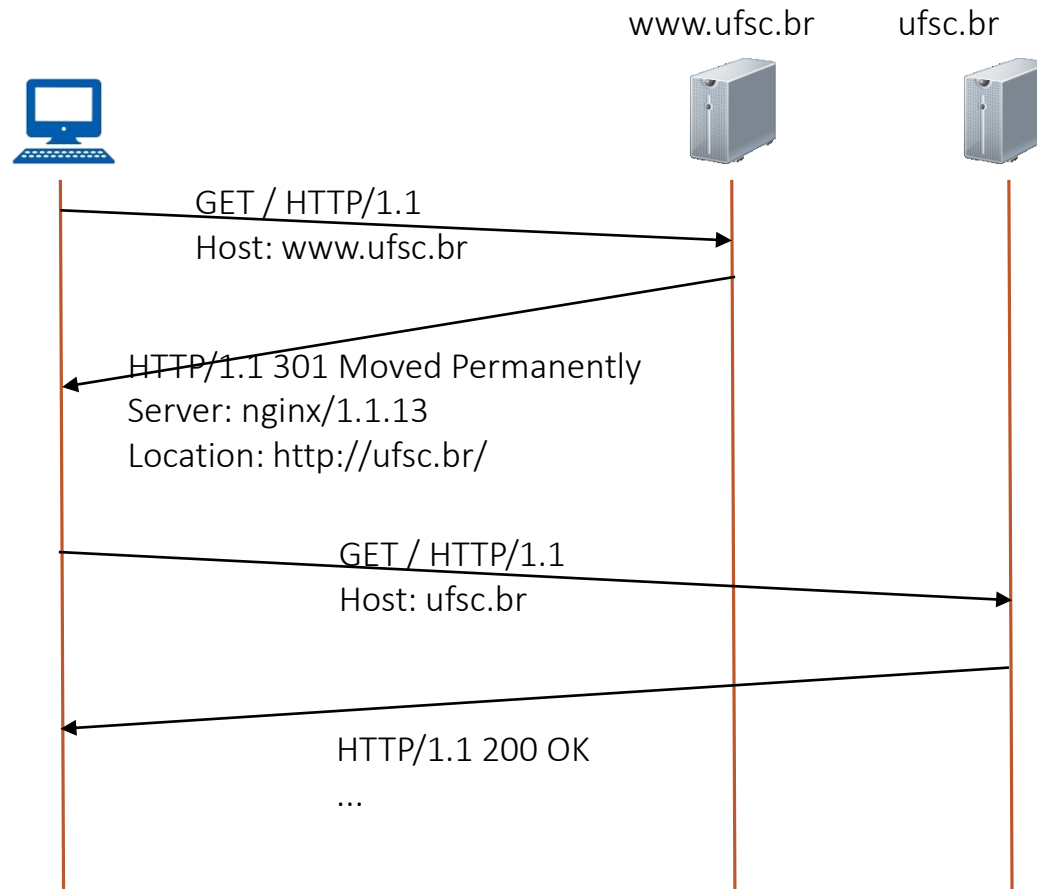
Funcionalidades do HTTP

HTTP oferece várias funcionalidades

- Redirecionamento
- Autenticação
- Cookies
- Get condicional

Redirecionamento

HTTP oferece um modo simples para indicar que uma página web foi movida



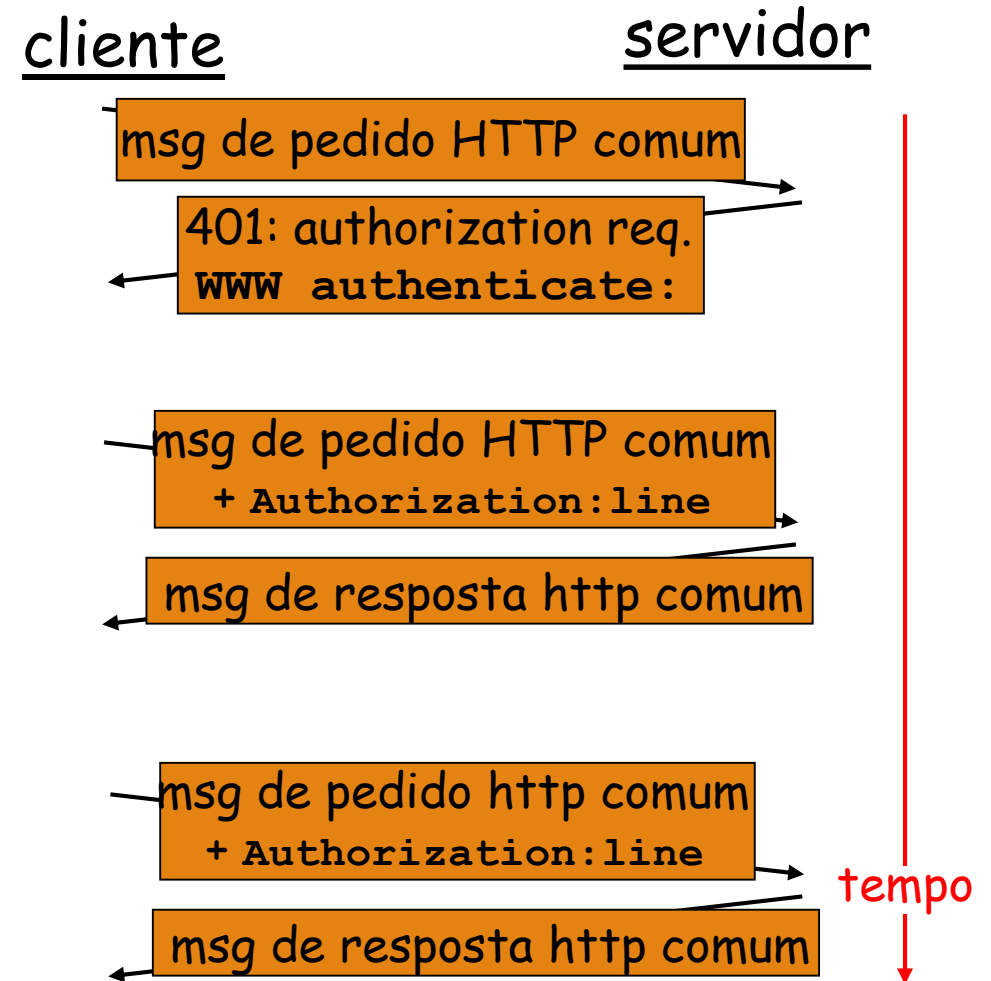
Interação usuário-servidor: autenticação

Meta da autenticação: controle de acesso aos documentos do servidor

- sem estado: cliente deve apresentar autorização com cada pedido
- autorização: tipicamente nome, senha
 - *authorization*: linha de cabeçalho no pedido
 - se não for apresentada autorização, servidor nega acesso, e coloca no cabeçalho da resposta
 - *WWW authenticate*:

Browser guarda nome e senha para

- evitar que sejam pedidos ao usuário a cada acesso.



Interação usuário-servidor: autenticação

Exemplo <http://www.inf.ufsc.br/~roberto.willrich/Ensino/INE5639/>

- Usuário teste e senha teste1
- Pedido do Browser

GET /~roberto.willrich/Ensino/INE5639/ HTTP/1.1

Host: www.inf.ufsc.br

Connection: keep-alive

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/73.0.3683.103 Safari/537.36

...

- Resposta do Servidor

HTTP/1.1 401 Unauthorized

Date: Wed, 17 Apr 2019 12:58:16 GMT

Server: Apache

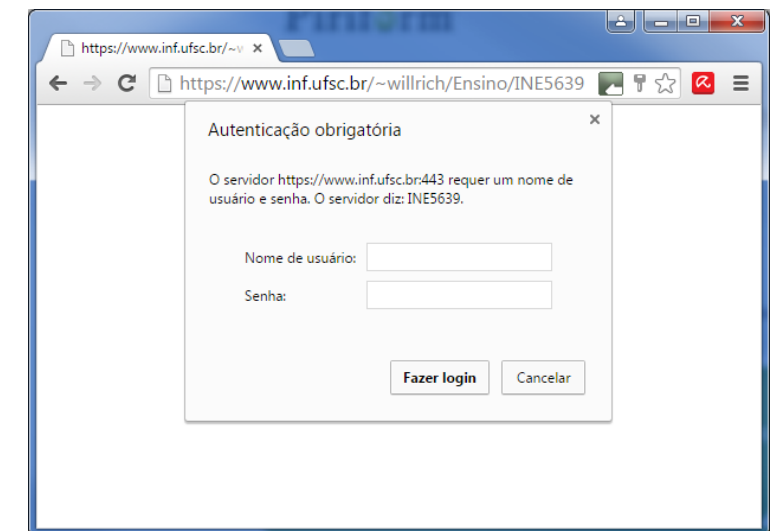
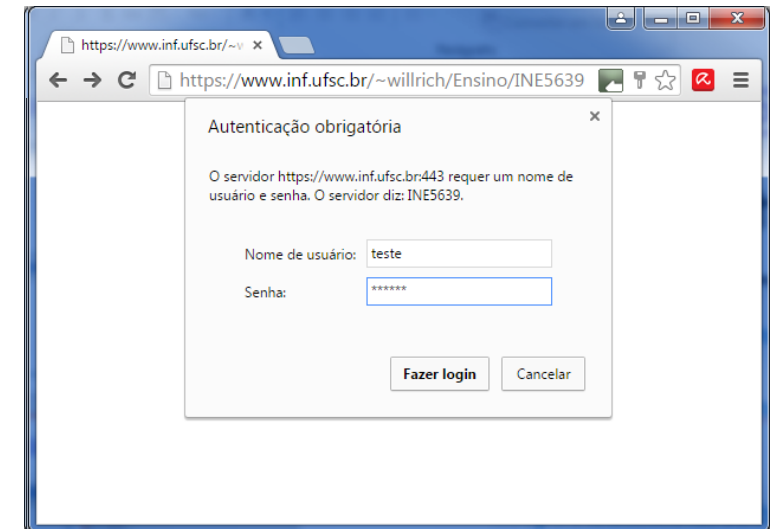
WWW-Authenticate: Basic realm="Acesso Restrito a Usuarios"

Content-Length: 381

Keep-Alive: timeout=5, max=100

Connection: Keep-Alive

Content-Type: text/html; charset=iso-8859-1



Interação usuário-servidor: autenticação

Exemplo (Cont.)

- Pedido do browser

GET /~roberto.willrich/Ensino/INE5639/ HTTP/1.1

Host: www.inf.ufsc.br

Connection: keep-alive

Authorization: Basic dGVzdGU6dGVzdGUx

Experimente decodificar:

<https://www.base64decode.org/>

Codificação base64 de
teste:teste1

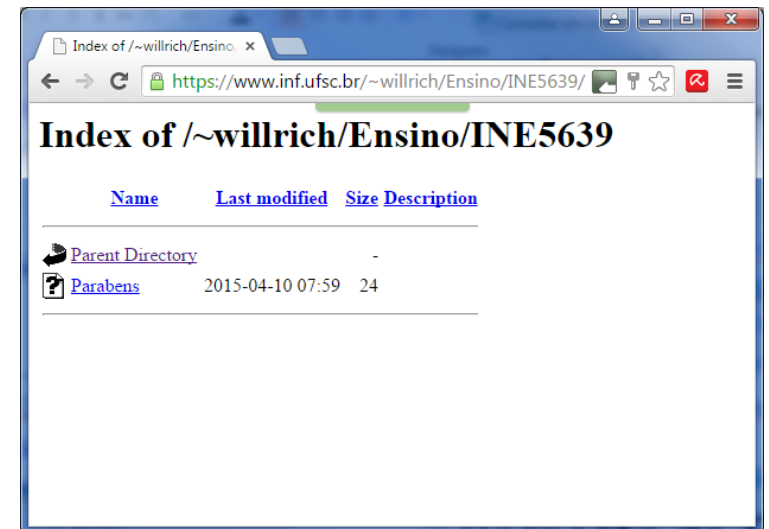
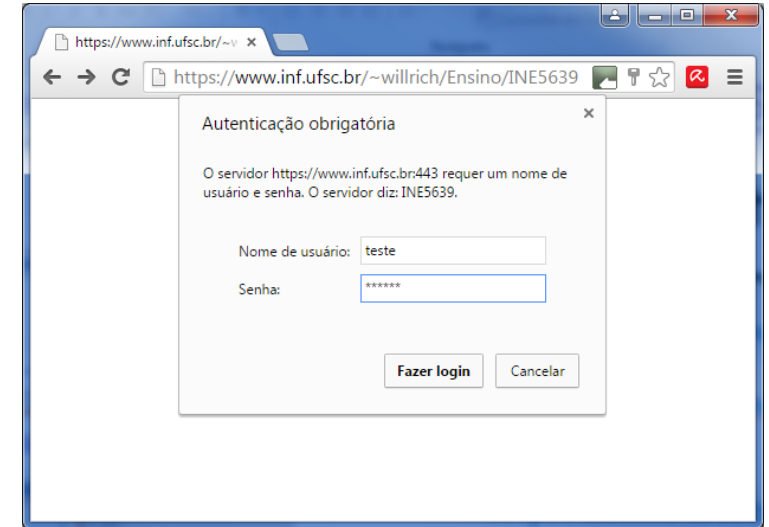
- Resposta do servidor

HTTP/1.1 200 OK

Date: Wed, 17 Apr 2019 12:58:33 GMT

Server: Apache

...



Interação usuário-servidor: autenticação

Exemplo (Cont.)

- Pedido do browser

GET /%7Ewillrich/Ensino/INE5615/Parabens
HTTP/1.1

Host: www.inf.ufsc.br

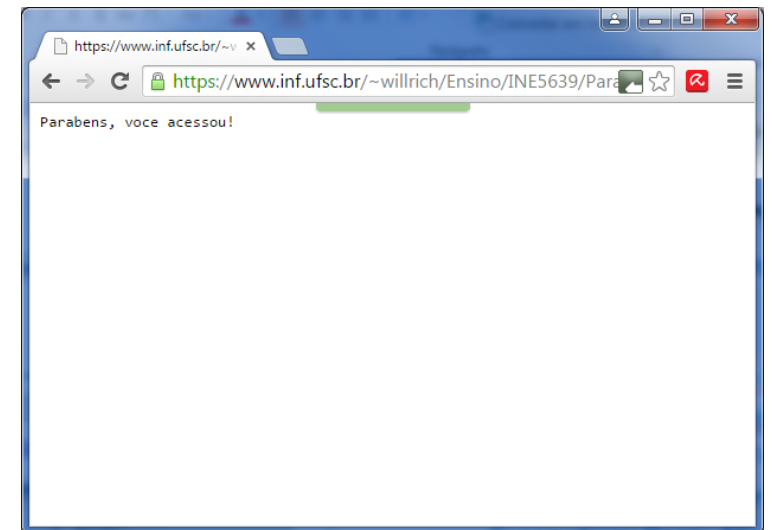
Connection: Keep-Alive

Authorization: Basic dGVzdGU6dGVzdGUx

- Resposta do browser

HTTP/1.1 200 OK

...



Interação usuário-servidor: autenticação

.htpasswd

- Arquivo que servirá como base para as senhas

```
htpasswd -c .htpasswd <login>
```

.htaccess

- Arquivo de orientação/leitura utilizado pelo servidor Apache
- Para controle de acesso:

```
AuthName "Acesso Restrito à Usuários"  
AuthType Basic  
AuthUserFile /<pasta>/.htpasswd  
<Limit GET POST>  
  require user <login>  
</Limit>
```

Ou

```
require valid-user
```

Interação usuário-servidor: cookies

Cookies

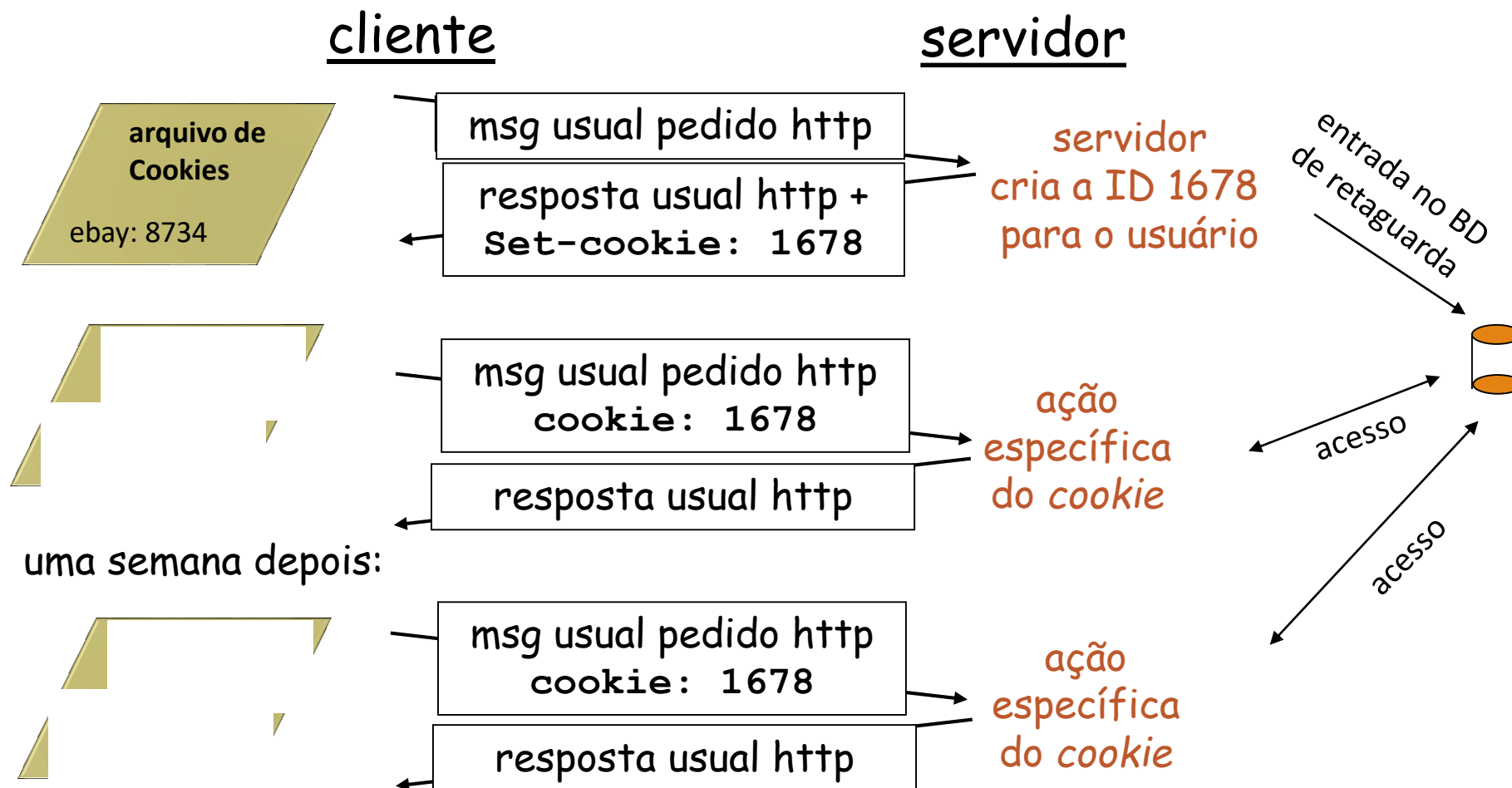
- Mecanismo que os sites utilizam para obter informações dos usuários
- Acessando um site usando cookies pela primeira vez
 - Resposta inclui um cabeçalho Set-cookie: #
 - # é um número de identificação
- Recebendo a mensagem o cliente inclui o nome do servidor e o número de identificação associado ao usuário
- Acessando o mesmo site posteriormente
 - Pedido do usuário inclui um cabeçalho Cookie: #
 - Não sabe o nome do usuário, mas pode manter um estado do usuário

Interação usuário-servidor: cookies

Finalidades

- Quando servidor necessita de identificação, mas não quer perturbar o usuário com um pedido de nome e senha
- Quando servidor quer lembrar das preferências de um usuário de modo que possa realizar uma adaptação do conteúdo
- Se o usuário estiver fazendo compras em um site, pode ser usado para anotar os itens que o usuário está comprando

Cookies: manutenção do “estado” (cont.)



Cookies: manutenção do “estado” (cont.)

Exemplo

- Pedido do browser

GET / HTTP/1.1

Host: www.inf.ufsc.br

...

- Resposta do browser

HTTP/1.1 200 OK

Date: Fri, 08 May 2009 14:23:35 GMT

...

Set-Cookie: PHPSESSID=be38e6f3ed5a5e7191f18d0fc816c7a1; path=/

- Pedidos subsequentes incluem cookie:

GET /wp-content/themes/k2/js/k2.functions.js.php?ver=223 HTTP/1.1

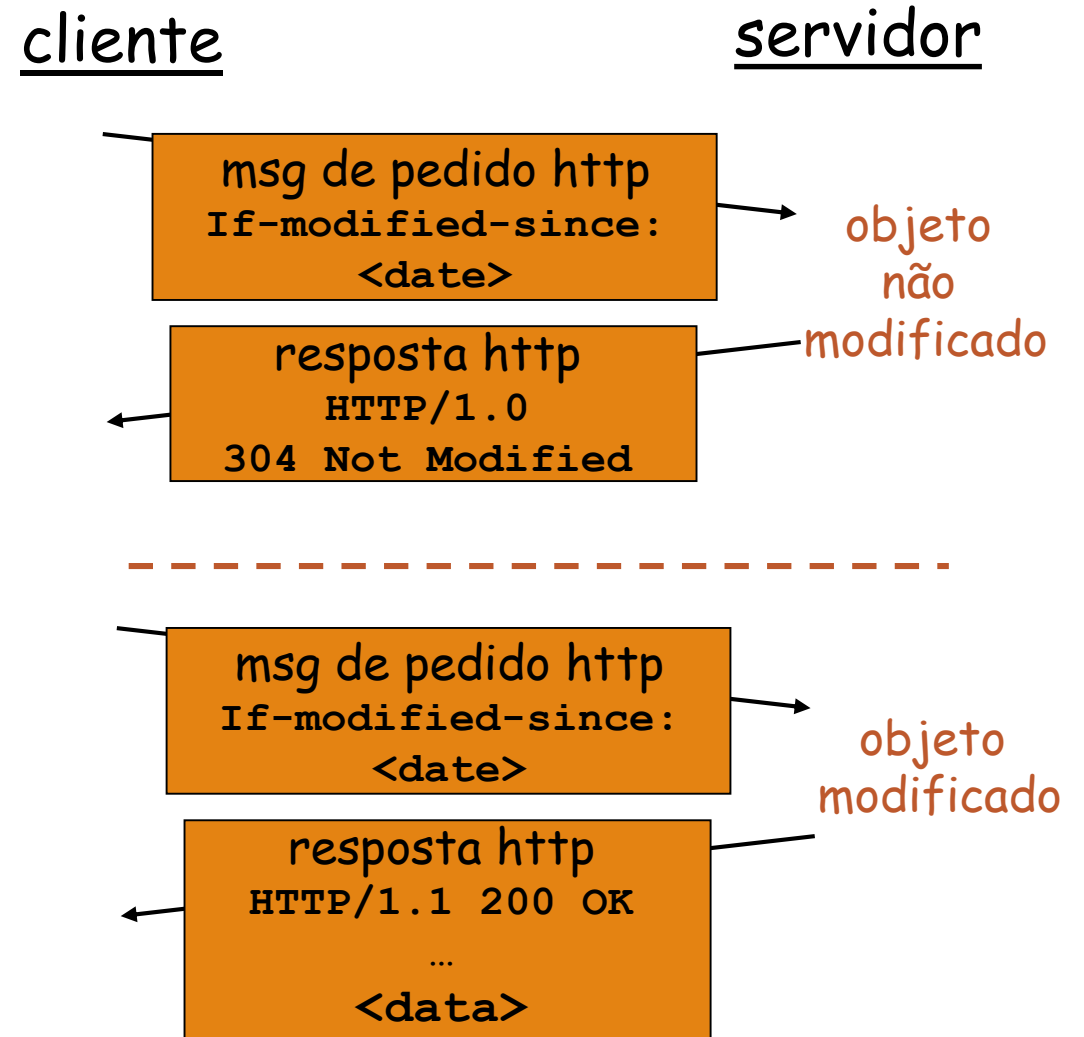
Host: www.inf.ufsc.br

Cookie: PHPSESSID=be38e6f3ed5a5e7191f18d0fc816c7a1

Interação usuário-servidor: GET condicional

Meta: não enviar objeto se cliente já tem (no cache) versão atual

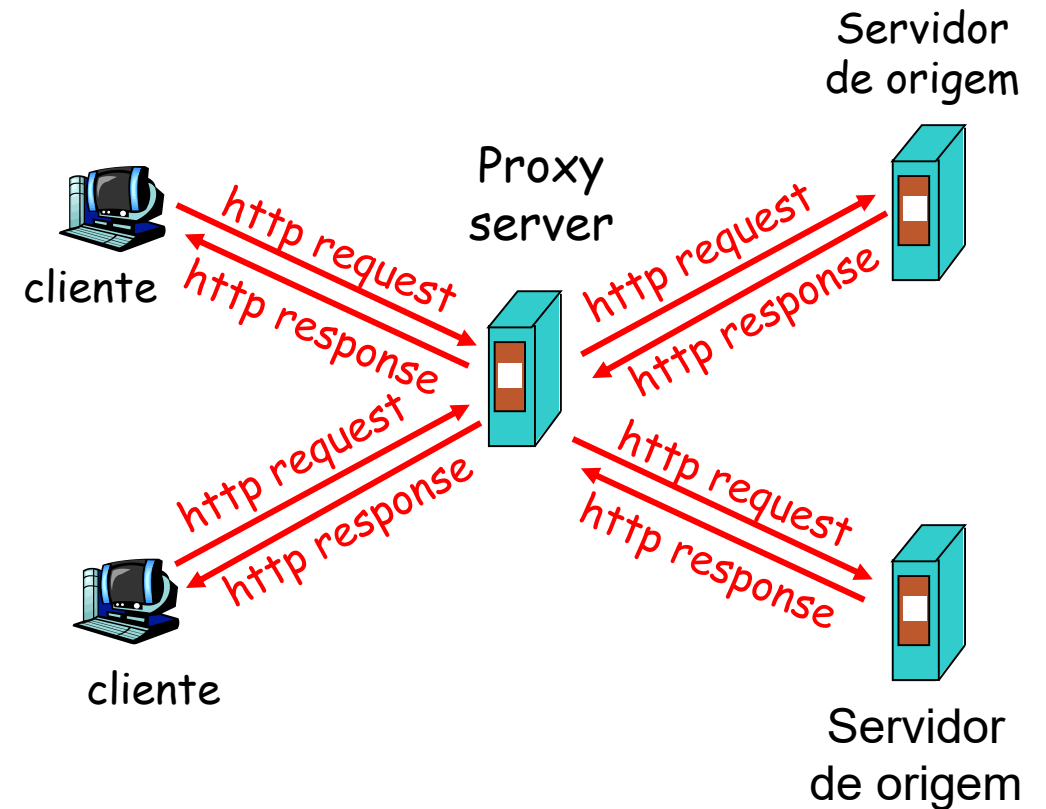
- cliente: especifica data da cópia no cache no pedido http
 - If-modified-since: <date>
- servidor: resposta não contém objeto se cópia no cache é atual:
 - HTTP/1.0 304 Not Modified



Web Caches (proxy server)

Meta: satisfazer pedido do cliente sem envolver o servidor de origem

- Usuário configura o browser: acessos web via cache web
- Cliente envia todos os pedidos http para a cache web
- Se o objeto está na cache ele é imediatamente retorna uma resposta http
- Senão o objeto é pedido para o servidor de origem, então retorna a resposta para o cliente



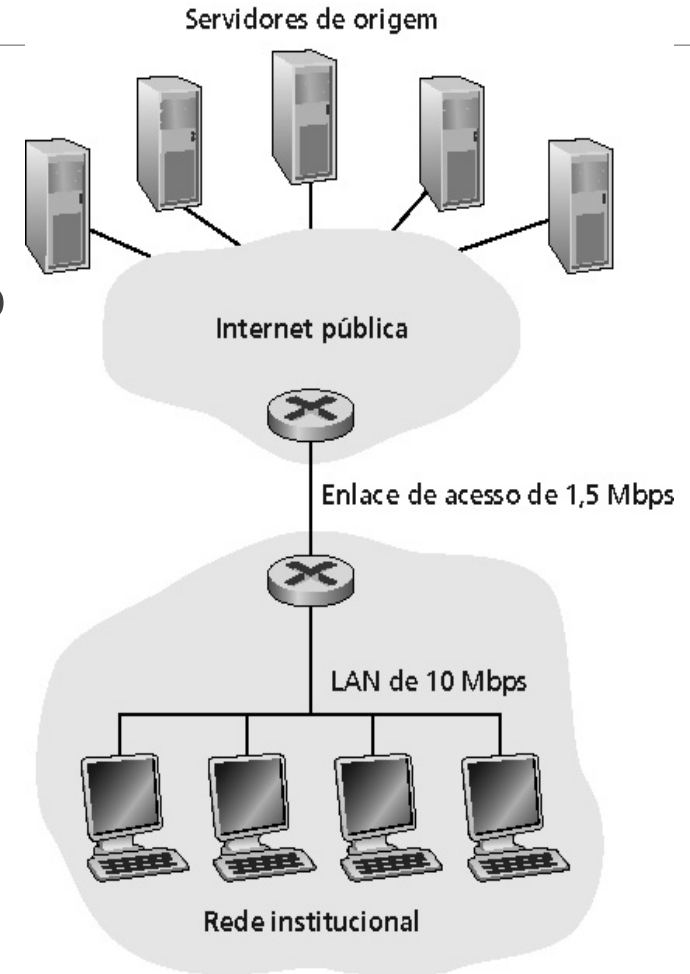
Exemplo de caching

Suponha:

- Tamanho médio objeto = 100.000 bits
- Taxa média de requisições dos browsers da instituição para os servidores de origem = 15/s
- Atraso do roteador institucional para ir a qualquer servidor de origem e retornar ao roteador = 2 s

Consequências:

- Utilização da LAN = 15%
- Utilização do link de acesso = 100%
- Atraso total = atraso da Internet + atraso de acesso + atraso da LAN



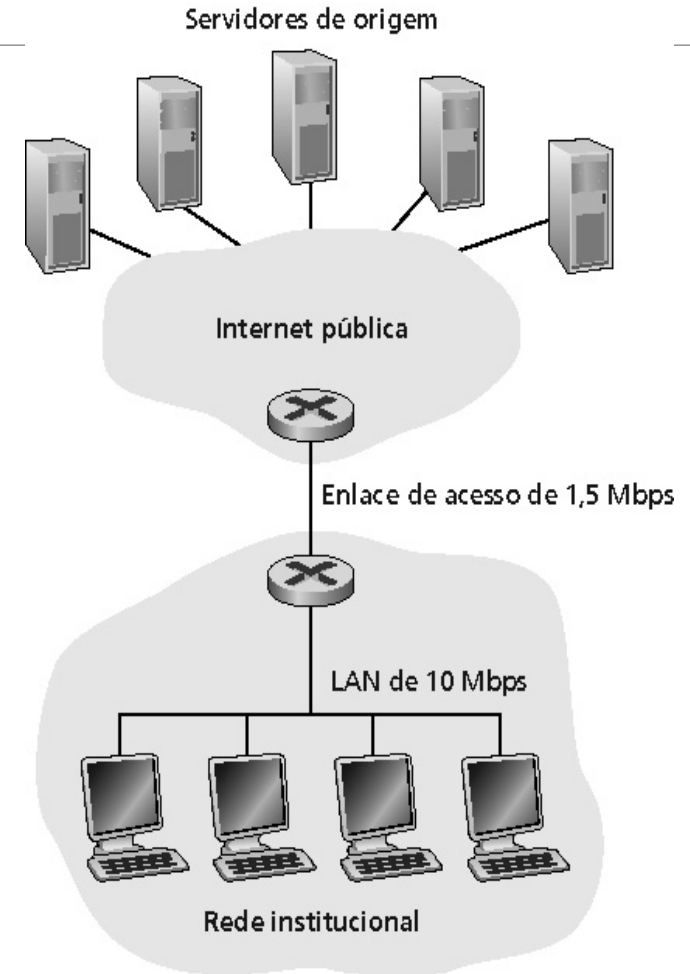
Exemplo de caching

Solução possível

- Aumentar a largura de banda do enlace de acesso, como, 10 Mbps

Consequências

- Utilização da LAN = 15%
- Utilização do enlace de acesso = 15%
- Atraso total = atraso da Internet + atraso de acesso + atraso da LAN
 - Tempo de acesso é reduzido pelo aumento da taxa da conexão
- Frequentemente é um upgrade caro



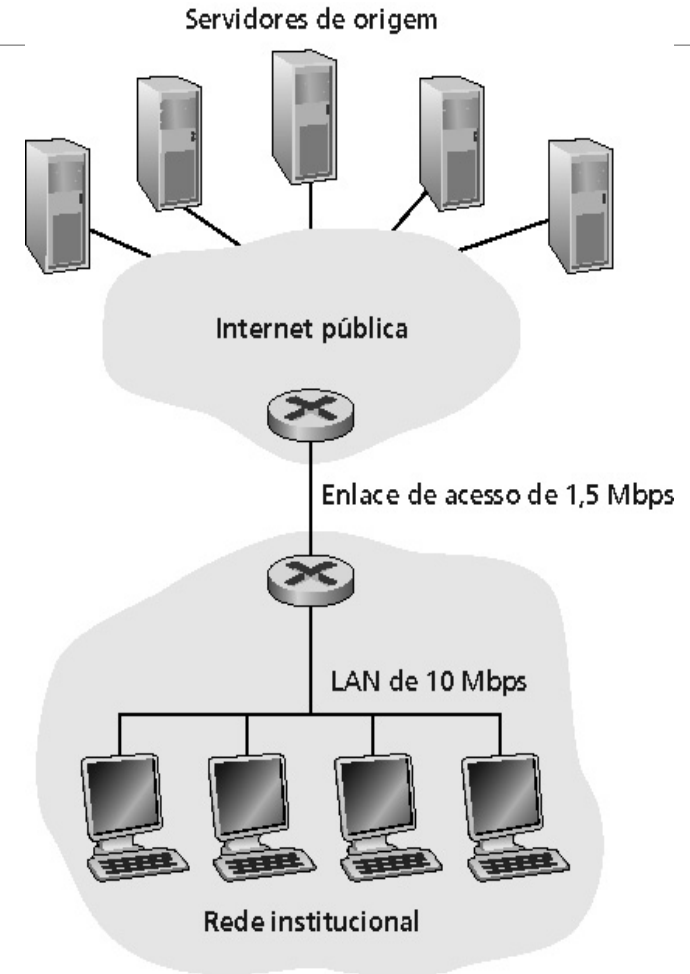
Exemplo de caching

Instalação do cache

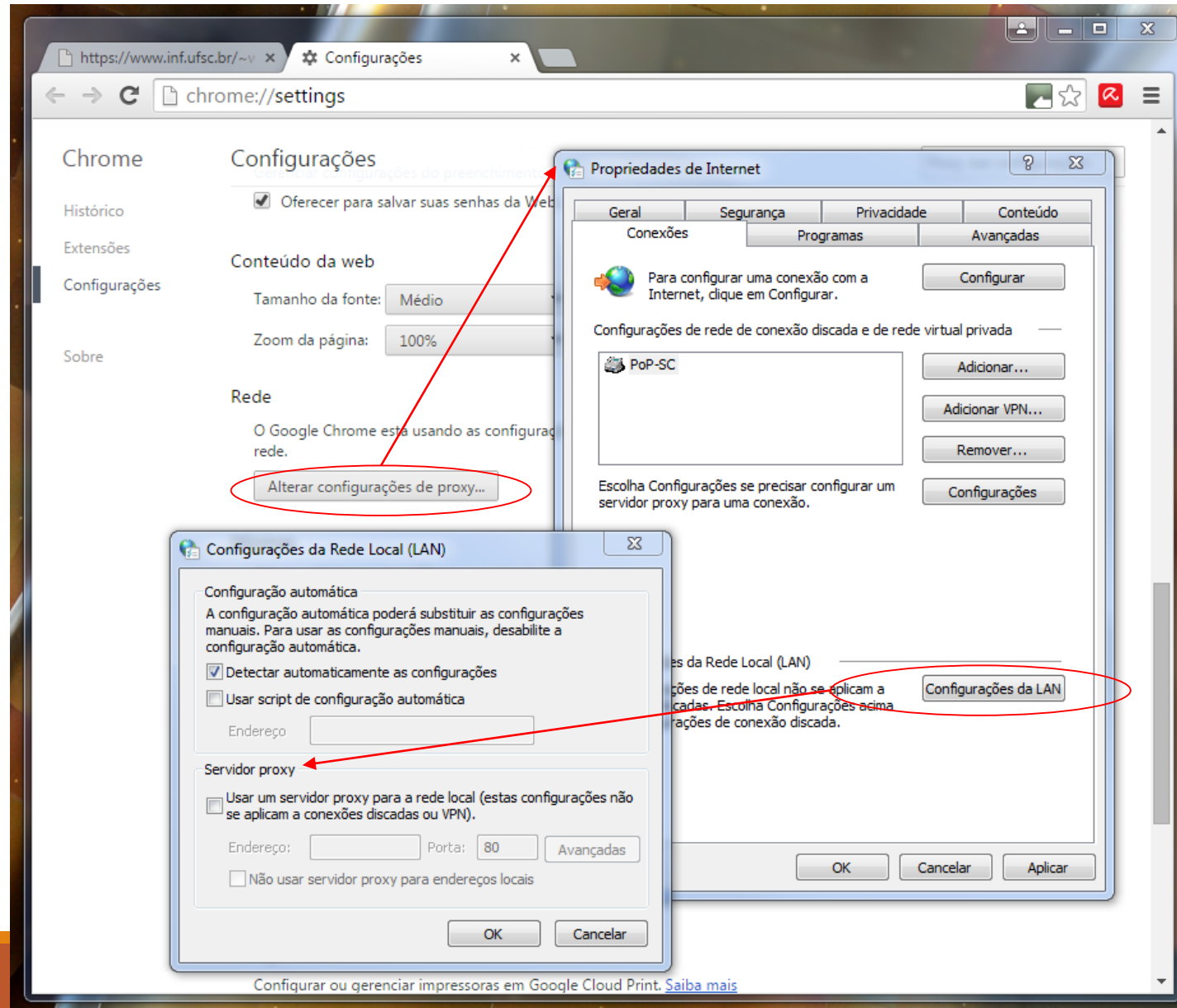
- Suponha que a taxa de acertos seja .4

Consequência

- 40% das requisições serão satisfeitas quase que imediatamente
- 60% das requisições serão satisfeitas pelo servidor de origem
- Utilização do enlace de acesso reduzida para 60%, resultando em atrasos insignificantes (como 10 ms)
- Média de atraso total é reduzir com a redução do tráfego de saída

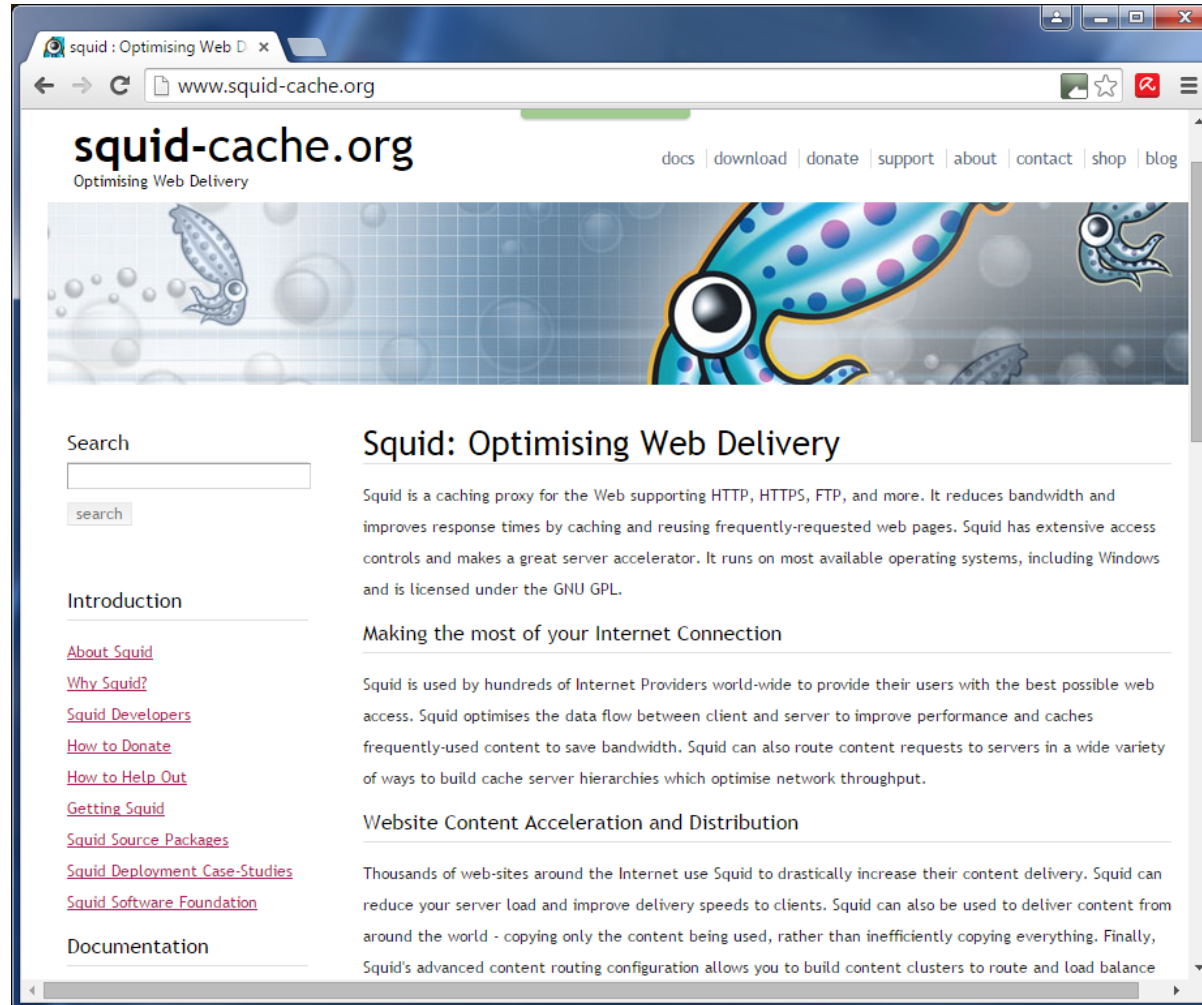


Configuração no Browser



Cache: Implementações

- Squid é um servidor proxy que suporta HTTP, HTTPS, FTP e outros



Pontos Importantes

Protocolo HTTP

- Entender o funcionamento das funcionalidades apresentadas

CAP 4. CAMADA DE APLICAÇÃO

AULA 9: HTTP – REVISÃO DE DNS E HTTP COM O CISCO PACKET TRACER

INE5422 REDES DE COMPUTADORES II
PROF. ROBERTO WILLRICH (INE/UFSC)
ROBERTO.WILLRICH@UFSC.BR
[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

CAP 4. CAMADA DE APLICAÇÃO

AULA 10: HTTPS E O HTTP/2

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Protocolo HTTPS

HTTPS (HyperText Transfer Protocol Secure)

- Implementação do protocolo HTTP sobre uma camada SSL ou do TLS.
 - permite que os dados sejam transmitidos através de uma conexão criptografada e que se verifique a autenticidade do servidor e do cliente através de certificados digitais.
- Porta TCP default do protocolo HTTPS é a 443.
 - Browser cria conexão com a porta 443 do servidor quando digitamos URL iniciando com 'https://'.

Usando quando...

- Se deseja evitar que a informação transmitida entre o cliente e o servidor seja visualizada por terceiros
 - Compras online, acesso a bancos
- A existência na barra de tarefas de um cadeado demonstra a certificação de página segura (SSL).

HTTP/2

Nova versão aprovada em maio de 2015 ([RFC 7540](#))

- Desenvolvido com base no protocolo experimental SPDY (Google)

Objetivos

- Cliente e servidor negociam versão do protocolo (1.1, 2 ou outro protocolo não HTTP)
- Mantém compatibilidade com o HTTP 1.1: métodos, códigos de status, URIs, e muitos campos de cabeçalho
- Reduzir latência da carga da página

HTTP/2

Codificação é binária (não ASCII) e compactada

- Reduz tamanho e aumenta eficiência na análise
- HPACK: Header Compression for HTTP/2 ([RFC 7541](#))

Elimina repetição de campos em pedidos na mesma conexão

- HTTP 1.1: repetição de todos os campos a cada requisição
- HTTP/2: requisições na mesma conexão não precisam repetir os campos, enviados apenas quando há alteração

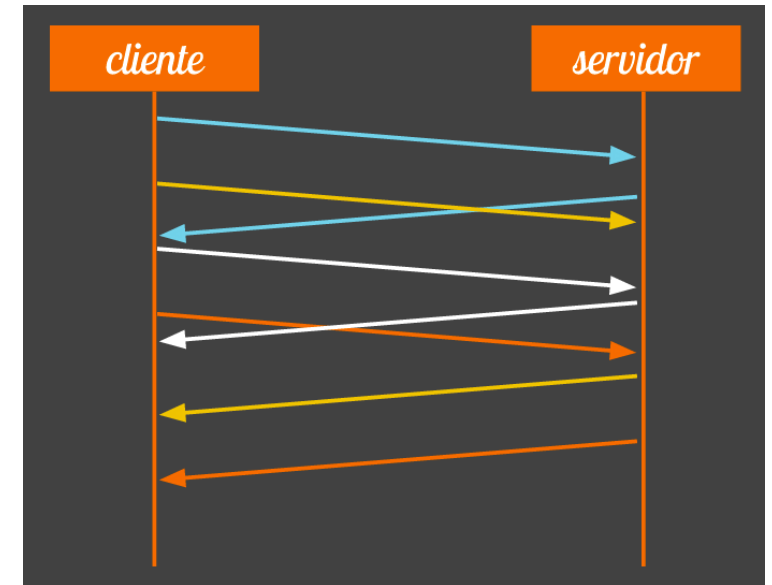
HTTP/2

Compressão automática

- HTTP 1.1: requer habilitação do gzip no servidor
- HTTP 2.0: gzip é padrão e obrigatório

head-of-line blocking do HTTP 1.x

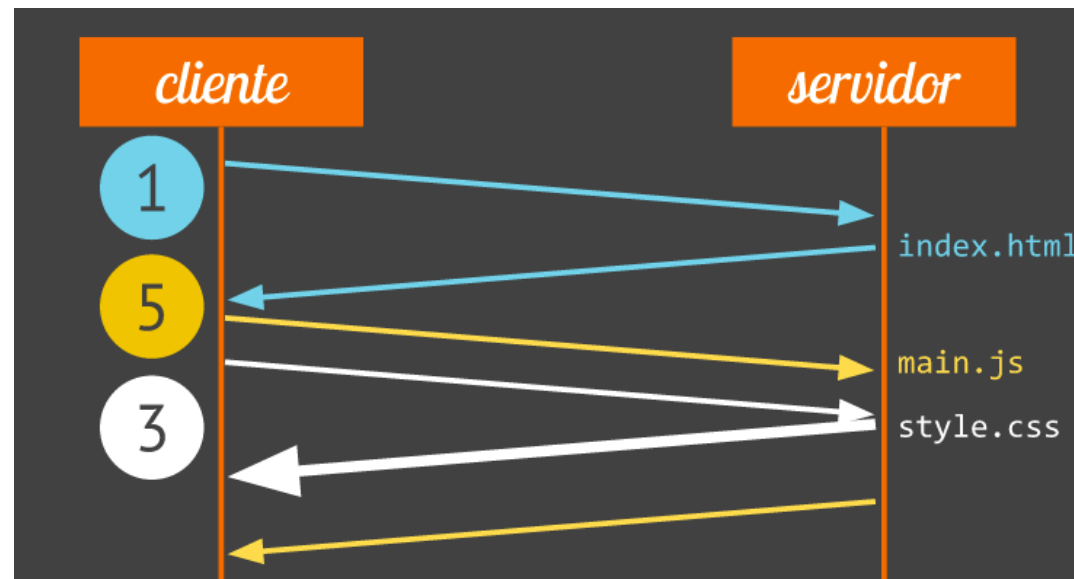
- HTTP 1.1: mesmo paralelizando pedido, o atendimento segue uma fila FIFO
- HTTP/2: respostas são enviadas quando pedidos estão prontos



HTTP/2

Priorização de pedidos

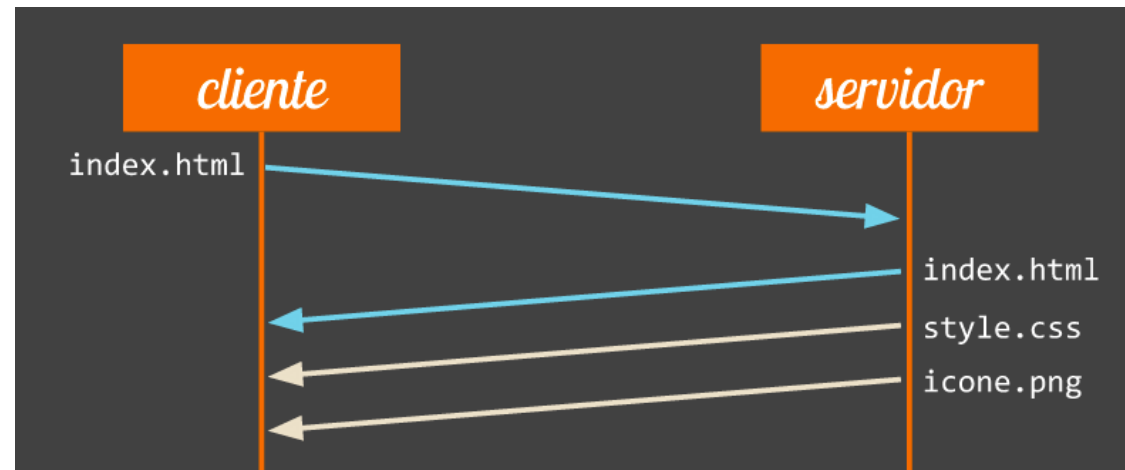
- Browser pode indicar prioridade do recurso, garantindo uma visualização inicial básica rapidamente



HTTP/2.0

Serviço Push

- HTTP 1.1: Cada pedido possui uma resposta
 - Programadores incluem recursos inline. Exemplo: embutir CSS no HTML com uma tag `<style>` ao invés de referenciar arquivo externo que exige um novo request (mas impede cacheamento no browser)
- HTTP 2.0: servidor pode mandar recursos sem eles serem solicitados



Pontos Importantes

Protocolo HTTPS

- Características principais

HTTP/2

- Saber características apresentadas e diferenças com HTTP1.1

CAP 4. CAMADA DE APLICAÇÃO

AULA 11: REDES DE DISTRIBUIÇÃO DE CONTEÚDOS (CDNS)

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

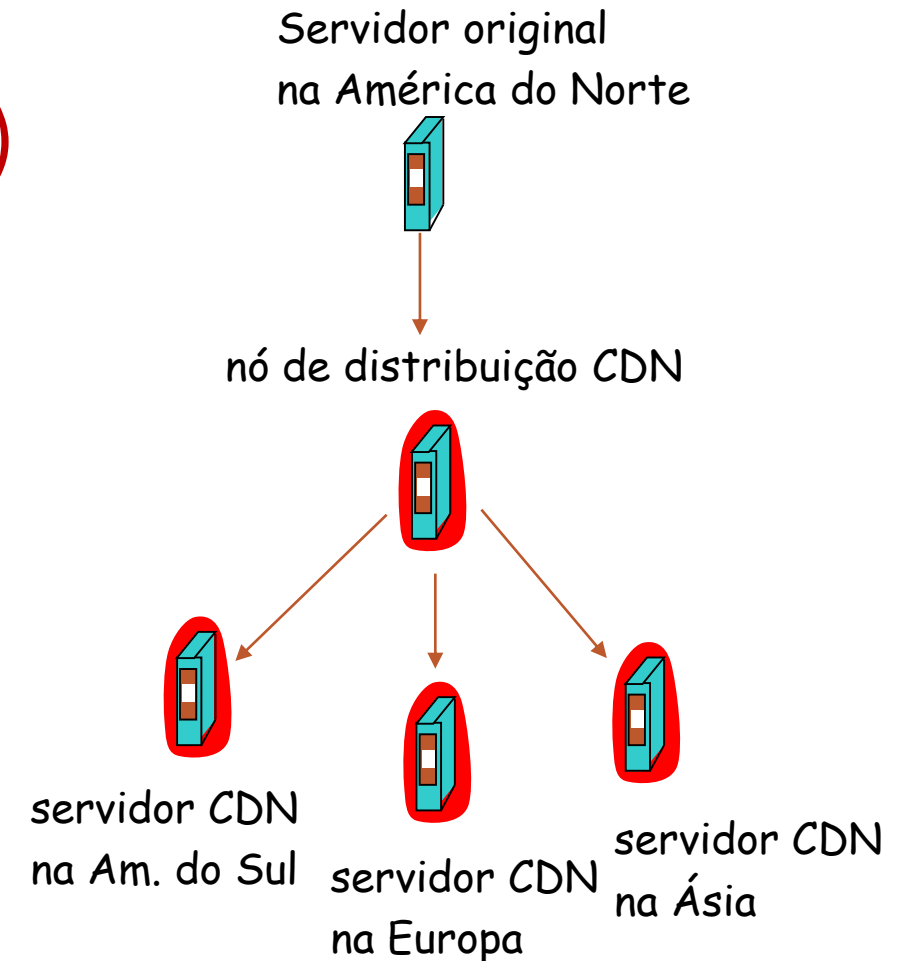
ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

CDNs

Redes de distribuição de conteúdos (CDNs - Content distribution networks)

- Provedores de conteúdo são clientes da CDN
- Empresa CDN:
 - Instala centenas de servidores através da Internet
 - Em ISPs próximos aos usuários
 - Replica conteúdo dos clientes nos servidores CDN
 - Atualização do conteúdo: quando o provedor atualiza o conteúdo, a CDN atualiza seus servidores



CDNs

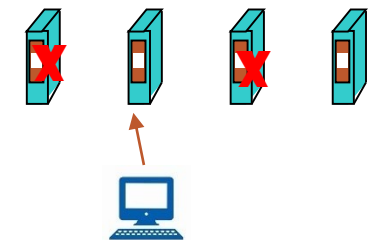
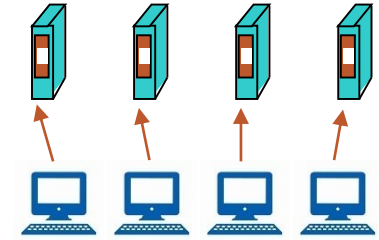
Empresas oferecendo o serviço

- <https://www.akamai.com/uk/en/solutions/intelligent-platform/visualizing-akamai/media-delivery-map.jsp>
- <https://www.cloudflare.com/pt-br/lp/cdn-a/>
- <https://www.stackpath.com/products/cdn/>
- <https://aws.amazon.com/pt/cloudfront/features/>
- E muitas outras

CDN: Seleção do Servidor

Que Servidor?

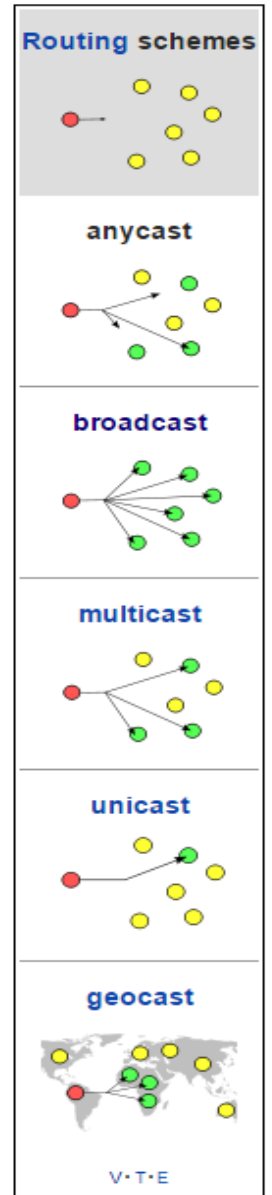
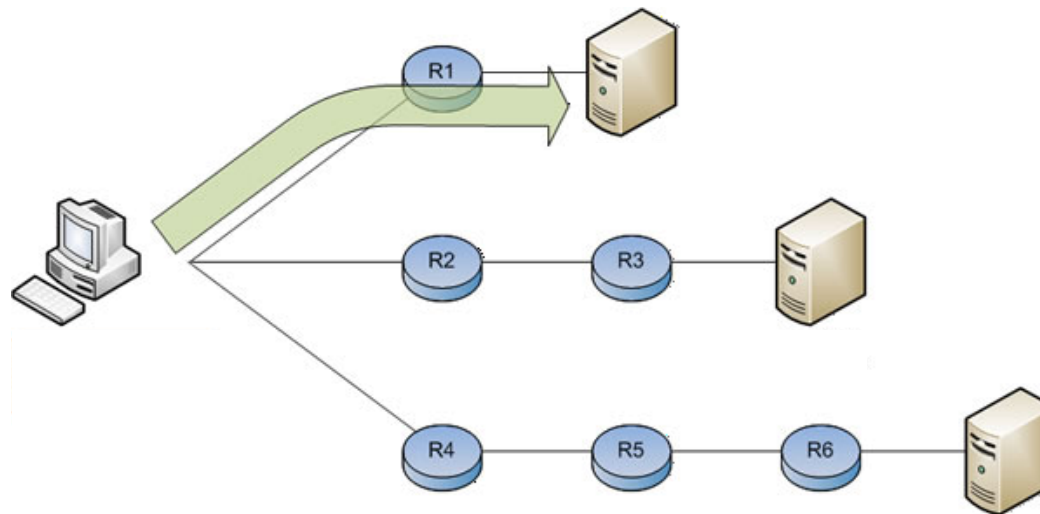
- Carga mais baixa → balancear a carga nos servidores
- Melhor desempenho → melhorar o desempenho do cliente
 - Baseado na geografia? RTT? Vazão? Carga?
- Qualquer nó vivo → oferece tolerância a faltas



CDN: Redirecionando pedido do cliente

Como redirecionar o cliente para um servidor particular?

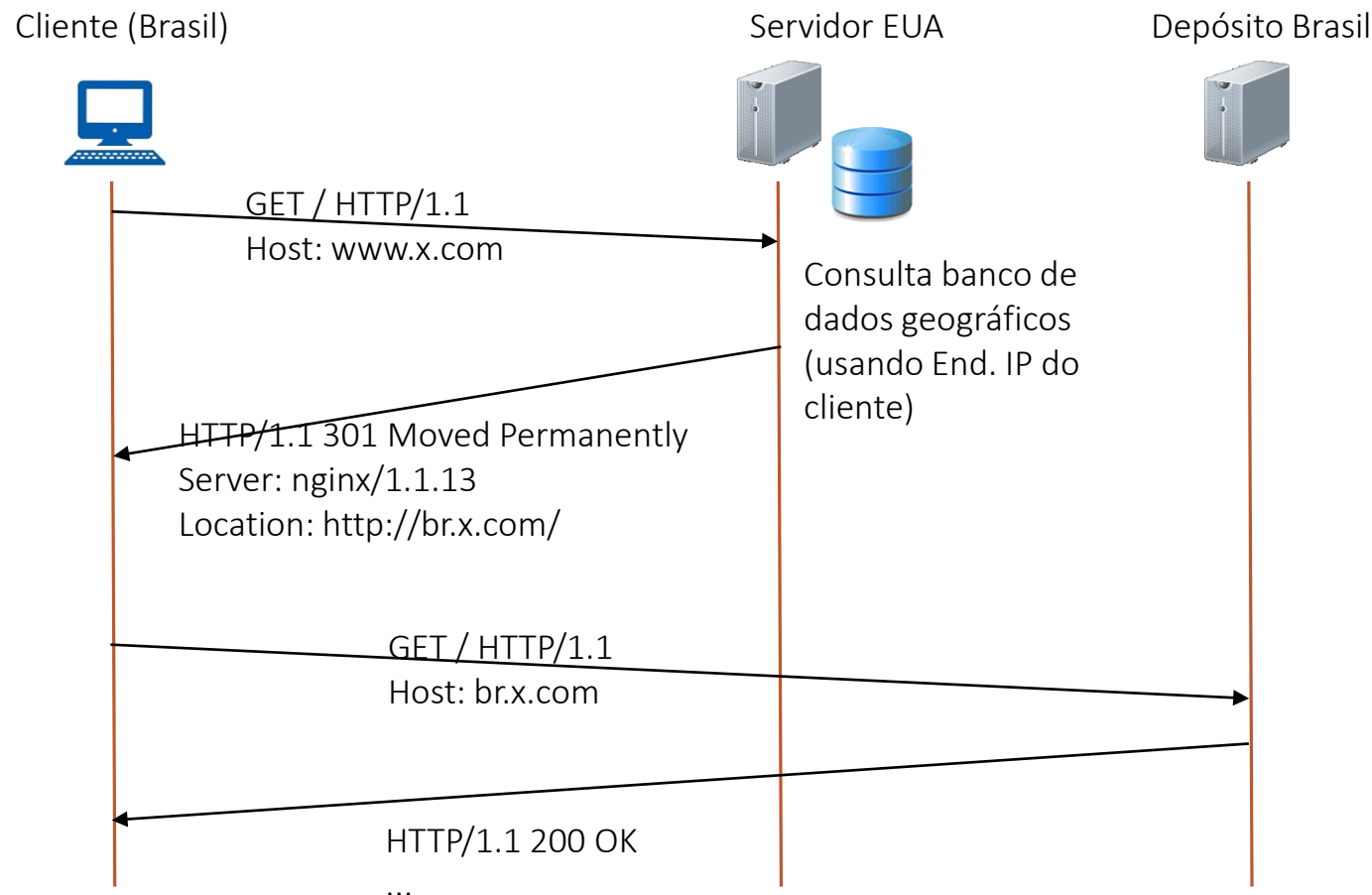
- Como parte do roteamento → anycast
- Anycast
 - Endereçamento de rede e metodologia de roteamento que os pacotes de um emissor são roteados para um servidor topologicamente mais próximo



CDN: Redirecionando pedido do cliente

Como redirecionar o cliente para um servidor particular?

- Como parte da aplicação → HTTP redireciona



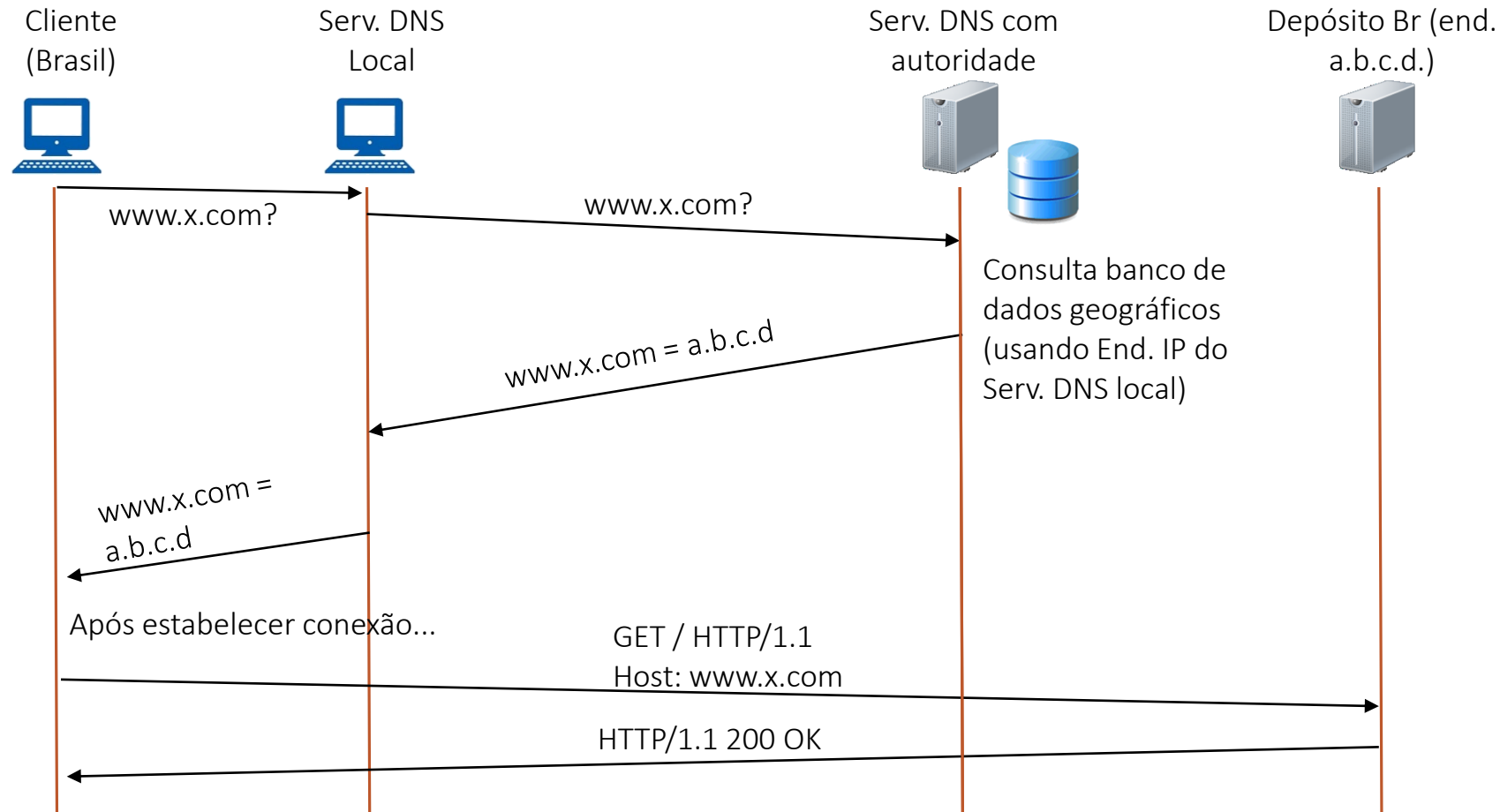
CDN: Redirecionando pedido do cliente

Utilizando DNS

- Cliente usa DNS para resolução de nomes
- Servidor de nomes escolhe o endereço de servidor Web selecionado
 - Dependendo do cliente
- Quais informações pode servir para tomar a decisão?
 - DNS round robin (RFC 1794)
 - Métricas [Semi-]estáticas
 - Geografia
 - Métricas de rota

CDN: Redirecionando pedido do cliente

Utilizando DNS



Como Akamai Trabalha

Clientes buscam o documento html do servidor principal

- Ex. busca index.html de cnn.com

URLs para conteúdos replicados são substituídos no html

- E.x.
 - ``
 - substituído por
 - ``

Cliente é forçado a resolver
aXYZ.g.akamaitech.net

Como Akamai Trabalha

Que conteúdo é replicado?

- Akamai apenas replica conteúdos estáticos

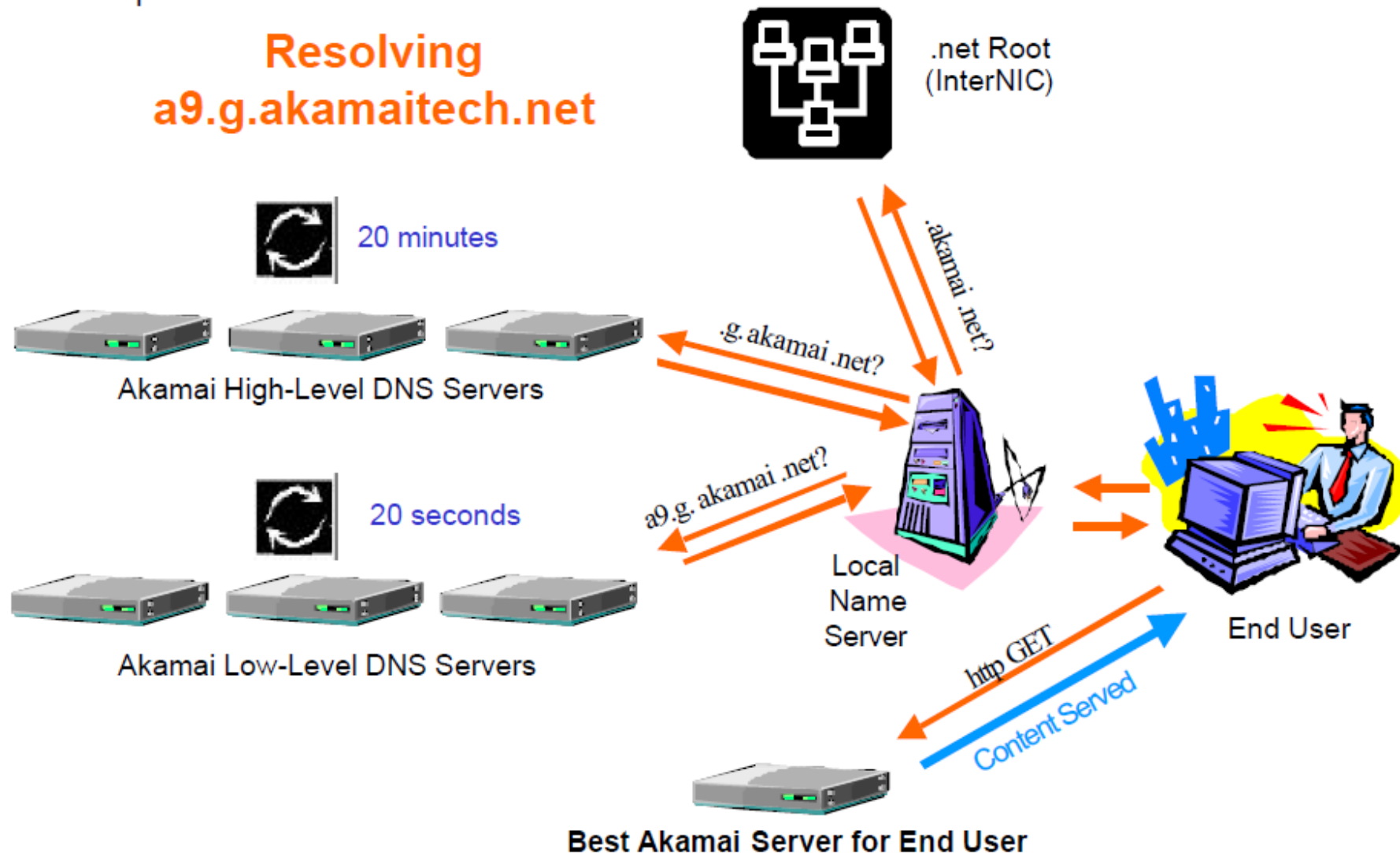
Nome modificado contém o nome do arquivo original

- Ex.: ``

Resolução de nome:

- Servidor Raíz resolve o TLD .net
- Servidor .net resolve o nome de akamaitech.net
- Servidor de nomes akamaitech.net resolve o nome de g.akamaitech.net
- Servidor de nome g.akamaitech.net escolhe o servidor da região

Como Akamai Trabalha



Mais sobre CDNs

Roteamento de pedidos

- A CDN cria um “mapa”, indicando as distâncias entre os ISPs folhas e os nós CDN
- quando a solicitação chega num servidor DNS oficial :
 - o servidor determina qual é o ISP de onde provém o pedido
 - usa o “mapa” para determinar qual o melhor servidor CDN

Não são apenas páginas Web

- fluxos de áudio/vídeo armazenados
- fluxos de áudio/vídeo de tempo real

Nós CDN criam uma rede sobreposta na camada de aplicação

Pontos Importantes

CDNs

- Entender finalidade
- Como as réplicas são selecionadas
- Como os clientes são redirecionados

CAP 4. CAMADA DE APLICAÇÃO

AULA 12: TELNET

INE5422 REDES DE COMPUTADORES II
PROF. ROBERTO WILLRICH (INE/UFSC)
ROBERTO.WILLRICH@UFSC.BR
[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Telnet

Protocolo cliente-servidor da camada de aplicação

- Permite acesso remoto via um terminal remoto
- Telnet [RFC 854] não possui criptografia
- Via conexão TCP entre cliente e servidor (porta 23)
 - Dados e controle na mesma conexão

Permite a um usuário em um computador logar-se em outro computador na Internet

- Servidor e cliente em qualquer parte da Internet.
- Conectado, máquina do cliente emula um terminal da máquina remota

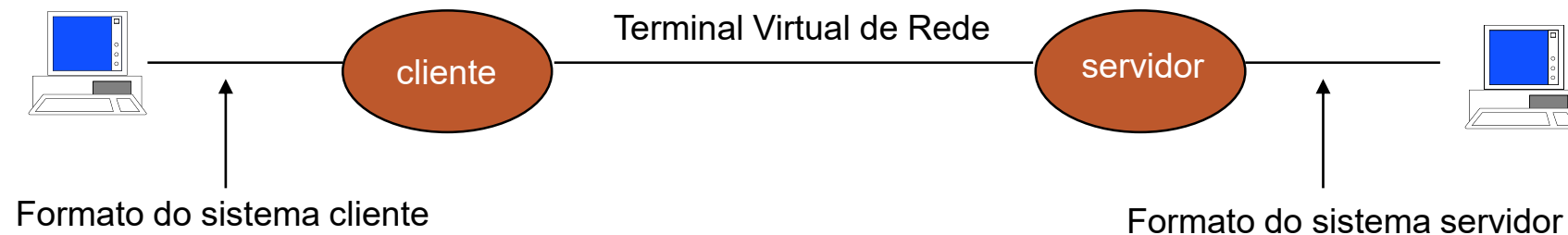
Comando: telnet nome-da-máquina-remota

- Exemplo:
 - telnet venus.inf.ufsc.br

Telnet

Funcionamento do Telnet

- Papel do cliente:
 - Cria conexão TCP com o servidor
 - Recebe dados de entrada do usuário
 - Adapta os dados de entrada num formato padrão para transmissão
 - Recebe dados de saída do servidor num formato padrão
 - formata dados de saída para exibição no terminal
- Papel do servidor:
 - Aceita conexões de clientes
 - Aguarda ocorrência de solicitações de serviço
 - Envia eco/resultado para o cliente num formato padrão
 - Entra em processo de espera



Telnet

Baseado em três ideias principais

- Conceito de Terminal Virtual de Rede (NVT)
- Princípio de Negociação de Opções
- Tratamento Equivalente de Terminais e Processos

Conceito de Terminal Virtual de Rede (NVT)

- NVT é um dispositivo lógico que fornece uma representação padronizada de um terminal
 - Elimina a necessidade das máquinas conhecerem as características do terminal usado por seus parceiros
- Servidor e clientes mapeiam as características e convenções de seus terminais locais nas características do NVT

Telnet

Princípio de Negociação de Opções

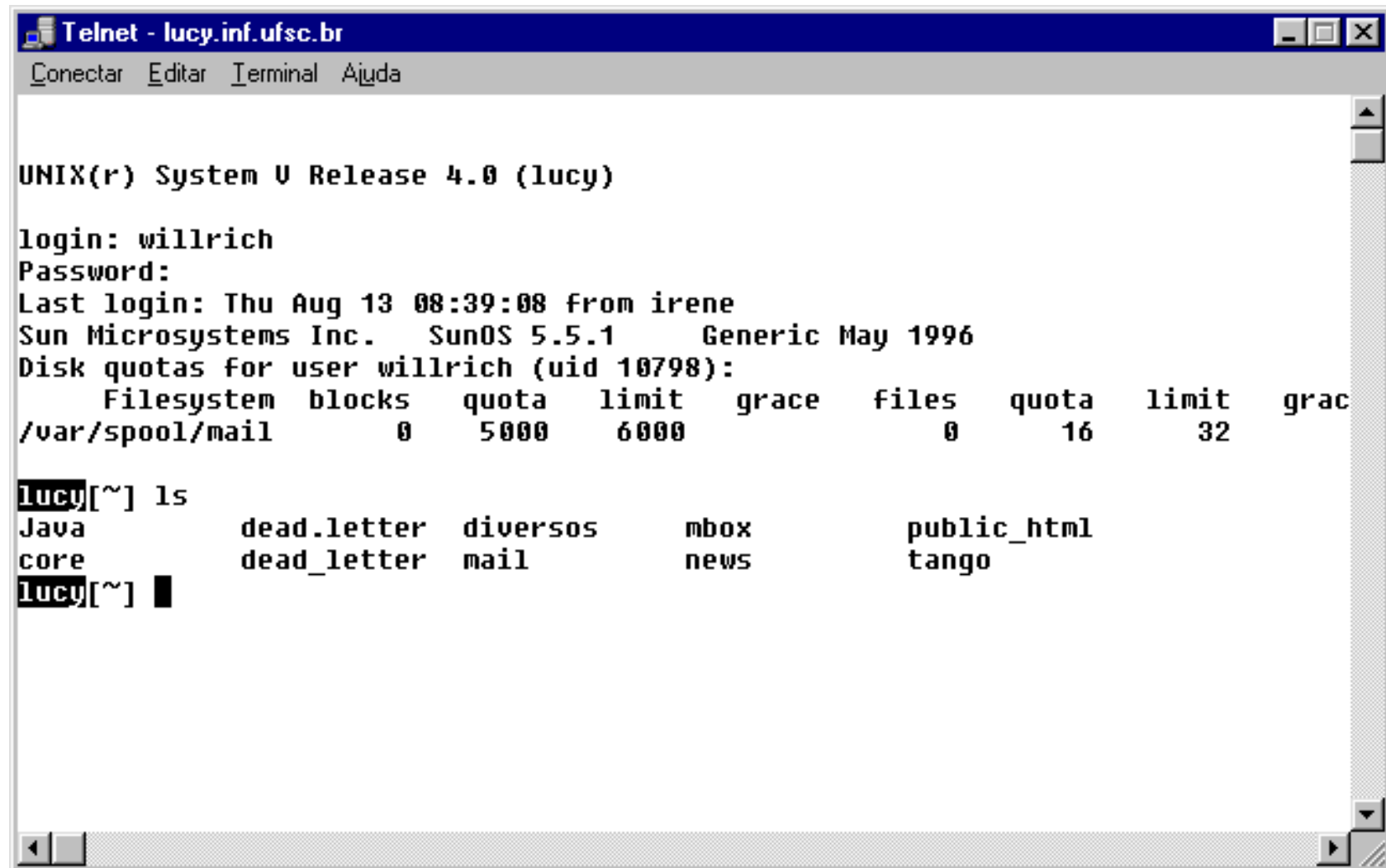
- Permite que cliente e servidor negociem as opções que definem o comportamento do NVT
 - P.e. formato de representação de caractere, modo de operação (half ou full-duplex)
- Ambas as extremidades podem negociar as opções

Tratamento Equivalente de Terminais e Processos

- Protocolo TELNET trata os terminais e o processo remoto de forma equivalente e simétrica
 - O cliente não precisa ser necessariamente um terminal
 - Pode ser um processo de aplicação qualquer

Telnet

Exemplo:



```
Telnet - lucy.inf.ufsc.br
Conectar  Editar  Terminal  Ajuda

UNIX(r) System V Release 4.0 (lucy)

login: willrich
Password:
Last login: Thu Aug 13 08:39:08 from irene
Sun Microsystems Inc.   SunOS 5.5.1   Generic May 1996
Disk quotas for user willrich (uid 10798):
      Filesystem  blocks    quota   limit   grace   files   quota   limit   grac
/var/spool/mail      0     5000    6000         0        16     32

lucy[~] ls
Java          dead.letter  diversos    mbox         public_html
core          dead_letter  mail        news         tango
lucy[~]
```

Exemplo de Telnet

telnet.pcap - Wireshark

File Edit View Go Capture Analyze Statistics Telephony Tools Help

Filter: Expression... Clear Apply

No.	Time	Source	Destination	Protocol	Info
83	5.479	150.162.59.226	150.162.59.211	TCP	50452 > telnet [SYN] Seq=0 win=8192 Len=0 MSS=1460 ws
84	5.479	150.162.59.211	150.162.59.226	TCP	telnet > 50452 [SYN, ACK] Seq=0 Ack=1 win=8192 Len=0
85	5.479	150.162.59.226	150.162.59.211	TCP	50452 > telnet [ACK] Seq=1 Ack=1 win=65700 Len=0
86	5.480	150.162.59.211	150.162.59.226	TELNET	Telnet Data ...
87	5.480	150.162.59.211	150.162.59.226	TELNET	Telnet Data ...
88	5.480	150.162.59.211	150.162.59.226	TELNET	Telnet Data ...
89	5.481	150.162.59.226	150.162.59.211	TELNET	Telnet Data ...
90	5.481	150.162.59.211	150.162.59.226	TELNET	Telnet Data ...
91	5.481	150.162.59.226	150.162.59.211	TELNET	Telnet Data ...

Pedido de conexão TCP com a porta 23

Frame 83: 66 bytes on wire (528 bits), 66 bytes captured (528 bits)

Ethernet II, Src: QuantaCo_79:99:7d (00:1e:68:79:99:7d), Dst: QuantaCo_95:70:bf (00:1b:24:95:70:bf)

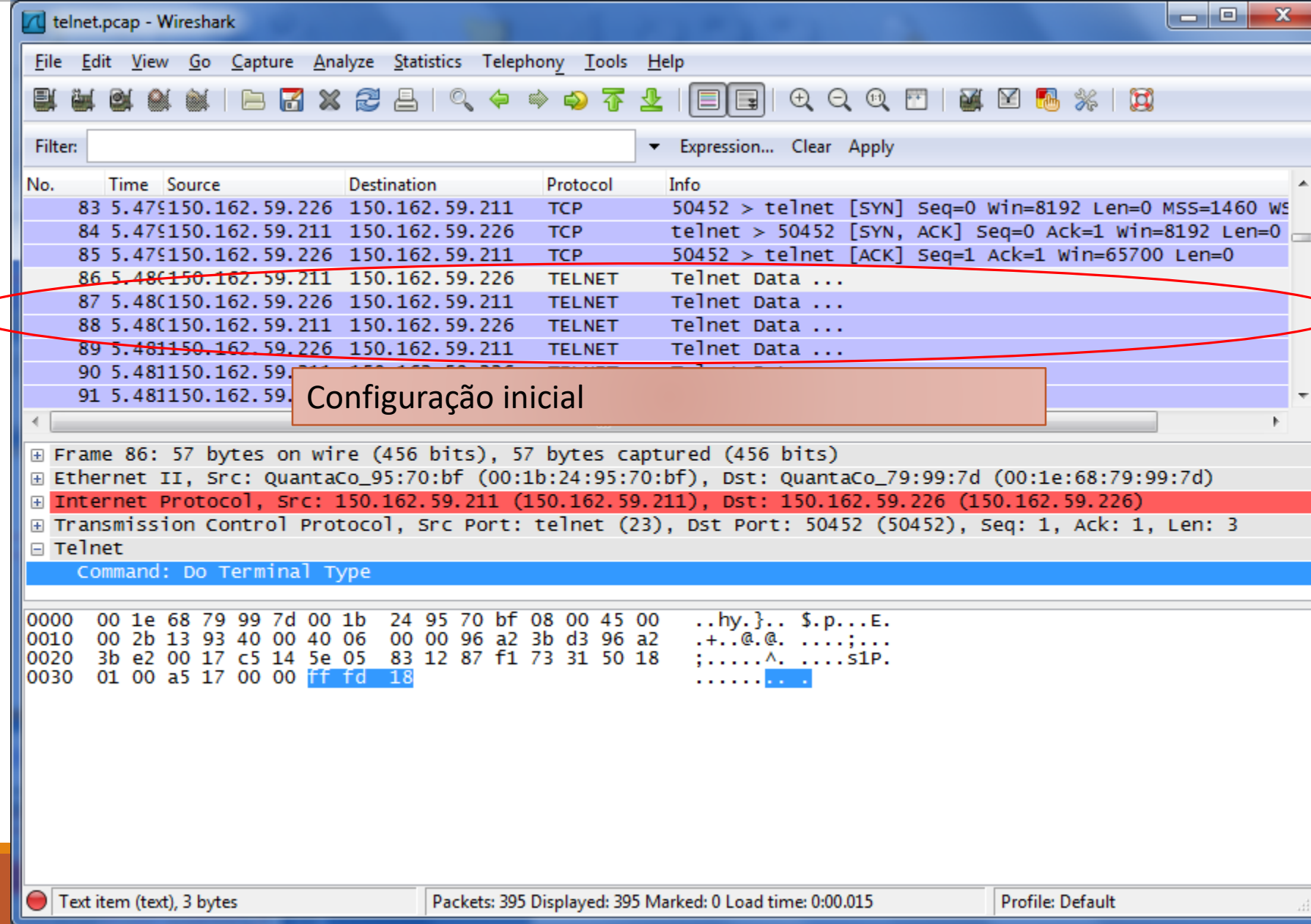
Internet Protocol, Src: 150.162.59.226 (150.162.59.226), Dst: 150.162.59.211 (150.162.59.211)

Transmission Control Protocol, Src Port: 50452 (50452), Dst Port: telnet (23), Seq: 0, Len: 0

```
0000 00 1b 24 95 70 bf 00 1e 68 79 99 7d 08 00 45 00  ..$.p... hy.}...E.
0010 00 34 54 0b 40 00 80 06 01 bf 96 a2 3b e2 96 a2  .4T.@... ..;...
0020 3b d3 c5 14 00 17 87 f1 73 30 00 00 00 00 80 02  ;..... s0.....
0030 20 00 e9 ce 00 00 02 04 05 b4 01 03 03 02 01 01  .....
0040 04 02  ..
```

File: "C:\Users\willrich\Documents\My Drop... Packets: 395 Displayed: 395 Marked: 0 Load time: 0:00.015 Profile: Default

Exemplo de Telnet



telnet.pcap - Wireshark

File Edit View Go Capture Analyze Statistics Telephony Tools Help

Filter: Expression... Clear Apply

No.	Time	Source	Destination	Protocol	Info
83	5.479	150.162.59.226	150.162.59.211	TCP	50452 > telnet [SYN] Seq=0 win=8192 Len=0 MSS=1460 ws
84	5.479	150.162.59.211	150.162.59.226	TCP	telnet > 50452 [SYN, ACK] Seq=0 Ack=1 win=8192 Len=0
85	5.479	150.162.59.226	150.162.59.211	TCP	50452 > telnet [ACK] Seq=1 Ack=1 win=65700 Len=0
86	5.480	150.162.59.211	150.162.59.226	TELNET	Telnet Data ...
87	5.480	150.162.59.226	150.162.59.211	TELNET	Telnet Data ...
88	5.480	150.162.59.211	150.162.59.226	TELNET	Telnet Data ...
89	5.481	150.162.59.226	150.162.59.211	TELNET	Telnet Data ...
90	5.481	150.162.59.211	150.162.59.226	TELNET	Telnet Data ...
91	5.481	150.162.59.226	150.162.59.211	TELNET	Telnet Data ...

Configuração inicial

Frame 86: 57 bytes on wire (456 bits), 57 bytes captured (456 bits)

Ethernet II, Src: QuantaCo_95:70:bf (00:1b:24:95:70:bf), Dst: QuantaCo_79:99:7d (00:1e:68:79:99:7d)

Internet Protocol, Src: 150.162.59.211 (150.162.59.211), Dst: 150.162.59.226 (150.162.59.226)

Transmission Control Protocol, Src Port: telnet (23), Dst Port: 50452 (50452), Seq: 1, Ack: 1, Len: 3

Telnet

Command: Do Terminal Type

```
0000  00 1e 68 79 99 7d 00 1b 24 95 70 bf 08 00 45 00  ..hy.}.. $.p...E.
0010  00 2b 13 93 40 00 40 06 00 00 96 a2 3b d3 96 a2  .+..@.@. ....;...
0020  3b e2 00 17 c5 14 5e 05 83 12 87 f1 73 31 50 18  ;.....^.....s1P.
0030  01 00 a5 17 00 00 ff fd 18                      ..... ..
```

Text item (text), 3 bytes

Packets: 395 Displayed: 395 Marked: 0 Load time: 0:00.015

Profile: Default

Telnet: Estrutura do Comando

Comandos iniciam com um caractere especial chamado Interpret as Command escape character (IAC).

- Código IAC é 255 (FFh)
- Se um 255 é enviado como dado, ele deve ser seguido por um outro 255.

Cada receptor deve olhar em cada byte na chegada para ver o IAC

- Se um IAC é encontrado e o próximo byte é também IAC, ele é enviado para a aplicação/terminal
- Se o IAC é seguido por outro código
 - A camada TELNET interpreta como um comando

Telnet: Funções de controle

TELNET inclui uma série de funções

- Interrupt Process (IP)
 - suspende/aborta processo.
- Abort Output (AO)
 - Processo pode completar, mas não envia mais saídas ao terminal do usuário
- Are You There (AYT)
 - Checa para ver se o sistema está executando
- Erase Character (EC)
 - Apaga último caractere enviado
- Erase Line (EL)
 - Apaga todas as entradas na linha atual

Telnet: Códigos dos comandos

❑ IP 243

❑ AO 244

❑ AYT 245

❑ EC 246

❑ EL 247

❑ WILL 251

❑ WON'T 252

❑ DO 253

❑ DON'T 254

❑ IAC 255

Emulador de terminal – TELNET

Transmissão de caracteres digitados

[10.0.0.138]	[10.0.0.1]	TCP: D=1135 S=23 ACK=88950803 WIN=4095
[10.0.0.138]	[10.0.0.1]	Telnet: R PORT=1135 User :
[10.0.0.1]	[10.0.0.138]	TCP: D=23 S=1135 ACK=1743040018 WIN=1636
[10.0.0.1]	[10.0.0.138]	Telnet: C PORT=1135 u
[10.0.0.138]	[10.0.0.1]	TCP: D=1135 S=23 ACK=88950804 WIN=4095
[10.0.0.138]	[10.0.0.1]	Telnet: R PORT=1135 u
[10.0.0.1]	[10.0.0.138]	TCP: D=23 S=1135 ACK=1743040019 WIN=1636
[10.0.0.1]	[10.0.0.138]	Telnet: C PORT=1135 s
[10.0.0.138]	[10.0.0.1]	TCP: D=1135 S=23 ACK=88950805 WIN=4095
[10.0.0.138]	[10.0.0.1]	Telnet: R PORT=1135 s
[10.0.0.1]	[10.0.0.138]	TCP: D=23 S=1135 ACK=1743040020 WIN=1636
[10.0.0.1]	[10.0.0.138]	Telnet: C PORT=1135 e
[10.0.0.138]	[10.0.0.1]	TCP: D=1135 S=23 ACK=88950806 WIN=4095
[10.0.0.138]	[10.0.0.1]	Telnet: R PORT=1135 e
[10.0.0.1]	[10.0.0.138]	TCP: D=23 S=1135 ACK=1743040021 WIN=1636
[10.0.0.1]	[10.0.0.138]	Telnet: C PORT=1135 r
[10.0.0.138]	[10.0.0.1]	TCP: D=1135 S=23 ACK=88950807 WIN=4095
[10.0.0.138]	[10.0.0.1]	Telnet: R PORT=1135 r
[10.0.0.1]	[10.0.0.138]	TCP: D=23 S=1135 ACK=1743040022 WIN=1636

Testes com Telnet

Telnet pode ser usado para testes simples

telnet é um cliente TCP genérico

- Envia o que voce teclar para o socket TCP
- Imprime o que chega do socket TCP
- Util para testar servidores TCP e usam protocolos baseados em ASCII

Alguns servidores TCP

Muitos sistemas Unix tem estes servidores rodando (por default):

- echo porta 7
 - Usado para teste e medidas de atraso
- discard porta 9
 - Usado para testes e medidas
 - Não há resposta
- daytime porta 13
 - Usado para testes e medidas
 - Servidor retorna a data e hora
- chargen porta 19
 - Character generator
 - Para teste e medidas

Definindo a porta

> telnet rcs.rpi.edu 7

Trying 128.113.113.33...

Connected to cortez.sss.rpi.edu (128.113.113.33).

Escape character is '^['.

Hi dave

Hi dave

stop it

stop it

^]

telnet> quit

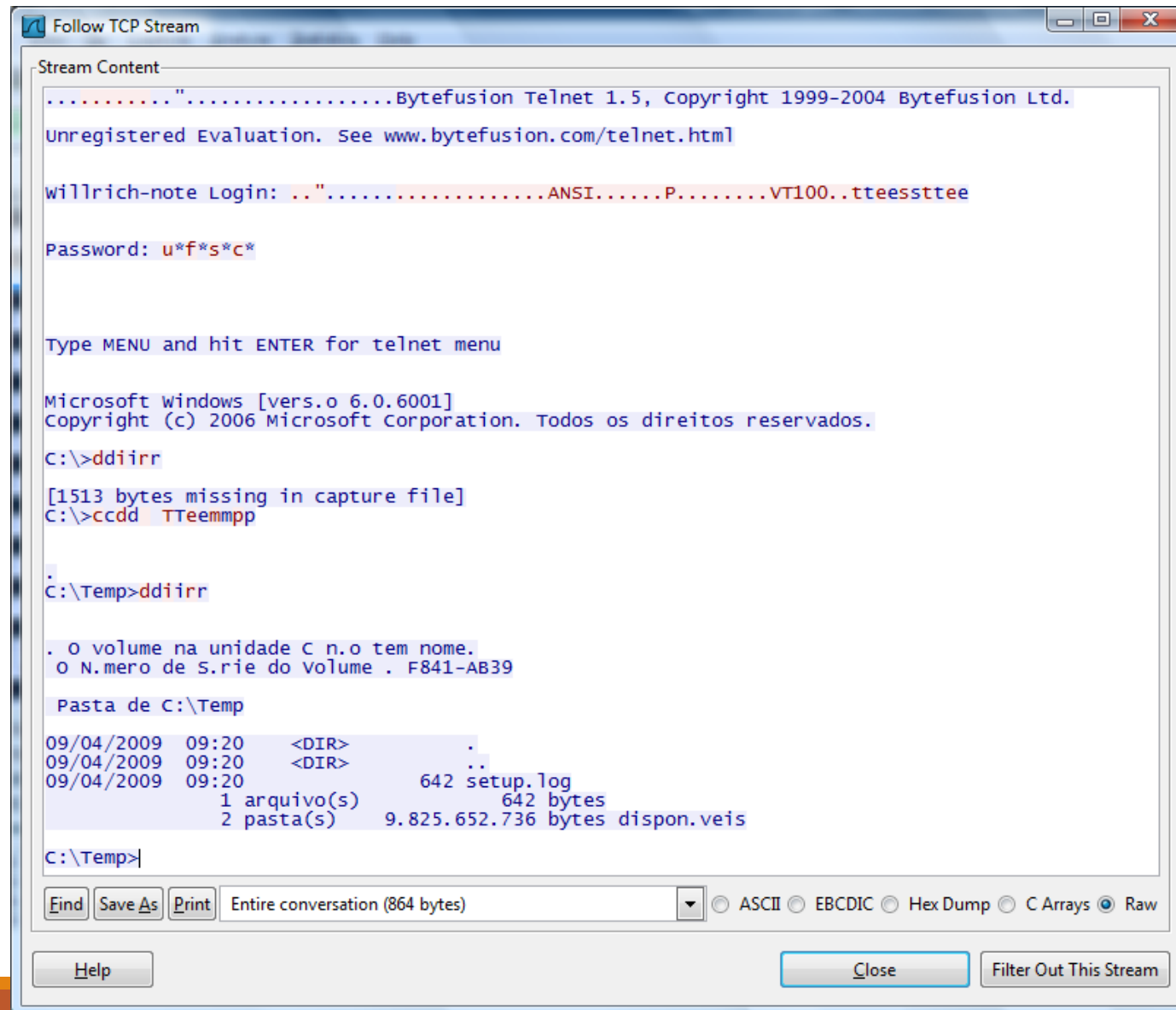
Connection closed.

Telnet

TELNET é um serviço inseguro

- sua senha estará circulando sem criptografia na rede!!!

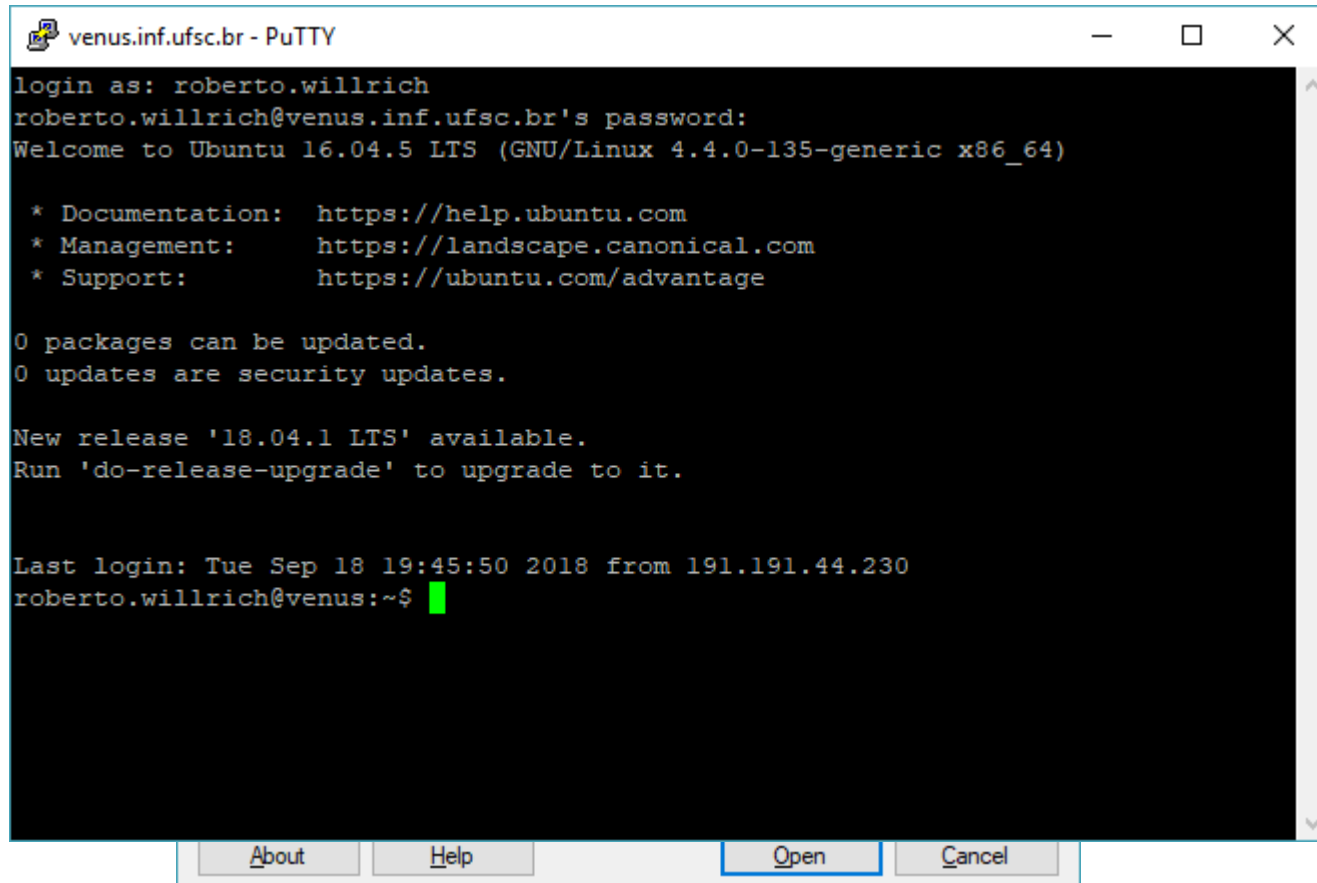
Telnet



SSH

secure shell client (remote login program)

- torna a comunicação segura



The image shows a PuTTY terminal window titled "venus.inf.ufsc.br - PuTTY". The terminal displays the following text:

```
login as: roberto.willrich
roberto.willrich@venus.inf.ufsc.br's password:
Welcome to Ubuntu 16.04.5 LTS (GNU/Linux 4.4.0-135-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

0 packages can be updated.
0 updates are security updates.

New release '18.04.1 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue Sep 18 19:45:50 2018 from 191.191.44.230
roberto.willrich@venus:~$
```

At the bottom of the window, there are four buttons: "About", "Help", "Open", and "Cancel". The "Open" button is highlighted with a blue border.

Pontos Importantes

Telnet

- Finalidades
- Características básicas
- Problemas de segurança

CAP 4. CAMADA DE APLICAÇÃO

AULA 13: FTP

INE5422 REDES DE COMPUTADORES II
PROF. ROBERTO WILLRICH (INE/UFSC)
ROBERTO.WILLRICH@UFSC.BR
[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

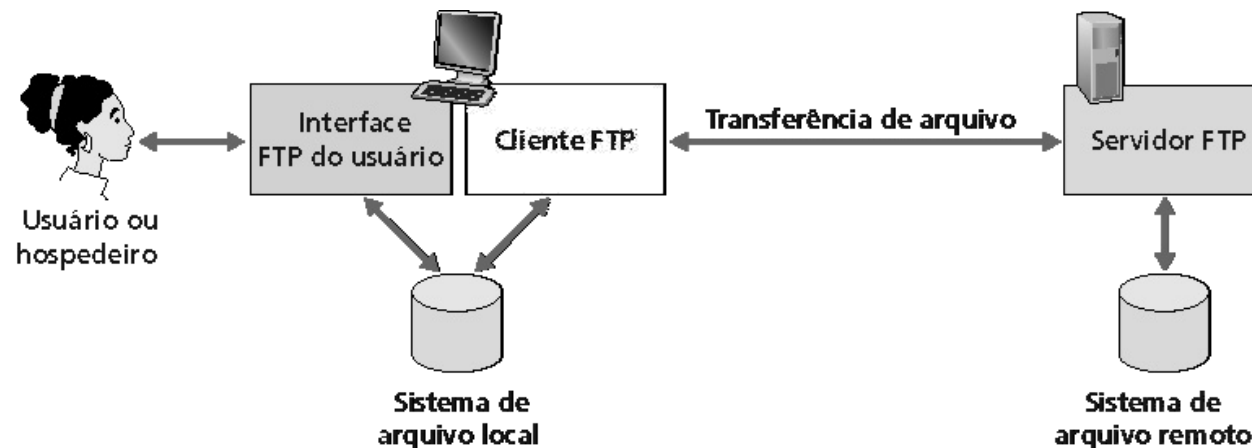
FTP: File Transfer Protocol

FTP (RFC 959) permite

- Transferir, renomear ou remover arquivos remotos
- Criar, remover e modificar diretórios remotos

Modelo cliente/servidor

- Cliente inicia transferência passando o nome (login) e sua senha
- Servidor: host remoto
 - Acessível via a porta 21
 - Servidor usa porta 20 para conexão de dados (modo ativo)



FTP: conexões separadas p/ controle, dados

Conexão de Controle (Porta 21)

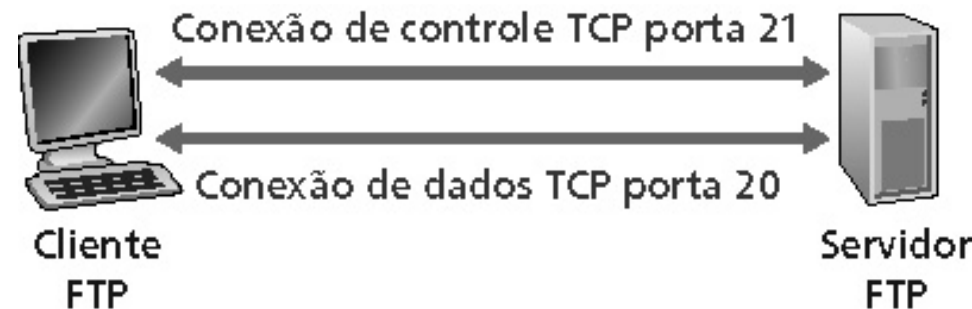
- Aberta pelo cliente FTP
- Cliente obtém autorização através da conexão de controle
- Cliente consulta o diretório remoto enviando comandos através da conexão de controle

Conexão de dados (porta 20)

- Servidor (modo ativo) ou Cliente (modo passivo) abre uma segunda conexão TCP para transferir dados
- Após a transmissão de um arquivo o servidor fecha a conexão de dados

Servidor FTP mantém o “estado”:

- diretório atual, autenticação anterior



FTP: File Transfer Protocol

Duas formas de se conectar a um servidor FTP

- Forma autenticada (nome do usuário e password)
 - direitos de acesso do usuário
- Forma anônima
 - Nome de login: anonymous
 - Password: E-mail
 - Têm-se acesso a repositórios públicos de arquivos
 - qualquer pessoa pode acessar
 - não é preciso cadastrar-se
 - Direitos autorais:
 - repositórios públicos contém apenas arquivos em domínio público

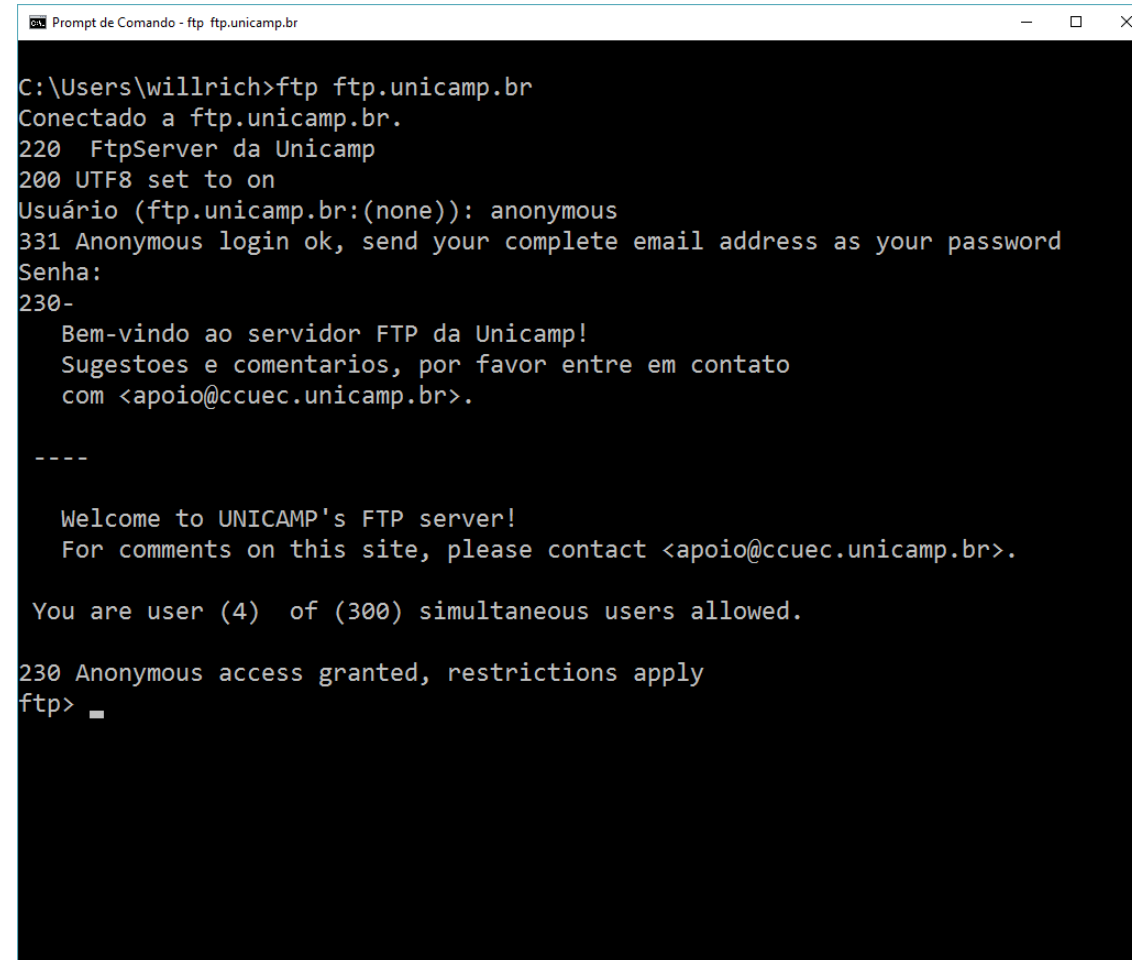
FTP: File Transfer Protocol

Programa FTP

- Implementa o protocolo FTP
- Existem várias implementações fornecendo interfaces gráficas ou não

Forma não gráfica (console)

- Conectando a uma máquina: ftp nome-da-máquina-remota



```
Prompt de Comando - ftp ftp.unicamp.br

C:\Users\willrich>ftp ftp.unicamp.br
Conectado a ftp.unicamp.br.
220 FtpServer da Unicamp
200 UTF8 set to on
Usuário (ftp.unicamp.br:(none)): anonymous
331 Anonymous login ok, send your complete email address as your password
Senha:
230-
    Bem-vindo ao servidor FTP da Unicamp!
    Sugestoes e comentarios, por favor entre em contato
    com <apoio@ccuec.unicamp.br>.

----

Welcome to UNICAMP's FTP server!
For comments on this site, please contact <apoio@ccuec.unicamp.br>.

You are user (4) of (300) simultaneous users allowed.

230 Anonymous access granted, restrictions apply
ftp> _
```

FTP: File Transfer Protocol

Comandos FTP

- Conhecendo o conteúdo das máquinas via ftp
 - ftp> dir [nome-do-diretório] [nome-do-arquivo]
 - ftp> ls [nome-do-diretório] [nome-do-arquivo]
 - ftp> !dir (LOCAL)
- Trabalhando com diretórios
 - ftp> lcd [nome-do-diretório] (Local)
 - ftp> cd [nome-do diretório] (Remoto)
 - ftp> pwd (Diretório remoto atual)
/home/venus/willrich

FTP: File Transfer Protocol

Comandos FTP

- Transferência de arquivos de texto (ASCII) e binários

```
ftp> binary
```

```
200 Type set to I.
```

```
ftp> ascii
```

```
200 Type set to A.
```

- Exemplos de arquivos e modos de transferência
 - Binários
 - Programas, etc...
 - ASCII
 - texto, mensagens de correio eletrônico, PostScript, etc...

FTP: File Transfer Protocol

Comandos FTP

- Transferindo arquivos: comandos get e put

```
ftp> get arquivo-fonte [arquivo-destino]
```

```
ftp> put arquivo-fonte [arquivo-destino]
```

- Exemplo de transferência de arquivo:

```
ftp> get comentario
```

```
200 PORT command successful.
```

```
150 ASCII data connection for comentario (150.162.60.1,3516) (1588 bytes)
```

```
226 ASCII Transfer complete.
```

```
1634 bytes received in 0.052 seconds (30 Kbytes/s)
```

```
ftp> quit
```

```
221 Goodbye.
```

Protocolo FTP: comandos, respostas

Comandos típicos:

- enviados em texto ASCII pelo canal de controle
- USER nome
- PASS senha
- LIST devolve lista de arquivos no diretório atual
- RETR arquivo recupera (lê) arquivo remoto
- STOR arquivo armazena (escreve) arquivo no hospedeiro remoto
- PASV indica o modo passivo
- PORT h1, h2, h3, h4, p1, p2 porta informa a porta de dados
 - H1.h2.h3.h4 é o endereço IP
 - P1*256+p2 é a porta

Códigos de retorno típicos

- código e frase de status (como para http)
- 331 Username OK, password required
- 125 data connection already open; transfer starting
- 425 Can't open data connection
- 452 Error writing file

Tente: telnet ftp.unicamp.br 21

Respostas FTP

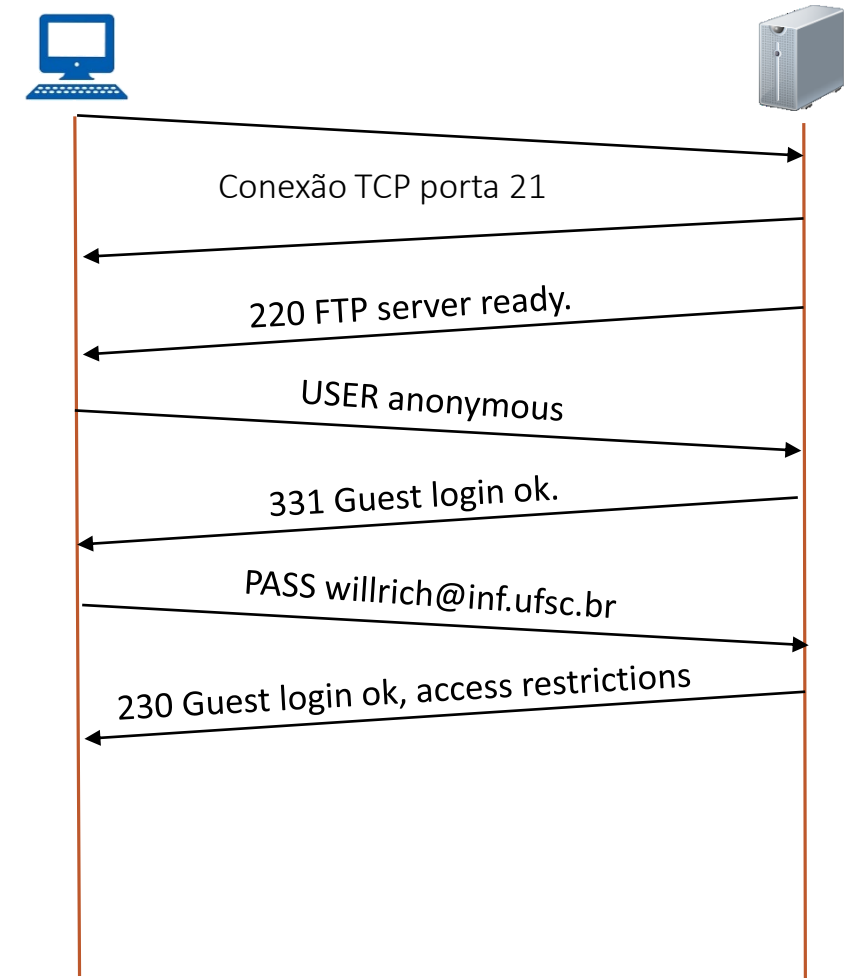
Enviadas sobre o canal de controle

- Linhas únicas contendo:
 - 3 dígitos de status
 - Mensagem de texto
- Primeiro dígito do status indica o tipo de resposta
 - '1': Resposta positiva preliminar (ok, mas espere)
 - '2': Resposta positiva (sucesso)
 - '3': Resposta positiva intermediária (esperando mais informações).
 - '4': Resposta negativa transiente (erro – tente novamente)
 - '5': Resposta negativa permanente (erro – não pode fazer).

FTP: Modo Ativo

Exemplo de conexão FTP modo ativo

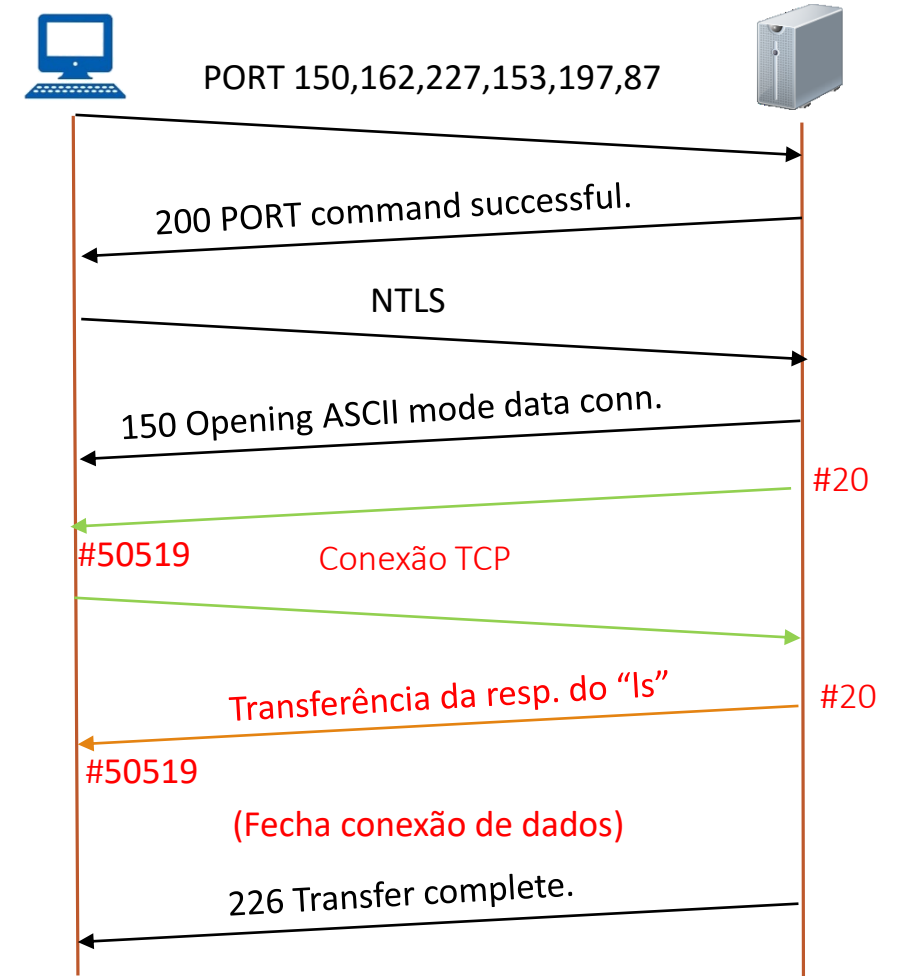
- Inicia conexão de controle FTP com servidor
- Servidor cumprimenta cliente
- Cliente faz o login (anônimo)
 - Note que a senha passar sem criptografia
- Agora cliente quer solicitar o conteúdo do diretório remoto
 - Necessário abrir conexão de dados antes



FTP: Modo Ativo

Exemplo de conexão FTP modo ativo

- Abrindo conexão de dados via comando PORT
 - Cliente informa ao servidor que ele quer receber conexão de dados no endereço IP 150.162.227.153 e na porta $197 \times 256 + 87 = 50519$
 - Servidor solicita conexão com o end. IP e porta informados
- Vendo o conteúdo do diretório
 - Comando NTLS
 - Conteúdo enviado na conexão de dados
 - Conexão de dados é fechada



Exemplo de FTP

The image displays a Wireshark packet capture window titled 'ftp1.pcap - Wireshark' with a filter set to 'tcp'. The packet list shows several TCP and FTP packets. A red circle highlights packets 22, 23, and 24, which represent the initial TCP connection setup. Packet 22 is a SYN from 192.168.0.197 to 128.113.126.13. Packet 23 is a SYN, ACK from 128.113.126.13 to 192.168.0.197. Packet 24 is an ACK from 192.168.0.197 to 128.113.126.13. A text box with the text 'Pedido de conexão TCP com a porta 21' points to these packets. Below the packet list, the packet details for packet 22 are expanded, showing the Ethernet II, Internet Protocol, and Transmission Control Protocol layers. A command prompt window is overlaid on the bottom right, showing the execution of the command 'ftp ftp.cs.rpi.edu'. The prompt shows the user connecting to the server, logging in as 'anonymous', and listing the directory contents, which include files like 'welcome.msg', '.pathmsg', '.index.html', '.snap', 'private', '.notar', 'mirrors', 'pub', and 'etc'.

Filter: tcp

No.	Time	Source	Destination	Protocol	Info
22	3.438	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=2 SACK_PERM=1
23	3.600	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1380 WS=3
24	3.606	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
25	3.770	128.113.126.13	192.168.0.197	FTP	Response: 220 FTP server ready.
26	3.976	192.168.0.197	128.113.126.13	FTP	Request: USER anonymous
30	9.322	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
31	9.484	128.113.126.13	192.168.0.197	FTP	Response: 331 Guest login ok, send your email address as password.
32	9.686	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=17 Ack=82 win=8108 Len=0
33	13.381	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
34	13.611	128.113.126.13	192.168.0.197	FTP	Response: 230 Guest login ok, access restrictions apply.
35	13.821	192.168.0.197	128.113.126.13	FTP	Request: ls
36	14.521	192.168.0.197	128.113.126.13	FTP	Request: cd pub

Pedido de conexão TCP com a porta 21

```
C:\Users\willrich>ftp ftp.cs.rpi.edu
Conectado a valvet.cs.rpi.edu.
220 FTP server ready.
Usuário (valvet.cs.rpi.edu:(none)): anonymous
331 Guest login ok, send your email address as password.
Senha:
230 Guest login ok, access restrictions apply.
ftp> ls
200 PORT command successful.
150 Opening ASCII mode data connection for 'file list'.
.login_conf
etc
bin
welcome.msg
.pathmsg
.index.html
.snap
private
.notar
mirrors
pub
226 Transfer complete.
ftp: 97 bytes recebidos em 0.00Segundos 97000.00Kbytes/s.
ftp> cd pub
250 CWD command successful.
```

Exemplo de FTP

The image shows a Wireshark packet capture of an FTP session. The filter is set to 'tcp'. A red circle highlights packets 22, 23, and 24, which represent the TCP connection establishment. Packet 22 is a SYN from 192.168.0.197 to 128.113.126.13. Packet 23 is a SYN, ACK from 128.113.126.13 to 192.168.0.197. Packet 24 is an ACK from 192.168.0.197 to 128.113.126.13. A text box with the text 'Pedido de conexão TCP com a porta 21' is overlaid on the capture. Below the packet list, the details of packet 22 are shown, including the Ethernet II header, Internet Protocol header, and Transmission Control Protocol header. The bottom of the window shows the packet bytes and their hexadecimal representation.

No.	Time	Source	Destination	Protocol	Info
22	3.438	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=2 SACK_PERM=1
23	3.600	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1380 WS=3
24	3.606	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
25	3.770	128.113.126.13	192.168.0.197	FTP	Response: 220 FTP server ready.
26	3.976	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=17 Ack=82 win=8108 Len=0
30	9.322	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
31	9.484	128.113.126.13	192.168.0.197	FTP	Response: 331 Guest login ok, send your email address as password.
32	9.686	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=17 Ack=82 win=8108 Len=0
33	13.38	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
34	13.611	128.113.126.13	192.168.0.197	FTP	Response: 230 Guest login ok, access restrictions apply.
35	13.82	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=44 Ack=130 win=8060 Len=0
36	14.52	192.168.0.197	128.113.126.13	MSNMS	PNG

Pedido de conexão TCP com a porta 21

Frame 22: 66 bytes on wire (528 bits), 66 bytes captured (528 bits)
Ethernet II, Src: IntelCor_2a:f0:71 (00:1f:3c:2a:f0:71), Dst: D-Link_c0:ce:df (00:1c:f0:c0:ce:df)
Internet Protocol, Src: 192.168.0.197 (192.168.0.197), Dst: 128.113.126.13 (128.113.126.13)
Transmission Control Protocol, Src Port: taurus-wh (1610), Dst Port: ftp (21), Seq: 0, Len: 0

0000 00 1c f0 c0 ce df 00 1f 3c 2a f0 71 08 00 45 00 <*.q..E.
0010 00 34 4c 81 40 00 80 06 ee 56 c0 a8 00 c5 80 71 .4L.@... .V....q
0020 7e 0d 06 4a 00 15 55 94 00 bd 00 00 00 00 80 02 ~..J..U.
0030 20 00 32 7a 00 00 02 04 05 b4 01 03 03 02 01 01 .2z....
0040 04 02 ..

File: "C:\Users\willrich\Documents\My Drop... Packets: 120 Displayed: 110 Marked: 0 Load time: 0:00.000 Profile: Default

Exemplo de FTP

The image shows a Wireshark packet capture of an FTP session. The filter is set to 'tcp'. The packet list shows several packets, with packet 25 highlighted in blue and circled in red. This packet is an FTP response from the server (128.113.126.13) to the client (192.168.0.197), indicating the server is ready (220 FTP server ready.). A red box with the text 'Mensagem de boas vindas do servidor' is overlaid on this packet. The packet details pane shows the structure of the packet: Frame 25 (77 bytes on wire, 77 bytes captured), Ethernet II, Internet Protocol, Transmission Control Protocol, and File Transfer Protocol (FTP) with the message '220 FTP server ready.\r\n'. The packet bytes pane shows the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Info
22	3.438	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=2 SACK_PERM=1
23	3.600	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1380 WS=3
24	3.600	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
25	3.770	128.113.126.13	192.168.0.197	FTP	Response: 220 FTP server ready.
26	3.978	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=24 win=8168 Len=0
30	9.322	192.168.0.197	128.113.126.13	FTP	Request: USER anonymous
31	9.484	128.113.126.13	192.168.0.197	FTP	Response: 331 Please send your email address as password.
32	9.686	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
34	13.611	128.113.126.13	192.168.0.197	FTP	Response: 230 Guest login ok, access restrictions apply.
35	13.821	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=44 Ack=130 win=8060 Len=0
36	14.521	192.168.0.197	128.113.126.13	MSNMS	PNG

Frame 25: 77 bytes on wire (616 bits), 77 bytes captured (616 bits)

Ethernet II, Src: D-Link_c0:ce:df (00:1c:f0:c0:ce:df), Dst: IntelCor_2a:f0:71 (00:1f:3c:2a:f0:71)

Internet Protocol, Src: 128.113.126.13 (128.113.126.13), Dst: 192.168.0.197 (192.168.0.197)

Transmission Control Protocol, Src Port: ftp (21), Dst Port: taurus-wh (1610), Seq: 1, Ack: 1, Len: 23

File Transfer Protocol (FTP)

220 FTP server ready.\r\n

0000 00 1f 3c 2a f0 71 00 1c f0 c0 ce df 08 00 45 00 ..<*.q..E.
0010 00 3f 84 45 40 00 2e 06 08 88 80 71 7e 0d c0 a8 .?.E@... ..q~...
0020 00 c5 00 15 06 4a da fa 15 d2 55 94 00 be 50 18J.. ..U...P.
0030 20 58 1b 84 00 00 32 32 30 20 46 54 50 20 73 65 X....22 0 FTP se
0040 72 76 65 72 20 72 65 61 64 79 2e 0d 0a rver rea dy...

File: "C:\Users\willrich\Documents\My Drop..." Packets: 120 Displayed: 110 Marked: 0 Load time: 0:00.000 Profile: Default

Exemplo de FTP

The image displays a Wireshark packet capture of an FTP session and a corresponding Windows command prompt window.

Wireshark Packet Capture:

- Filter: tcp
- Packet 25 (highlighted in blue and circled in red):

No.	Time	Source	Destination	Protocol	Info
25	3.77	128.113.126.13	192.168.0.197	FTP	Response: 220 FTP server ready.

Windows Command Prompt:

```
C:\Users\willrich>ftp ftp.cs.rpi.edu
Conectado a valvet.cs.rpi.edu.
220 FTP server ready.
Usuário (valvet.cs.rpi.edu:(none)): anonymous
231 Guest login ok, send your email address as password.
Senha:
230 Guest login ok, access restrictions apply.
ftp> ls
200 PORT command successful.
150 Opening ASCII mode data connection for 'file list'.
.login_conf
etc
bin
welcome.msg
.pathmsg
.index.html
.snap
private
.notar
mirrors
pub
226 Transfer complete.
ftp: 97 bytes recebidos em 0.00Segundos 97000.00Kbytes/s.
ftp> cd pub
250 CWD command successful.
```

A red circle highlights the "220 FTP server ready." message in both the Wireshark packet list and the command prompt output.

Exemplo de FTP

The image shows a Wireshark capture of an FTP session. The filter is set to 'tcp'. The packet list shows several packets, with packet 30 highlighted in blue and circled in red. Packet 30 is an FTP request: 'Request: USER anonymous'. A red box highlights the text 'Comando USER' over the packet details. The packet details pane shows the structure of the packet: Ethernet II, Internet Protocol, Transmission Control Protocol, and File Transfer Protocol (FTP). The FTP details show 'USER anonymous\r\n'. The packet bytes pane shows the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Info
22	3.438	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=2 SACK_PERM=1
23	3.600	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1380 WS=3
24	3.600	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
25	3.770	128.113.126.13	192.168.0.197	FTP	Response: 220 FTP server ready.
26	3.976	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=24 win=8168 Len=0
30	9.327	192.168.0.197	128.113.126.13	FTP	Request: USER anonymous
31	9.484	128.113.126.13	192.168.0.197	FTP	Response: 331 Guest login ok, send your email address as password.
32	9.686	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=17 Ack=82 win=8108 Len=0
33	13.381	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
34	13.611	128.113.126.13	192.168.0.197	FTP	Response: 332 Password required for willrich@inf.ufsc.br. Restrictions apply. Password may be changed after login.
35	13.821	192.168.0.197	128.113.126.13	FTP	Request: USER anonymous
36	14.571	192.168.0.197	128.113.126.13	FTP	Response: 331 Guest login ok, send your email address as password.

Comando USER

Frame 30: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface 0
Ethernet II, Src: IntelCor_2a:f0:71 (00:1f:3c:2a:f0:71), Dst: D-Link_c0:ce:df (00:1c:f0:c0:ce:df)
Internet Protocol, Src: 192.168.0.197 (192.168.0.197), Dst: 128.113.126.13 (128.113.126.13)
Transmission Control Protocol, Src Port: taurus-wh (1610), Dst Port: ftp (21), Seq: 1, Ack: 24, Len: 16
File Transfer Protocol (FTP)
USER anonymous\r\n

0000 00 1c f0 c0 ce df 00 1f 3c 2a f0 71 08 00 45 00 <*.q..E.
0010 00 38 4c 84 40 00 80 06 ee 4f c0 a8 00 c5 80 71 .8L.@... .O.....q
0020 7e 0d 06 4a 00 15 55 94 00 be da fa 15 e9 50 18 ~..J..U.P.
0030 07 fa 12 65 00 00 55 53 45 52 20 61 6e 6f 6e 79 ...e..US ER anony
0040 6d 6f 75 73 0d 0a mous..

Text item (text), 16 bytes Packets: 120 Displayed: 110 Marked: 0 Load time: 0:00.000 Profile: Default

Exemplo de FTP

The image displays a Wireshark packet capture window titled 'ftp1.pcap - Wireshark' and a Windows Command Prompt window showing an FTP session.

Wireshark Packet Capture:

- Filter:** tcp
- Table:**

No.	Time	Source	Destination	Protocol	Info
22	3.438	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=2 SACK_PERM=1
23	3.600	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1380 WS=3
24	3.600	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
25	3.770	128.113.126.13	192.168.0.197	FTP	Response: 220 FTP server ready.
26	3.976	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=24 win=8168 Len=0
30	9.327	192.168.0.197	128.113.126.13	FTP	Request: USER anonymous
31	9.484	128.113.126.13	192.168.0.197	FTP	Response: 331 Guest login ok, send your email address as password.
32	9.686	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=17 Ack=82 win=8108 Len=0
33	13.381	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
34	13.611	128.113.126.13	192.168.0.197	FTP	Response: 332 Password accepted. Restrictions apply.
35	13.821	192.168.0.197	128.113.126.13	FTP	Request: CWD /
36	14.571	192.168.0.197	128.113.126.13	FTP	Response: 200 CWD successful.

Command Prompt:

```
C:\Users\willrich>ftp ftp.cs.rpi.edu
Conectado a valvet.cs.rpi.edu.
220 FTP server ready.
Usuário (valvet.cs.rpi.edu:(none)): anonymous
331 Guest login ok, send your email address as password.
Senha:
230 Guest login ok, access restrictions apply.
ftp> ls
200 PORT command successful.
150 Opening ASCII mode data connection for 'file list'.
.login_conf
etc
bin
welcome.msg
.pathmsg
.index.html
.snap
private
.notar
mirrors
pub
```

Exemplo de FTP

ftp1.pcap - Wireshark

Filter: tcp

No.	Time	Source	Destination	Protocol	Info
22	3.438	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=2 SACK_PERM=1
23	3.600	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1380 WS=3
24	3.600	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
25	3.770	128.113.126.13	192.168.0.197	FTP	Response: 220 FTP server ready.
26	3.976	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=24 win=8168 Len=0
30	9.322	192.168.0.197	128.113.126.13	FTP	Request: USER anonymous
31	9.484	128.113.126.13	192.168.0.197	FTP	Response: 331 Guest login ok, send your email address as password.
32	9.686	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=17 Ack=82 win=8108 Len=0
33	13.381	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
34	13.611	128.113.126.13	192.168.0.197	FTP	Response: 230 Guest login ok, access restrictions apply.
35	13.821	192.168.0.197			
36	14.521	192.168.0.197			

Resposta do comando USER

Frame 31: 112 bytes on wire (896 bits), 112 bytes captured (896 bits)

Ethernet II, Src: D-Link_c0:ce:df (00:1c:f0:c0:ce:df), Dst: IntelCor_2a:f0:71 (00:1f:3c:2a:f0:71)

Internet Protocol, Src: 128.113.126.13 (128.113.126.13), Dst: 192.168.0.197 (192.168.0.197)

Transmission Control Protocol, Src Port: ftp (21), Dst Port: taurus-wh (1610), Seq: 24, Ack: 17, Len: 58

File Transfer Protocol (FTP)

331 Guest login ok, send your email address as password.\r\n

0000 00 1f 3c 2a f0 71 00 1c f0 c0 ce df 08 00 45 00 ..<*.q..E.
0010 00 62 84 7b 40 00 2e 06 08 2f 80 71 7e 0d c0 a8 .b.{@... ./..q~...
0020 00 c5 00 15 06 4a da fa 15 e9 55 94 00 ce 50 18J.. ..U...P.
0030 20 58 32 14 00 00 33 33 31 20 47 75 65 73 74 20 x2...33 1 Guest
0040 6c 6f 67 69 6e 20 6f 6b 2c 20 73 65 6e 64 20 79 login ok , send y
0050 6f 75 72 20 65 6d 61 69 6c 20 61 64 64 72 65 73 our email address
0060 73 20 61 73 20 70 61 73 73 77 6f 72 64 2e 0d 0a s as password...

File: "C:\Users\willrich\Documents\My Drop... Packets: 120 Displayed: 110 Marked: 0 Load time: 0:00.000 Profile: Default

Exemplo de FTP

The image displays a Wireshark packet capture window titled 'ftp1.pcap - Wireshark' and a Windows Command Prompt window showing an FTP session.

Wireshark Packet Capture:

- Filter:** tcp
- Table:**

No.	Time	Source	Destination	Protocol	Info
22	3.438	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=2 SACK_PERM=1
23	3.600	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1380 WS=3
24	3.600	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
25	3.770	128.113.126.13	192.168.0.197	FTP	Response: 220 FTP server ready.
26	3.976	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=24 win=8168 Len=0
30	9.322	192.168.0.197	128.113.126.13	FTP	Request: USER anonymous
31	9.484	128.113.126.13	192.168.0.197	FTP	Response: 331 Guest login ok, send your email address as password.
32	9.686	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=17 Ack=82 win=8108 Len=0
33	13.381	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
34	13.611	128.113.126.13	192.168.0.197	FTP	Response: 230 Guest login ok, access restrictions apply.
35	13.821	192.168.0.197			
36	14.521	192.168.0.197			

Command Prompt:

```
C:\Users\willrich>ftp ftp.cs.rpi.edu
Conectado a valvet.cs.rpi.edu.
220 FTP server ready.
Usuário (valvet.cs.rpi.edu:(none)): anonymous
331 Guest login ok, send your email address as password.
Senha:
230 Guest login ok, access restrictions apply.
ftp> ls
200 PORT command successful.
150 Opening ASCII mode data connection for 'file list'.
.login_conf
etc
bin
welcome.msg
.pathmsg
.index.html
.snap
private
.notar
mirrors
pub
```

Annotations:

- A red oval highlights packet 31 in the Wireshark list: "Response: 331 Guest login ok, send your email address as password."
- A red oval highlights the corresponding command prompt output: "331 Guest login ok, send your email address as password."
- An orange box with the text "Resposta do comando USER" points to the highlighted packet 31.

Exemplo de FTP

The image shows a Wireshark capture of an FTP session. The filter is set to 'tcp'. The packet list shows several packets, with packet 33 highlighted in blue. A red oval highlights packet 33, and a red box with the text 'Comando USER' is placed over it. The packet details pane shows the structure of packet 33, which is an FTP 'PASS' command. The packet bytes pane shows the raw data of packet 33.

Filter: tcp

No.	Time	Source	Destination	Protocol	Info
22	3.438	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=2 SACK_PERM=1
23	3.600	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1380 WS=3
24	3.600	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
25	3.770	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [ACK] Seq=1 Ack=1 win=8192 Len=0
26	3.976	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
30	9.322	192.168.0.197	128.113.126.13	FTP	Request: USER anonymous
31	9.484	128.113.126.13	192.168.0.197	FTP	Response: 331 Guest login ok, send your email address as password.
32	9.686	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=17 Ack=82 win=8108 Len=0
33	13.381	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
34	13.611	128.113.126.13	192.168.0.197	FTP	Response: 230 Guest login ok, access restrictions apply.
35	13.821	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=44 Ack=130 win=8060 Len=0
36	14.571	192.168.0.197	128.113.126.13	MSNMS	PNG

Frame 33: 81 bytes on wire (648 bits), 81 bytes captured (648 bits)

Ethernet II, Src: IntelCor_2a:f0:71 (00:1f:3c:2a:f0:71), Dst: D-Link_c0:ce:df (00:1c:f0:c0:ce:df)

Internet Protocol, Src: 192.168.0.197 (192.168.0.197), Dst: 128.113.126.13 (128.113.126.13)

Transmission Control Protocol, Src Port: taurus-wh (1610), Dst Port: ftp (21), Seq: 17, Ack: 82, Len: 27

File Transfer Protocol (FTP)

PASS willrich@inf.ufsc.br\r\n

0000 00 1c f0 c0 ce df 00 1f 3c 2a f0 71 08 00 45 00 <*.q..E.
0010 00 43 4c 86 40 00 80 06 ee 42 c0 a8 00 c5 80 71 .CL.@... .B.....q
0020 7e 0d 06 4a 00 15 55 94 00 ce da fa 16 23 50 18 ~..J..U.#P.
0030 07 eb cb 96 00 00 50 41 53 53 20 77 69 6c 6c 72PA SS willr
0040 69 63 68 40 69 6e 66 2e 75 66 73 63 2e 62 72 0d ich@inf. ufsc.br.
0050 0a .

File: "C:\Users\willrich\Documents\My Drop..." Packets: 120 Displayed: 110 Marked: 0 Load time: 0:00.000 Profile: Default

Exemplo de FTP

The image displays a Wireshark packet capture window titled 'ftp1.pcap - Wireshark' and a terminal window titled 'Prompt de Comando'.

Wireshark Packet Capture:

- Filter:** tcp
- Table:**

No.	Time	Source	Destination	Protocol	Info
22	3.438	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=2 SACK_PERM=1
23	3.600	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1380 WS=3
24	3.600	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
25	3.770	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [ACK] Seq=1 Ack=1 win=8192 Len=0
26	3.976	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
30	9.322	192.168.0.197	128.113.126.13	FTP	Request: USER anonymous
31	9.484	128.113.126.13	192.168.0.197	FTP	Response: 331 Guest login ok, send your email address as password.
32	9.686	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=17 Ack=82 win=8108 Len=0
33	13.381	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
34	13.611	128.113.126.13	192.168.0.197	FTP	Response: 230 Guest login ok, access restrictions apply.
35	13.821	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=44 Ack=130 win=8060 Len=0
36	14.571	192.168.0.197	128.113.126.13	MSNMS	PNG

- Packet 33 Details:**
 - Frame 33: 81 bytes on wire (648 bits) - 81 bytes captured (648 bits)
 - Ethernet II, Src: IntelCor_2a:f0:71
 - Internet Protocol, Src: 192.168.0.1
 - Transmission Control Protocol, Src: 192.168.0.197
 - File Transfer Protocol (FTP)
 - PASS willrich@inf.ufsc.br\r\n

Prompt de Comando:

```
C:\Users\willrich>ftp ftp.cs.rpi.edu
Conectado a valvet.cs.rpi.edu.
220 FTP server ready.
Usuário (valvet.cs.rpi.edu:(none)): anonymous
331 Guest login ok, send your email address as password.
Senha:
230 Guest login ok, access restrictions apply.
ftp> ls
200 PORT command successful.
150 Opening ASCII mode data connection for 'file list'.
.login_conf
etc
bin
welcome.msg
.pathmsg
.index.html
.snap
private
.notar
mirrors
pub
```

Exemplo de FTP

The image shows a Wireshark packet capture of an FTP session. The filter is set to 'tcp'. The packet list shows several packets, with packet 34 highlighted in blue and circled in red. A red box with the text 'Resposta do comando USER' is placed over packet 33. The packet details pane shows the structure of packet 34, which is an FTP response. The packet bytes pane shows the raw data of packet 34.

No.	Time	Source	Destination	Protocol	Info
22	3.438	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=2 SACK_PERM=1
23	3.600	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1380 WS=3
24	3.600	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
25	3.770	128.113.126.13	192.168.0.197	FTP	Request: 220 FTP server ready
26	3.976	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=1 Ack=1 win=8192 Len=0
30	9.322	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=17 Ack=82 win=8108 Len=0
31	9.484	128.113.126.13	192.168.0.197	FTP	Response: 331 Guest login ok, send your email address as password.
32	9.686	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=17 Ack=82 win=8108 Len=0
33	13.381	192.168.0.197	128.113.126.13	FTP	Request: PASS willrich@inf.ufsc.br
34	13.611	128.113.126.13	192.168.0.197	FTP	Response: 230 Guest login ok, access restrictions apply.
35	13.821	192.168.0.197	128.113.126.13	TCP	taurus-wh > ftp [ACK] Seq=44 Ack=130 win=8060 Len=0
36	14.571	192.168.0.197	128.113.126.13	MSNMS	PNG

Frame 34: 102 bytes on wire (816 bits), 102 bytes captured (816 bits)

Ethernet II, Src: D-Link_c0:ce:df (00:1c:f0:c0:ce:df), Dst: IntelCor_2a:f0:71 (00:1f:3c:2a:f0:71)

Internet Protocol, Src: 128.113.126.13 (128.113.126.13), Dst: 192.168.0.197 (192.168.0.197)

Transmission Control Protocol, Src Port: ftp (21), Dst Port: taurus-wh (1610), Seq: 82, Ack: 44, Len: 48

File Transfer Protocol (FTP)

230 Guest login ok, access restrictions apply.\r\n

0000 00 1f 3c 2a f0 71 00 1c f0 c0 ce df 08 00 45 00 ..<*.q..E.
0010 00 58 84 a1 40 00 2e 06 08 13 80 71 7e 0d c0 a8 .X..@... ..q~...
0020 00 c5 00 15 06 4a da fa 16 23 55 94 00 e9 50 18J.. .#U...P.
0030 20 58 d5 77 00 00 32 33 30 20 47 75 65 73 74 20 X.w..23 0 Guest
0040 6c 6f 67 69 6e 20 6f 6b 2c 20 61 63 63 65 73 73 login ok , access
0050 20 72 65 73 74 72 69 63 74 69 6f 6e 73 20 61 70 restric tions ap
0060 70 6c 79 2e 0d 0a ply...

File: "C:\Users\willrich\Documents\My Drop..." Packets: 120 Displayed: 110 Marked: 0 Load time: 0:00.000 Profile: Default

Exemplo de FTP

The image shows a Wireshark capture of an FTP session. The filter is set to 'tcp'. The packet list shows several packets, with packets 39 and 40 circled in red. Packet 39 is an FTP Request: PORT 192,168,0,197,6,75. Packet 40 is an FTP Response: 200 PORT command successful. A red box highlights the text: Comando Port: IP: 192.168.0.197 Porta Y: 6*256+75=1611. The packet details pane shows the structure of the selected packet, including Ethernet II, Internet Protocol, Transmission Control Protocol, and File Transfer Protocol (FTP). The packet bytes pane shows the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Info
39	15.13	192.168.0.197	128.113.126.13	FTP	Request: PORT 192,168,0,197,6,75
40	15.25	128.113.126.13	192.168.0.197	FTP	Response: 200 PORT command successful.
41	15.25	192.168.0.197	128.113.126.13	FTP	Request: NLST
42	15.45	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [SYN] Seq=0 win=65535 Len=0 MSS=536 WS=3 SACK_PERM=1 T...
43	15.55	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [ACK] Seq=160 Ack=75 win=66240 Len=0
45	18.45	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [SYN] Seq=0 win=65535 Len=0 MSS=536 WS=3 SACK_PERM=1 T...
46	18.45	192.168.0.197	128.113.126.13	TCP	ftp > taurus-wh [ACK] Seq=160 Ack=75 win=66240 Len=0
47	18.62	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [SYN] Seq=0 win=65535 Len=0 MSS=536 WS=3 SACK_PERM=1 T...
48	18.62	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [ACK] Seq=160 Ack=75 win=66240 Len=0
49	18.62	192.168.0.197	128.113.126.13	TCP	ftp-data > ill [SYN] Seq=0 win=65535 Len=0 MSS=536 WS=3 SACK_PERM=1 T...
50	18.62	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [ACK] Seq=160 Ack=75 win=66240 Len=0
51	18.62	128.113.126.13	192.168.0.197	FTP	Response: 220 Transfer complete.

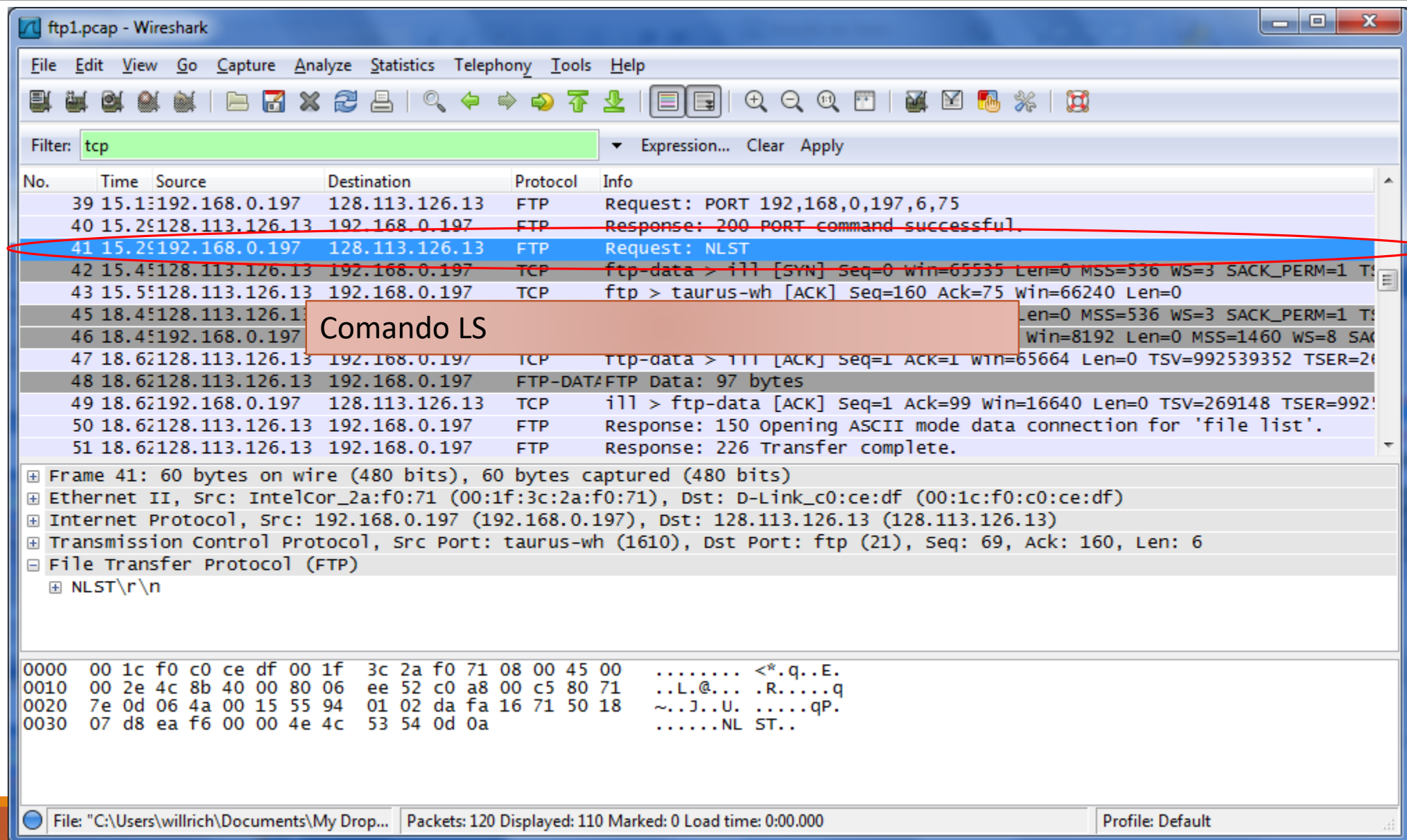
Comando Port:
IP: 192.168.0.197
Porta Y: $6 \times 256 + 75 = 1611$

Frame 39: 79 bytes on wire (632 bits), 79 bytes captured (632 bits)
Ethernet II, Src: IntelCor_2a:f0:71 (00:1f:3c:2a:f0:71), Dst: D-Link_c0:ce:df (00:1c:f0:c0:ce:df)
Internet Protocol, Src: 192.168.0.197 (192.168.0.197), Dst: 128.113.126.13 (128.113.126.13)
Transmission Control Protocol, Src Port: taurus-wh (1610), Dst Port: ftp (21), Seq: 44, Ack: 130, Len: 25
File Transfer Protocol (FTP)
PORT 192,168,0,197,6,75\r\n

0000 00 1c f0 c0 ce df 00 1f 3c 2a f0 71 08 00 45 00 <*.q..E.
0010 00 41 4c 8a 40 00 80 06 ee 40 c0 a8 00 c5 80 71 .AL.@... .@.....q
0020 7e 0d 06 4a 00 15 55 94 00 e9 da fa 16 53 50 18 ~..J..U.SP.
0030 07 df 12 3b 00 00 50 4f 52 54 20 31 39 32 2c 31 ...;...PO RT 192,1
0040 36 38 2c 30 2c 31 39 37 2c 36 2c 37 35 0d 0a 68,0,197 ,6,75..

File: "C:\Users\willrich\Documents\My Drop... Packets: 120 Displayed: 110 Marked: 0 Load time: 0:00.000 Profile: Default

Exemplo de FTP



The image shows a Wireshark packet capture window titled "ftp1.pcap - Wireshark". The filter is set to "tcp". The packet list shows several packets, with packet 41 highlighted in blue and circled in red. Packet 41 is an FTP Request: NLST. A red box with the text "Comando LS" is overlaid on packet 41. The packet details pane shows the following structure:

- Frame 41: 60 bytes on wire (480 bits), 60 bytes captured (480 bits)
- Ethernet II, Src: IntelCor_2a:f0:71 (00:1f:3c:2a:f0:71), Dst: D-Link_c0:ce:df (00:1c:f0:c0:ce:df)
- Internet Protocol, Src: 192.168.0.197 (192.168.0.197), Dst: 128.113.126.13 (128.113.126.13)
- Transmission Control Protocol, Src Port: taurus-wh (1610), Dst Port: ftp (21), Seq: 69, Ack: 160, Len: 6
- File Transfer Protocol (FTP)
 - NLST\r\n

The packet bytes pane shows the raw data of the NLST command:

```
0000  00 1c f0 c0 ce df 00 1f 3c 2a f0 71 08 00 45 00  ..... <*.q..E.
0010  00 2e 4c 8b 40 00 80 06 ee 52 c0 a8 00 c5 80 71  ..L.@... .R.....q
0020  7e 0d 06 4a 00 15 55 94 01 02 da fa 16 71 50 18  ~..J..U. ....qP.
0030  07 d8 ea f6 00 00 4e 4c 53 54 0d 0a                .....NL ST..
```

The status bar at the bottom indicates: File: "C:\Users\willrich\Documents\My Drop...", Packets: 120 Displayed: 110 Marked: 0 Load time: 0:00.000, Profile: Default.

Exemplo de FTP

The image shows a Wireshark capture of an FTP session. The filter is set to 'tcp'. The packet list shows several packets, with packet 42 highlighted. A red circle highlights packets 42, 43, 45, 46, and 47. A text box is overlaid on the packet details for packet 42, stating: "Estabelece conexão TCP com porta origem 20 (FTP-DATA) e destino 1611". The packet details for packet 42 show: Ethernet II, Src: D-Link_c0:ce:df (00:1c:f0:c0:ce:df), Dst: IntelCor_2a:f0:71 (00:1f:3c:2a:f0:71); Internet Protocol, Src: 128.113.126.13 (128.113.126.13), Dst: 192.168.0.197 (192.168.0.197); Transmission Control Protocol, Src Port: ftp-data (20), Dst Port: ill (1611), Seq: 0, Len: 0.

No.	Time	Source	Destination	Protocol	Info
39	15.13	192.168.0.197	128.113.126.13	FTP	Request: PORT 192,168,0,197,6,75
40	15.25	128.113.126.13	192.168.0.197	FTP	Response: 200 PORT command successful.
41	15.25	192.168.0.197	128.113.126.13	FTP	Request: NLST
42	15.45	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [SYN] Seq=0 win=65535 Len=0 MSS=536 WS=3 SACK_PERM=1 TSV=992539352 TSER=269148
43	15.55	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [ACK] Seq=160 Ack=75 win=66240 Len=0
45	18.45	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [SYN] Seq=0 win=65535 Len=0 MSS=536 WS=3 SACK_PERM=1 TSV=992539352 TSER=269148
46	18.45	192.168.0.197	128.113.126.13	TCP	ill > ftp-data [SYN, ACK] Seq=0 Ack=1 win=8192 Len=0 MSS=1460 WS=8 SACK_PERM=1 TSV=269148 TSER=992539352
47	18.62	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [ACK] Seq=1 Ack=1 win=65664 Len=0 TSV=992539352 TSER=269148
48	18.62	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [ACK] Seq=1 Ack=1 win=65664 Len=0 TSV=992539352 TSER=269148
49	18.62	192.168.0.197	128.113.126.13	TCP	ill > ftp-data [ACK] Seq=0 Ack=1 win=8192 Len=0 MSS=1460 WS=8 SACK_PERM=1 TSV=269148 TSER=992539352
50	18.62	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [ACK] Seq=1 Ack=1 win=65664 Len=0 TSV=992539352 TSER=269148
51	18.62	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [ACK] Seq=1 Ack=1 win=65664 Len=0 TSV=992539352 TSER=269148

Frame 42: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0
Ethernet II, Src: D-Link_c0:ce:df (00:1c:f0:c0:ce:df), Dst: IntelCor_2a:f0:71 (00:1f:3c:2a:f0:71)
Internet Protocol, Src: 128.113.126.13 (128.113.126.13), Dst: 192.168.0.197 (192.168.0.197)
Transmission Control Protocol, Src Port: ftp-data (20), Dst Port: ill (1611), Seq: 0, Len: 0

0000 00 1f 3c 2a f0 71 00 1c f0 c0 ce df 08 00 45 00 ..<*.q..E.
0010 00 3c 84 af 40 00 2e 06 08 21 80 71 7e 0d c0 a8 <..@... !.q~...
0020 00 c5 00 14 06 4b 96 b3 47 81 00 00 00 00 a0 02K.. G.....
0030 ff ff 85 7c 00 00 02 04 02 18 01 03 03 03 04 02 ...|.
0040 08 0a 3b 28 e6 7b 00 00 00 00 ..;(.{. ..

File: "C:\Users\willrich\Documents\My Drop... Packets: 120 Displayed: 110 Marked: 0 Load time: 0:00.000 Profile: Default

Exemplo de FTP

The image shows a Wireshark capture of an FTP session. The packet list pane displays several packets, with packet 48 highlighted in blue. A red circle is drawn around packet 48 and packet 49. A red box is drawn around the packet details for packet 48, which is labeled "Resposta do ls".

Filter: tcp

No.	Time	Source	Destination	Protocol	Info
43	15.55	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [ACK] Seq=160 Ack=75 win=66240 Len=0
45	18.45	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [SYN] Seq=0 win=65535 Len=0 MSS=536 WS=3 SACK_PERM=1
46	18.45	192.168.0.197	128.113.126.13	TCP	ill > ftp-data [SYN, ACK] Seq=0 Ack=1 win=8192 Len=0 MSS=1460 WS=8
47	18.62	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [ACK] Seq=1 Ack=1 win=65664 Len=0 TSV=992539352 TSER
48	18.62	128.113.126.13	192.168.0.197	FTP-DATA	FTP Data: 97 bytes
49	18.62	192.168.0.197	128.113.126.13	TCP	ill > ftp-data [ACK] Seq=1 Ack=99 win=16640 Len=0 TSV=269148 TSER=9
50	18.62	128.113.126.13	192.168.0.197	FTP	Response: 150 Opening ASCII mode data connection for 'file list'.
51	18.62	128.113.126.13	192.168.0.197	FTP	Response: 226 Transfer complete.

Frame 48: 163 bytes on wire (1304 bits) captured (163 bytes) over Ethernet II
Ethernet II, Src: D... (08:00:27:3c:2a:f0:71), Dst: 192.168.0.197 (08:00:27:3c:2a:f0:71)
Internet Protocol, Src: 128.113.126.13 (128.113.126.13), Dst: 192.168.0.197 (192.168.0.197)
Transmission Control Protocol, Src Port: ftp-data (20), Dst Port: ill (1611), Seq: 1, Ack: 1, Len: 97
FTP Data

Resposta do ls

```
0000  00 1f 3c 2a f0 71 00 1c f0 c0 ce df 08 00 45 00  ..<*.q.. .....E.
0010  00 95 84 c0 40 00 2e 06 07 b7 80 71 7e 0d c0 a8  ....@... ..q~...
0020  00 c5 00 14 06 4b 96 b3 47 82 1e 5c f5 db 80 19  ....K.. G.. \....
0030  20 10 14 6f 00 00 01 01 08 0a 3b 28 f2 d8 00 04  ..o.... .;(. ...
0040  1b 4b 2e 6c 6f 67 69 6e 5f 63 6f 6e 66 0d 0a 65  .K.login _conf..e
0050  74 63 0d 0a 62 69 6e 0d 0a 77 65 6c 63 6f 6d 65  tc..bin. .welcome
0060  2e 6d 73 67 0d 0a 2e 70 61 74 68 6d 73 67 0d 0a  .msg...p athmsg..
0070  2e 69 6e 64 65 78 2e 68 74 6d 6c 0d 0a 2e 73 6e  .index.h tml...sn
0080  61 70 0d 0a 70 72 69 76 61 74 65 0d 0a 2e 6e 6f  ap..priv ate...no
0090  74 61 72 0d 0a 6d 69 72 72 6f 72 73 0d 0a 70 75  tar..mir rors..pu
00a0  62 0d 0a  b..
```

File: "C:\Users\willrich\Documents\My Drop... Packets: 120 Displayed: 110 Marked: 0 Load time: 0:00.000 Profile: Default

Exemplo de FTP

The image displays a network packet capture in Wireshark and a corresponding Windows command prompt window, illustrating an FTP session.

Wireshark Packet Capture:

- Filter:** tcp
- Table:**

No.	Time	Source	Destination	Protocol	Info
43	15.5	128.113.126.13	192.168.0.197	TCP	ftp > taurus-wh [ACK] Seq=160 Ack=75 win=66240 Len=0
45	18.4	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [SYN] Seq=0 win=65535 Len=0 MSS=536 WS=3 SACK_PERM=1
46	18.4	192.168.0.197	128.113.126.13	TCP	ill > ftp-data [SYN, ACK] Seq=0 Ack=1 win=8192 Len=0 MSS=1460 WS=8
47	18.6	128.113.126.13	192.168.0.197	TCP	ftp-data > ill [ACK] Seq=1 Ack=1 win=65664 Len=0 TSV=992539352 TSER
48	18.6	128.113.126.13	192.168.0.197	FTP-DATA	FTP Data: 97 bytes
49	18.6	192.168.0.197	128.113.126.13	TCP	ill > ftp-data [ACK] Seq=1 Ack=99 win=16640 Len=0 TSV=269148 TSER=9
50	18.6	128.113.126.13	192.168.0.197	FTP	Response: 150 Opening ASCII mode data connection for 'file list'.
51	18.6	128.113.126.13	192.168.0.197	FTP	Response: 226 Transfer complete.

- Packet 48 Details:**
 - Frame 48: 163 bytes
 - Ethernet II, Src: D-LINK_CV
 - Internet Protocol, Src: 128.113.126.13
 - Transmission Control Protocol
 - FTP Data

Windows Command Prompt:

```
C:\Users\willrich>ftp ftp.cs.rpi.edu
Conectado a valvet.cs.rpi.edu.
220 FTP server ready.
Usuário (valvet.cs.rpi.edu:(none)): anonymous
331 Guest login ok, send your email address as password.
Senha:
230 Guest login ok, access restrictions apply.
ftp> ls
200 PORT command successful.
150 Opening ASCII mode data connection for 'file list'.
.login_conf
etc
bin
welcome.msg
.pathmsg
.index.html
.snap
private
.notar
mirrors
pub
226 Transfer complete.
ftp: 97 bytes recebidos em 0,00Segundos 97000,00Kbytes/s.
ftp> cd pub
250 CWD command successful.
```

Redacted Area: A red box labeled "Resposta do ls" (ls response) covers the details of the FTP data packet in the Wireshark interface.

FTP: Modo Ativo

Geralmente não funciona: Firewalls bloqueiam pedidos de conexão vinda de fora da rede local

- Cliente abre o canal de comandos da porta cliente
- Para transferência de dados, cliente envia comando PORT indicando porta para receber dados
- Servidor cria conexão de dados (bloqueada pelo firewall)



FTP: Modo Ativo

```
Prompt de Comando - ftp ftp.unicamp.br

C:\Users\willrich>ftp ftp.unicamp.br
Conectado a ftp.unicamp.br.
220 FtpServer da Unicamp
200 UTF8 set to on
Usuário (ftp.unicamp.br:(none)): anonymous
331 Anonymous login ok, send your complete email address as your pass
Senha:
230-
    Bem-vindo ao servidor FTP da Unicamp!
    Sugestoes e comentarios, por favor entre em contato
    com <apoio@ccuec.unicamp.br>.

----

Welcome to UNICAMP's FTP server!
For comments on this site, please contact <apoio@ccuec.unicamp.br>.

You are user (4) of (300) simultaneous users allowed.

230 Anonymous access granted, restrictions apply
ftp> ls
200 PORT command successful
425 Unable to build data connection: Operation timed out
ftp> █
```

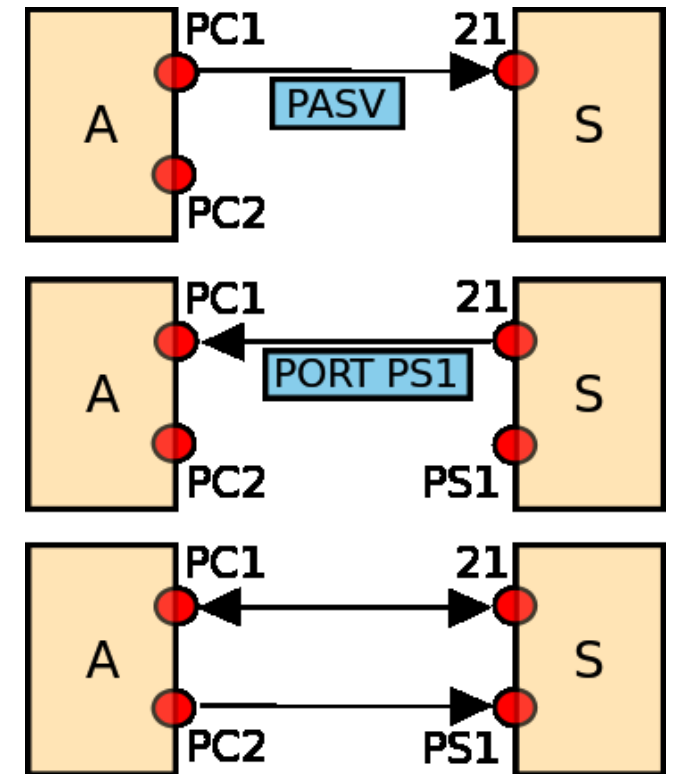
FTP Ativo (com Firewall)



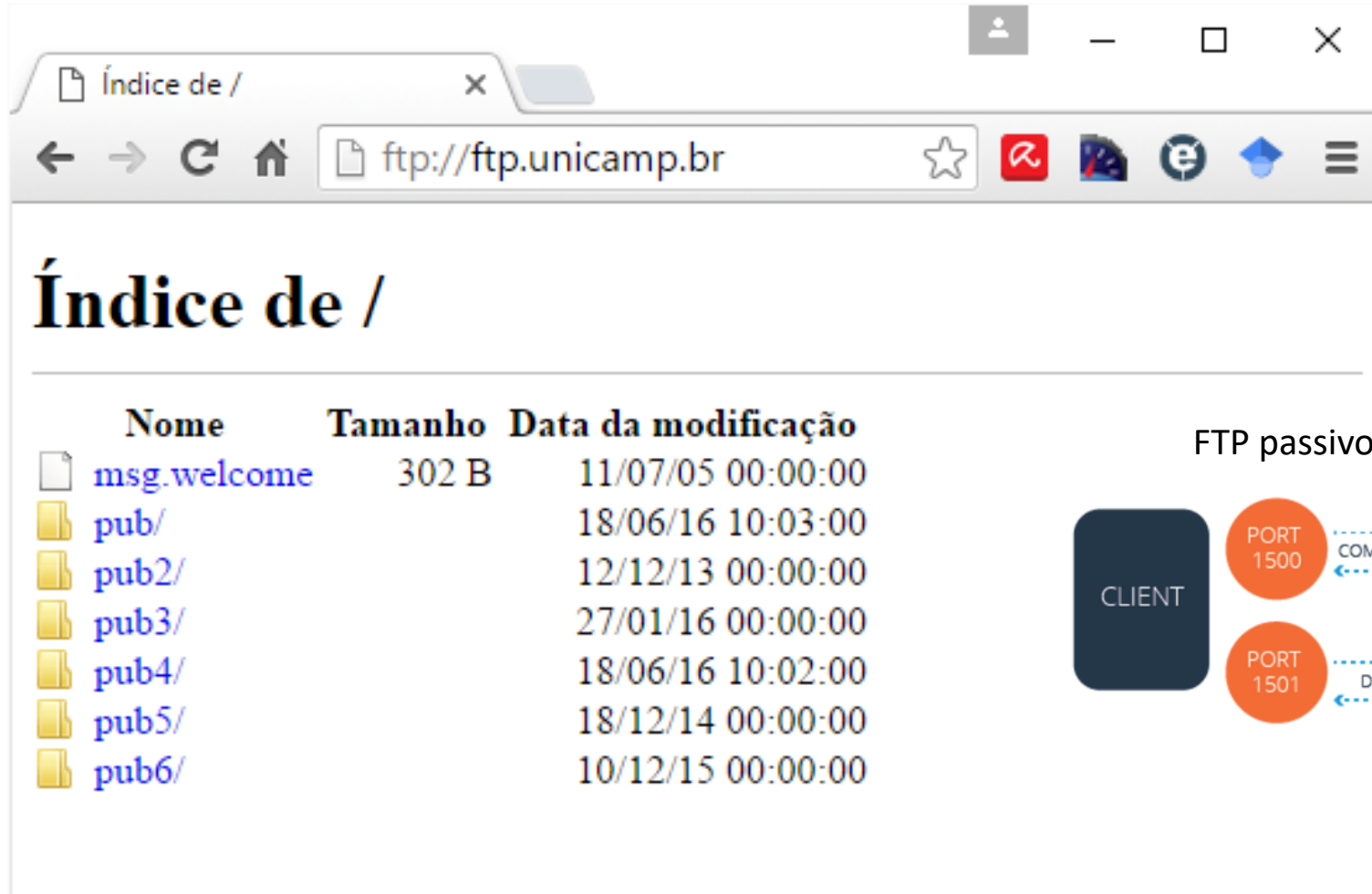
FTP: Modo Passivo

Operação

- Cliente abre o canal de comandos
- Cliente envia o comando “PASV” para o servidor através do canal de comandos
 - Esse comando muda a transmissão para o modo passivo;
 - Servidor responde através do canal de comandos a porta que irá escutar para o canal de dados (PS1);
- Cliente abre o canal de dados para a porta PS1 do servidor

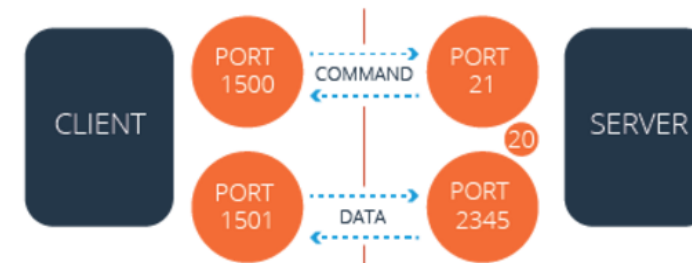


FTP: Modo passivo



Nome	Tamanho	Data da modificação
msg.welcome	302 B	11/07/05 00:00:00
pub/		18/06/16 10:03:00
pub2/		12/12/13 00:00:00
pub3/		27/01/16 00:00:00
pub4/		18/06/16 10:02:00
pub5/		18/12/14 00:00:00
pub6/		10/12/15 00:00:00

FTP passivo (com Firewall)



Características do FTP

Autenticação FTP

- Para usar o FTP é preciso ter uma conta (login) no host onde o servidor está rodando
- Servidores FTP possuem um usuário padrão chamado “anonymous”. Quando habilitado, usuários não precisam ter uma conta naquele host
- Usuário “anonymous” não tem senha: o servidor pede normalmente o e-mail do usuário (mas não checa)

Características do FTP

Desvantagens do FTP

- Passwords e conteúdo de arquivos trafegam na rede de forma “plana” (não criptografados)
- Negociação da “porta de dados” entre cliente e servidor no modo ativo
 - dificulta o trabalho de configuração de firewall
- No modo ativo é possível dizer ao servidor para estabelecer a “conexão de dados” com um terceiro host, indicando IP e porta

FTP Seguro

FTPS

- Uma extensão do FTP que adiciona suporte ao TLS (Transport Layer Security), e anteriormente SSL (proibido pela [RFC7568](#))
 - TLS permite a certificação do servidor e criptografia dos dados
 - Modo implícito: usa conexão segura desde o início
 - Modo explícito: conexão FTP inicia aberta, e o cliente pede explicitamente segurança

FTP Seguro

SFTP (SSH FTP)

- Não é o protocolo FTP seguro!
- Um protocolo de rede que fornece acesso, transferência e gerenciamento de arquivos sobre um canal confiável.
- Um protocolo binário e usa apenas uma conexão (não separa conexão de comando e dados)
- É tecnologicamente superior ao FTPS

Pontos Importantes

FTP

- Finalidades
- Características básicas
- Problemas de segurança

CAP 4. CAMADA DE APLICAÇÃO

AULA 14: NFS

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

NFS (Network File System)

NFS

- Desenvolvido pela Sun Microsystems
 - RFCs: [RFC 1094](#), [RFC 1813](#) e [RFC 3530](#) (que tornou obsoleta a [RFC 3010](#)).
- NFS é um sistema que permite a montagem de sistemas de arquivos remotos através de uma rede TCP-IP
 - Permite que um sistema tenha acesso a arquivos localizados remotamente de um modo integrado e transparente (como se fosse local)
 - NFS fornece a ilusão que os discos, impressoras ou outros dispositivos, fisicamente localizados em um sistema remoto, estão diretamente conectados ao sistema local



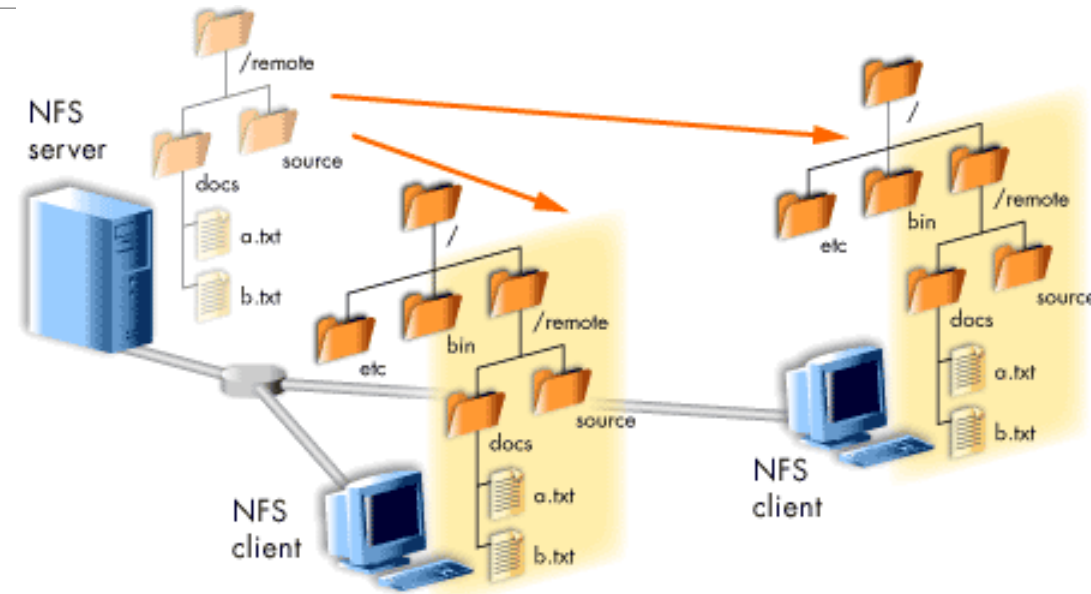
NFS (Network File System)

Servidor NFS

- Um servidor de arquivos NFS determina os recursos locais que serão compartilhados com outras máquinas

Cliente NFS

- Quando solicitado pelo usuário, o cliente faz uma requisição ao servidor que, dependendo das permissões do cliente, responde confirmando a requisição.
- Com NFS a hierarquia de diretórios e arquivos remotos passa a fazer parte do sistema de arquivos local da máquina.

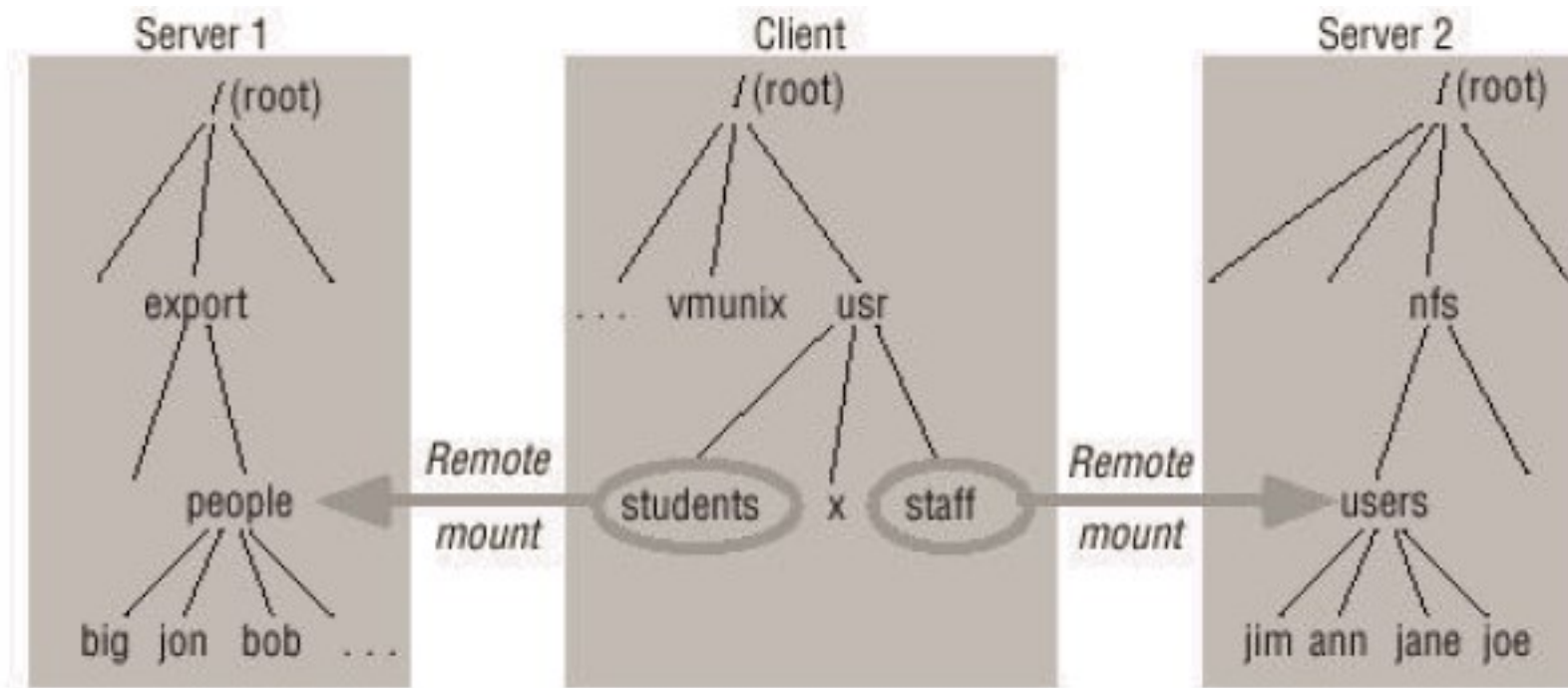


The NFS server exports the /remote filesystem to two NFS clients.

NFS (Network File System)

Transparência de Localização:

- Arquivos remotos podem estar associados a diferentes caminhos em diferentes clientes;



Benefícios do NFS

Arquivos centralizados

- Os arquivos estão localizados no servidor
- Uma cópia do arquivo está disponível a vários usuários simultaneamente

Softwares comuns

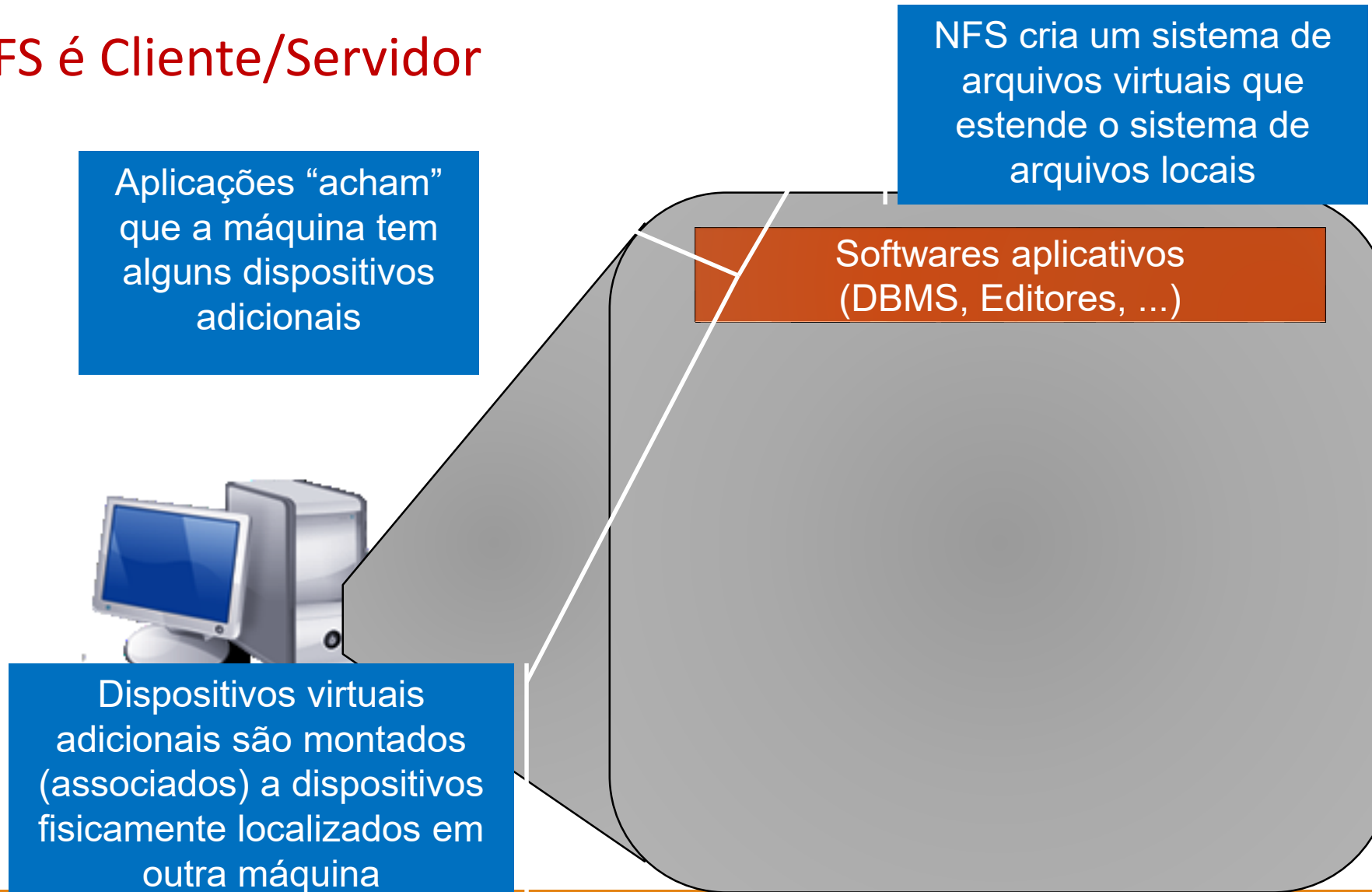
- Pacotes de software podem ser compartilhados
- Diminui o espaço gasto em disco e facilita a gerência.

Os arquivos parecem ser locais

- A distribuição de arquivos é transparente para o usuário e as aplicações

NFS (Network File System)

NFS é Cliente/Servidor



NFS (Network File System)

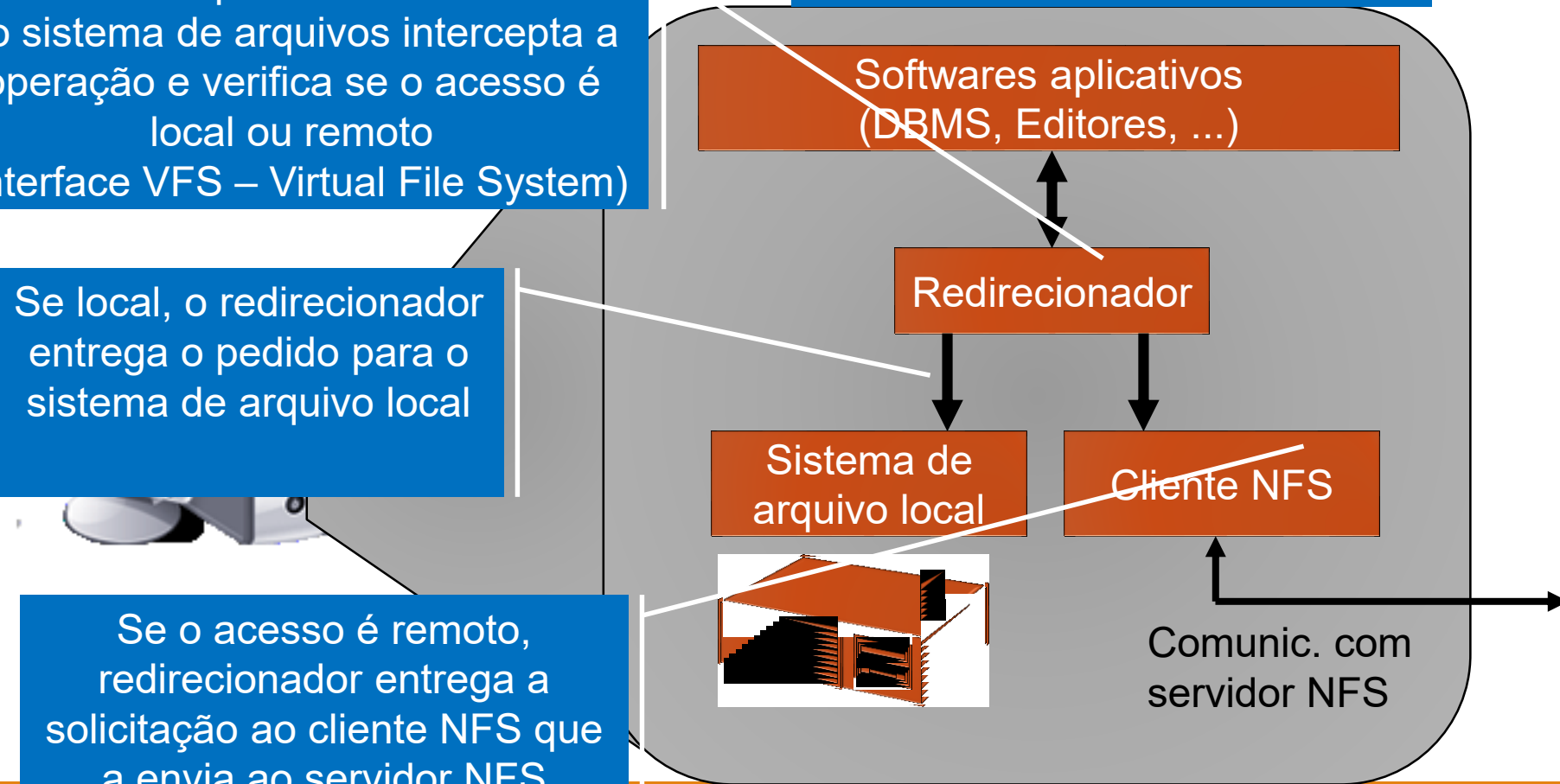
NFS é Cliente/Servidor

Mecanismo que controla o acesso ao sistema de arquivos intercepta a operação e verifica se o acesso é local ou remoto (Interface VFS – Virtual File System)

Se local, o redirecionador entrega o pedido para o sistema de arquivo local

Se o acesso é remoto, redirecionador entrega a solicitação ao cliente NFS que a envia ao servidor NFS

Servidor NFS realiza o pedido e envia resposta ao cliente NFS, que a entrega à aplicação

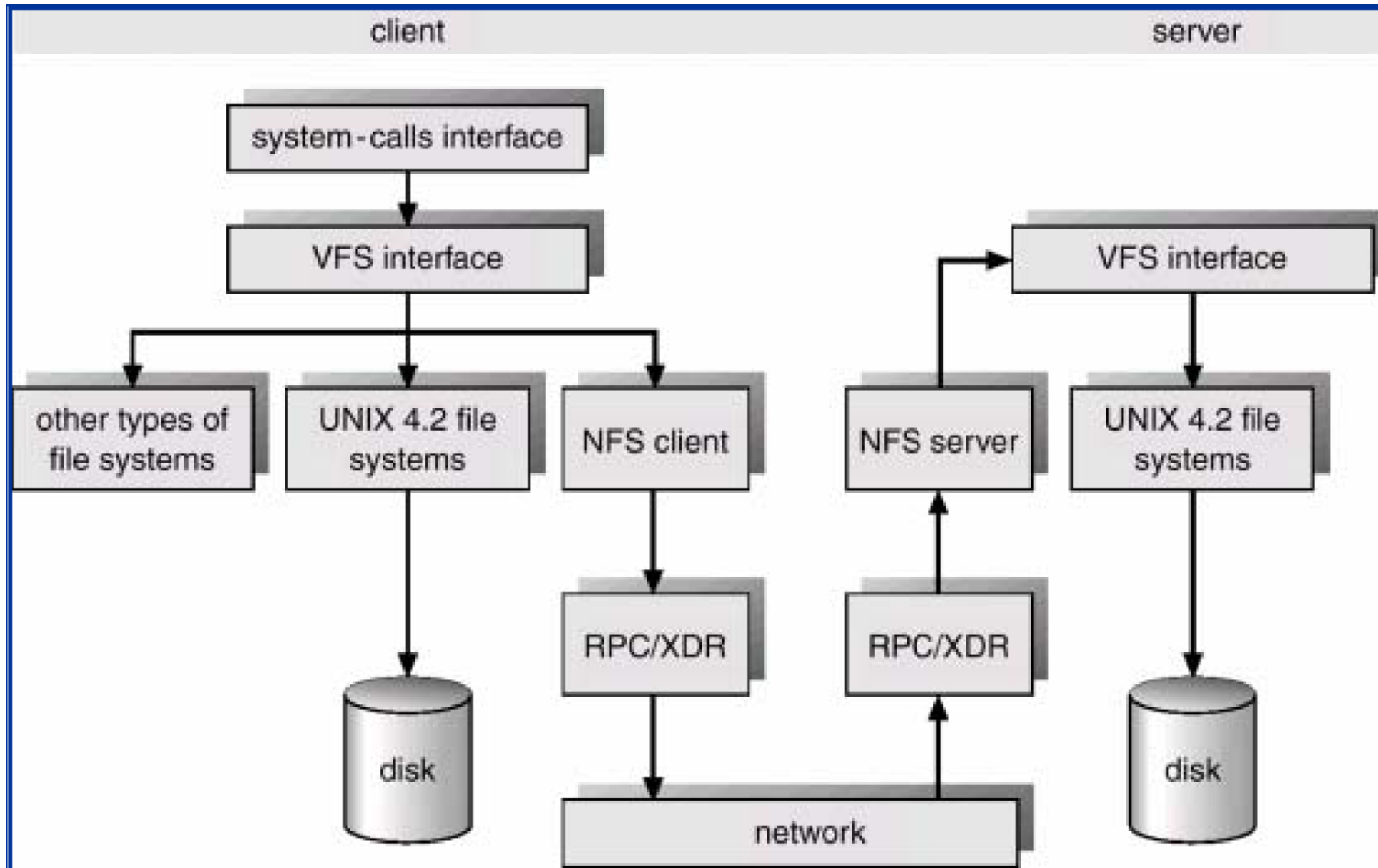


NFS (Network File System)

NFS é dividido em três componentes

- Protocolo NFS
- RPC (Remote Procedure Call)
 - Ferramenta genérica que controla a execução de chamadas a procedimentos remotos
 - Permite o NFS ser utilizado em uma grande variedade de tipos de máquinas, sistemas operacionais e arquiteturas de rede.
- XDR (External Data Representation)
 - Ferramenta que compatibiliza formatos com representação diferentes de dados
 - Permite aos programadores escreverem aplicações distribuídas onde são trocados dados entre máquinas que os representam de forma distinta

NFS (Network File System)



Protocolo NFS [RFCs 1094, 1813,3530]

Protocolo de Montagem dos arquivos

- Montagem automática durante o boot da máquina cliente;
- Automontagem:
 - Nenhum diretório remoto é montado durante o boot;
 - Montagem dinâmica de um sistema de arquivos;

Protocolo para Acesso a Diretórios e a Arquivos:

- Clientes enviam mensagens aos servidores solicitando autorização para ler ou escrever em arquivos;

Protocolo NFS

Proteção dos arquivos

- Utiliza o mecanismo de proteção do UNIX:
 - bits *rwx*;
- Utilização de sistema de criptografia:
 - validação do cliente e do servidor a cada pedido e resposta;
 - Chaves usadas na autenticação: mantidas pelo NIS (*Network Information Service*);

NFS no Ubuntu

Instalação

Em um terminal entre o seguinte comando para instalar o servidor NFS:

```
sudo apt-get install nfs-kernel-server
```

Configuração

Você pode configurar os diretórios a serem exportados adicionando-os ao arquivo `/etc/exports`. Por exemplo:

```
/ubuntu *(ro,sync,no_root_squash)
/home *(rw,sync,no_root_squash)
```

Você pode substituir o `*` com um dos formatos de nome de host. Faça a declaração de nome de host o mais específica possível para que sistemas indesejados não acessem a montagem NFS.

Para iniciar o servidor NFS, você pode executar o seguinte comando em um terminal:

```
sudo service nfs-kernel-server start
```

Configuração do Cliente NFS

Use o comando `mount` para montar um diretório compartilhado NFS de outra máquina, digitando num prompt de terminal um comando similar ao seguinte:

```
sudo mount exemplo.nomedamáquina.com:/ubuntu /local/ubuntu
```



O diretório ponto de montagem `/local/ubuntu` deve existir. Não devem existir arquivos ou subdiretórios dentro do diretório `/local/ubuntu`.

Uma forma alternativa para montar uma partição compartilhada NFS remota é adicionar uma linha ao arquivo `/etc/fstab`. A linha deve conter o nome do servidor NFS host, o diretório no servidor sendo exportado, e o diretório local onde a partição NFS deve ser montado.

A sintaxe normal para a linha no arquivo `/etc/fstab` é a seguinte:

```
exemplo.nomedohost.com:/ubuntu /local/ubuntu nfs rsize=8192,wsiz=8192,timeo=14,intr
```

Se você tem problemas montando um compartilhamento NFS, certifique-se que o pacote `nfs-common` está instalado no cliente. Para instalar o `nfs-common` digite o seguinte comando em um prompt de terminal:

```
sudo apt-get install nfs-common
```


Protocolo NFS

Três versões em uso

- NFSv2
 - Originalmente operado inteiramente sobre UDP
 - Implementadores usavam TCP
 - No NFSv2 e 3: Não guarda informações sobre o estado dos arquivos abertos (stateless);
- NFSv3
 - Suporta tamanhos de arquivo de 64-bits (grandes arquivos)
 - Suporta escrita assíncrona, para melhorar o desempenho
 - Protocolo sem estado (stateless)
 - Utiliza TCP
- NFSv4
 - Melhoras em desempenho e segurança
 - Mantém o estado

Pontos Importantes

NFS

- Finalidades
- Características básicas

CAP 4. CAMADA DE APLICAÇÃO

AULA 15: EMAIL – PROTOCOLO SMTP

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

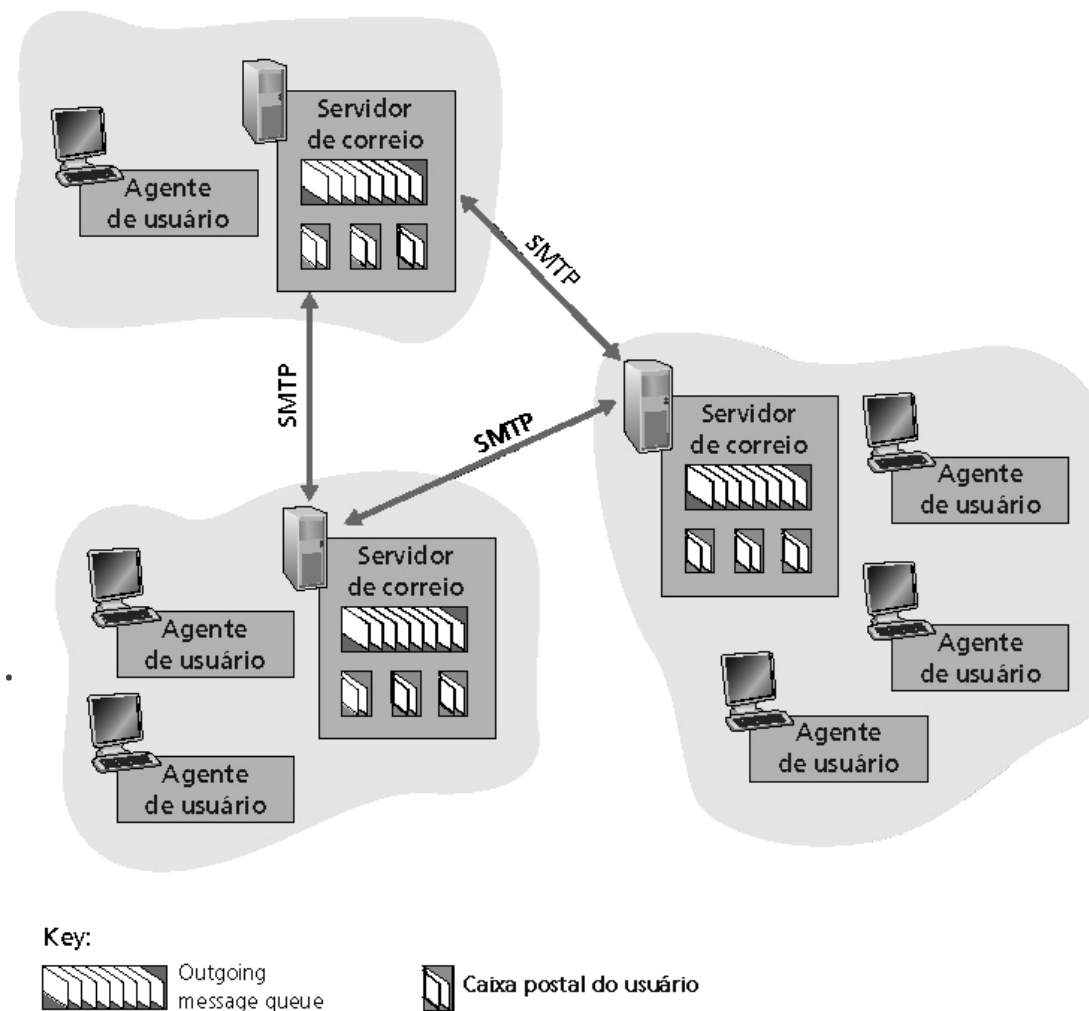
Correio Eletrônico

Três grandes componentes:

- Agentes de usuário (UA)
- Servidores de correio
- Simple Mail Transfer Protocol: SMTP

Agente de Usuário

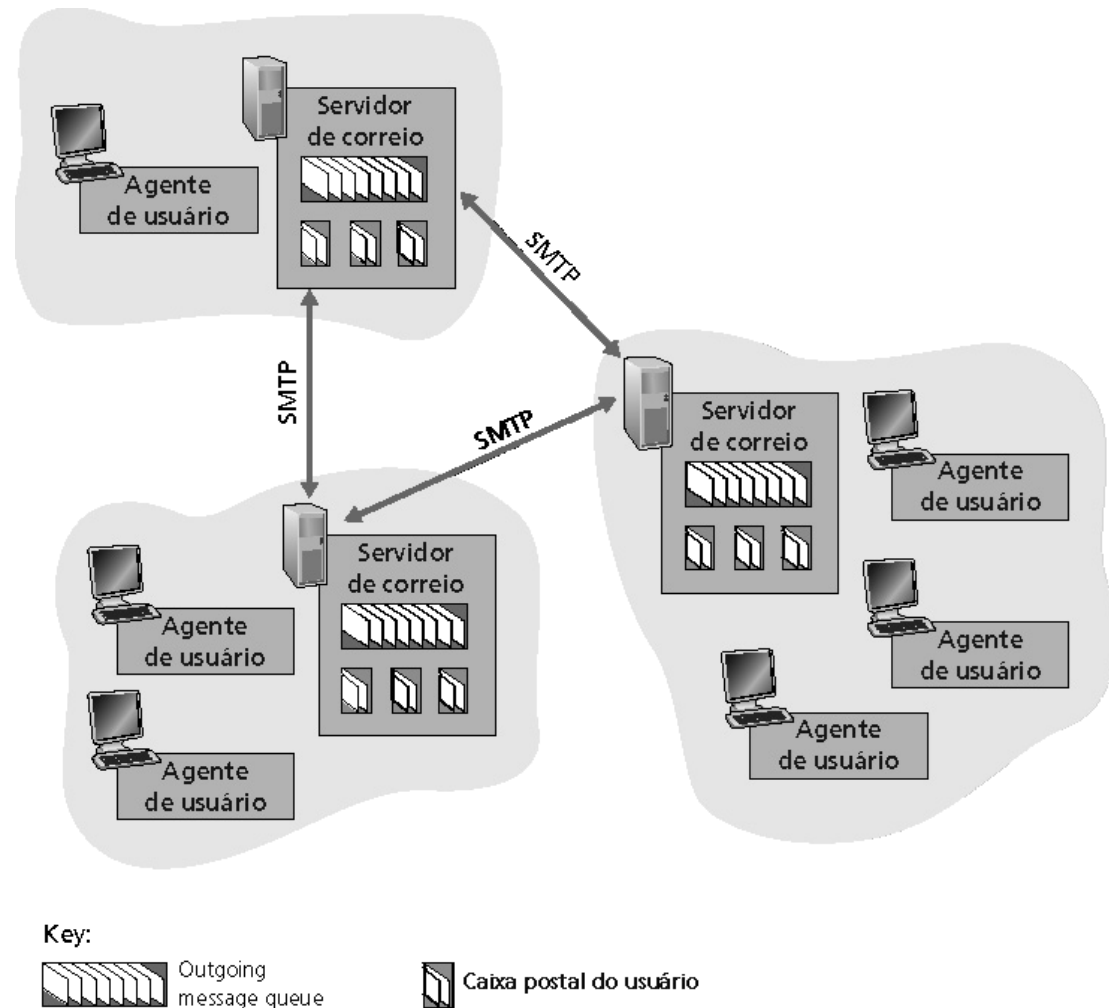
- “Leitor de correio”
- Criar, ler e enviar mensagens de correio
- p.ex., Mozilla Thunderbird, Opera, Outlook, ...



Correio Eletrônico

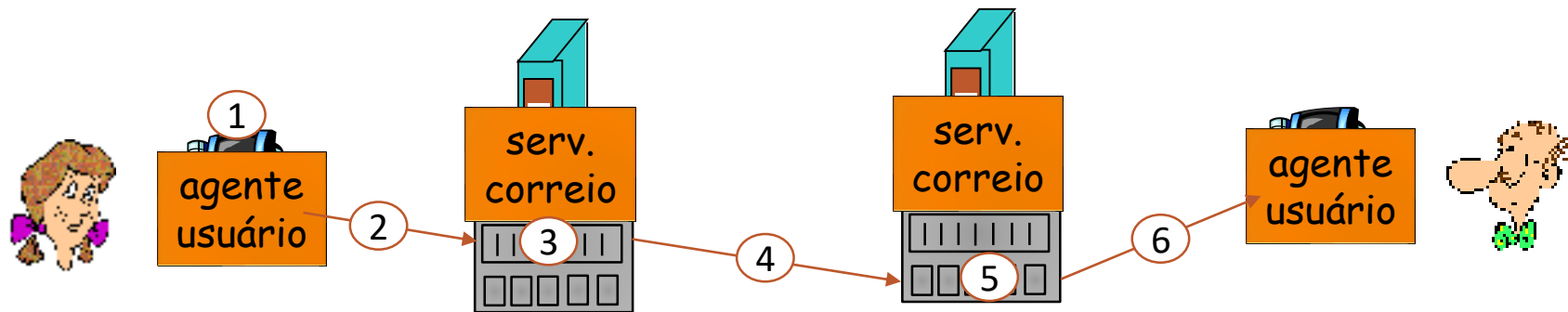
Servidores de Email

- **Mailbox** (caixa de correio) contem mensagem que chegaram para cada usuário
- **Fila de mensagens de saída** (a ser enviada)
- **Protocolo SMTP** entre usado para envio de mensagens entre servidores de email
 - cliente: servidor emissor
 - “servidor”: servidor receptor



Cenário: Alice envia uma msg para Bob

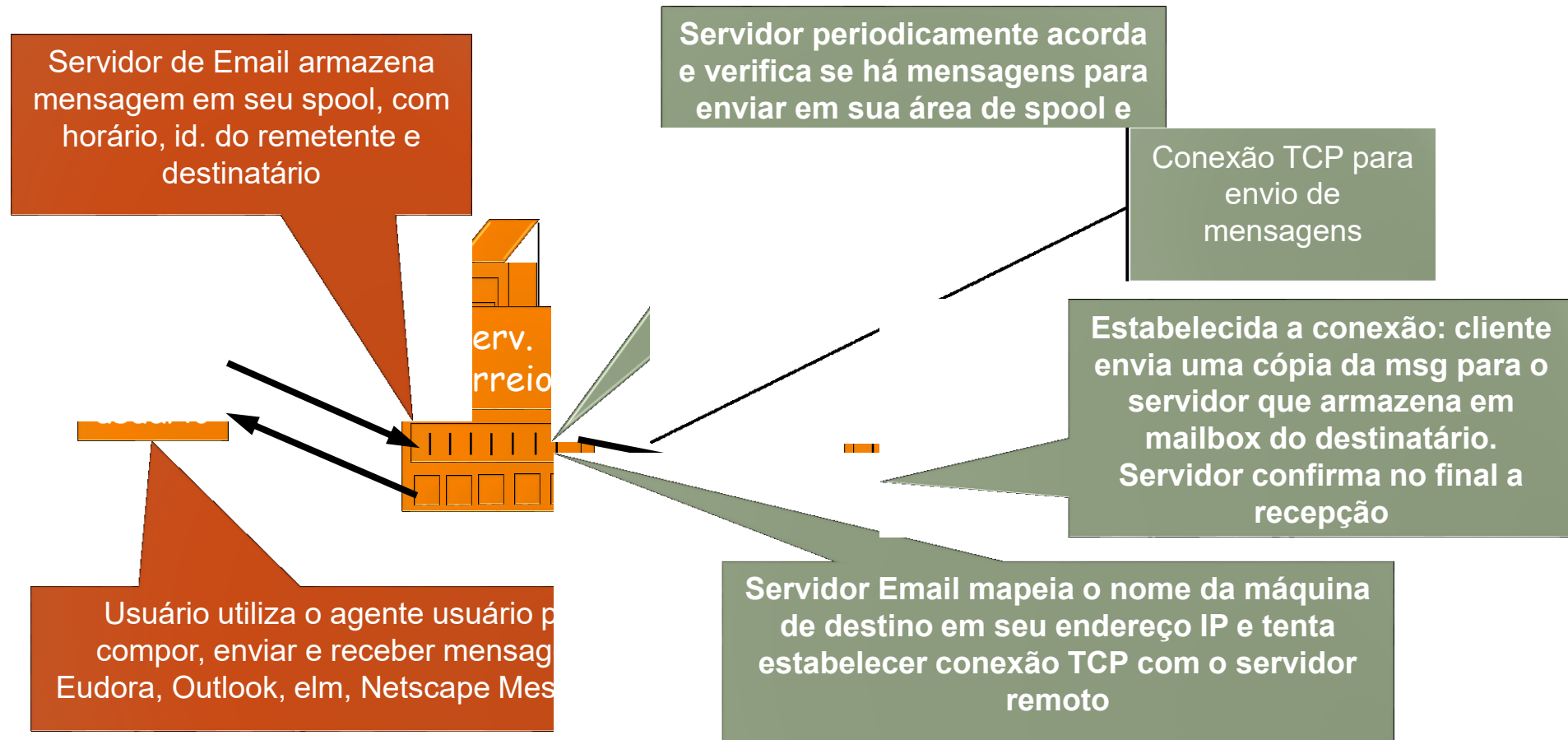
- 1) Alice usa o UA para compor uma mensagem “para” bob@xyz.com.br
- 2) O UA de Alice envia a mensagem para o seu servidor de correio; a mensagem é colocada na fila de mensagens
- 3) O lado cliente do SMTP abre uma conexão TCP com o servidor de correio de Bob
- 4) O cliente SMTP envia a mensagem de Alice através da conexão TCP
- 5) O servidor de correio de Bob coloca a mensagem na caixa de entrada de Bob
- 6) Bob chama o seu UA para ler a mensagem



SMTP (Simple Mail Transfer Protocol)

Protocolo usado no sistema de correio eletrônico na arquitetura TCP/IP

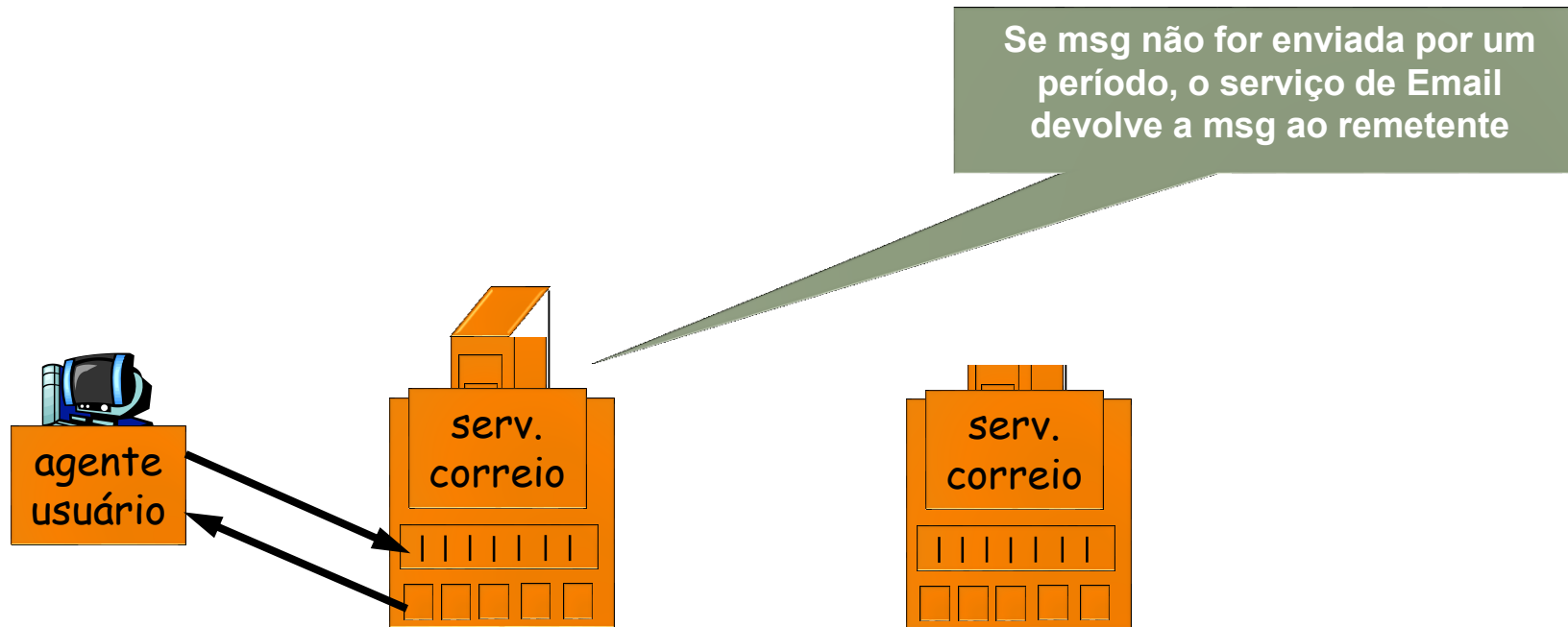
Componentes Essenciais



SMTP (Simple Mail Transfer Protocol)

Protocolo usado no sistema de correio eletrônico na arquitetura TCP/IP

Componentes Essenciais



SMTP e DNS

Registros MX (Mail Exchanger)

- usado para resolver o nome de domínio para o endereço IP do servidor SMTP do receptor.

```
literatura@literaturadigital:~$ dig ufsc.br MX

; <<>> DiG 9.18.18-0ubuntu0.22.04.2-Ubuntu <<>> ufsc.br MX
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 27684
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: fa1f1e5193f5099101000000663e22a808925f98d600a9df (good)
;; QUESTION SECTION:
;ufsc.br.                IN      MX

;; ANSWER SECTION:
ufsc.br.                  42972   IN      MX      10 mx.mail.ufsc.br.

;; ADDITIONAL SECTION:
mx.mail.ufsc.br.          369     IN      A        150.162.2.75

;; Query time: 0 msec
;; SERVER: 150.162.1.33#53(150.162.1.33) (UDP)
;; WHEN: Fri May 10 10:35:36 -03 2024
;; MSG SIZE rcvd: 111
```

Protocolo SMTP [RFC 2821]

Protocolo e porta

- Usa TCP para transferir de modo confiável a mensagem de e-mail do cliente ao servidor, porta 25
- Transferência direta: servidor de envio ao servidor de recepção

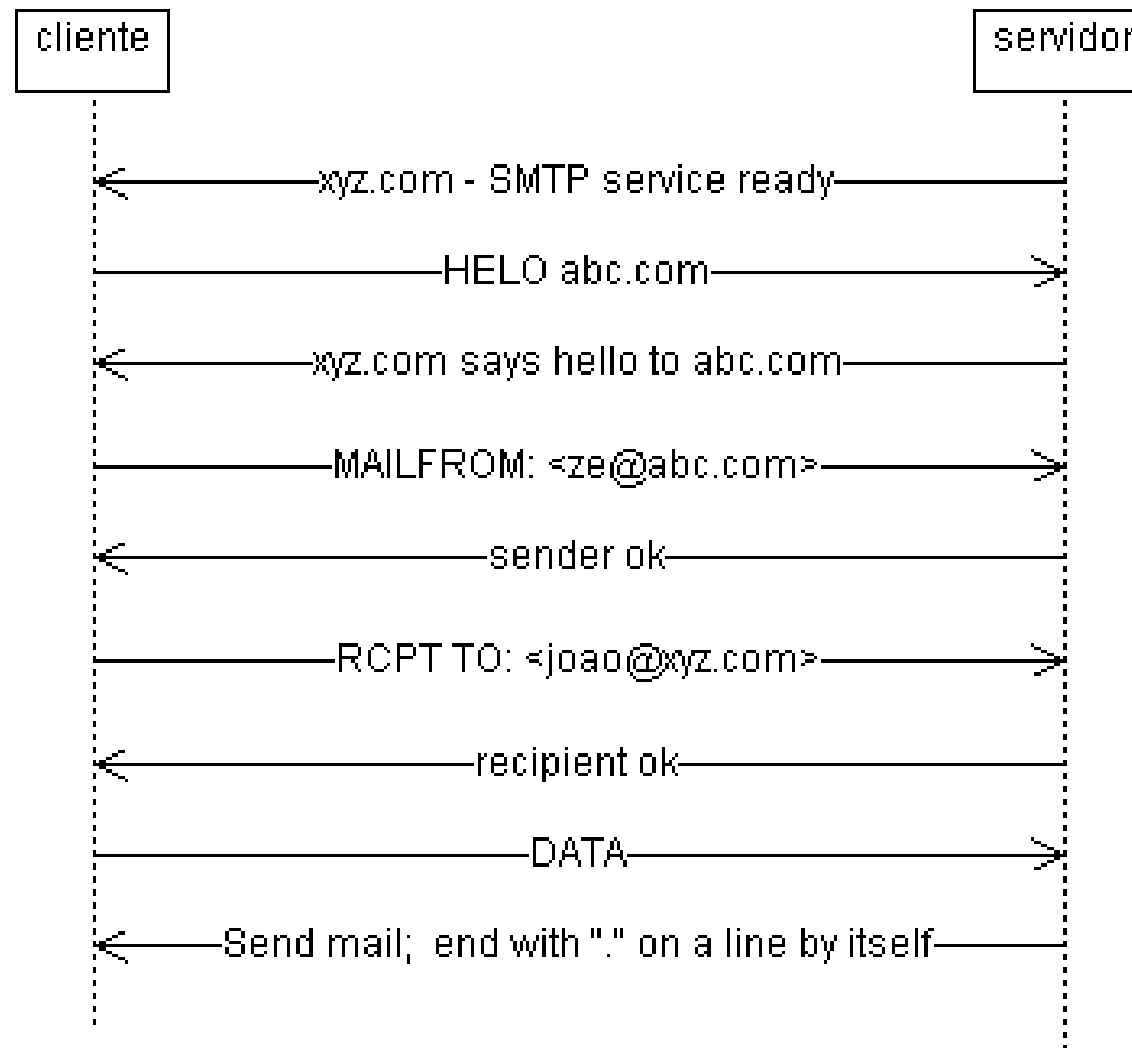
Três fases da transferência

- handshaking (saudação)
- transferência de mensagens
- fechamento

Interação comando/resposta

- comandos: texto ASCII
- resposta: código e frase de estado
- Mensagens devem estar em ASCII

Email: smtp [RFC 821]



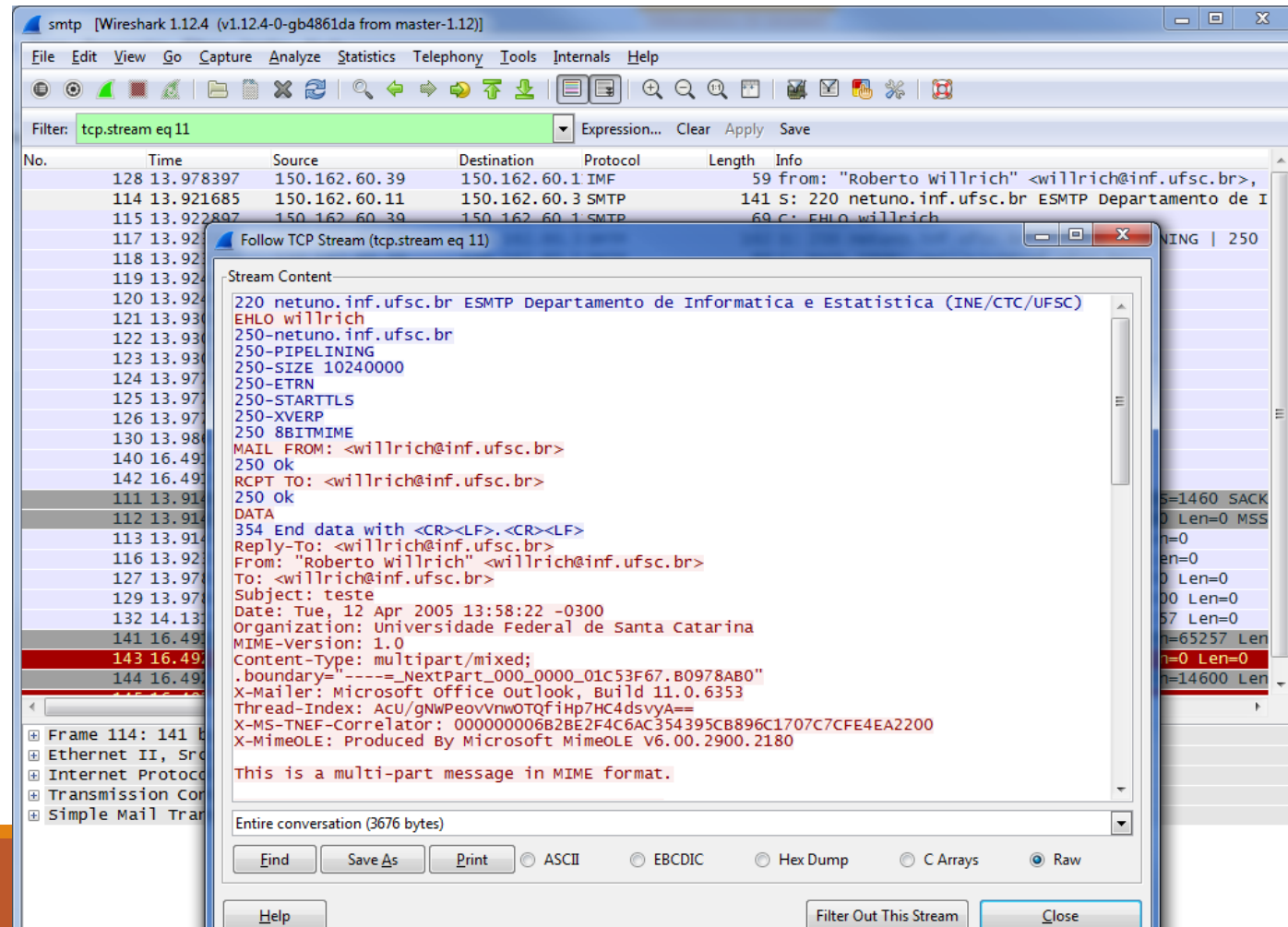
Interação SMTP

```
S: 220 terra.inf.ufsc.br
C: HELO mail.inf.ufsc.br
S: 250 terra.inf.ufsc.br
C: MAIL FROM: <teste@teste.com>
S: 250 ok
C: RCPT TO: <roberto.willrich@gmail.com>
S: 250 ok
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: Como voce vai?
C: Aqui está tudo bem.
C: .
S: 250 Ok: queued as 6E03D1A615
C: QUIT
S: 221 Bye
```

SMTP: Segurança

Usando SMTP na porta 25, originalmente

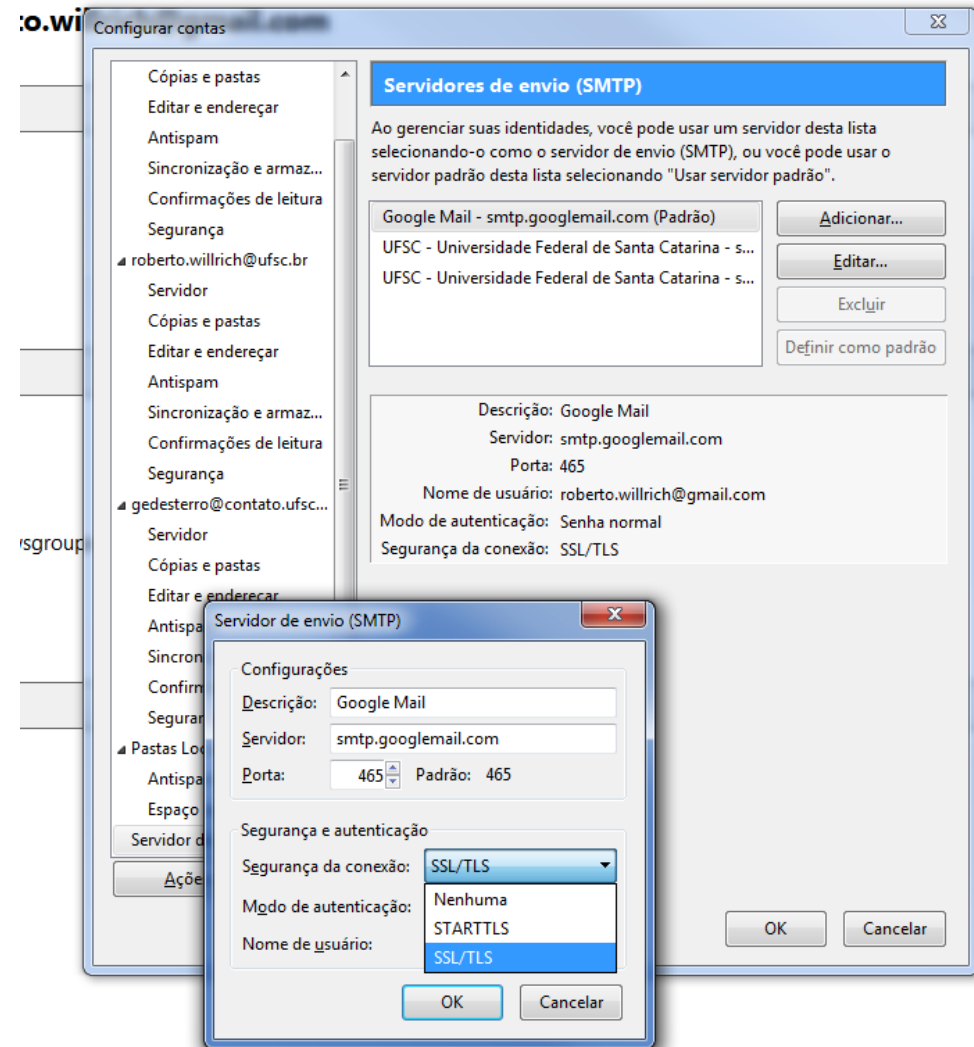
- Dados são enviados sem criptografia
- Cliente podem enviar dados sem autenticação
- Facilita Spam e falsificação!



SMTP: Segurança

Usando uma conexão segura

- STARTTLS
 - Porta 587
- SSL/TLS
 - Porta 465



SMTP: Segurança

Usando uma conexão segura SSL/TLS

- Oferece criptografia entre dois hospedeiros
 - TLS é o sucessor do SSL
- Conexão inicia com a negociação da criptografia antes de qualquer transferência
 - Mensagens SMTP ocorrem após o estabelecimento da conexão segura

Usando uma conexão segura STARTTLS

- Utiliza uma conexão criptografada (TLS ou SSL)
- Conexão inicial não criptografada e SMTP negocia STARTTLS

SMTP: Segurança

Usando uma conexão segura STARTTLS

S: <espera conexão na porta TCP 25>

C: <abre conexão>

S: 220 mail.example.org ESMTP
service ready

C: EHLO client.example.org

S: 250-mail.example.org
uma mensagem de boas vindas

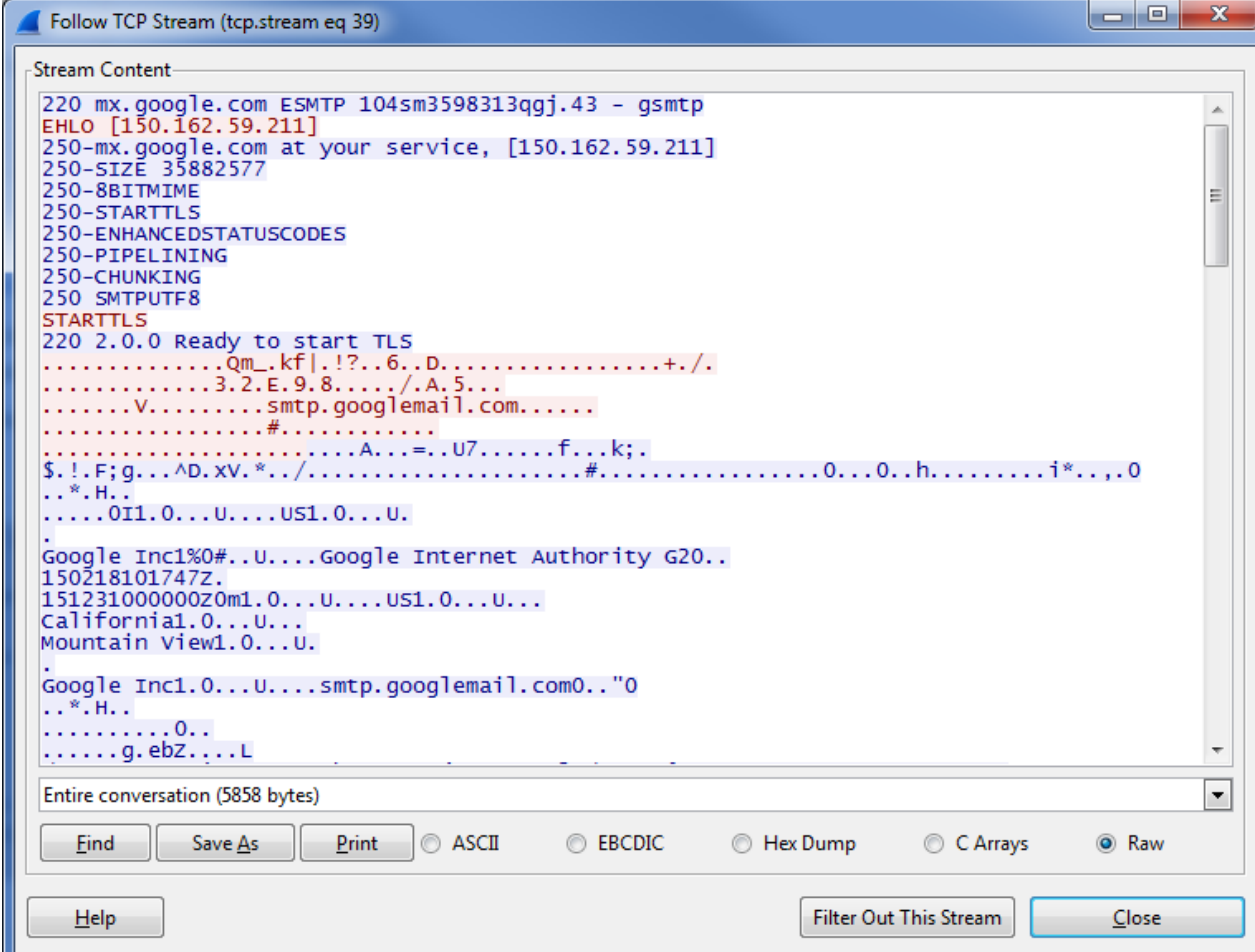
S: 250 STARTTLS

C: STARTTLS

S: 220 Ready to start TLS

C: <inicia negociação TLS >

...



```
220 mx.google.com ESMTP 104sm3598313qgj.43 - gsmt
EHLO [150.162.59.211]
250-mx.google.com at your service, [150.162.59.211]
250-SIZE 35882577
250-8BITMIME
250-STARTTLS
250-ENHANCEDSTATUSCODES
250-PIPELINING
250-CHUNKING
250 SMTPUTF8
STARTTLS
220 2.0.0 Ready to start TLS
.....Qm_kf|.!?..6..D.....+./..
.....3.2.E.9.8...../A.5...
.....V.....smtp.googlemail.com.....
.....#.....
.....A...=..U7.....f...k;.
$.!.F;g...^D.xV.*../.....0...0..h.....i*...0
..*.H..
.....0i1.0...U....US1.0...U.
.
Google Inc1%0#..U....Google Internet Authority G20..
150218101747Z.
151231000000Z0m1.0...U....US1.0...U...
California1.0...U...
Mountain View1.0...U.
.
Google Inc1.0...U....smtp.googlemail.com0.."0
..*.H..
.....0..
.....g.ebZ...L
```


SMTP: Segurança

Extensão SMTP AUTH

- Uma extensão do SMTP onde o cliente SMTP deve se logar usando o mecanismo de autenticação suportado pelo servidor SMTP

S: 220 smtp.example.com ESMTP Server

C: EHLO client.example.com

S: 250-smtp.example.com Hello client.example.com

S: 250-AUTH GSSAPI DIGEST-MD5

S: 250-ENHANCEDSTATUSCODES

S: 250 STARTTLS

C: STARTTLS

S: 220 Ready to start TLS

... Negociação TLS. Comandos a seguir são protegidos pela camada TLS ...

C: EHLO client.example.com

S: 250-smtp.example.com Hello client.example.com

S: 250 AUTH GSSAPI DIGEST-MD5 PLAIN

C: AUTH PLAIN dGVzdAB0ZXN0ADEyMzQ=

S: 235 2.7.0 Authentication successful

Mensagem SMTP

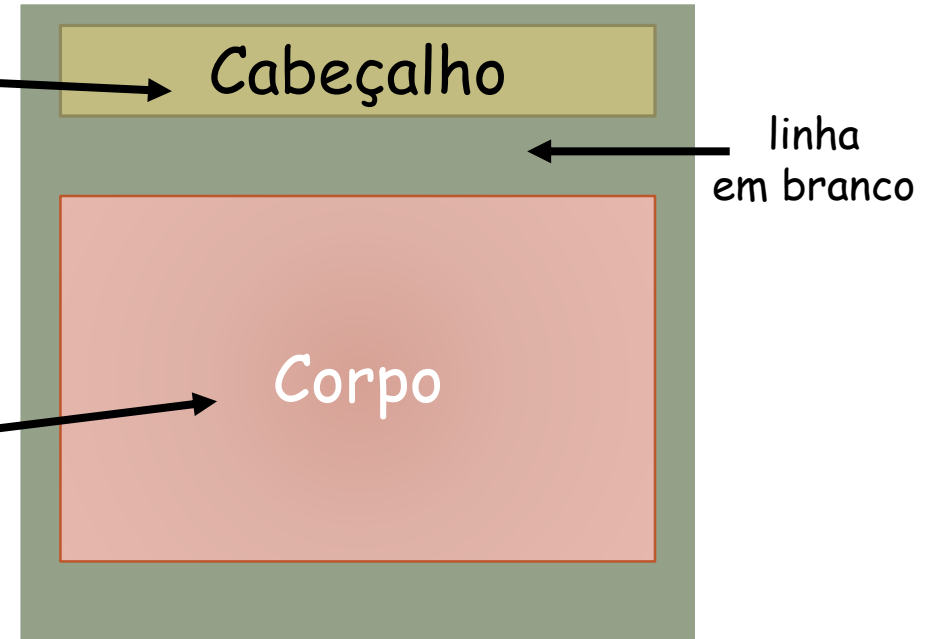
RFC 822: padrão para formato de mensagens de texto:

Linhas do cabeçalho, exemplo:

- To:
- From:
- Subject:

Corpo

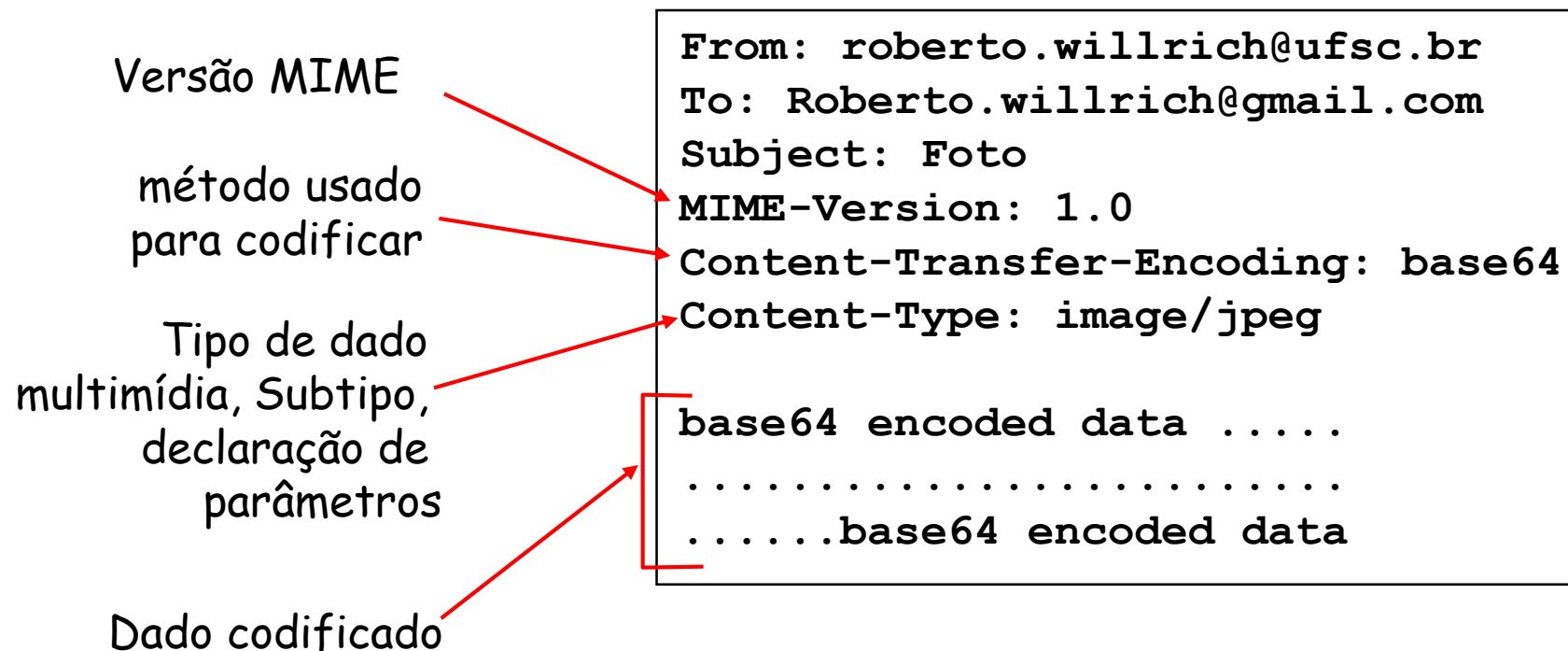
- a “mensagem” no formato texto



Formato de Msg: Extensões Multimídia

Linhas adicionais no cabeçalho declaram o tipo de conteúdo MIME

- MIME: multimedia mail extension, RFC 2045, 2056



Tipos MIME

Type	Subtype	Description
Text	Plain	Unformatted text
	Richtext	Text including simple formatting commands
Image	Gif	Still picture in GIF format
	Jpeg	Still picture in JPEG format
Audio	Basic	Audible sound
Video	Mpeg	Movie in MPEG format
Application	Octet-stream	An uninterpreted byte sequence
	Postscript	A printable document in PostScript
Message	Rfc822	A MIME RFC 822 message
	Partial	Message has been split for transmission
	External-body	Message itself must be fetched over the net
Multipart	Mixed	Independent parts in the specified order
	Alternative	Same message in different formats
	Parallel	Parts must be viewed simultaneously
	Digest	Each part is a complete RFC 822 message

Fig. 7-45. The MIME types and subtypes defined in RFC 1521.

Tipo Multiparte

From: abc-silva@gmail.com
To: roberto.willrich@ufsc.br
Subject: Foto da Casa
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary=98766789

--98766789
Content-Transfer-Encoding: quoted-printable
Content-Type: text/plain

Caro Roberto,
Veja abaixo a foto da casa.
--98766789
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data
.....
.....base64 encoded data
--98766789--

Protocolo SMTP

Palavras finais

- SMTP usa conexões persistentes
- SMTP requer que a mensagem (cabeçalho e corpo) esteja em ASCII
- servidor SMTP usa CRLF.CRLF para determinar fim da mensagem
- Comparação com HTTP:
 - HTTP: puxa
 - SMTP: empurra
 - ambos têm interação de comando/resposta em ASCII, códigos de estado
 - HTTP: cada objeto encapsulado em sua própria mensagem de resposta
 - SMTP: múltiplos objetos enviados na mensagem multiparte

Pontos Importantes

SMTP

- Finalidades
- Características básicas

CAP 4. CAMADA DE APLICAÇÃO

AULA 16: EMAIL –POP, IMAP E WEBMAIL

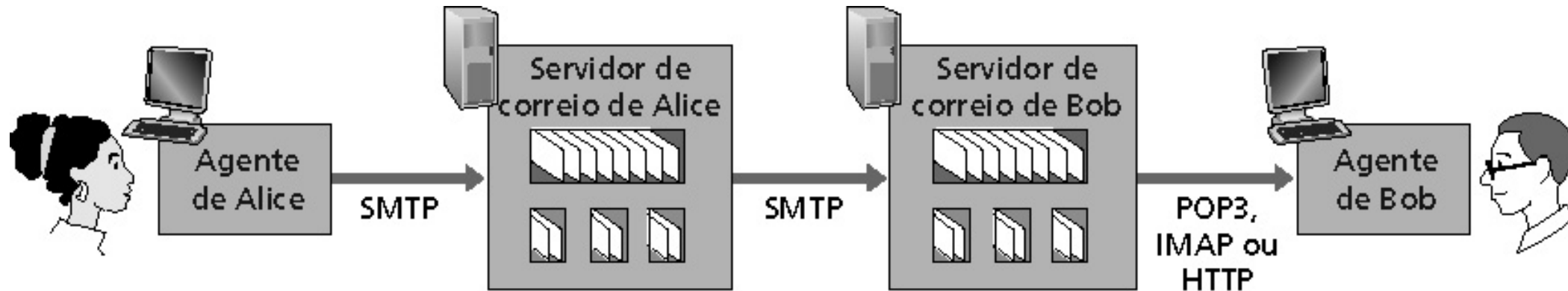
INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

SMTP (Simple Mail Transfer Protocol)



SMTP: envia/armazena msg para servidores

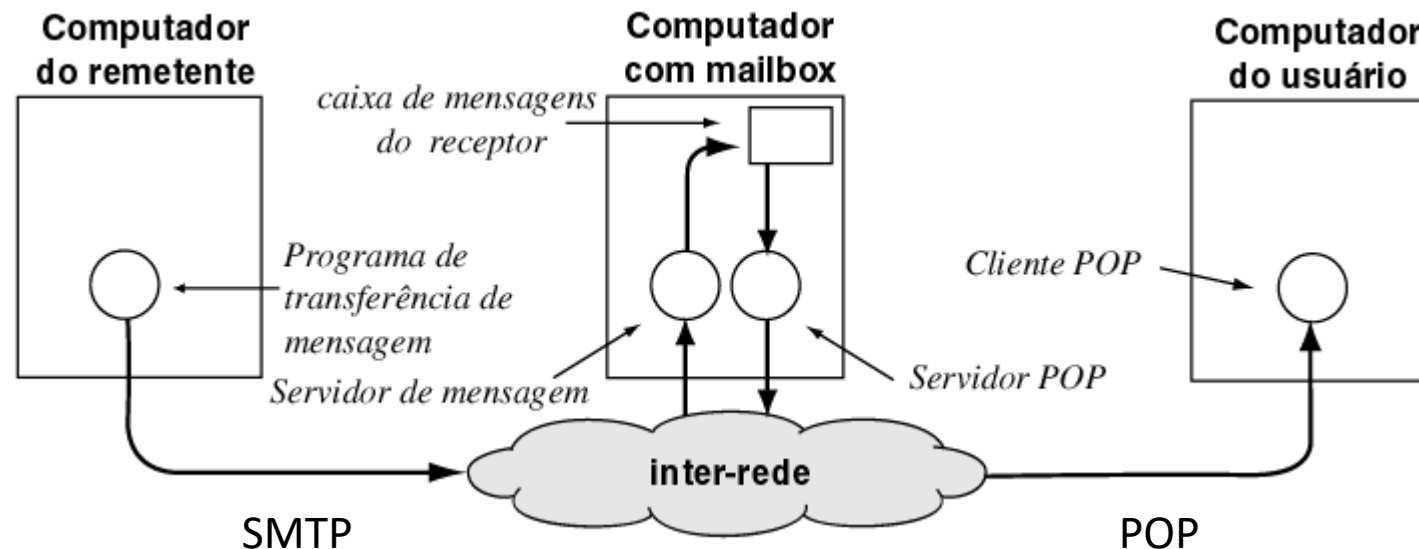
Protocolo de acesso a Mail: obtém msg de servidores

- POP: Post Office Protocol [RFC 1939]
 - autorização (agente <-->servidor) e download
- IMAP: Internet Mail Access Protocol [RFC 1730]
 - Mais características (mais complexo)
 - Manipulação de msgs armazenadas no servidor
- HTTP: Gmail, Hotmail, Yahoo! Mail, etc.

Protocolo POP3

Características

- Um dos protocolos utilizados por leitores de email para buscar mensagens no servidor de email
- Começa quando o agente usuário (cliente) abre uma conexão TCP com o servidor de Email (servidor) na porta 110
- Quando a conexão é estabelecida, POP3 prossegue em 3 fases
 - Autenticação, transação, atualização



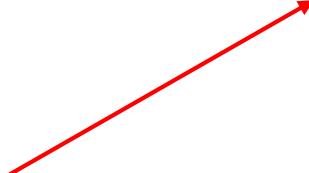
Protocolo POP3

Fase autorização

- Comandos do cliente:
 - user: declara username
 - pass: password
- Respostas do servidor
 - +OK
 - -ERR
- Fase de transação, cliente:
 - list: lista de números de msgs
 - retr: obtém msg pelo número
 - dele: apagar
- quit



```
S: +OK POP3 server ready
C: user alice
S: +OK
C: pass hungry
S: +OK user successfully logged on
```



```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

Protocolo POP3

Dois modos de operação

- **download-e-apaga**: Mensagens são transferidas para o computador do cliente e apagadas do servidor
 - Ruim para usuários nômades: não vai ter acesso aos emails já transferidos para o cliente
 - Após o comando quit o servidor entra na fase de atualização e remove mensagens apagadas do mailbox
 - Durante a seção POP3 o servidor mantém algumas informações de estado
 - Mantém quais mensagens foram marcadas como apagadas
- **download-e-manter**: agente usuário deixa as mensagens no servidor
 - Usuário pode reler seus emails de outros documentos

Protocolo POP3

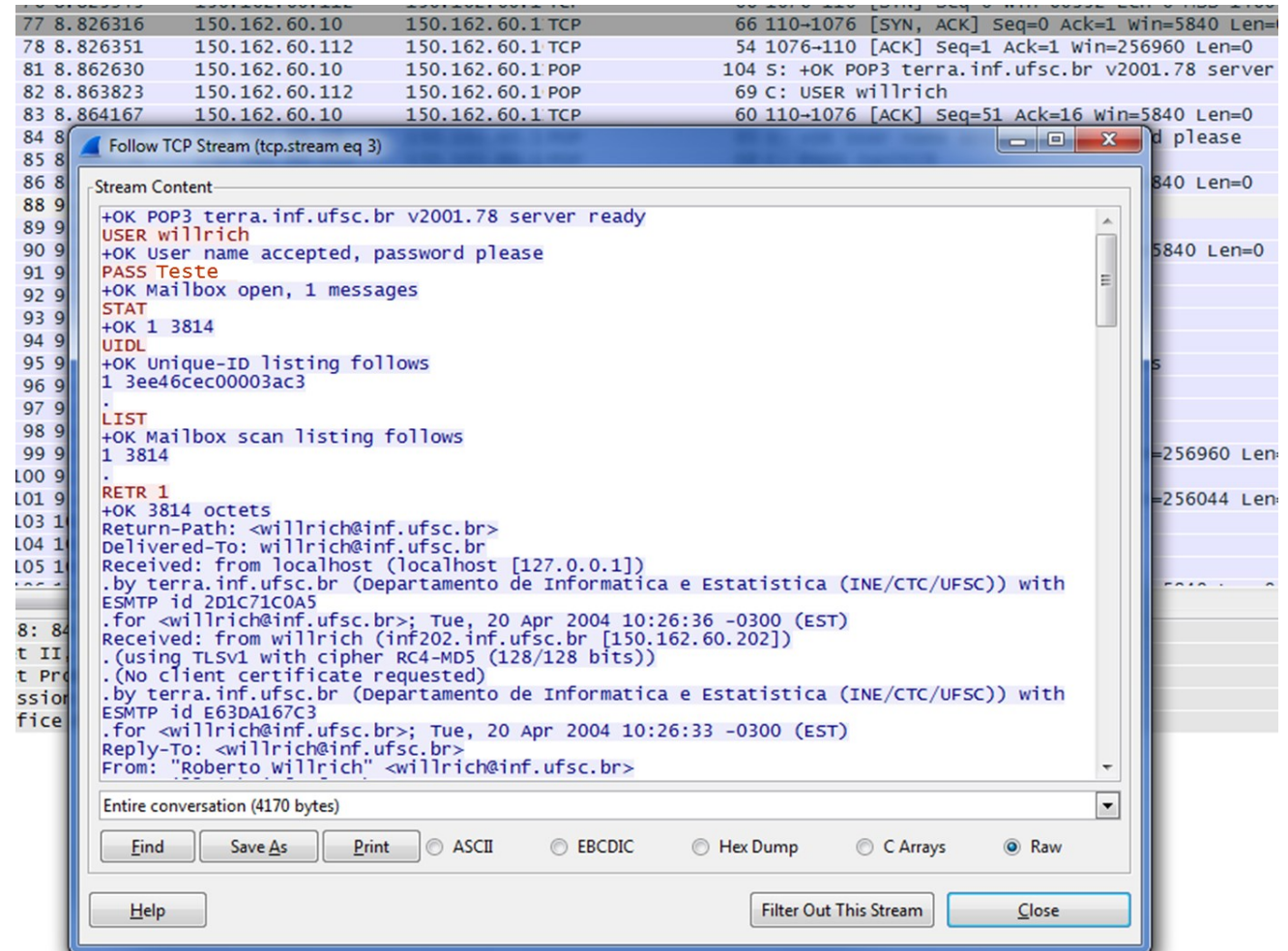
Características

- Mensagens são transferidas do servidor para o computador local quando o usuário se conecta ao servidor
- Após buscar as mensagens a conexão pode ser desfeita, procedendo-se à leitura das mensagens sem precisar estar conectado ao servidor
- Indicado no caso de se utilizar conexões de acesso discado (via linha telefônica convencional onde se paga impulsos em função do tempo de conexão).
- Servidor POP3 não transfere estados para outras seções POP3
 - Simplifica a implementação

Segurança em POP3

Usando POP na porta 110

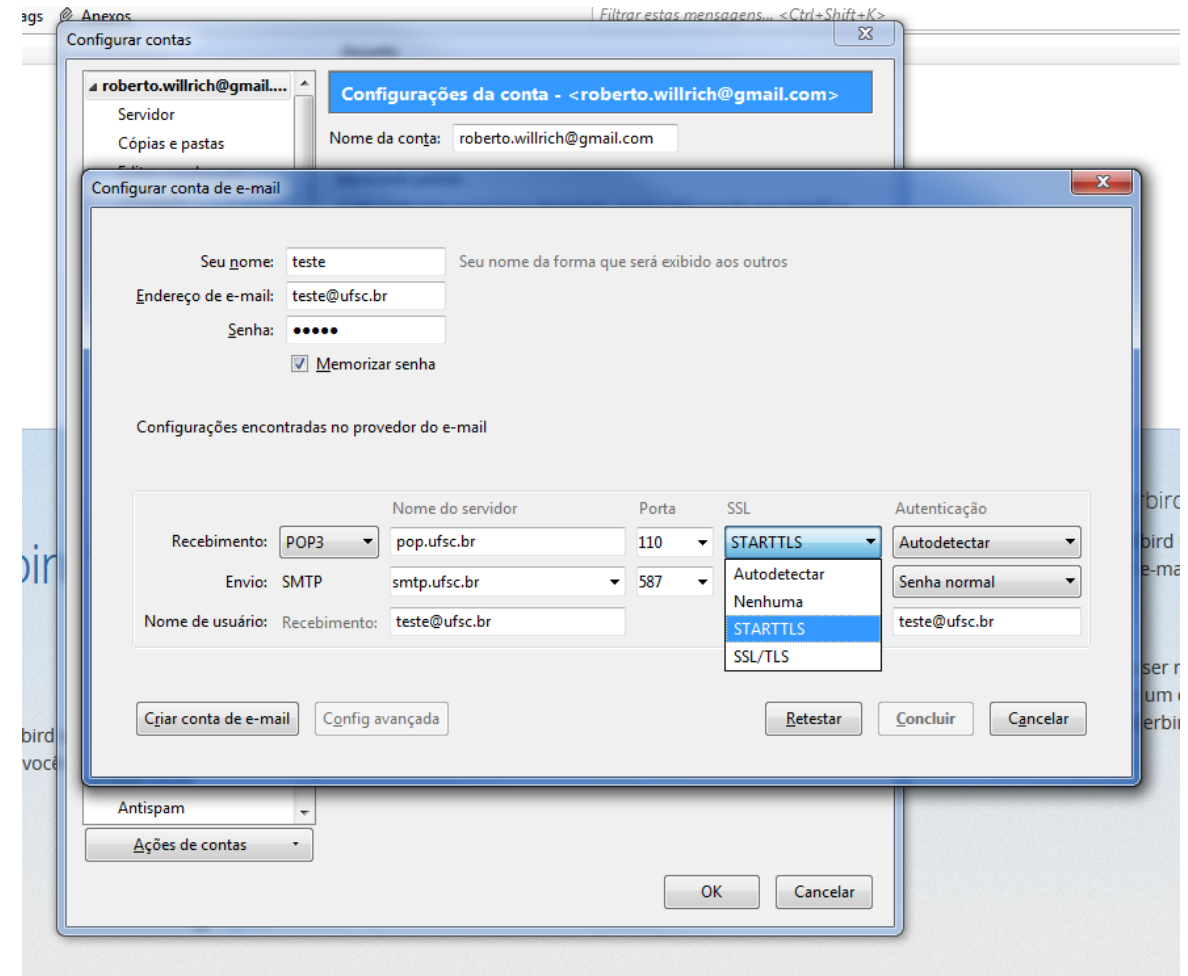
- Tráfego não é criptografado
- Facilitando captura do usuário e senha.
- Não garante confidencialidade



Segurança em POP3

Aumentando a segurança do POP

- Uso do STARTTLS, que pode usar a porta 110
- Uso do SSL/TLS, usa a porta 995 (POP3S)



Protocolo IMAP

Útil para usuários nômades

- Permite ao usuário manipular mailbox remoto como se ele fosse local
- Permite ao usuário criar e manter várias pastas no servidor de email
 - Usuário pode transferir mensagens de uma pasta para outra
- Fornece funcionalidades de busca de mensagens

Informações de estado persistem para as conexões que sucedem

Mais complexo que o POP3

- Implementações de cliente e servidores mais complexas

Protocolo IMAP

Sessão IMAP

- Estabelecimento da conexão TCP entre cliente e servidor IMAP
- Um contato inicial entre servidor e cliente
- Interações entre o cliente e servidor
 - Similar ao POP3 (mas bem mais rica)

Estados do servidor IMAP

- nonauthenticated state, estado inicial quando a conexão começa, e o usuário deve fornecer um nome de usuário e senha
- authenticated state, o usuário deve selecionar uma pasta
- selected state, o usuário pode emitir comandos que afetam mensagens (obter, mover, apagar, obter uma parte de uma mensagem multiparte, etc).
- logout state, no termino da seção

Protocolo IMAP

Útil para usuários nômades

- conexão entre o computador local e o servidor de email deve estar sempre ativa pois há uma constante interação entre eles
 - mensagens são mantidas do servidor de email, mas acessadas como se estivessem localmente.
 - útil para pessoas que lêem seus emails de diferentes computadores

Segurança do IMAP

Usando a porta 143

- Originalmente sem segurança (sem Criptografia)

Aumento da segurança com o uso de conexão criptografada

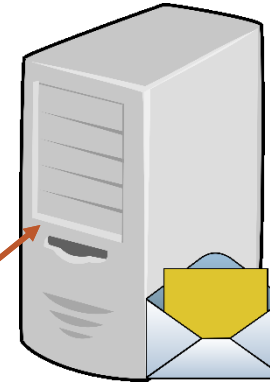
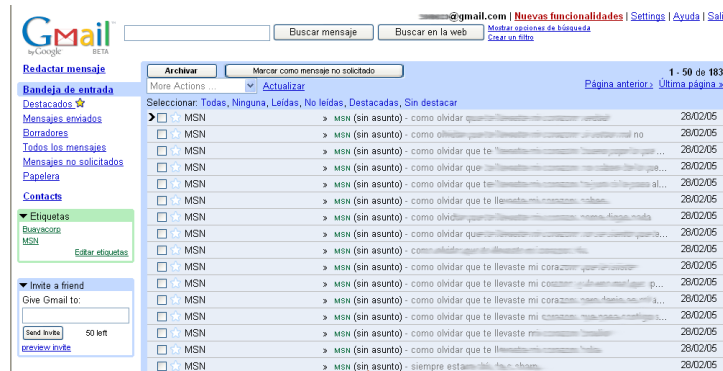
- STARTTLS, usando a porta 143
- SSL/TLS, usando a porta 993

WebMail

Muitos usuários utilizam serviços de email baseado em browser

- Usuário agente é um browser
 - Exemplos: Gmail, Hotmail, Yahoo!
- Usuário se comunica com seu mailbox no seu servidor de email via HTTP
 - Não com SMTP, POP ou IMAP
- Como no IMAP
 - Usuários podem organizar suas mensagens em hierarquias de folder no servidor remoto
- HTTP substitui o POP e o IMAP
 - Do ponto de vista do Agente de Usuário
 - POP/IMAP são úteis para importação de e-mails em outros agentes usuários e servidores

Comparação: HTTP, SMTP, POP, IMAP



SMTP: envio do email para destinatários

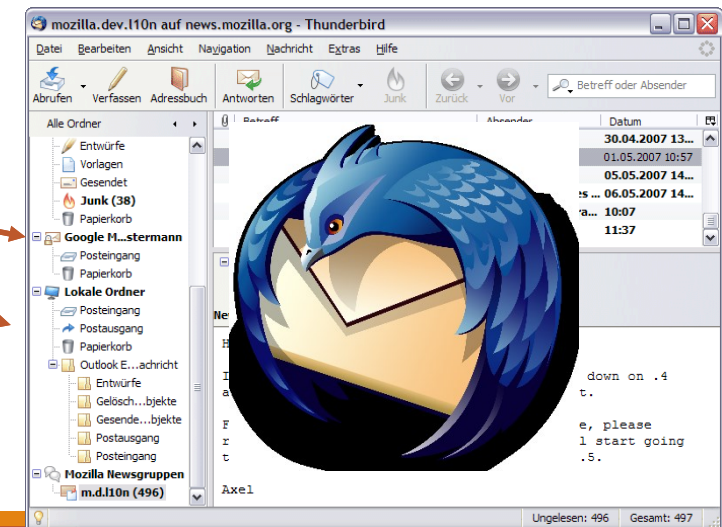
HTTP: leitura, organização e envio de emails



SMTP

POP/IMAP

Uso de outro agente usuário para acesso à solução WebMail



Pontos Importantes

POP e IMAP

- Finalidades
- Características básicas

Webmail

- Entender relação com SMTP, POP e IMAP

CAP 4. CAMADA DE APLICAÇÃO

AULA 17: PEER-TO-PEER (P2P) – DEFINIÇÃO E REDE SOBREPOSTA

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

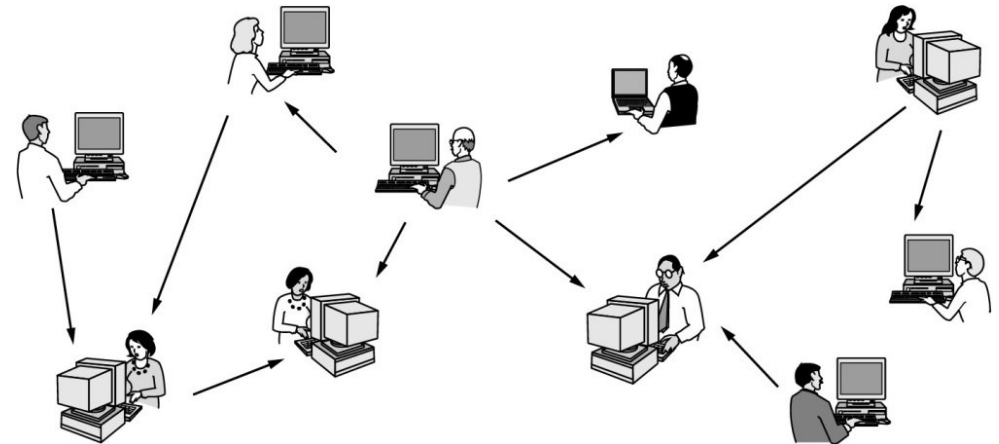
ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Peer-to-peer: Paradigma

Tecnologia peer-to-peer (P2P)

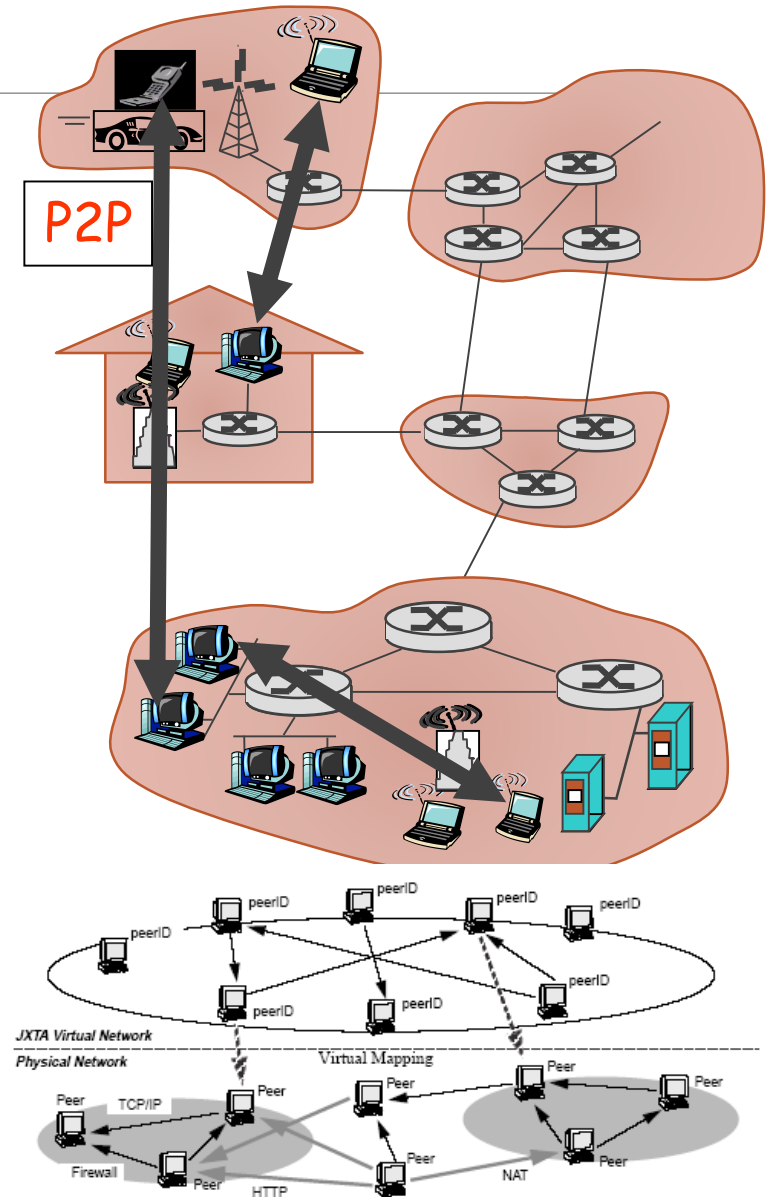
- Surgiu para mudar o paradigma existente (Modelo Cliente/Servidor)
- Baseiam-se na capacidade de processamento e na vazão de rede dos participantes da rede ao invés de se concentrar em um pequeno número de servidores
- Rede P2P pura:
 - São sistemas totalmente distribuídos
 - Não depende de uma organização central ou hierárquica
 - Nos quais todos os nós têm as mesmas funcionalidades e tarefas
 - Qualquer dispositivo pode acessar diretamente os recursos de outro, sem nenhum controle centralizado



Peer-to-peer: Definição

Definições

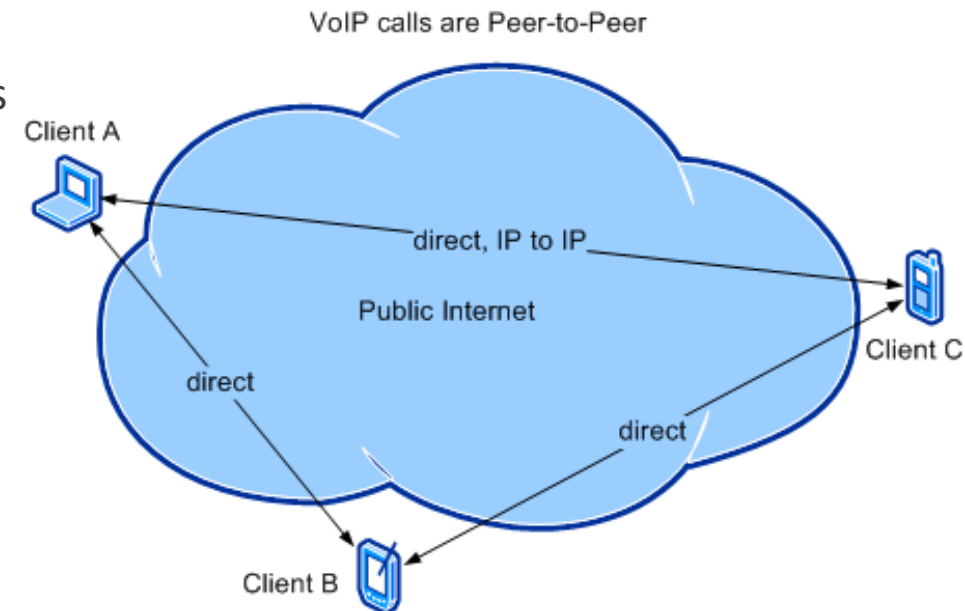
- IETF: “o compartilhamento de recursos e serviços computacionais diretamente entre sistemas”
- São sistemas distribuídos
 - formados por nodos interconectados capazes de se auto organizar em redes sobrepostas,
 - mantendo conectividade aceitável e desempenho mediante população transiente de nodos,
 - sem necessitar da intermediação ou apoio de uma entidade central
- Objetivos
 - Compartilhar recursos como conteúdo, ciclos de CPU, armazenamento e largura de banda,



Peer-to-peer: Definição

Não há consenso

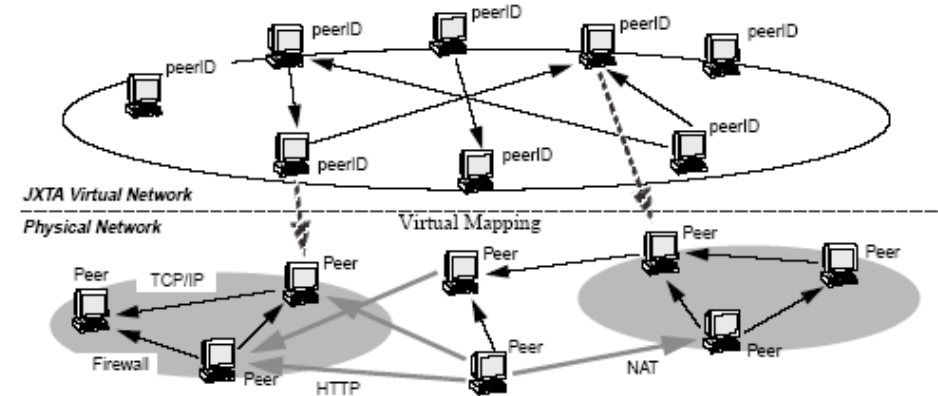
- Existem muitas arquiteturas diferentes para sistemas P2P
 - Diversas arquiteturas impuras: algumas utilizam supernós outras utilizam servidores centralizados para executar certas funções.
- Muitos sistemas são chamados de P2P pela forma como são percebidos pelos usuários e não pela sua arquitetura.
 - Exemplo: Voz sobre IP em sistemas convencionais (SIP)
 - Os usuários realizam chamadas entre si utilizando servidores
 - Após a sessão estabelecida a comunicação (áudio) é entre os pares



Peer-to-peer: Requisitos

Requisitos

- Nós podem estar localizados na borda das redes;
- Capacidade dos nós se comunicarem diretamente uns com os outros.
- Nós com conectividade variável ou temporária e endereços também variáveis (dinâmicos);
- Capacidade de lidar com diferentes taxas de transmissão entre nós;
- Nós com autonomia parcial ou total em relação a um servidor centralizado;
 - Natureza distribuída da rede P2P garante robustez no caso de falhas devido a replicação dos dados através de múltiplos pares
 - Em sistemas P2P puros, pela capacidade dos pares de localizar dados sem necessitar de um servidor de indexação centralizado, evita existência de pontos de falha únicos no sistema
- Assegurar que os nós possuem capacidades iguais de fornecer e consumir recursos de seus peers;
- Rede deve ser escalável;



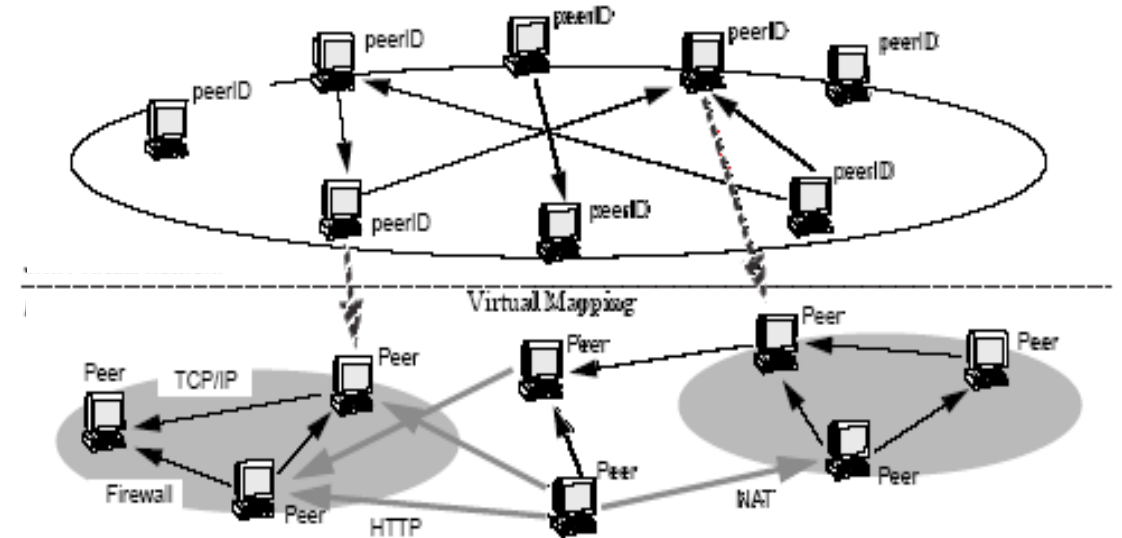
Peer-to-peer: Rede Sobreposta

Pares formam uma rede sobreposta à rede IP auto-organizada

- Rede sobreposta é uma rede construída sobre outra rede
- Nós da rede sobreposta podem ser vistos como sendo conectados através de conexões virtuais
 - cada conexão corresponde a um caminho através de um número de conexões físicas na rede subjacente

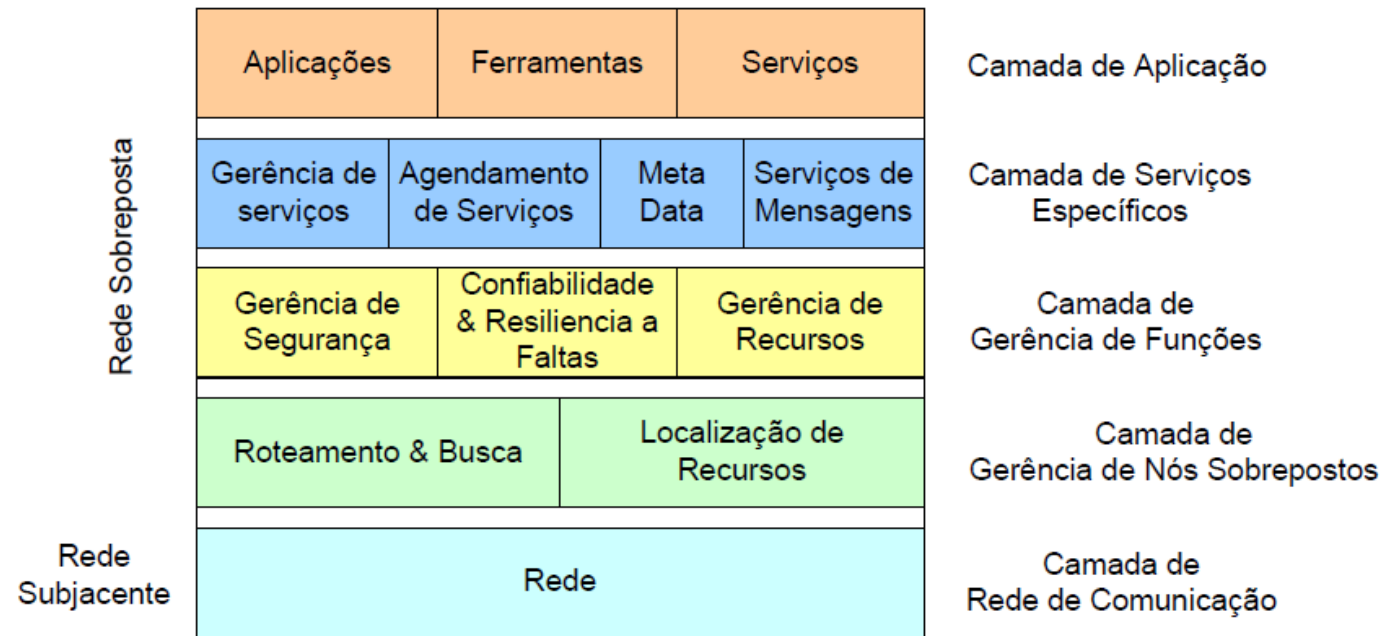
Rede sobreposta oferece funcionalidades aos seus componentes:

- arquitetura de roteamento WAN robusta,
- busca eficiente de dados,
- seleção de pares próximos,
- armazenamento redundante,
- durabilidade dos dados,
- autenticação,
- anonimato,
- escalabilidade e tolerância à faltas.



Peer-to-peer: Rede Sobreposta

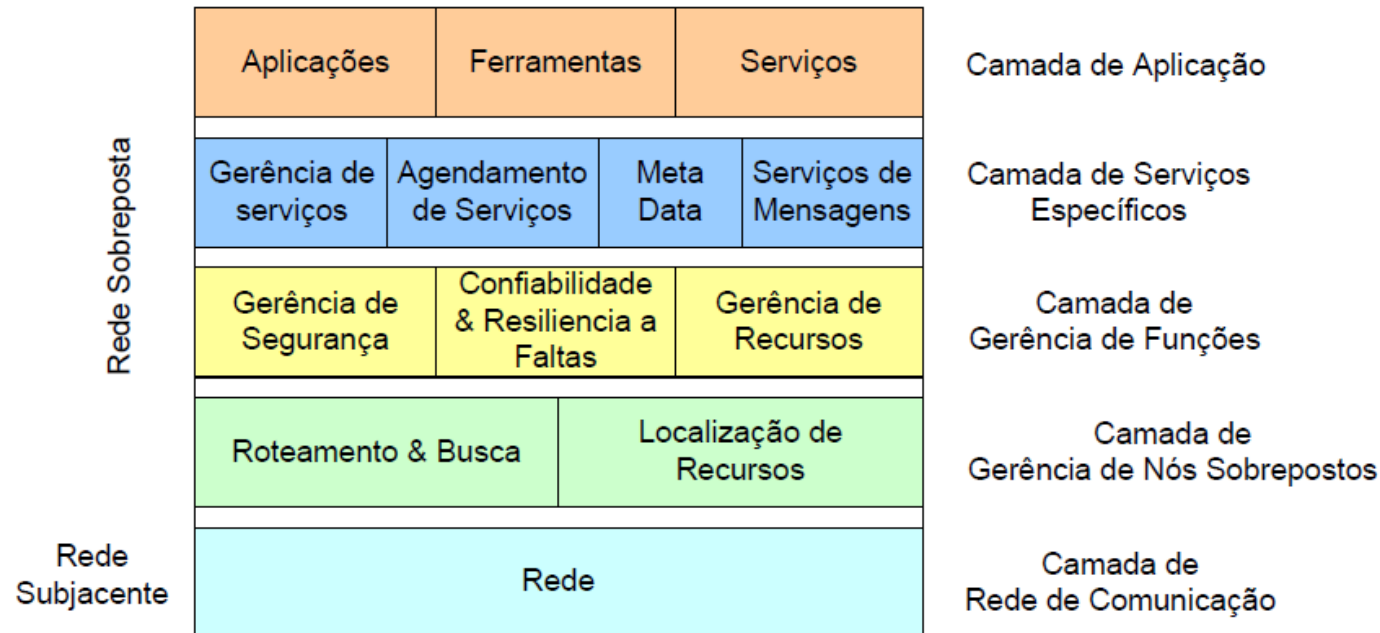
Arquitetura de redes P2P sobreposta



- **Camada de rede de comunicação:** a rede que provê a comunicação para a rede sobreposta

Peer-to-peer: Rede Sobreposta

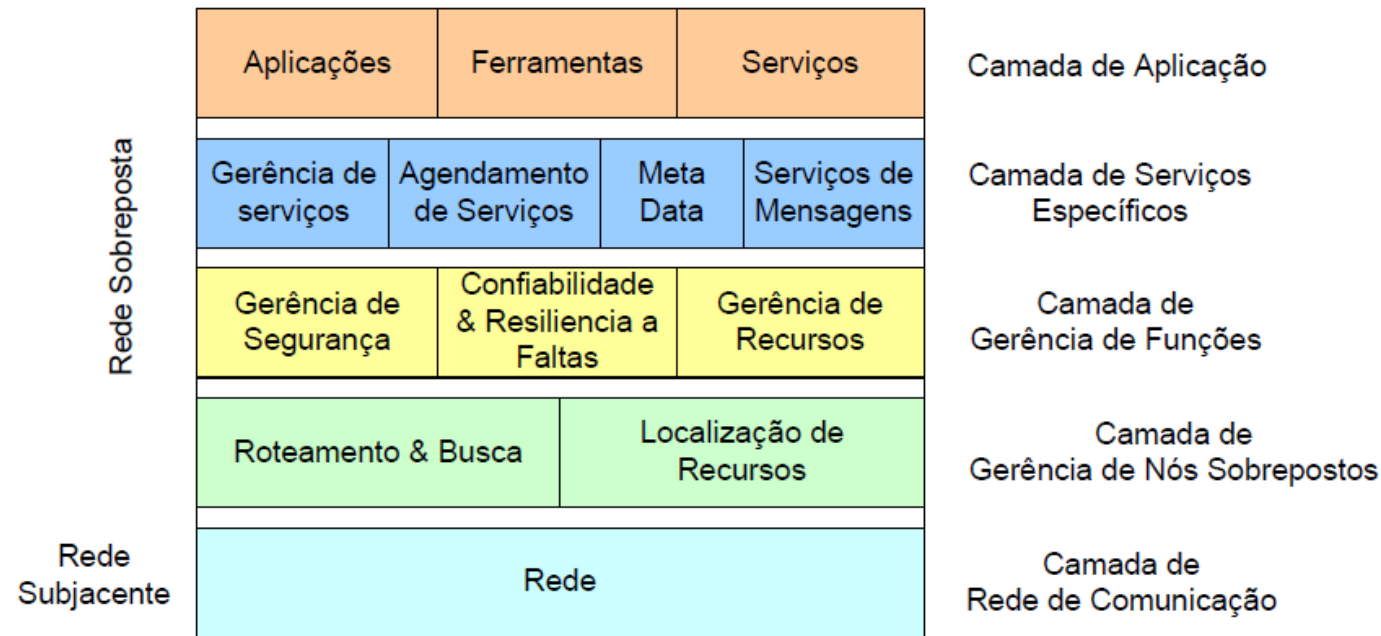
Arquitetura de redes P2P sobreposta



- **Camada de Gerência de Nós Sobrepostos** gerencia os pares, incluindo detecção de pares e algoritmos de roteamento para otimização do sistema, além da localização de recursos

Peer-to-peer: Rede Sobreposta

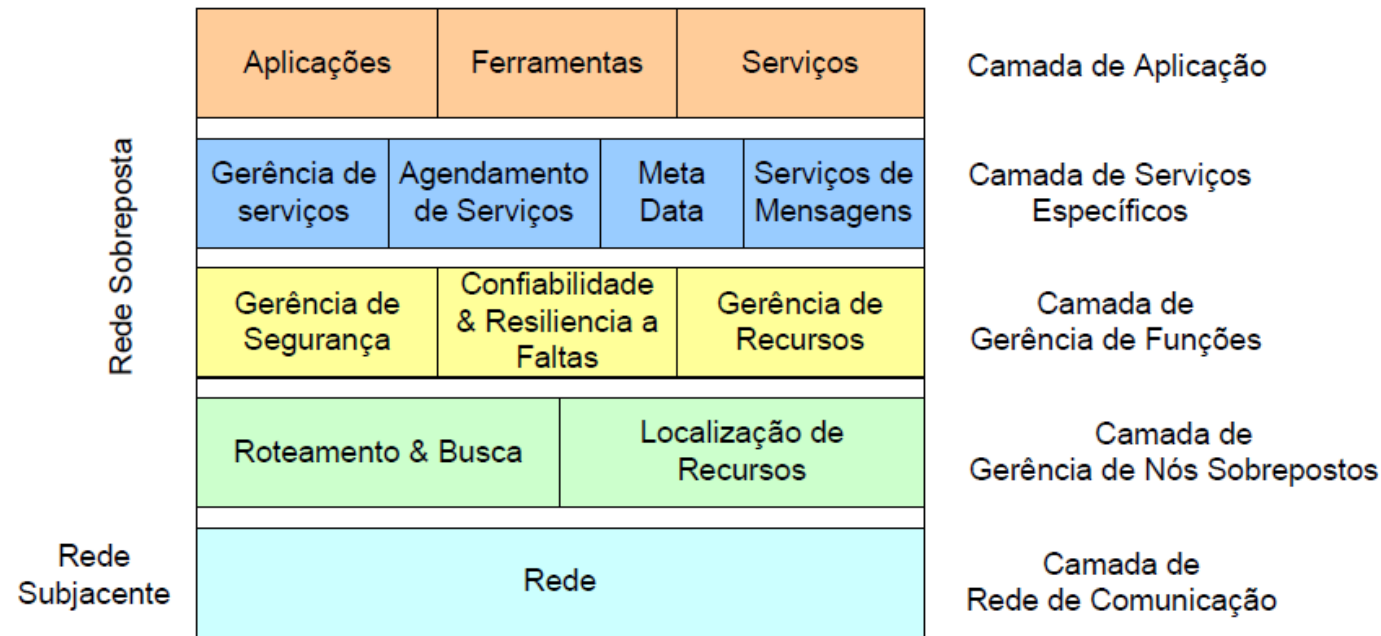
Arquitetura de redes P2P sobreposta



- **Camada de Gerência de Funções** lida com segurança, confiabilidade, tolerância a faltas e aspectos de disponibilidade de recursos para a manutenção da robustez do sistema P2P

Peer-to-peer: Rede Sobreposta

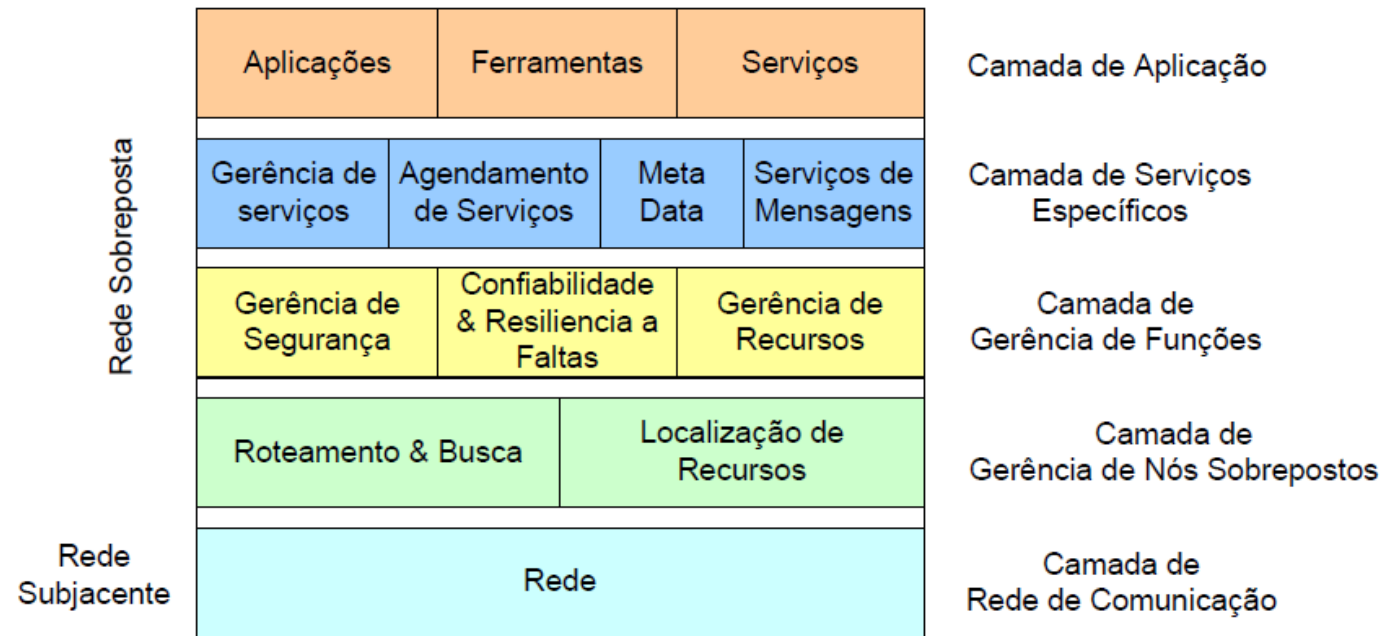
Arquitetura de redes P2P sobreposta



- **Camada de Serviços Específicos** dá suporte à infraestrutura P2P subjacente e aos componentes específicos das aplicações através de agendamento de serviços, gerência de conteúdo e arquivos.

Peer-to-peer: Rede Sobreposta

Arquitetura de redes P2P sobreposta



- **Camada de Aplicação** composta pelas ferramentas, aplicações e serviços que são implementados com funcionalidades específicas sobre a infraestrutura P2P sobreposta

Aplicações P2P

Compartilhamento de arquivos

- BitTorrent
- Retroshare
- Freenet
- GNUnet
- gtk-gnutella
- Filetopia
- Shareaza
- DC++
- Vuze
- WireShare
- Ares Galaxy
- Emule
- Azureus
- Kademlia
- SOULSEEK
- YaCy
- FastTrack
- BearShare
- Pichat
- Gnutella
- Kazaa
- BitComet
- Deluge

Aplicações P2P

Mensagens instantâneas P2P:

- Pidgin, P2P Messenger, FireChat, Briar, ICQ, MSN, Skype

Computação distribuída

- SETI@home (Search for Extra-Terrestrial Intelligence)
 - Objetivou analisar o máximo de sinais de rádio captados por radiotelescópios
 - <https://setiathome.ssl.berkeley.edu/>
- Folding@home
 - Enovelamento de proteína, mal formações,..
 - <https://foldingathome.org/>
- World Community Grid, OurGrid
 - Grade computacionais
 - <https://www.worldcommunitygrid.org/>

Aplicações P2P

CDNs (Content Delivery Networks)

- BitTorrent DNA, Internap, Pando Networks, Rawflow

Armazenamento distribuído

- Freenet: para compartilhar arquivos e publicar sites livres (<https://www.hyphanet.org/index.html>)
 - ofuscação do nó que origina uma consulta e do nó que responde
 - cada nó só conhece seus vizinhos na rota

Anonimato, resistência a censura

- Mixmaster remailers: anonimato no envio de mensagens.
- Tor: defesa contra monitoramento da rede (<https://www.torproject.org/>)

Aplicações P2P

Aplicação de trabalho colaborativo

- Aplicações deste gênero são conhecidas como groupware application
- É um software que suporta diversos pontos, tais como:
 - Comunicação e coordenação de usuários
 - E-mail
 - Calendário
 - Espaço de trabalho
 - Listas de discussão
 - Gerência de documentos
 - Vídeo conferência
- Microsoft SharePoint Workspace
 - Sistema colaborativo permitindo os usuários se comunicarem e interagirem diretamente de modo síncrono

Outros...

Aplicações P2P

Criptomoedas

- Bitcoin: uma criptomoeda descentralizada
 - sistema econômico alternativo (peer-to-peer electronic cash system), responsável pelo ressurgimento do sistema bancário livre
- Blockchain ("Cadeia de Blocos" em inglês)
 - Tipo de banco de dados distribuído, que tem a função de livro-razão de contabilidade pública onde são registradas as transações bitcoin.
 - Gerenciado por uma rede P2P, permitindo que os dados sejam transmitidos entre todos os participantes da rede (nós P2P) de maneira descentralizada e transparente

Pontos Importantes

P2P: Definição e Tipos de Aplicações

- Entender o que é rede P2P,
- vantagens
- e requisitos

P2P: Rede Sobreposta

- Entender o conceito e serviços

CAP 4. CAMADA DE APLICAÇÃO

AULA 18: PEER-TO-PEER (P2P) – CLASSIFICAÇÕES

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

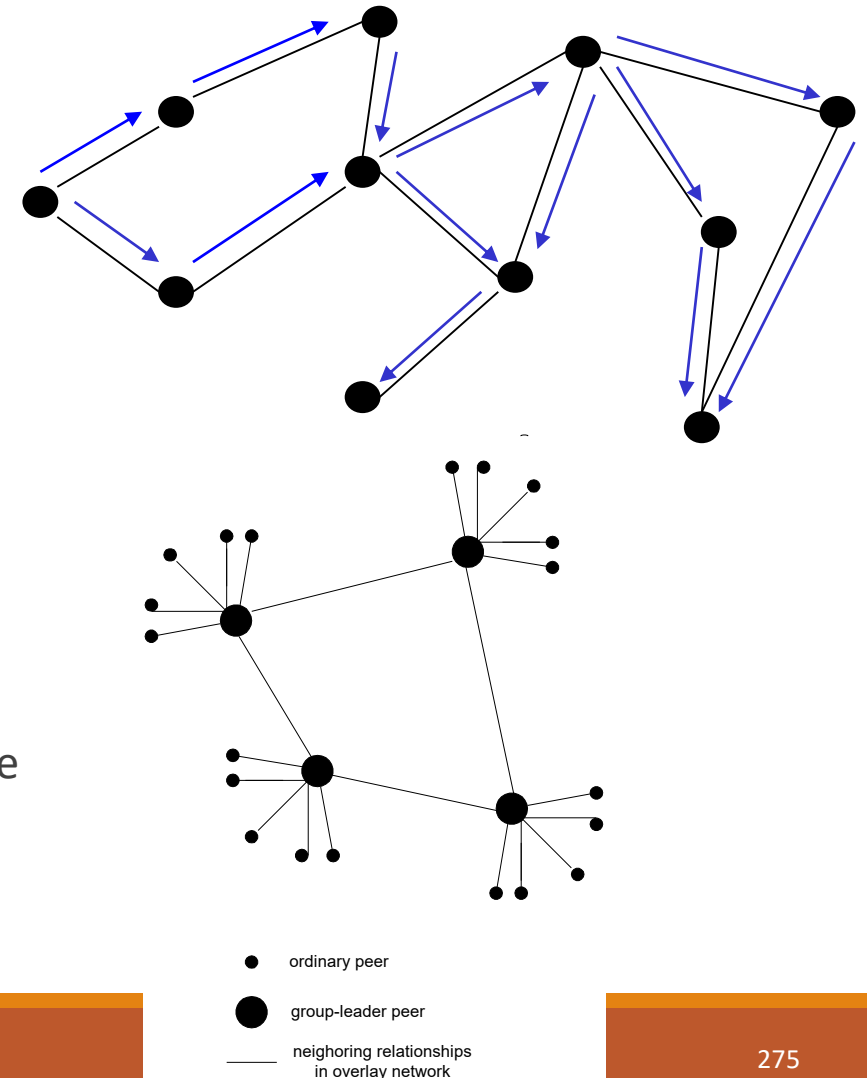
ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Peer-to-peer: Classificação

Modelos de Arquitetura P2P (1ª Classificação): quanto a distinção das funções realizadas pelos nós

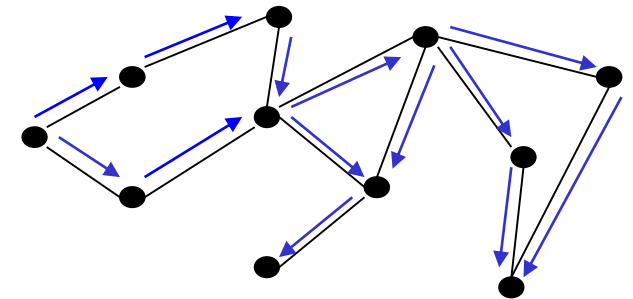
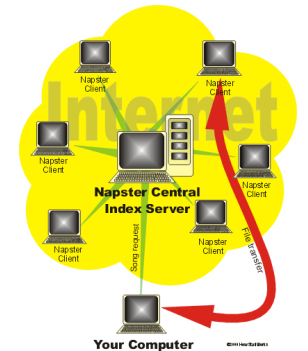
- **Descentralizado (P2P pura)**
 - Rede onde não há um nó central: todo nó na rede P2P tem a mesma função
 - Nós são autônomos e responsáveis por troca de recursos e por controle (gerenciamento) de recursos.
- **Semicentralizado**
 - Existem nós com funções específicas
 - nós centrais para controle
 - Conjunto de supernós
 - Utilizados na busca de informações
 - Mantém dados dos nós folhas para a busca, normalmente usando TabelaHash Distribuída (DHT)



Peer-to-peer: Classificação

Modelos de Arquitetura P2P (2ª Classificação): quanto ao modelo de busca

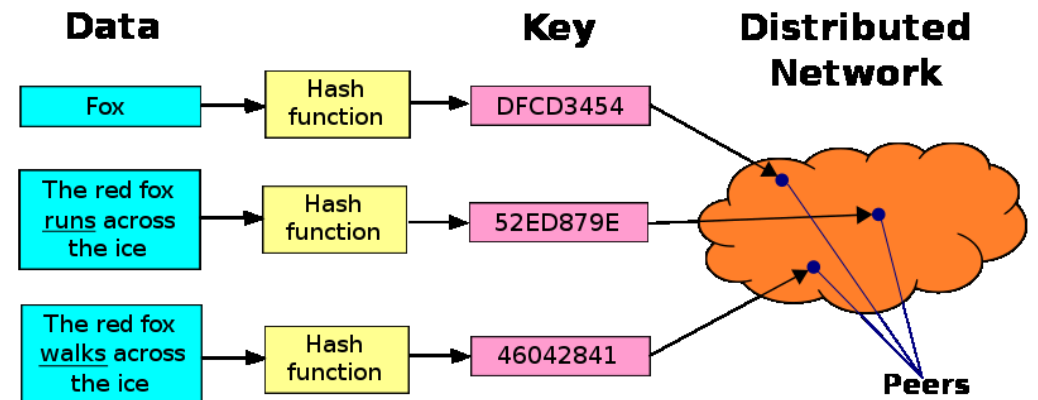
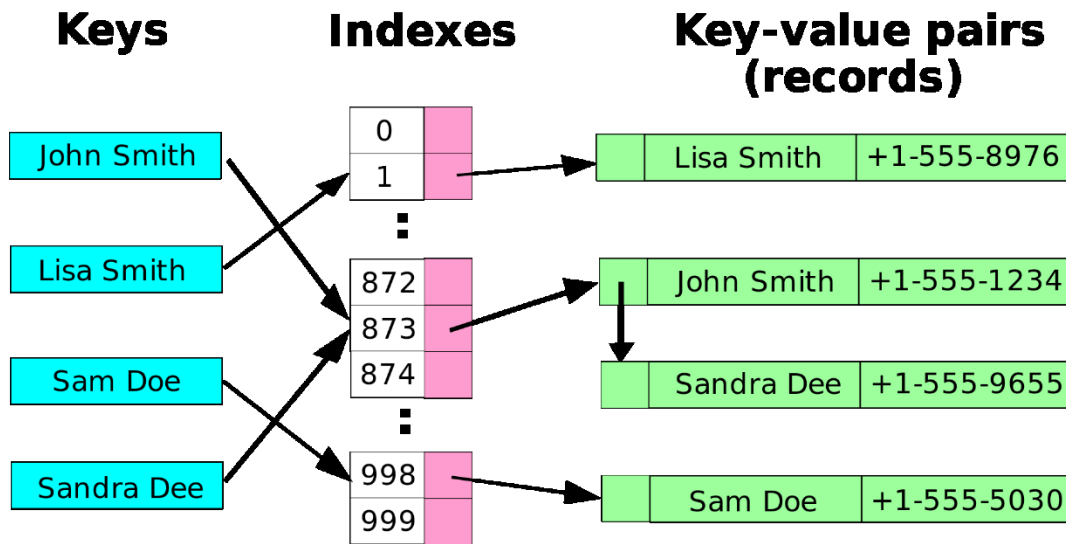
- **Busca centralizada**
 - Rede com um ponto central de busca e nós consultam o ponto central
 - Troca de informações é diretamente entre os nós;
- **Busca por inundação**
 - Normalmente a busca é limitada à vizinhança mais próxima do nó que fez a busca
 - busca é escalável, mas não é completa
- **Busca por tabela hash distribuída (DHT)**



Peer-to-peer: Classificação

Modelos de Arquitetura P2P (2ª Classificação): quanto ao modelo de busca

- Busca por tabelas hash distribuídas (DHTs)
 - Provê um serviço de lookup similar a uma tabela hash: pares (chave, valor)
 - Qualquer nó participante pode eficientemente recuperar o valor associado a uma dada chave
 - Responsabilidade de manter o mapeamento de chaves para valores é distribuída entre os nós



Peer-to-peer: Classificação

Modelos de Arquitetura P2P (3ª Classificação)

- **P2P Centralizada**
 - Modelo centralizado (busca centralizada) onde um nó central mantém um índice central com informações atualizadas.
 - Exemplo: sistemas de compartilhamento de arquivos Napster
- **P2P Descentralizada e Estruturada**
- **P2P Descentralizada e não Estruturada**

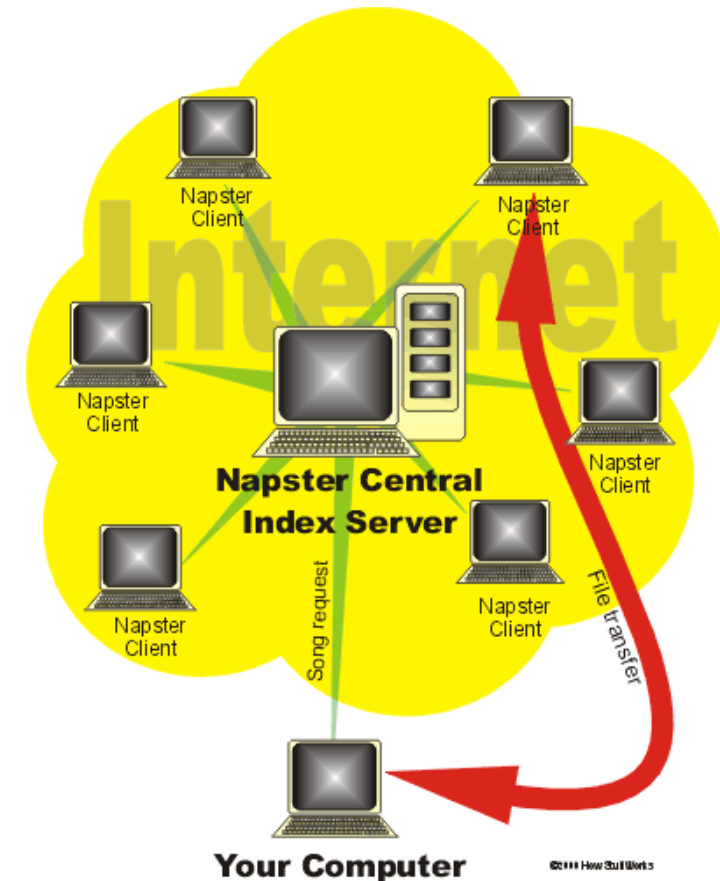
P2P Centralizado



Representante mais conhecido: Napster (original)

- Solução híbrida: parte centralizada, parte distribuída
- Índice centralizado, recuperação distribuída

Serviço de diretório pode limitar desempenho



P2P Centralizado

Exemplo: Napster

- Máquina de busca dedicada a encontrar arquivos MP3.
- Servidor central era utilizado para armazenar uma lista com as músicas disponibilizadas pelos usuários e onde elas estão localizadas.
- Programa cliente, instalado no computador dos usuários, faz uma consulta ao servidor Napster para obter informações sobre o arquivo desejado.
- Servidor Napster responde se existe o arquivo desejado e onde ele está localizado
 - Caso exista, uma conexão direta é estabelecida com o computador onde o arquivo está armazenado para que seja efetuado o download.

P2P Centralizado

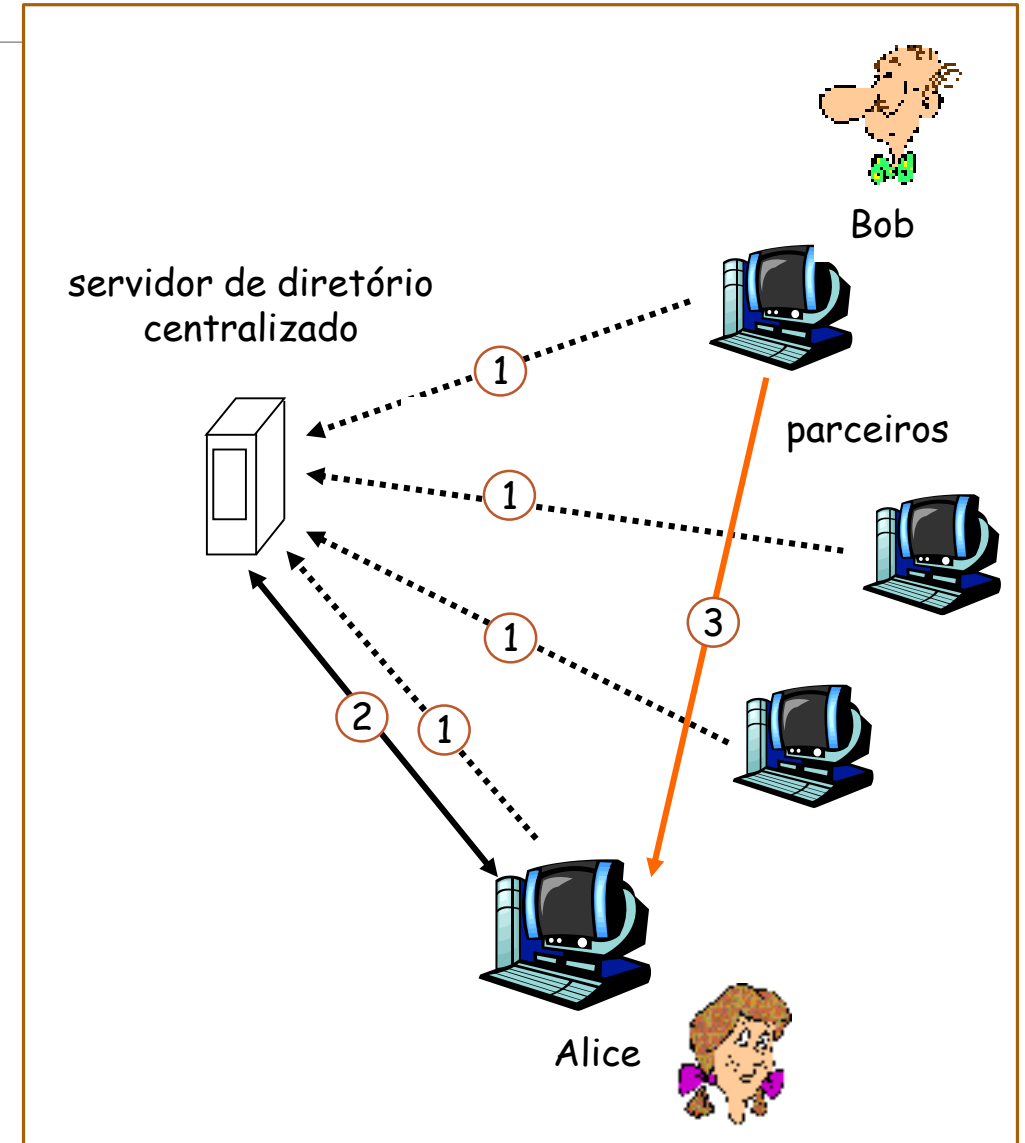
Projeto original do “Napster”

1) quando um parceiro conecta ele informa ao servidor central o seu:

- endereço IP
- conteúdo

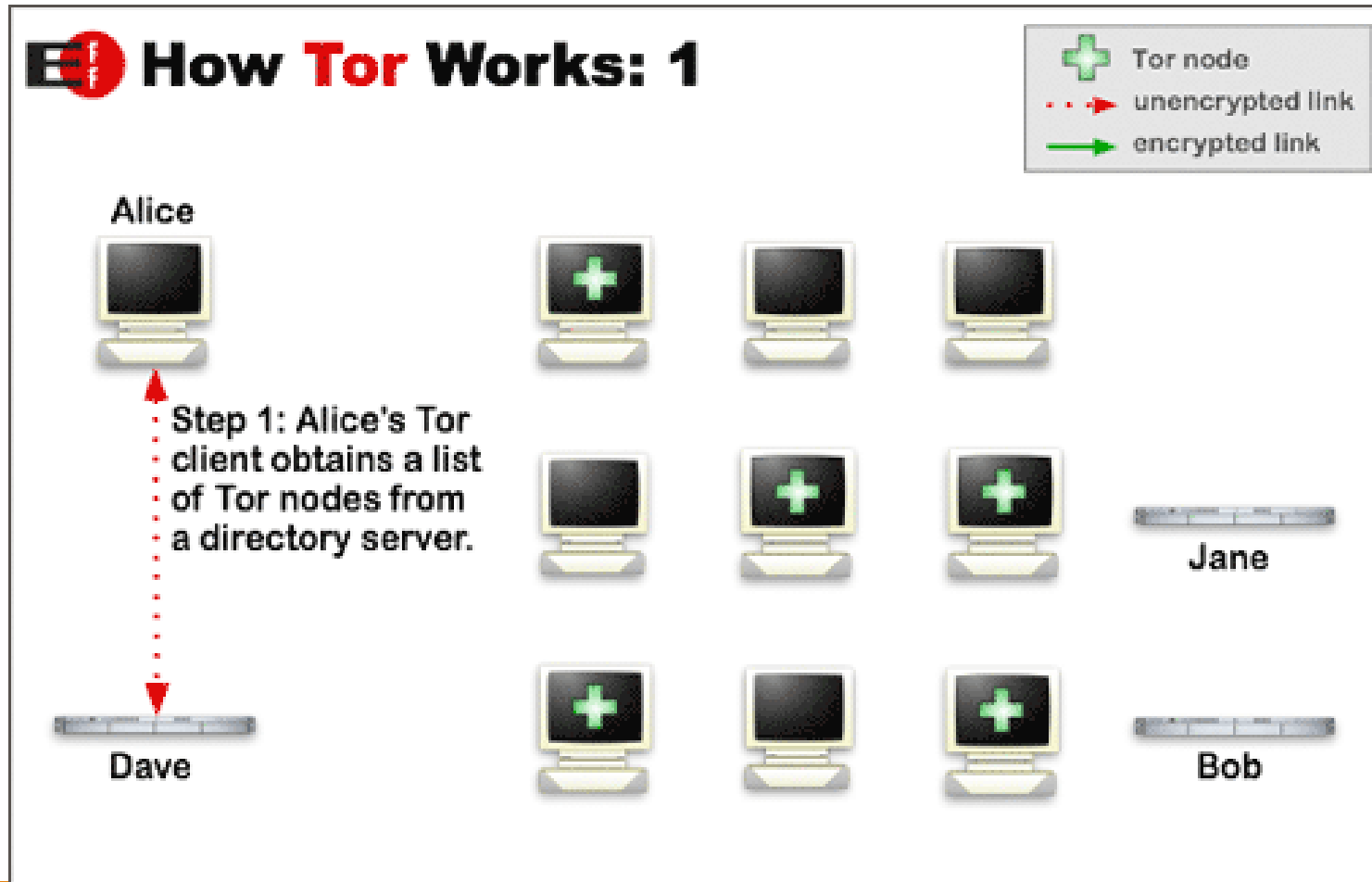
2) Alice consulta sobre a música “Hey Jude”

3) Alice solicita o arquivo a Bob



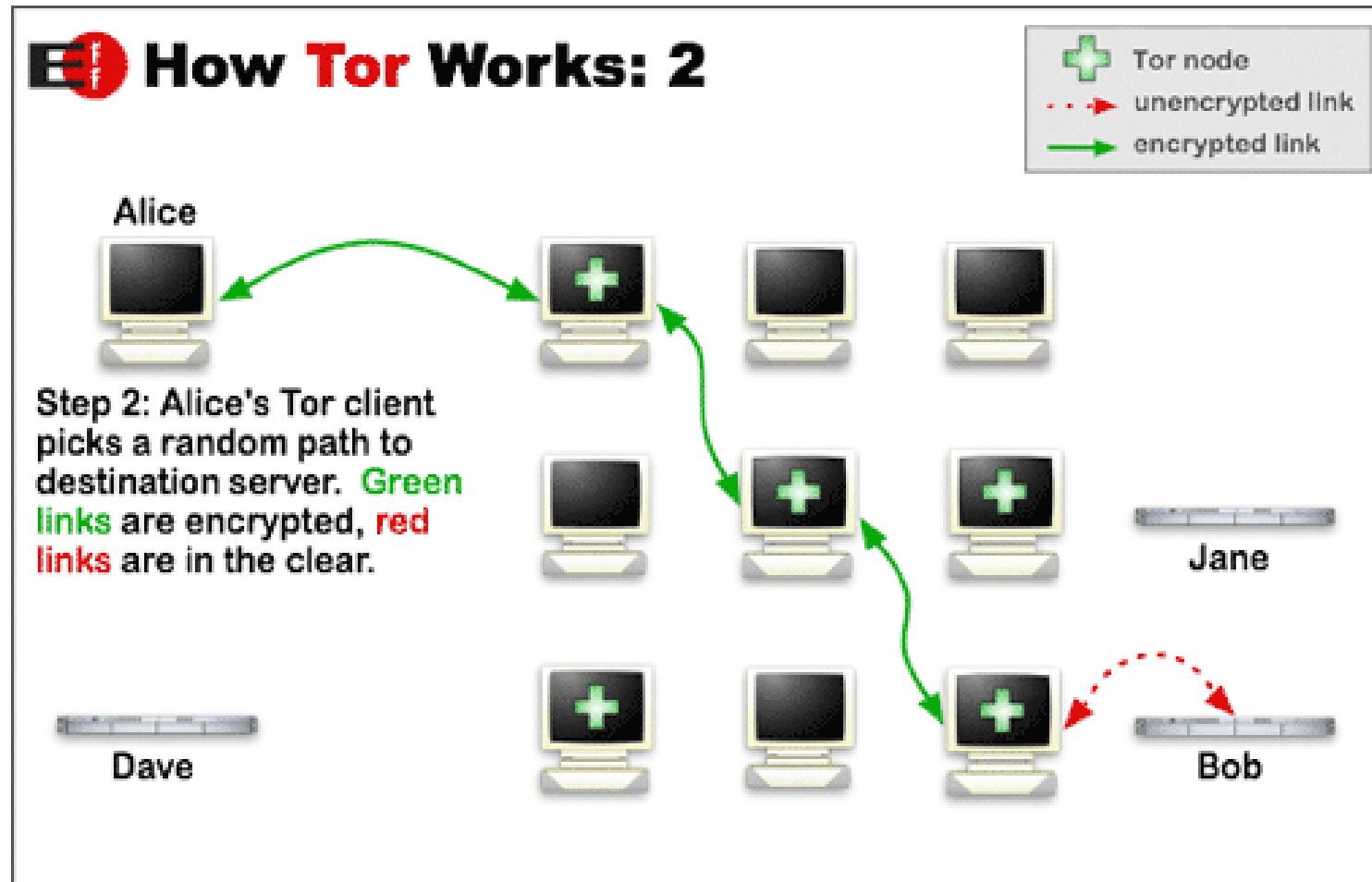
P2P Centralizado

Como Tor trabalha para garantir anonimato
(<https://www.torproject.org/>)



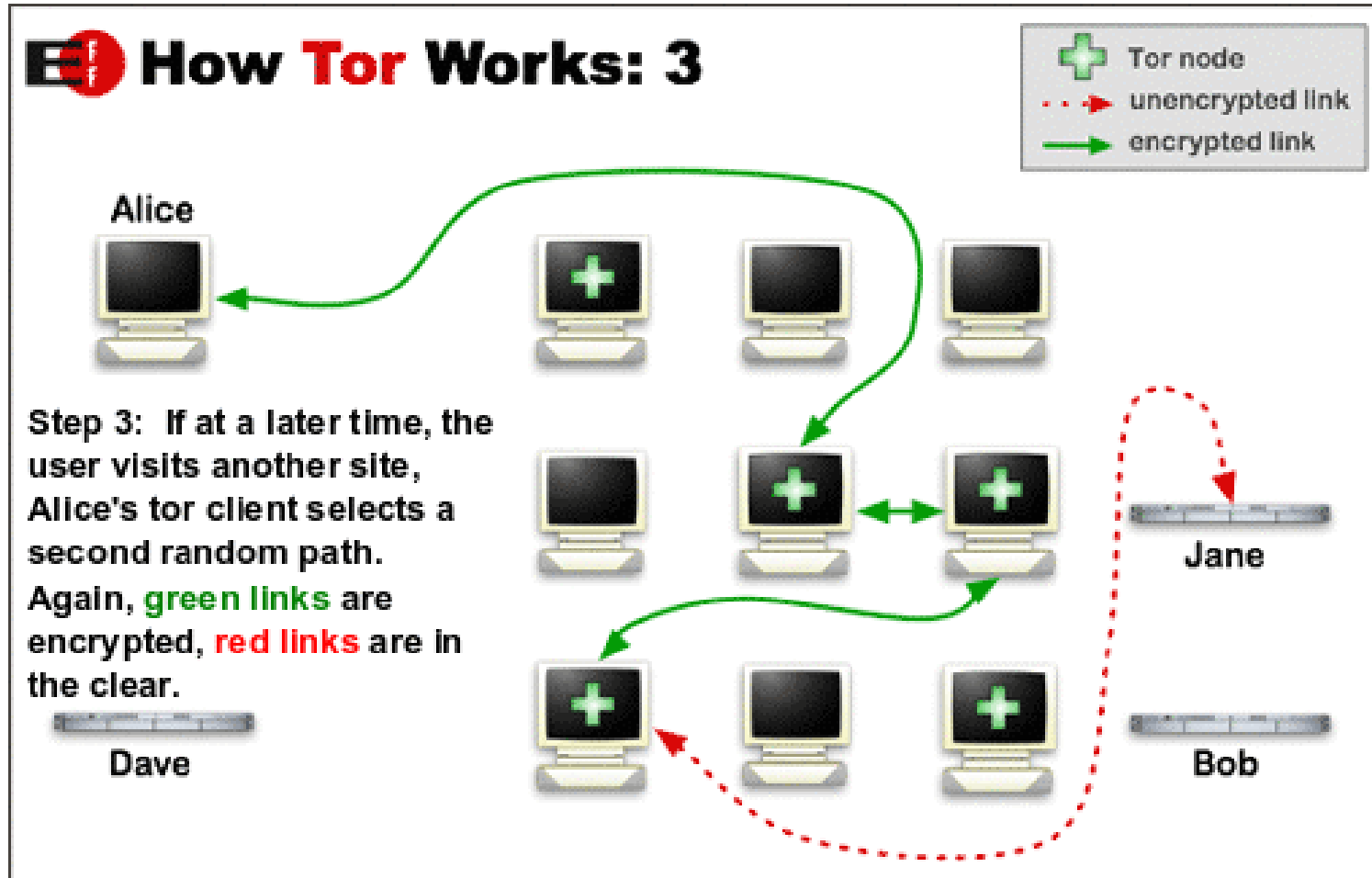
P2P Centralizado

Como Tor trabalha para garantir anonimato



P2P Centralizado

Como Tor trabalha para garantir anonimato



P2P Centralizado

Problemas

- Ponto único de falha
- Gargalo de desempenho
- Violação de Direitos Autorais
- A transferência de arquivo é descentralizada, mas a localização do conteúdo é altamente centralizada.

Peer-to-peer: Classificação

Modelos de Arquitetura P2P (3ª Classificação)

- P2P Centralizado
- P2P Descentralizada e Estruturada
 - Rede que não possui um servidor centralizado de diretório de informações
 - mas que tem uma estruturação entre os nós.
 - Topologia da rede é controlada e os documentos/arquivos são posicionados em locais que posteriormente tornam fácil a sua localização.
 - Em geral utiliza busca baseada em DHTs (Tabelas de espalhamento distribuídas)
 - Protocolos e implementações: Chord, Kademlia, Pastry, Tapestry e CAN, Viceroy
 - Aplicações: BitTorrent, eMule
- P2P Descentralizada e não Estruturada

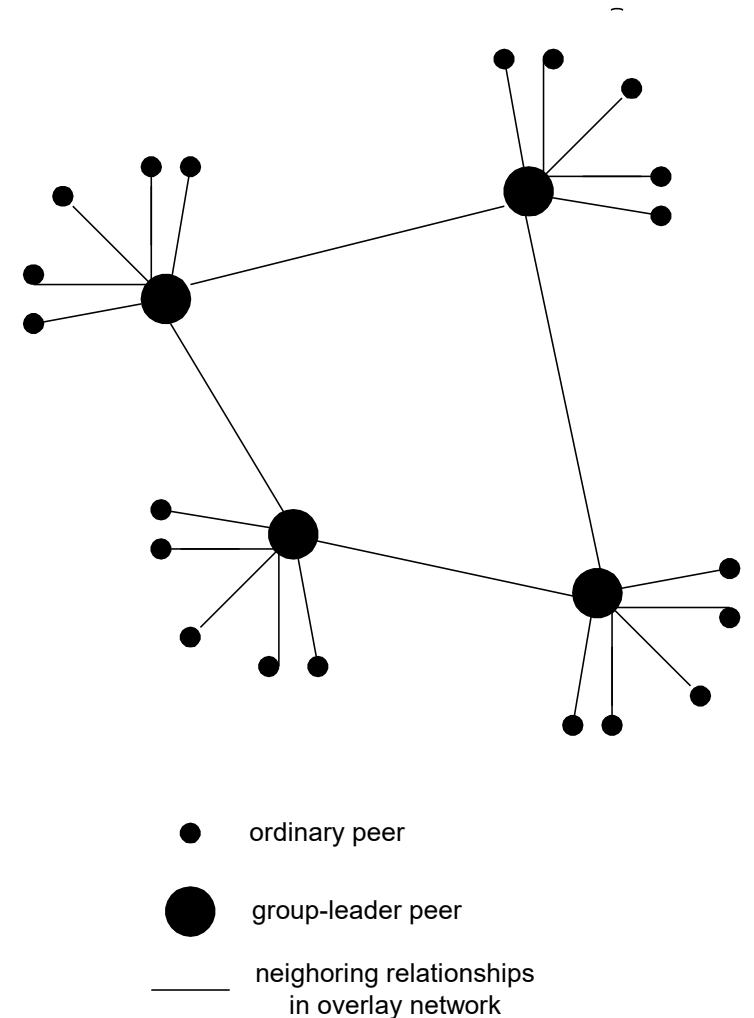
P2P Descentralizada e Estruturada

Cada nó é um líder de grupo ou está associado a um líder de grupo (nó filho)

Líder de um grupo segue o conteúdo de todos os seus filhos

Nó consulta líder do grupo

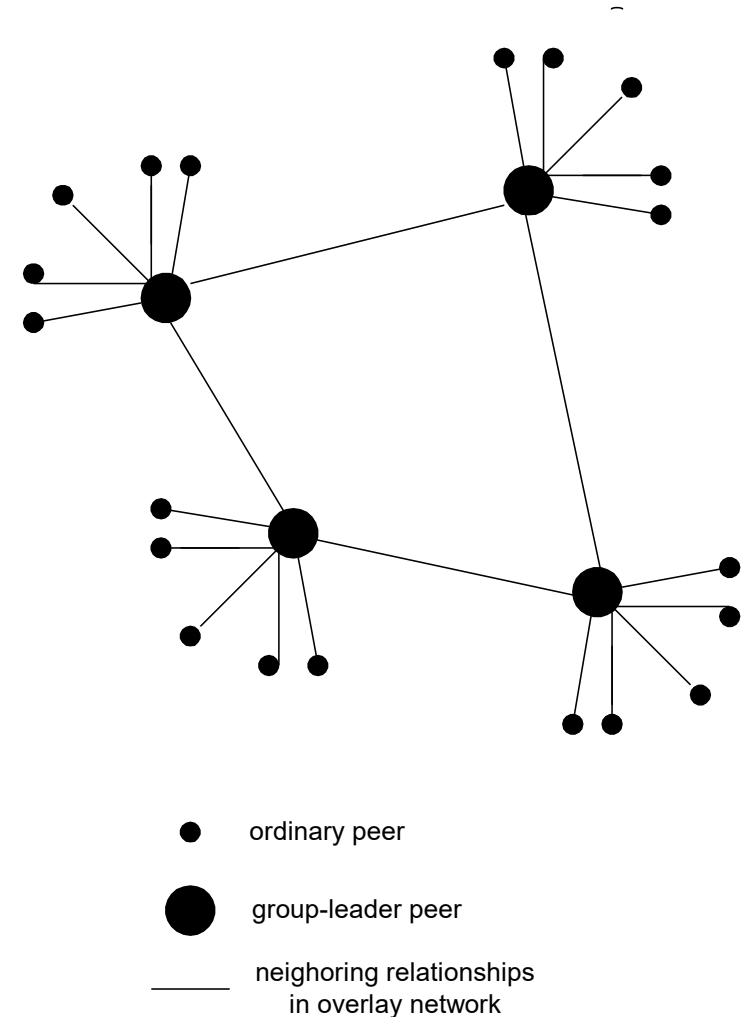
- Líder por sua vez pode consultar outros líderes de grupo.



P2P Descentralizada e Estruturada

Gnutella2

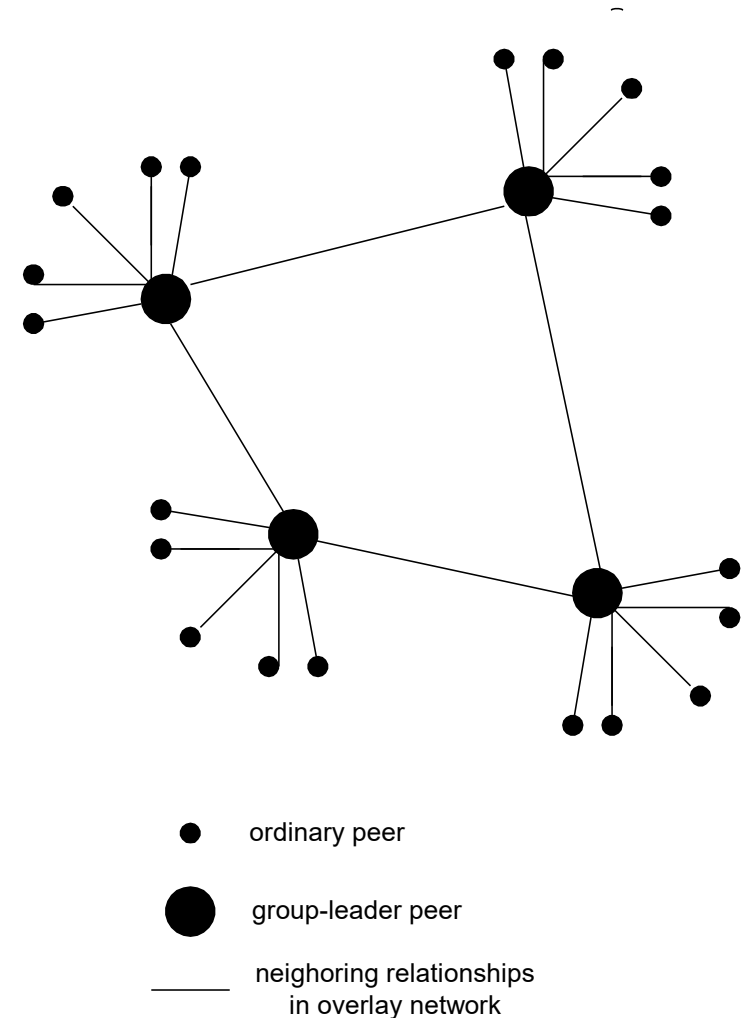
- Divide nós em dois grupos: folhas (leaves) e concentradores (hubs).
- As folhas mantêm uma ou duas conexões com os concentradores,
- Os concentradores aceitam milhares de folhas e muitas conexões de outros concentradores.
- Quando uma pesquisa é iniciada
 - nó obtém uma lista de concentradores se necessário, e contacta todos os concentradores da lista até o fim da lista ou se atingir um limite definido na consulta
 - permite um usuário encontrar um arquivo desejado facilmente sem sobrecarregar a rede
- Implementações: Limeware (descontinuada)



P2P Descentralizada e Estruturada

KaZaA (descontinuado)

- Utiliza o conceito de supernós para melhorar o desempenho da rede.
- Uma máquina que participa da rede (nó) e que possui um alto poder computacional e rápidas conexões com a Internet.
- Mantêm uma lista contendo os arquivos disponibilizados por outros usuários e o local onde eles estão armazenados.
- Quando uma busca é executada
 - Aplicação KaZaA procura primeiro no supernó mais próximo do usuário que iniciou a consulta; este supernó retorna um conjunto de respostas para o usuário e encaminha a consulta para outros supernós.
 - Uma vez localizado o usuário que possui o arquivo, uma conexão é estabelecida diretamente entre os peers para que seja efetuado o download.



P2P Descentralizada e Estruturada

Rede sobreposta

- parceiros são nós
- arcos entre os parceiros e seus líderes de grupo
- arcos entre alguns pares de líderes de grupo
- vizinhos virtuais

Nó de inicialização

- o parceiro que está se conectando ou é alocado a um líder de grupo ou é designado como líder

Vantagens da abordagem

- Ausência de servidor de diretórios centralizado
 - serviço de localização distribuído entre os parceiros
 - mais difícil de ser tirado do ar

Desvantagens da abordagem

- é necessário um nó de inicialização
- os líderes de grupo podem ficar sobrecarregados

Peer-to-peer: Classificação

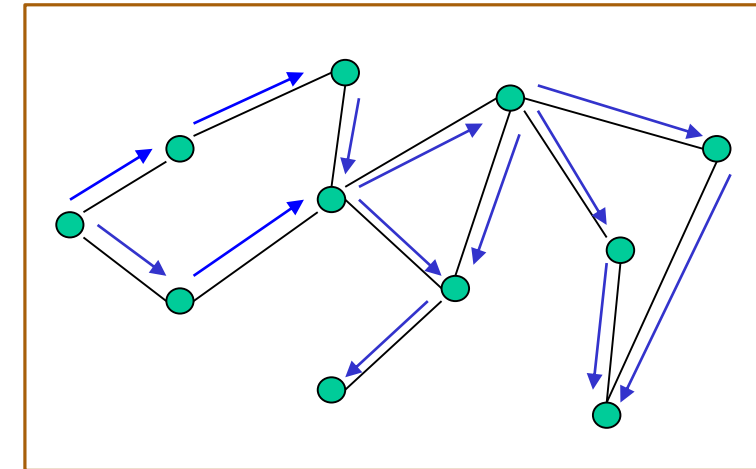
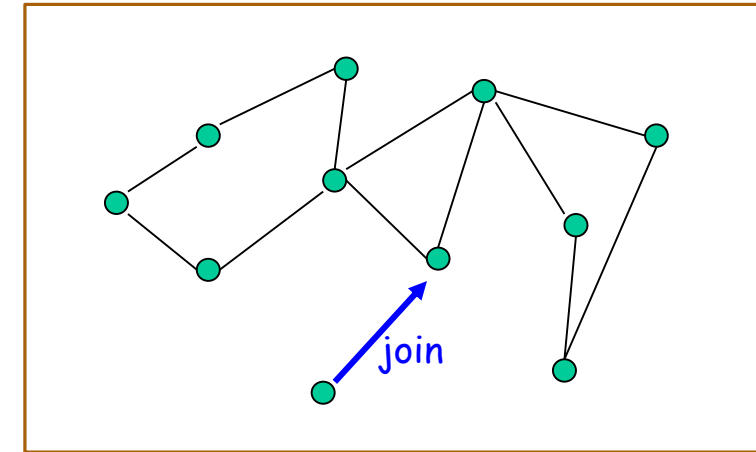
Modelos de Arquitetura P2P (3ª Classificação)

- P2P Centralizado
- P2P Descentralizada e Estruturada
- P2P Descentralizada e não Estruturada
 - Rede que não possui servidor centralizado, nem controle preciso sobre a topologia e localização/busca de documentos.

P2P Descentralizada e não Estruturada: Inundação de consultas

Gnutella

- sem hierarquias
 - usa nó de inicialização para aprender sobre os demais
 - mensagem de união (join)
- Envia consulta aos vizinhos
 - Vizinhos encaminham a consulta
 - Se o parceiro consultado possui o objeto, ele devolve a mensagem para o parceiro que fez a consulta



P2P Descentralizada e não Estruturada: Inundação de consultas

Prós

- os parceiros têm responsabilidades similares: sem líderes de grupo
- altamente descentralizado
- nenhum parceiro mantém informação de diretório

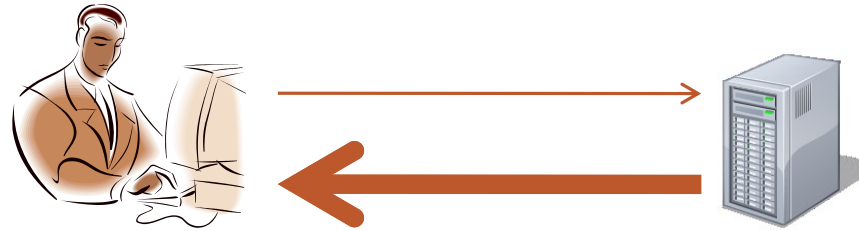
Contras

- tráfego excessivo de consultas
- raio da consulta: pode não ter o conteúdo quando estiver presente
- nó de inicialização
- manutenção da rede sobreposta

P2P: Problema para a Internet

P2P é tráfego Simétrico

- Normalmente o tráfego da Internet é assimétrico



- P2P é uma aplicação simétrica

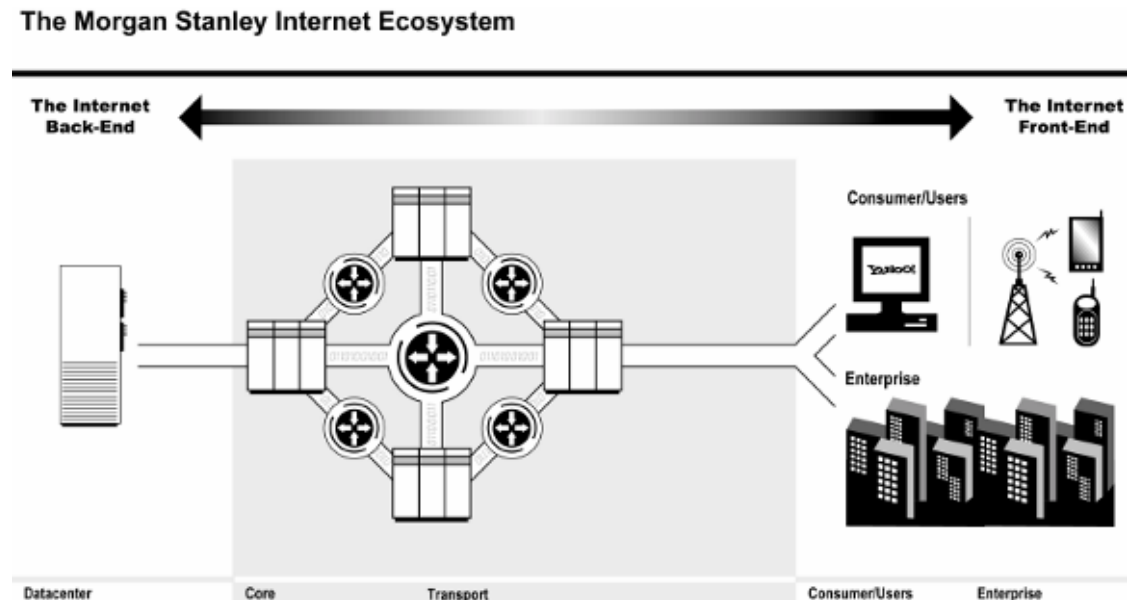


Internet de hoje não é projetada para tráfego simétrico

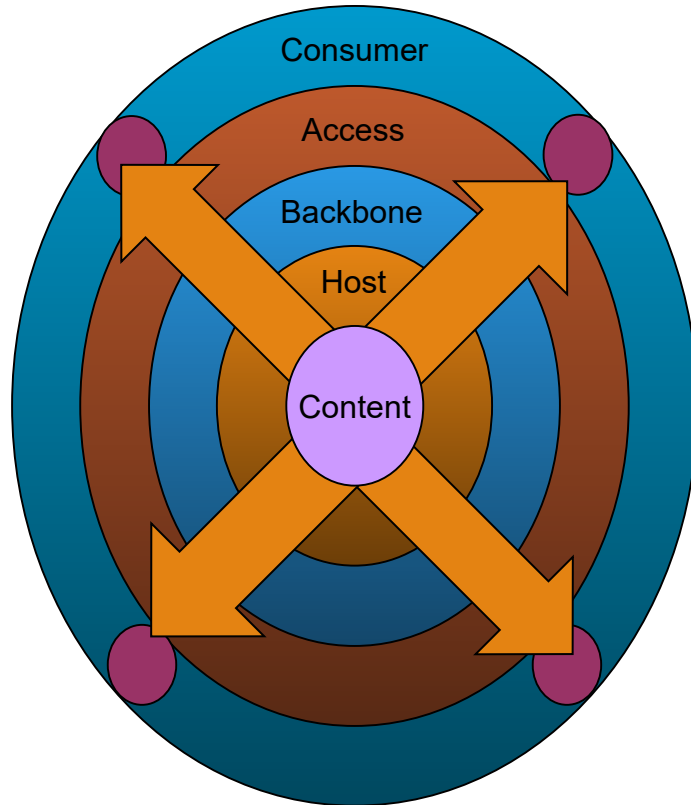
Internet foi construído com a premissa de distribuição núcleo-para-borda

- Centros de dados de alta capacidade, High Capacity Data Centres, núcleos de alta capacidade, bordas pequenas assimétricas
- Custo e desempenho otimizado pelos provedores

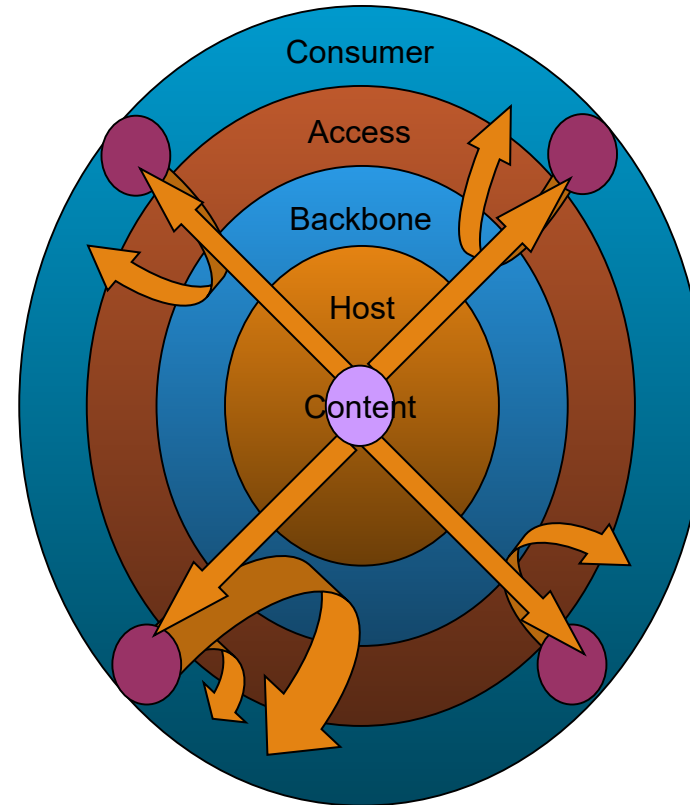
P2P coloca nas bordas o conteúdo da distribuição nas redes de acesso



Internet de hoje não é projetada para tráfego simétrico



Fluxos de Conteúdo tradicionais



Fluxos de conteúdos no P2P

Redes de Acesso Banda Larga Projeto- DSL

ADSL e Cable modem foram projetados para tráfego assimétrico

- Web, VoD

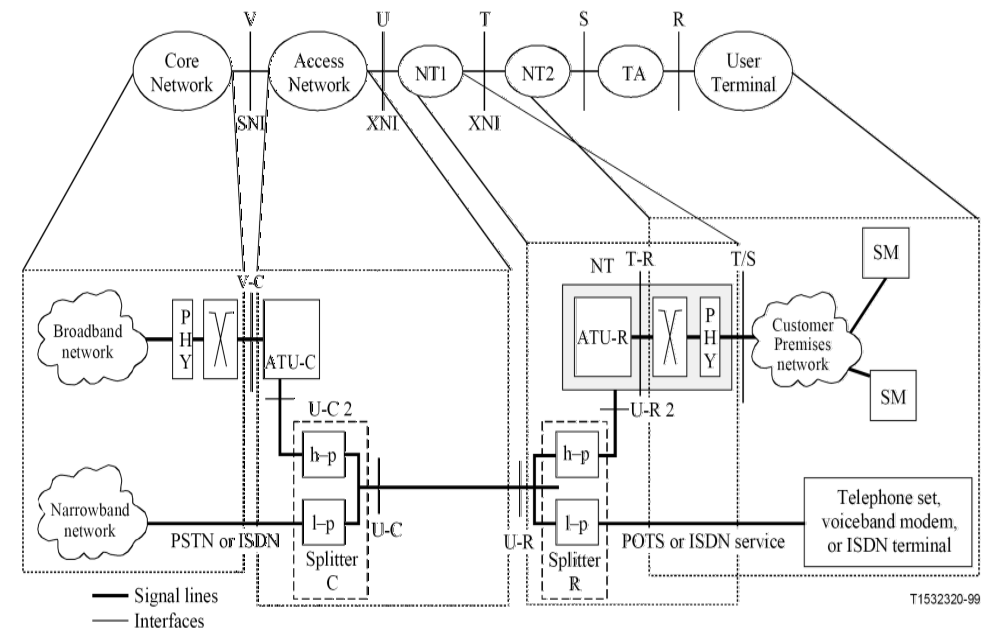
ADSL (varios padrões)

- Menor: 1.5Mbps download, 0,5 Mbps upload (3:1)
- Maior: 12Mbps download, 1,3 Mbps upload (9:1)

ADSL2+

- Download <24Mbps
- Upload < 3.3Mbps
- 7:1

Assimetria está aumentando



Impacto do P2P nos Provedores

Maior parte dos protocolos tradicionais são orientados a download (HTTP)

- Isto torna o P2P um problema para os provedores de última milha (mas perdendo o destaque)

Top Apps by Upstream Volume – Fixed				Top Apps by Upstream Volume – Mobile			
Upstream Volume				Upstream Volume			
	Application	% US Vol	Sub. Volume		Application	% US Vol	Sub. Volume
1	iCloud	9%	101 MB	1	Tik Tok	8%	10 MB
2	FaceTime	5%	56 MB	2	Facebook Messenger	8%	10 MB
3	Google Cloud Storage	4%	47 MB	3	Snapchat	6%	7 MB
4	Google	4%	46 MB	4	Amazon AWS	6%	7 MB
5	YouTube	4%	46 MB	5	YouTube	5%	6 MB
6	BitTorrent	4%	46 MB	6	Facebook	5%	6 MB
7	Tik Tok	3%	40 MB	7	iCloud	5%	6 MB
8	Facebook	3%	31 MB	8	Google	4%	5 MB
9	Discord	2%	28 MB	9	WhatsApp	4%	5 MB
10	Amazon AWS	2%	25 MB	10	Instagram	3%	4 MB

Fonte: <https://www.sandvine.com>

Pontos Importantes

P2P

- 3ª Classificação
- Problema para os provedores/Internet

CAP 4. CAMADA DE APLICAÇÃO

AULA 19: PROGRAMAÇÃO VIA API SOCKET

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Programação via Sockets

Meta da aula

- Ensinar como construir aplicações cliente/servidor que se comunicam usando sockets

Socket API

- Introduzido no BSD4.1 UNIX, 1981
- Seguem o paradigma cliente/servidor
- Dois tipos de serviço de transporte via socket API:
 - Datagrama não confiável
 - Confiável e orientado a fluxo de bytes

socket

Uma interface controlada pelo S.O., local ao host, que é criada e controlada pela aplicação

Uma porta na qual o processo de aplicação pode enviar e receber dados de outros processo de aplicação

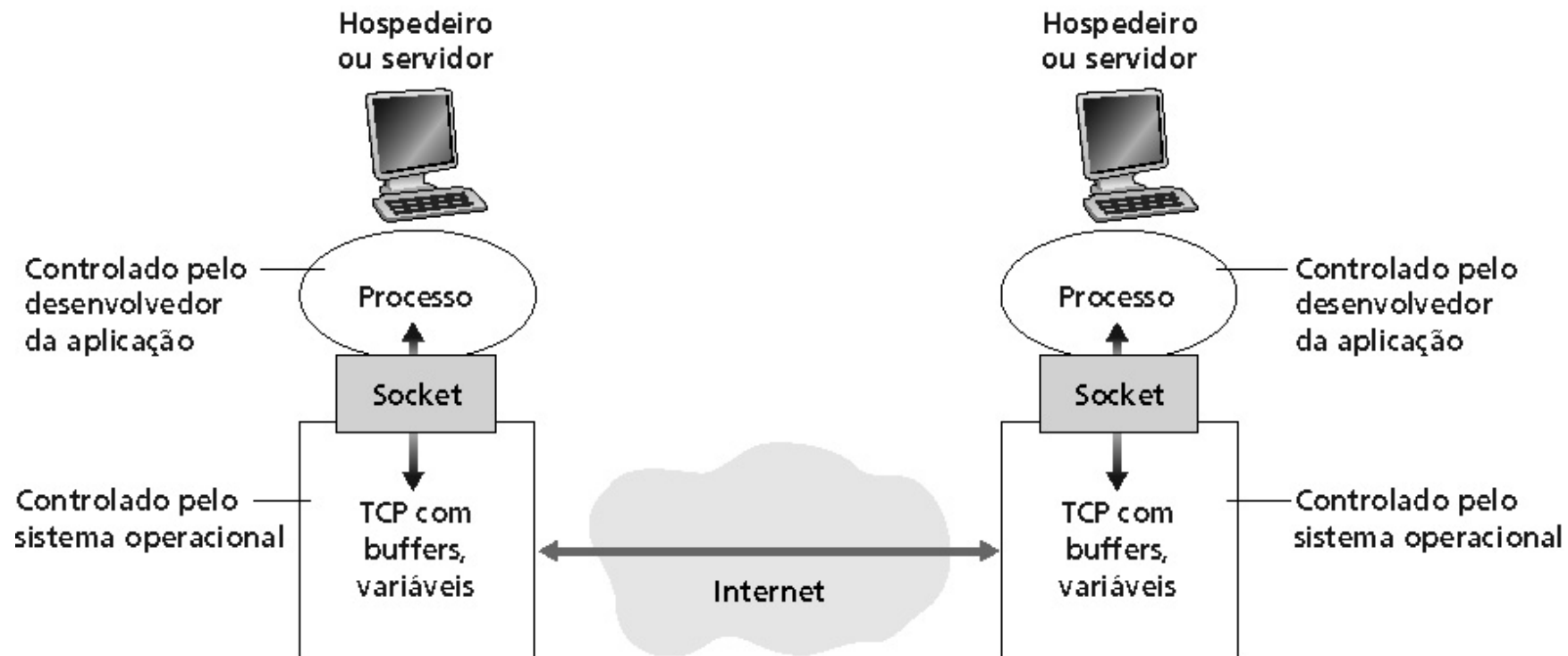
Programação de Socket usando TCP

Socket

- uma porta entre o processo de aplicação e o protocolo de transporte fim-a-fim (TCP ou UDP)

Serviço TCP

- transferência confiável de bytes de um processo para outro



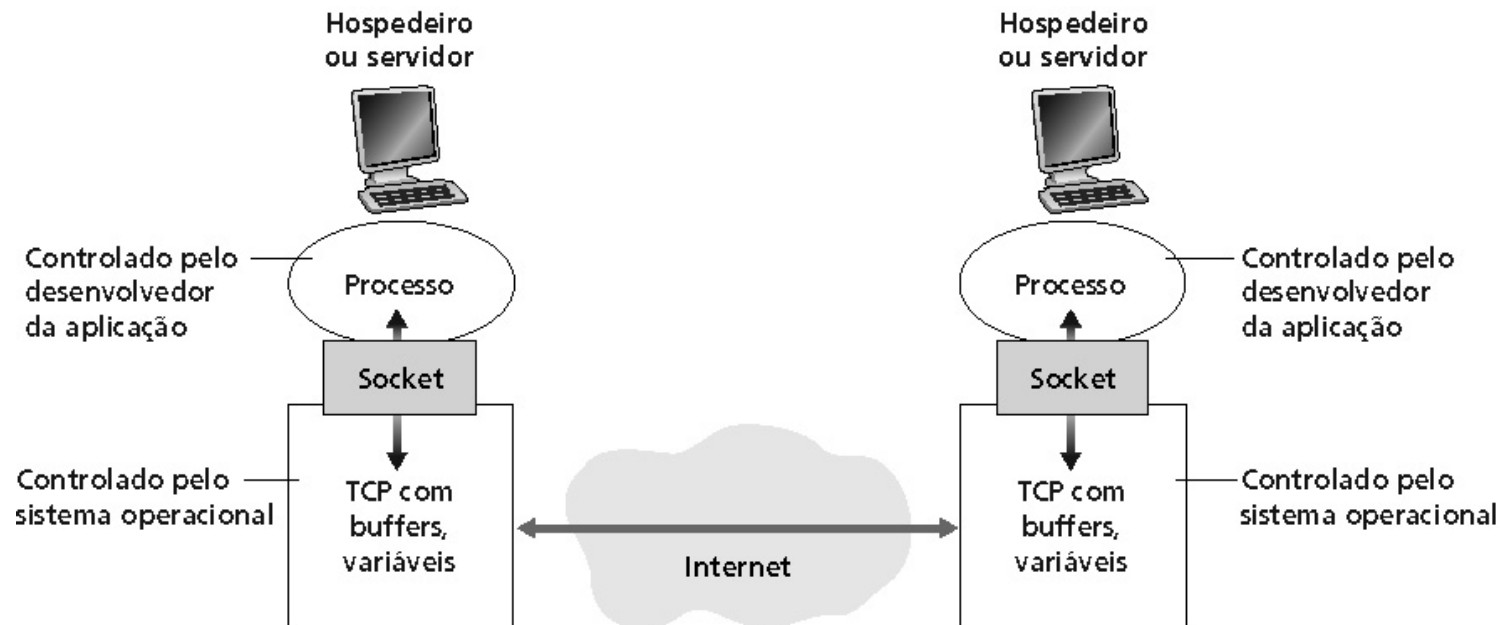
Programação de Socket usando TCP

Cliente deve contactar servidor

- processo servidor deve antes estar em execução
- servidor deve antes ter criado socket (porta) que aguarda contato do cliente

Cliente cria socket TCP:

- socket TCP local ao cliente
- especificar endereço IP, número de porta do processo servidor



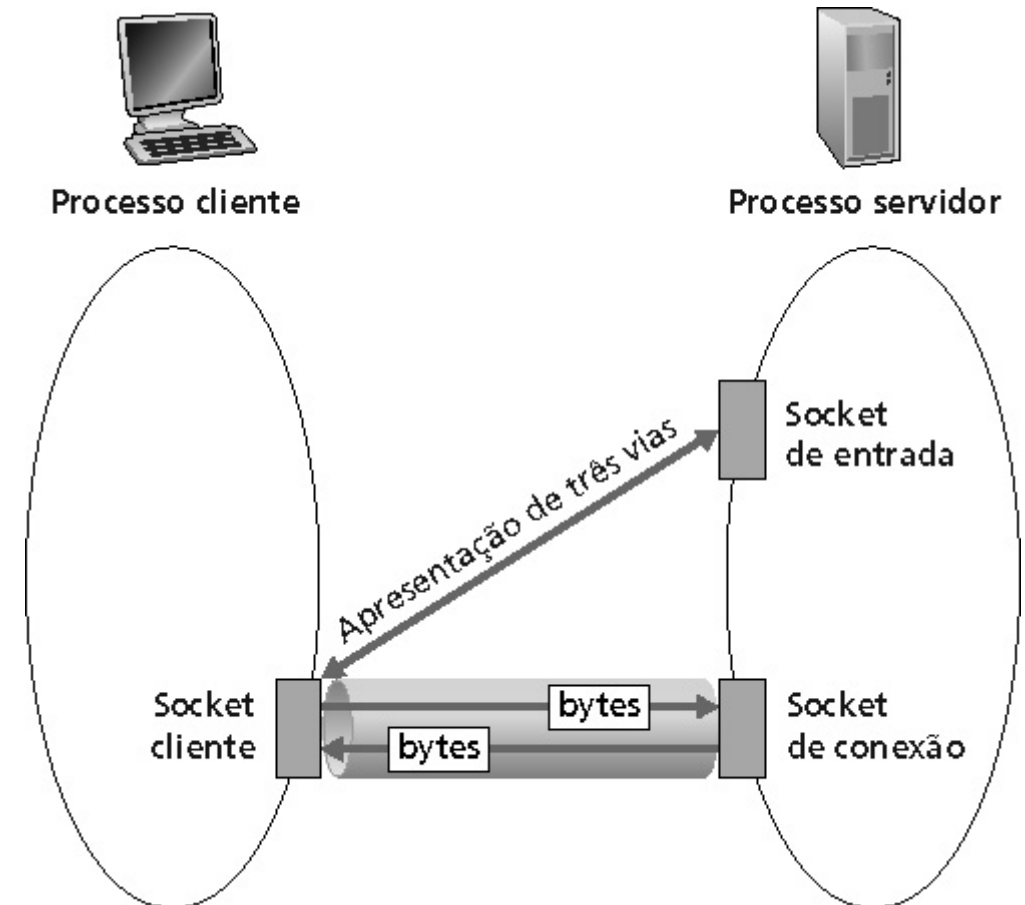
Programação de Socket usando TCP

Quando cliente cria socket

- TCP do cliente estabelece conexão com TCP do servidor

Quando contatado pelo cliente, o TCP do servidor

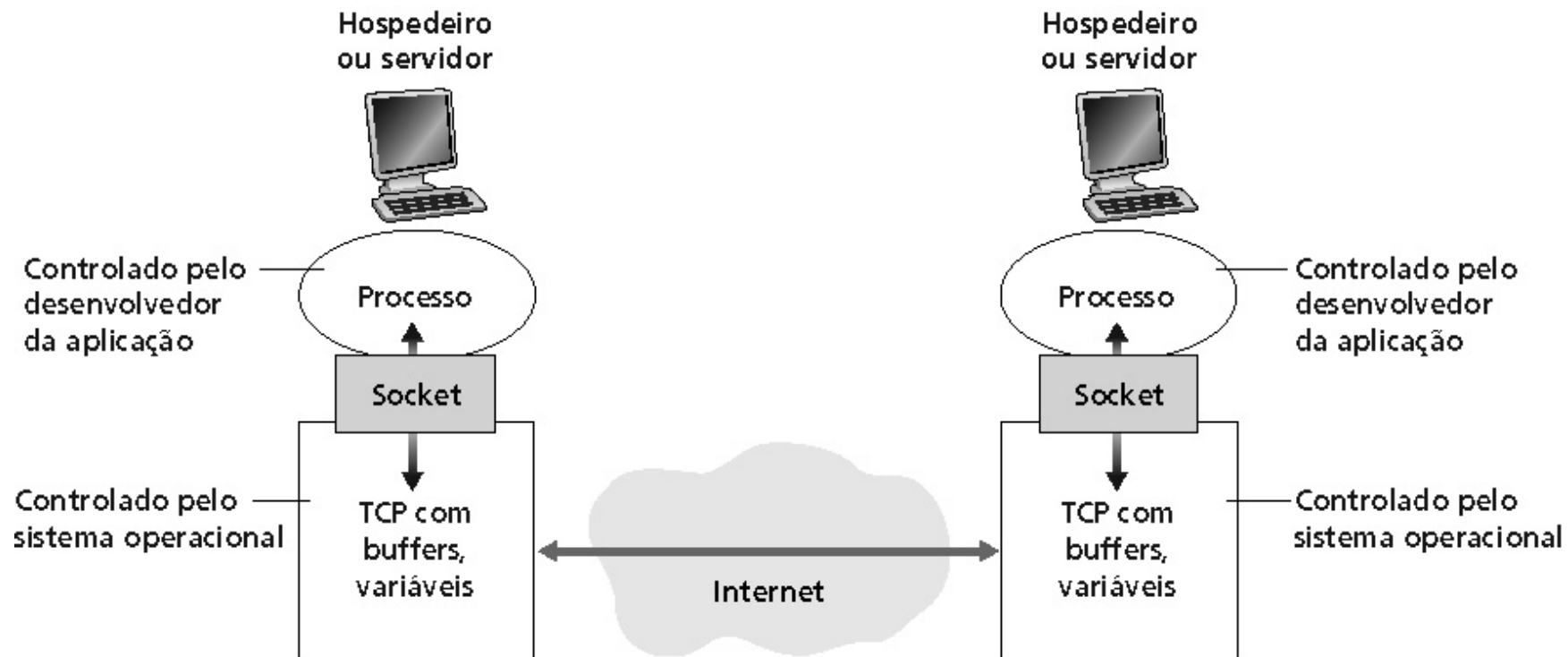
- cria socket novo para que o processo servidor possa se comunicar com o cliente
- permite que o servidor converse com múltiplos clientes



Programação de Socket usando TCP

Do ponto de vista da aplicação

- TCP provê transferência confiável, ordenada de bytes (“tubo”) entre cliente e servidor



Jargão stream

Stream

- é uma sequência de caracteres que fluem para dentro ou para fora de um processo

Stream de entrada

- é agregado a alguma fonte de entrada para o processo, ex.: teclado ou socket

Stream de saída

- é agregado a uma fonte de saída, ex.: monitor ou socket

Exemplo: Cliente e Servidor UDP

Cliente TCP

```
import socket

HOST = "127.0.0.1" # Nome ou end. IP do servidor
PORT = 65432      # Porta usada pelo servidor

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    # socket.AF_INET indica que é um socket IPv4
    # socket.SOCK_STREAM indica que o socket é TCP

    # estabelece a conexão
    s.connect((HOST, PORT))

    message = input("Digite uma string: ").encode('utf-8')
    # envia string codificada
    s.sendall(message)

    #recebe os dados da conexão, até 1024B e decodifica
    data = s.recv(1024).decode('utf-8')

print(data)
```

Servidor TCP

```
import socket

HOST = "127.0.0.1" # localhost
PORT = 65432      # porta alocada

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    # socket.AF_INET indica que é um socket IPv4
    # socket.SOCK_STREAM indica que o socket é TCP

    s.bind((HOST, PORT)) # Liga o socket ao endereço/porta
    s.listen()           # bloqueia aguardando conexão
    conn, addr = s.accept() # Aceita a conexão

    with conn:
        print(f"Connected by {addr}")
        while True:
            data = conn.recv(1024) # leitura de até 1024B
            if not data:
                break
            conn.sendall(data.decode('utf-8').upper().encode('utf-8'))
```

Exemplo: Cliente e Servidor TCP

Cliente UDP

```
import socket

HOST = "127.0.0.1" # Nome ou end. IP do servidor
PORT = 5000        # Porta usada pelo servidor
dest = (HOST, PORT) # Destino da mensagem

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as udp:
    # socket.AF_INET indica que é um socket IPv4
    # socket.SOCK_DGRAM indica que o socket é UDP

    message = input("Digite uma string: ").encode('utf-8')
    udp.sendto(message, dest)
    msg, server = udp.recvfrom(1024)
    print(msg.decode('utf-8'))
    udp.close()
```

Servidor UDP

```
import socket

HOST = "127.0.0.1" # localhost
PORT = 5000        # porta alocada

with socket.socket(socket.AF_INET, SOCK_DGRAM) as udp:
    # socket.AF_INET indica que é um socket IPv4
    # socket.SOCK_DGRAM indica que o socket é UDP

    udp.bind((HOST, PORT)) # Liga o socket ao endereço/porta
    msg, cliente = udp.recvfrom(1024)
    print(f"Received from {cliente}")
    response = msg.decode('utf-8').upper().encode('utf-8')
    udp.sendto(response, cliente)
```

Exemplo: Web Server Python

```
import socket
# Define socket host e porta
SERVER_HOST = '0.0.0.0'
SERVER_PORT = 8000

# Create socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((SERVER_HOST, SERVER_PORT))
server_socket.listen(1)
print('Listening on port %s ...' % SERVER_PORT)

while True:
    # Espera por conexões
    client_connection, client_address = server_socket.accept()

    # obté o pedido do cliente
    request = client_connection.recv(1024).decode()
    # Parse HTTP headers
    headers = request.split('\n')
    command = headers[0].split()
    method = command[0]
    filename = command[1]

    ...
```

```
#Analisa pedido
if method != 'GET':
    response = 'HTTP/1.0 501 NOT IMPLEMENTED\n\nNot
                Implemented Method'
else:
    # Trata no me arquivo
    if filename == '/':
        filename = 'index.html'
    else:
        filename = filename[1:] # Remove caractere "/"
    # Obtém recurso
    try:
        fin = open(filename)
        content = fin.read()
        fin.close()
        # Monta resposta
        response = 'HTTP/1.0 200 OK\n\n' + content
    except FileNotFoundError:
        response = 'HTTP/1.0 404 NOT FOUND\n\nFile Not
                    Found'
    client_connection.sendall(response.encode())
    client_connection.close()

# Fecha socket
server_socket.close()
```

Pontos Importantes

Programação com API Socket

- Conhecimentos gerais para entendimento de como é selecionado e usado o protocolo da camada de transporte